



MORGAN & CLAYPOOL PUBLISHERS

KNOWLEDGE GRAPHS

Aidan Hogan | Eva Blomqvist | Michael Cochez
Claudia d'Amato | Gerard de Melo | Claudio Gutierrez
Sabrina Kirrane | José Emilio Labra Gayo | Roberto Navigli
Sebastian Neumaier | Axel-Cyrille Ngonga Ngomo
Axel Polleres | Sabbir M. Rashid | Anisa Rula
Lukas Schmelzeisen | Juan Sequeda
Steffen Staab | Antoine Zimmermann

SYNTHESIS LECTURES ON
DATA, SEMANTICS AND KNOWLEDGE

Ying Ding and Paul Groth, *Series Editors*

KNOWLEDGE

Give us feedback!

**Aidan Hogan**

IMFD, DCC, Universidad de Chile

Eva Blomqvist

Linköping University

Michael Cochez

Vrije Universiteit Amsterdam and Discovery Lab, Elsevier

Claudia d'Amato

University of Bari

Gerard de Melo

HPI, University of Potsdam and Rutgers University

Claudio Gutierrez

IMFD, DCC, Universidad de Chile

Sabrina Kirrane

WU Vienna

José Emilio Labra Gayo

Universidad de Oviedo

Roberto Navigli

Sapienza University of Rome

Sebastian Neumaier

St. Pölten University of Applied Sciences

Axel-Cyrille Ngonga Ngomo

DICE, Universität Paderborn

Axel Polleres

WU Vienna

Sabbir M. Rashid

Tetherless World Constellation, Rensselaer Polytechnic Institute

Anisa Rula

University of Brescia

Lukas Schmelzeisen

Universität Stuttgart

Juan Sequeda

data.world

Steffen Staab

Universität Stuttgart and University of Southampton

Antoine Zimmermann

École des mines de Saint-Étienne

About the book

The book is published by Springer in the series *Synthesis Lectures on Data, Semantics, and Knowledge* edited by Ying Ding and Paul Groth. The book and series was previously published by Morgan & Claypool. Please cite the book as:

Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, Antoine Zimmermann (2021) *Knowledge Graphs*, Synthesis Lectures on Data, Semantics, and Knowledge, No. 22, 1–237, DOI: 10.2200/S01125ED1V01Y202109DSK022, Springer.

BibTeX entry of this book:

```
@book{kg-book,
  author = {Hogan, Aidan and Blomqvist, Eva and Cochez, Michael and
d'Amato, Claudia and de Melo, Gerard and Guti\errez, Claudio and
Kirrane, Sabrina and Labra Gayo, Jos\'e Emilio and Navigli, Roberto and
Neumaier, Sebastian and Ngonga Ngomo, Axel-Cyrille and Polleres, Axel and
Rashid, Sabbir M. and Rula, Anisa and Schmelzeisen, Lukas and
Sequeda, Juan F. and Staab, Steffen and Zimmermann, Antoine},
  doi = {10.2200/S01125ED1V01Y202109DSK022},
  isbn = {9783031007903},
  language = {English},
  number = {22},
  numpages = {237},
  publisher = {Springer},
  series = {Synthesis Lectures on Data, Semantics, and Knowledge},
  title = {{K}nowledge {G}raphs},
  url = {https://kgbook.org/},
  year = {2021}
}
```

ISBN paperback:

9783031007903

ISBN ebook:

9783031019180

Copyright © 2021 by Springer. All rights reserved.

Access options

HTML version:

You are currently reading the free HTML version of the book, the most recent of which is available at <https://kgbook.org/>.^{*}

PDF Version:

You can download or buy the book from Springer (the book was previously published by Morgan & Claypool). Academic and Corporate licences are available.

Hard copy:

You can order from Springer or Amazon.

note * You can see the [comments](#) as new issue tutorial [at] go can be found on GitHub

SYNTHESIS LECTURES ON ON DATA, SEMANTICS, AND KNOWLEDGE #22

📖 Abstract 📖

Give us feedback!

This book provides a comprehensive and accessible introduction to knowledge graphs, which have recently garnered notable attention from both industry and academia.

Knowledge graphs are founded on the principle of applying a graph-based abstraction to data, and are now broadly deployed in scenarios that require integrating and extracting value from multiple, diverse sources of data at large scale. The book defines knowledge graphs and provides a high-level overview of how they are used. It presents and contrasts popular graph models that are commonly used to represent data as graphs, and the languages by which they can be queried before describing how the resulting data graph can be enhanced with notions of schema, identity, and context. The book discusses how ontologies and rules can be used to encode knowledge as well as how inductive techniques — based on statistics, graph analytics, machine learning, etc. — can be used to encode and extract knowledge. It covers techniques for the creation, enrichment, assessment, and refinement of knowledge graphs and surveys recent open and enterprise knowledge graphs and the industries or applications within which they have been most widely adopted. The book closes by discussing the current limitations and future directions along which knowledge graphs are likely to evolve.

This book is aimed at students, researchers, and practitioners who wish to learn more about knowledge graphs and how they facilitate extracting value from diverse data at large scale. To make the book accessible for newcomers, running examples and graphical notation are used throughout. Formal definitions and extensive references are also provided for those who opt to delve more deeply into specific topics.

Keywords

knowledge graphs, graph databases, knowledge graph embeddings, graph neural networks, ontologies, knowledge graph refinement, knowledge graph quality, knowledge bases, artificial intelligence, semantic web, machine learning

Preface

The origins of this book can be traced back to a Dagstuhl Seminar, held in 2018, on the topic of Knowledge Graphs. At the time of the seminar, the topic was quickly becoming mainstream in academia and industry, but there were conflicting messages as to what a “knowledge graph” was. Much of the discussion of the seminar centred on this question, and there were divergent opinions as to how knowledge graphs could (or should) be defined; how they relate to previous concepts such as graph databases, knowledge bases, ontologies, RDF graphs, property graphs, semantic networks, etc.; and how the emerging area of Knowledge Graphs should be positioned with respect to the established areas of Artificial Intelligence, Big Data, Databases, Graph Theory, Logic, Machine Learning, Knowledge Representation, Natural Language Processing, Networks (in their various forms), and the Semantic Web. As the discussion continued, a consensus began to emerge: Knowledge Graphs, as a topic, involves a novel confluence of techniques stemming from previously disparate scientific communities, with the unifying goal of developing novel graph-based techniques for better integrating and extracting value from diverse knowledge sources at large scale.

As a follow-up to the seminar, the attendees agreed that in order to foster this unifying view of Knowledge Graphs, there was a need for a manuscript that would serve as a general introduction to the area. This manuscript would:

- motivate knowledge graphs and the value of abstracting data as graphs;
- survey the historical context of knowledge graphs and the key initiatives leading to their popularisation;

[Give us feedback!](#)

- draw together disparate views of knowledge graphs into a unifying definition;
- provide an introduction to the key techniques that knowledge graphs enable, relating to querying, validation, reasoning, learning, refinement, enrichment, quality assessment, and more besides;
- describe how knowledge graphs are used in practice, surveying the companies using knowledge graphs, the applications they are used for, the open knowledge graphs that have been published, etc.;
- delineate future research directions for knowledge graphs.

The manuscript would then serve as an introductory text for students, practitioners and researchers new to the area, helping to form a consensus in terms of what is a knowledge graph, laying the foundations for future developments.

The goal of preparing this manuscript was an ambitious one, and involved drawing together and distilling down a vast amount of literature on a diverse range of topics into a set of key concepts described in an accessible way. For this reason, the manuscript has been prepared by many authors, who have lent their knowledge and expertise to the preparation of specific sections. A short version of the manuscript was first published as a tutorial paper [Hogan et al., 2021], consisting of an abridged version of the first five chapters of this book, along with a summary of how knowledge graphs are used in practice, and conclusions. However, there was not enough space to describe all of the important developments in the area. This led us to publish this book, which further includes topics relating to the creation, enrichment, quality assessment, refinement and publication of knowledge graphs, as well as formal definitions, a historical perspective, and extended discussion throughout.

The book is divided into ten chapters. The first chapter provides a general introduction to the area, defines the concept of a “knowledge graph”, and provides a high-level overview of how knowledge graphs are currently being used. The second chapter presents and contrasts popular graph models that are commonly used to represent data as graphs, and the languages by which they can be queried. The third chapter describes how the resulting data graph can be enhanced with notions of schema, identity and context. The fourth chapter discusses how ontologies and rules can be used to encode knowledge, and how they enable deductive forms of reasoning. The fifth chapter delves into how inductive techniques – based on statistics, graph analytics, machine learning, etc. – can be used to encode and extract knowledge. The sixth chapter is dedicated to techniques for the creation and enrichment of knowledge graphs from legacy sources of data. The seventh chapter enumerates a variety of quality measures that can be used to assess a knowledge graph in terms of its fitness for use in a variety of applications. The eighth chapter presents key methods for the refinement of knowledge graphs, with the goal of improving their completeness and correctness. The ninth chapter provides a survey of the open and enterprise knowledge graphs that have emerged in recent years, along with the industries within which, and the applications for which, they have been most widely adopted. The tenth chapter wraps up the book with discussion of the current limitations and future directions along which knowledge graphs are likely to evolve. An appendix further covers knowledge graphs from an historical perspective, establishing their significance in the broader context of the academic study of data and knowledge, as well as surveying prior definitions of “knowledge graphs” from the literature.

A key aim of this book is to be accessible to a broader audience. While background knowledge of related topics such as Databases, Logic, Machine Learning, Semantic Web, etc., will help to understand some of the particular topics mentioned, such a background is not necessary to follow the general concepts described within. The book aims to motivate and illustrate the various concepts it introduces from a practical perspective, and in order to be as accessible as possible, relies heavily on an example-driven presentation using a graphical notation. For the reader wishing to dig more into the technical minutiae, we complement this discussion with formal definitions throughout; however, the reader more interested in understanding the general concepts and their rationale will find the discussion to be self-contained if they choose to skip the definitions presented in visually distinctive boxes.

The book serves as an entry point for those new to the topic, and may thus serve as a useful textbook for university courses, for researchers who are venturing into the topic for the first time, and for practitioners who wish to understand more about how knowledge graphs might be of use within their company or organisation, or indeed, how to maximise the value of the knowledge graphs that they are currently developing. Readers who are already active within specific sub-areas Knowledge Graphs may further appreciate the technical definitions included, the references to other

[Give us feedback!](#)

literature provided, and the broader perspective that this book offers in terms of the other related sub-areas and how they complement each other.

By drawing together diverse techniques from disparate areas, Knowledge Graphs has become an exciting topic in terms of both research and applications. We expect to see growing interest on this topic as the years advance, and indeed hope that this book will help to more firmly establish the foundations of this topic, and to foster future developments upon these foundations, potentially by its readers.

Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, Antoine Zimmermann
September 2021

Acknowledgements

We thank the organisers and attendees of the Dagstuhl Seminar on “Knowledge Graphs”. We also thank those who provided feedback on this content.

Hogan was funded by Fondecyt Grant No. 1181896. Hogan & Gutierrez were funded by ANID – Millennium Science Initiative Program – Code ICN17_002. Cochez did part of the work while employed at Fraunhofer FIT, Germany and was later partially funded by Elsevier’s Discovery Lab. Kirrane, Ngonga Ngomo, Polleres & Staab received funding through the project “KnowGraphs” from the European Union’s Horizon programme under the Marie Skłodowska-Curie grant agreement No. 860801. Kirrane & Polleres were supported by the European Union’s Horizon 2020 research and innovation programme under grant 731601. Labra was supported by the Spanish Ministry of Economy and Competitiveness (Society challenges: TIN2017-88877-R). Navigli was supported by the MOUSSE ERC Grant No. 726487 under the European Union’s Horizon 2020 research and innovation programme. Rashid was supported by IBM Research AI through the AI Horizons Network. Schmelzeisen was supported by the German Research Foundation (DFG) grant STA 572/18-1.

Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, Antoine Zimmermann
September 2021

Chapter 1

Introduction

Though the phrase “knowledge graph” has been used in the literature since at least 1972 [Schneider, 1973], the modern incarnation of the phrase stems from the 2012 announcement of the Google Knowledge Graph [Singhal, 2012], followed by further announcements of knowledge graphs by Airbnb [Chang, 2018], Amazon [Krishnan, 2018], eBay [Pittman et al., 2017], Facebook [Noy et al., 2019], IBM [Devarajan, 2017], LinkedIn [He et al., 2016], Microsoft [Shrivastava, 2017].

Give us feedback!

Uber [Hamad et al., 2018], and more besides. The growing industrial uptake of the concept proved difficult for academia to ignore: more and more scientific literature is being published on knowledge graphs, which includes books (e.g., [Pan et al., 2017, Qi et al., 2021, Fensel et al., 2020, Kejriwal et al., 2021]), as well as papers outlining definitions (e.g., [Ehrlinger and Wöß, 2016]), novel techniques (e.g., [Pujara et al., 2013, Wang et al., 2014, Lin et al., 2015]), and surveys of specific aspects of knowledge graphs (e.g., [Paulheim, 2017, Wang et al., 2017]).

Underlying all such developments is the core idea of using graphs to represent data, often enhanced with some way to explicitly represent knowledge [Noy et al., 2019]. The result is most often used in application scenarios that involve integrating, managing and extracting value from diverse sources of data at large scale [Noy et al., 2019]. Employing a graph-based abstraction of knowledge has numerous benefits in such settings when compared with, for example, a relational model or NoSQL alternatives. Graphs provide a concise and intuitive abstraction for a variety of domains, where edges capture the (potentially cyclical) relations between the entities inherent in social data, biological interactions, bibliographical citations and co-authorships, transport networks, and so forth [Angles and Gutierrez, 2008]. Graphs allow maintainers to postpone the definition of a schema, allowing the data – and its scope – to evolve in a more flexible manner than typically possible in a relational setting, particularly for capturing incomplete knowledge [Abiteboul, 1997]. Unlike (other) NoSQL models, specialised graph query languages support not only standard relational operators (joins, unions, projections, etc.), but also navigational operators for recursively finding entities connected through arbitrary-length paths [Angles et al., 2017]. Standard knowledge representation formalisms – such as ontologies [Hitzler et al., 2012, Brickley and Guha, 2014, Mungall et al., 2012] and rules [Horrocks et al., 2004, Kifer and Boley, 2013] – can be employed to define and reason about the semantics of the terms used to label and describe the nodes and edges in the graph. Scalable frameworks for graph analytics [Malewicz et al., 2010, Xin et al., 2013a, Stutz et al., 2016] can be leveraged for computing centrality, clustering, summarisation, etc., in order to gain insights about the domain being described. Various representations have also been developed that support applying machine learning techniques both directly and indirectly over graphs [Wang et al., 2017, Wu et al., 2019].

In summary, the decision to build and use a knowledge graph opens up a range of techniques that can be brought to bear for integrating and extracting value from diverse sources of data at large scale. The goal of this book is to motivate and give a comprehensive introduction to knowledge graphs: to describe their foundational data models and how they can be queried; to discuss representations relating to schema, identity, and context; to discuss deductive and inductive ways to make knowledge explicit; to present a variety of techniques that can be used for the creation and enrichment of graph-structured data; to describe how the quality of knowledge graphs can be discerned and how they can be refined; to discuss standards and best practices by which knowledge graphs can be published; and to provide an overview of existing knowledge graphs found in practice. Our intended audience includes researchers and practitioners who are new to knowledge graphs. As such, we do not assume that readers have specific expertise on knowledge graphs.

Knowledge graph. The definition of a “*knowledge graph*” remains contentious [Ehrlinger and Wöß, 2016, Bonatti et al., 2018, Bergman, 2019], where a number of (sometimes conflicting) definitions have emerged, varying from specific technical proposals to more inclusive general proposals; we address these prior definitions in Appendix A. Herein we adopt an inclusive definition, where we view a knowledge graph as a *graph of data intended to accumulate and convey knowledge of the real world, whose nodes represent entities of interest and whose edges represent relations between these entities*. The graph of data (aka *data graph*) conforms to a graph-based data model, which may be a *directed edge-labelled graph*, a *property graph*, etc. (we discuss concrete alternatives in Chapter 2). By *knowledge*, we refer to something that is *known*. Such knowledge may be accumulated from external sources, or extracted from the knowledge graph itself. Knowledge may be composed of simple statements, such as “*Santiago is the capital of Chile*”, or quantified statements, such as “*all capitals are cities*”. Simple statements can be accumulated as edges in the data graph. If the knowledge graph intends to accumulate quantified statements, a more expressive way to represent knowledge – such as *ontologies* or *rules* – is required. *Deductive methods* can then be used to entail and accumulate further knowledge (e.g., “*Santiago is a city*”). Additional knowledge – based on simple or quantified statements – can also be extracted from and accumulated by the knowledge graph using *inductive methods*.

Knowledge graphs are often assembled from numerous sources, and as a result, can be highly diverse in terms of structure and granularity. To address this diversity, representations of *schema*, *identity*

Give us feedback!

and *context* often play a key role, where a *schema* defines a high-level structure for the knowledge graph, *identity* denotes which nodes in the graph (or in external sources) refer to the same real-world entity, while *context* may indicate a specific setting in which some unit of knowledge is held true. As aforementioned, effective methods for *extraction*, *enrichment*, *quality assessment*, and *refinement* are required for a knowledge graph to grow and improve over time.

In practice. Knowledge graphs aim to serve as an ever-evolving shared substrate of knowledge within an organisation or community [Noy et al., 2019]. We distinguish two types of knowledge graphs in practice: *open knowledge graphs* and *enterprise knowledge graphs*. Open knowledge graphs are published online, making their content accessible for the public good. The most prominent examples – DBpedia [Lehmann et al., 2015], Freebase [Bollacker et al., 2007b], Wikidata [Vrandečić and Krötzsch, 2014], YAGO [Hoffart et al., 2011], etc. – cover many domains and are either extracted from Wikipedia [Lehmann et al., 2015, Hoffart et al., 2011], or built by communities of volunteers [Bollacker et al., 2007b, Vrandečić and Krötzsch, 2014]. Open knowledge graphs have also been published within specific domains, such as media [Raimond et al., 2014], government [Hendler et al., 2012, Shadbolt and O'Hara, 2013], geography [Stadler et al., 2012], tourism [Lu et al., 2016, Kärle et al., 2018, Maturana et al., 2018, Zhang et al., 2019], life sciences [Callahan et al., 2013], and more besides. Enterprise knowledge graphs are typically internal to a company and applied for commercial use-cases [Noy et al., 2019]. Prominent industries using enterprise knowledge graphs include Web search (e.g., Bing [Shrivastava, 2017], Google [Singhal, 2012]), commerce (e.g., Airbnb [Chang, 2018], Amazon [Krishnan, 2018, Dong, 2019], eBay [Pittman et al., 2017], Uber [Hamad et al., 2018]), social networks (e.g., Facebook [Noy et al., 2019], LinkedIn [He et al., 2016]), finance (e.g., Accenture [Okorafor and Ray, 2019], Banca d'Italia [Bellomarini et al., 2019], Bloomberg [Meij, 2019], Capital One [Branum and Sehon, 2019], Wells Fargo [Newman, 2019]), among others. Applications include search [Shrivastava, 2017, Singhal, 2012], recommendations [Chang, 2018, Hamad et al., 2018, He et al., 2016, Noy et al., 2019], personal agents [Pittman et al., 2017], advertising [He et al., 2016], business analytics [He et al., 2016], risk assessment [Tobin, 2017, Dalglish, 2016], automation [Henson et al., 2019], and more besides. We will provide more details on the use of knowledge graphs in practice in Chapter 10.

Running example. To keep the discussion accessible, throughout the book, we present concrete examples in the context of a hypothetical knowledge graph relating to tourism in Chile (loosely inspired by related use-cases [Kärle et al., 2018, Lu et al., 2016]). The knowledge graph is managed by a tourism board that aims to increase tourism in the country and promote new attractions in strategic areas. The knowledge graph itself will eventually describe tourist attractions, cultural events, services, businesses, travel routes, etc. Some applications the organisation envisages are to:

- create a tourism portal that allows visitors to search for attractions, upcoming events, and other related services (in multiple languages);
- gain insights into tourism demographics in terms of season, nationalities, etc.;
- analyse sentiment about tourist attractions, including positive reviews, summaries of complaints about events and services, crime reports, etc.;
- understand tourism trajectories: the sequence of attractions, events, etc., that tourists often visit;
- cross-reference these tourism trajectories with currently available flights, buses, etc., to suggest new strategic routes for public transport;
- offer personalised recommendations of places to visit;
- and so forth.

Outline. The remainder of the book is structured as follows:

- Chapter 2:** outlines graph data models and the languages used to query them.
- Chapter 3:** describes representations of schema, identity, and context for graphs.
- Chapter 4:** presents deductive formalisms for representing and entailing knowledge.
- Chapter 5:** describes inductive techniques for learning from graphs.
- Chapter 6:** discusses the creation and enrichment of knowledge graphs.
- Chapter 7:** enumerates dimensions for assessing knowledge graph quality.
- Chapter 8:** discusses various techniques for knowledge graph refinement.

Chapter 9: introduces principles and protocols for publishing knowledge graphs.

Chapter 10: surveys some prominent knowledge graphs and their applications.

Chapter 11: concludes with future directions for knowledge graphs.

Appendix A: outlines the historical background for knowledge graphs.

Chapter 2

Data Graphs

At the foundation of any knowledge graph is the principle of first applying a graph abstraction to data, resulting in an initial data graph. We now discuss a selection of graph-structured data models that are commonly used in practice to represent data graphs. We then discuss the primitives that form the basis of graph query languages used to interrogate such data graphs.

2.1 MODELS

Leaving aside graphs, let us assume that the tourism board from our running example has not yet decided how to model relevant data about attractions, events, services, etc. The board first considers using a tabular structure – in particular, relational databases – to represent the required data, and though they do not know precisely what data they will need to capture, they begin to design an initial relational schema. They begin with an Event table with five columns:

Event(name, venue, type, start, end)

where name and start together form the primary key of the table in order to uniquely identify recurring events. But as they start to populate the data, they encounter various issues: events may have multiple names (e.g., in different languages), events may have multiple venues, they may not yet know the start and end date-times for future events, events may have multiple types, and so forth. Incrementally addressing these modelling issues as the data become more diverse, they generate internal identifiers for events and adapt their relational schema until they have:

EventName(id,name), EventStart(id,start), EventEnd(id,end), EventVenue(id,venue), EventType(id,type) (2.1)

With the above schema, the organisation can now model events with 0– n names, venues, and types, and 0–1 start dates and end dates (without needing relational nulls).

Along the way, the board has to incrementally change the schema several times in order to support new sources of data. Each such change requires a costly remodelling, reloading, and reindexing of data; here we only considered one table. The tourism board struggles with the relational model because they do not know, *a priori*, what data will need to be modelled or what sources they will use. But once they reach the latter relational schema, the board finds that they can integrate further sources without more changes: with minimal assumptions on *multiplicities* (1–1, 1– n , etc.) this schema offers a lot of flexibility for integrating incomplete and diverse data.

In fact, the refined, flexible schema that the board ends up with – as shown in (2.1) – is modelling a set of binary relations between entities, which indeed can be viewed as modelling a graph. By instead adopting a graph data model from the outset, the board could forgo the need for an upfront schema, and could define any (binary) relation between any pair of entities at any time.

We now introduce graph data models popular in practice [Angles et al., 2017].

2.1.1 DIRECTED EDGE-LABELLED GRAPHS

A directed edge-labelled graph (sometimes known as a *multi-relational graph* [Nickel and Tresp, 2013, Bordes et al., 2013, Balazevic et al., 2019a]) is defined as a set of nodes – like (Santiago), (Arica), (EID16), (2018-03-22 12:00) – and a set of directed labelled edges between those nodes, like (Santa Lucía) $\rightarrow$ (Santiago). In the case of knowledge graphs, nodes are used to represent entities and edges $\rightarrow$

Give us feedback!

used to represent (binary) relations between those entities. Figure 2.1 provides an example of how the tourism board could model some relevant event data as a directed edge-labelled graph. The graph includes data about the names, types, start and end date-times, and venues for events.¹ Adding information to such a graph typically involves adding new nodes and edges (with some exceptions discussed later). Representing incomplete information requires simply omitting a particular edge; for example, the graph does not yet define a start/end date-time for the Food Truck festival.

note 1 We represent bi depicts two directed while some naming c verb, such as has veni omit the “has” and “-

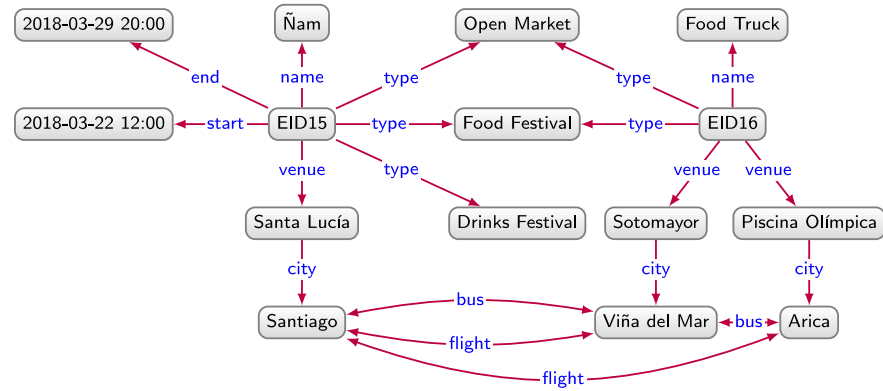


Figure 2.1: Directed edge-labelled graph describing events and their venues

Modelling data as a graph in this way offers more flexibility for integrating new sources of data, compared to the standard relational model, where a schema must be defined upfront and followed at each step. While other structured data models such as trees (XML, JSON, etc.) would offer similar flexibility, graphs do not require organising the data hierarchically (should venue be a parent, child, or sibling of type for example?). They also allow cycles to be represented and queried (e.g., note the directed cycle in the routes between Santiago, Arica, and Viña del Mar).

A standardised data model based on directed edge-labelled graphs is the Resource Description Framework (RDF) [Cyganiak et al., 2014], which has been recommended by the W3C. The RDF model defines different types of nodes, including *Internationalized Resource Identifiers* (IRIs) [Dürst and Suignard, 2005] which allow for global identification of entities on the Web; *literals*, which allow for representing strings (with or without language tags) and other datatype values (integers, dates, etc.); and *blank nodes*, which are anonymous nodes that are not assigned an identifier (for example, rather than create internal identifiers like EID15, EID16, in RDF, we have the option to use blank nodes). We will discuss these different types of nodes further in Section 3.2 when we speak about issues relating to identity.

We now formally define a directed edge-labelled graph, where we denote by $\mathbf{Con}$ a countably infinite set of constants.

Definition 2.1 (Directed edge-labelled graph)

A directed edge-labelled graph is a tuple $G = (V, E, L)$, where $V \subseteq \mathbf{Con}$ is a set of nodes, $L \subseteq \mathbf{Con}$ is a set of edge labels, and $E \subseteq V \times L \times V$ is a set of edges.

Example 2.1 — In reference to Figure 2.1, the set of nodes V has 15 elements, including Arica, EID16, etc. The set of edges E has 23 triples, including (Arica, flight, Santiago). Bidirectional edges are represented with two edges. The set of edge labels L has 8 elements, including start, flight, etc.

Definition 2.1 does not state that V and L are disjoint: though not present in the example, a node can also serve as an edge-label. The definition also permits that nodes and edge labels can be present without any associated edge. Either restriction could be explicitly stated – if necessary – in a particular application while still conforming to a directed edge-labelled graph.

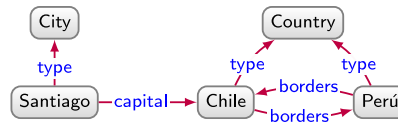
For ease of presentation, we may treat a set of (directed labelled) edges $E \subseteq V \times L \times V$ as a directed edge-labelled graph (V, E, L) , in which case we refer to the graph induced by E assuming that V and L contain all and only those nodes and edge labels, respectively, used in E . We may similarly apply set operators on directed edge-labelled graphs, which should be interpreted as

Give us feedback!

applying to their sets of edges; for example, given $G_1 = (V_1, E_1, L_1)$ and $G_2 = (V_2, E_2, L_2)$, by $G_1 \cup G_2$ we refer to the directed edge-labelled graph induced by $E_1 \cup E_2$.

2.1.2 HETEROGENEOUS GRAPHS

A heterogeneous graph [Hussein et al., 2018, Wang et al., 2019, Yang et al., 2020] (or *heterogeneous information network* [Sun et al., 2011, Sun and Han, 2012]) is a directed graph where each node and edge is assigned one type. Heterogeneous graphs are thus akin to directed edge-labelled graphs – with edge labels corresponding to edge types – but where the type of node forms part of the graph model itself, rather than being expressed with a relation (as seen in Figure 2.2). An edge is called *homogeneous* if it is between two nodes of the same type (e.g., *borders* in Figure 2.2); otherwise it is called *heterogeneous* (e.g., *capital* in Figure 2.2). Heterogeneous graphs allow for partitioning nodes according to their type, for example, for the purposes of machine learning tasks [Hussein et al., 2018, Wang et al., 2019, Yang et al., 2020]. Conversely, such graphs typically only support a many-to-one relation between nodes and types, which is not the case for directed edge-labelled graphs (see, for example, the node `Santiago` with zero types and `EID15` with multiple types in Figure 2.1).



(a) Directed edge-labelled graph



(b) Heterogenous graph

Figure 2.2: Comparing directed edge-labelled graphs and heterogeneous graphs

We next define the notion of a heterogeneous graph.

Definition 2.2 (Heterogeneous graph)

A heterogeneous graph is a tuple $G = (V, E, L, l)$, where $V \subseteq \mathbf{Con}$ is a set of nodes, $L \subseteq \mathbf{Con}$ is a set of edge/node labels, $E \subseteq V \times L \times V$ is a set of edges, and $l : V \rightarrow L$ maps each node to a label.

Example 2.2 — In reference to Figure 2.2b, the set of nodes V has three elements: Santiago, Chile, and Perú. The set of edges E has 3 triples, including $(\text{Santiago}, \text{capital}, \text{Chile})$. The set of edge labels L has 4 elements: capital, borders, City, Country. Finally, with respect to the node labels, $l(\text{Santiago}) = \text{City}$, $l(\text{Chile}) = \text{Country}$, and $l(\text{Perú}) = \text{Country}$.

In heterogeneous graphs, edge and node labels are often called *types*. By rather defining edges with labels as per directed edge-labelled graphs – rather than separately labelling edges with l – two nodes can be related by n edges with n different labels; for example, we can represent both $(\text{Santiago}, \text{capital}, \text{Chile})$ and $(\text{Santiago}, \text{country}, \text{Chile})$ as edges in the heterogeneous graph.

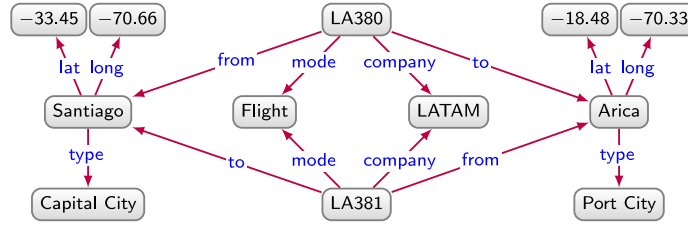
2.1.3 PROPERTY GRAPHS

Property graphs constitute an alternative graph model that offers additional flexibility when modelling more complex relations. Consider integrating incoming data that provide further details on which companies offer fares on which flights, allowing the board to better understand available routes between cities (for example, on national airlines). In the case of directed edge-labelled graph.

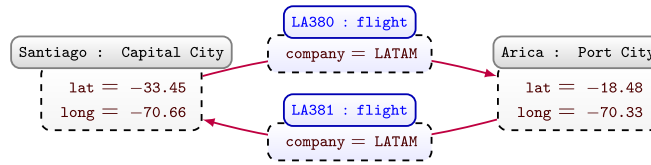
Give us feedback!

we cannot directly annotate an edge like $\text{Santiago} \xrightarrow{\text{flight}} \text{Arica}$ with the company (or companies) offering that route. But we could add a new node denoting a flight, connect it with the source, destination, companies, and mode, as shown in Figure 2.3a. Applying this modelling to all routes in Figure 2.1 would, however, involve significant changes.

The property graph model was thus proposed to offer additional flexibility when modelling data as a graph [Miller, 2013, Angles et al., 2017]. A property graph allows a set of *property-value* pairs and a *label* to be associated with both nodes and edges. Figure 2.3b depicts an example of a property graph with data analogous to Figure 2.3a. We use property-value pairs on edges to model the companies. The type of relation is captured by the label *flight*. We further use node labels to indicate the types of the two nodes, and property-value pairs for their latitude and longitude.



(a) Directed edge-labelled graph



(b) Property graph

Figure 2.3: Comparing directed edge-labelled graphs and property graphs

Property graphs are prominently used in graph databases, such as Neo4j [Miller, 2013, Angles et al., 2017]. Property graphs can be converted to/from directed edge-labelled graphs [Hernández et al., 2015, Angles et al., 2019] (per, e.g., Figure 2.3b). In summary, directed edge-labelled graphs offer a more minimal model, while property graphs offer a more flexible one. Often the choice of model will be secondary to other practical factors, such as the implementations available for different models, etc.

We formally define a property graph.

Definition 2.3 (Property graph)

A property graph is a tuple $G = (V, E, L, P, U, e, l, p)$, where $V \subseteq \mathbf{Con}$ is a set of node ids, $E \subseteq \mathbf{Con}$ is a set of edge ids, $L \subseteq \mathbf{Con}$ is a set of labels, $P \subseteq \mathbf{Con}$ is a set of properties, $U \subseteq \mathbf{Con}$ is a set of values, $e : E \rightarrow V \times V$ maps an edge id to a pair of node ids, $l : V \cup E \rightarrow 2^L$ maps a node or edge id to a set of labels, and $p : V \cup E \rightarrow 2^{P \times U}$ maps a node or edge id to a set of property-value pairs.

Example 2.3 — Returning to Figure 2.3b:

- the set V contains Santiago and Arica;
- the set E contains LA380 and LA381;
- the set L contains Capital City, Port City, and flight;
- the set P contains lat, long, and company;
- the set U contains -33.45, -70.66, LATAM, -18.48, and -70.33;
- the mapping e gives, for example, $e(\text{LA380}) = (\text{Santiago}, \text{Arica})$;
- the mapping l gives, for example, $l(\text{Santiago}) = \{\text{Capital City}\}$ and $l(\text{LA380}) = \{\text{flight}\}$;

Give us feedback!

- the mapping p gives, for example, $p(\text{LA380}) = \{(\text{company}, \text{LATAM})\}$ and $p(\text{Santiago}) = \{(\text{lat}, -33.45), (\text{long}, -70.66)\}$.

Unlike previous definitions [Angles et al., 2017], we allow a node or edge to have several values for a given property. In practice, systems like Neo4j [Miller, 2013] may rather support this by allowing a single array (i.e., list) of values.

2.1.4 GRAPH DATASET

Although multiple directed edge-labelled graphs can be merged by taking their union, it is often desirable to manage several graphs rather than one monolithic graph; for example, it may be beneficial to manage multiple graphs from different sources, making it possible to update or refine data from one source, to distinguish untrustworthy sources from more trustworthy ones, and so forth. A graph dataset then consists of a set of *named graphs* and a *default graph*. Each named graph is a pair of a graph ID and a graph. The default graph is a graph without an ID, and is referenced “by default” if a graph ID is not specified. Figure 2.4 provides an example where events and routes are stored in two named graphs, and the default graph manages metadata about the named graphs. Graph names can also be used as nodes in a graph. Furthermore, nodes and edges can be repeated across graphs, where the same node in different graphs will typically refer to the same entity, allowing data on that entity to be integrated when merging graphs. Though the example depicts a dataset of directed edge-labelled graphs, the concept generalises straightforwardly to datasets of other types of graphs.

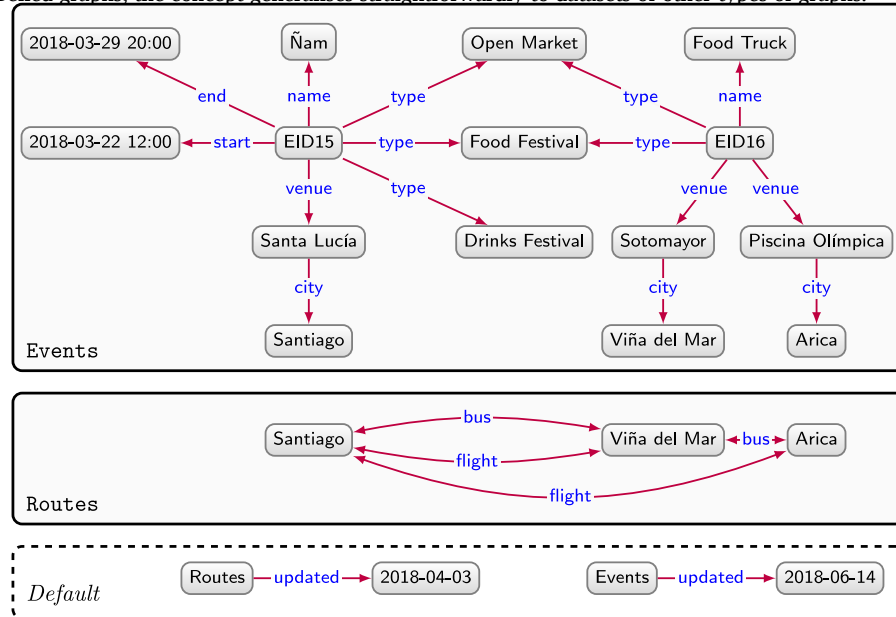


Figure 2.4: Graph dataset based on directed edge-labelled graphs with two named graphs and a default graph describing events and routes

An RDF dataset is a graph dataset model standardised by the W3C [Cyganiak et al., 2014] where each graph is an RDF graph, and graph names can be blank nodes or IRIs. A prominent use-case for RDF datasets is to manage and query *Linked Data* composed of interlinked documents of RDF graphs spanning the Web. When dealing with Web data, tracking the source of data becomes of key importance [Dividino et al., 2009, Bonatti et al., 2011, Zimmermann et al., 2012]. We will discuss Linked Data later in Section 3.2 and further discuss provenance in Section 3.3.

We more formally define a graph dataset. We assume that all data graphs featured in a given graph dataset follow the same model (directed edge-labelled graph, heterogeneous graph, property graph, etc).

Definition 2.4 (Graph dataset)

A *named graph* is a pair (n, G) where G is a data graph, and $n \in \mathbf{Con}$ is a graph name. A *graph dataset* is a pair $D = (G_D, N)$ where G_D is a data graph called the *default graph* and N is either the empty set, or a set of named graphs $\{(n_1, G_1), \dots, (n_k, G_k)\}$ ($k > 0$) such that if $i \neq j$ then $n_i \neq n_j$ (for all $1 \leq i \leq k, 1 \leq j \leq k$).

Give us feedback!

Example 2.4 — Figure 2.4 provides an example of a directed edge-labelled graph dataset D consisting of two named graphs and a default graph. The default graph does not have a name associated with it. The two graph names are *Events* and *Routes*; these are also used as nodes in the default graph.

2.1.5 OTHER GRAPH DATA MODELS

The previous models are popular examples of graph representations. Other graph data models exist with *complex nodes* that may contain individual edges [Angles and Gutierrez, 2008, Hartig and Thompson, 2014] or nested graphs [Angles and Gutierrez, 2008, Berners-Lee and Connolly, 2011] (sometimes called *hypernodes* [Levene and Poulouvasilis, 1989]). Likewise the mathematical notion of a *hypergraph* defines *complex edges* that connect sets rather than pairs of nodes. In our view, a knowledge graph can adopt any such graph data model based on nodes and edges: often data can be converted from one model to another (see Figure 2.3a vs. Figure 2.3b). In the rest of the book, we prefer discussing directed edge-labelled graphs given their relative succinctness, but most discussion extends naturally to other models.

2.1.6 GRAPH STORES

A variety of techniques have been proposed for storing and indexing graphs, facilitating the efficient evaluation of queries (as discussed next). Directed edge-labelled graphs can be stored in relational databases either as a single relation of arity three (*triple table*), as a binary relation for each property (*vertical partitioning*), or as n -ary relations for entities of a given type (*property tables*) [Wylot et al., 2018]. Custom (so-called *native*) storage techniques have also been developed for a variety of graph models, providing efficient access for finding nodes, edges and their adjacent elements [Angles and Gutierrez, 2008, Miller, 2013, Wylot et al., 2018]. A number of systems further allow for distributing graphs over multiple machines based on popular NoSQL stores or custom partitioning schemes [Wylot et al., 2018, Janke and Staab, 2018]. For further details we refer to the book chapter by Janke and Staab [2018] and the survey by Wylot et al. [2018] dedicated to this topic.

2.2 QUERYING

A number of languages have been proposed for querying graphs [Angles et al., 2017], including the SPARQL query language for RDF graphs [Harris et al., 2013]; and Cypher [Francis et al., 2018], Gremlin [Rodríguez, 2015], and G-CORE [Angles et al., 2018] for querying property graphs. We refer to Seifer et al. [2019] for an investigation of the popularity of these languages. Underlying these query languages are some common primitives, including (basic) graph patterns, relational operators, path expressions, and more besides [Angles et al., 2017]. We now describe these core features for querying graphs in turn, starting with basic graph patterns.

2.2.1 BASIC GRAPH PATTERNS

At the core of every structured query language for graphs lie *basic graph patterns* [Consens and Mendelzon, 1990, Angles et al., 2017], which follow the same model as the data graph being queried (see Section 2.1), additionally allowing variables as terms.² Terms in basic graph patterns are thus divided into constants, such as `(Arica)` or `venue`, and variables, which we prefix with question marks, such as `{?event}` or `?rel`. A basic graph pattern is then evaluated against the data graph by generating mappings from the variables of the graph pattern to constants in the data graph such that the image of the graph pattern under the mapping (replacing variables with the assigned constants) is contained within the data graph.

Figure 2.5 provides an example of a basic graph pattern looking for the venues of Food Festivals, along with the possible mappings generated by the graph pattern against the data graph of Figure 2.1. In some of the presented mappings (the last two listed), multiple variables are mapped to the same term, which may or may not be desirable depending on the application. Hence a number of semantics have been proposed for evaluating basic graph patterns [Angles et al., 2017], among which the most important are: *homomorphism-based semantics*, which allows multiple variables to

note 2 The terms of a $\mathcal{G}$ are the nodes and edges of a property graph $\mathcal{G}$ (nodes).

Give us feedback!

mapped to the same term such that all mappings shown in Figure 2.5 would be considered results; and *isomorphism-based semantics*, which requires variables on nodes and/or edges to be mapped to unique terms, thus excluding the latter three mappings of Figure 2.5 from the results. Different languages may adopt different semantics for evaluating basic graph patterns; for example, SPARQL adopts a homomorphism-based semantics, while Cypher adopts an isomorphism-based semantics specifically on edges (while allowing multiple variables to map to one node).

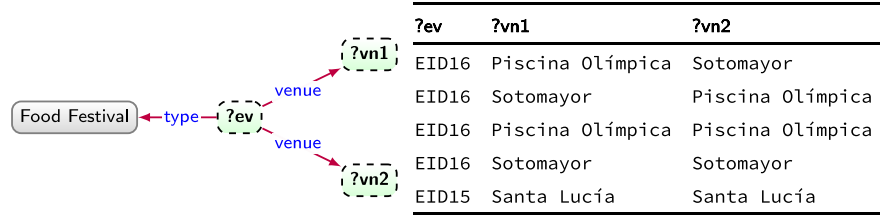


Figure 2.5: Basic graph pattern (left) with mappings generated over the directed edge-labelled graph of Figure 2.1 (right)

As we will see in later examples (particularly Figure 2.7), basic graph patterns may also form cycles (be they directed or undirected), and may replace edge labels with variables. Basic graph patterns in the context of other models – such as property graphs – can be defined analogously by allowing variables to replace constants in any position of the model.

We formalise basic graph patterns first for directed edge-labelled graphs, and subsequently for property graphs [Angles et al., 2017]. For these definitions, we introduce a countably infinite set of *variables* $\mathbf{Var}$ ranging over (but disjoint from: $\mathbf{Con} \cap \mathbf{Var} = \emptyset$) the set of constants. We refer generically to constants and variables as *terms*, denoted and defined as $\mathbf{Term} = \mathbf{Con} \cup \mathbf{Var}$. We define a basic graph pattern for a particular graph data model by simply replacing constants with terms (that may be variables). Though we focus on directed edge-labelled graphs and property graphs, basic graph patterns for other graph models can be defined analogously.

Definition 2.5 (Basic directed edge-labelled graph pattern)

We define a basic directed edge-labelled graph pattern as a tuple $Q = (V, E, L)$, where $V \subseteq \mathbf{Term}$ is a set of node terms, $L \subseteq \mathbf{Term}$ is a set of edge terms, and $E \subseteq V \times L \times V$ is a set of edges (triple patterns).

Example 2.5 — Returning to the example of Figure 2.5:

- the set V contains the constant `Food Festival` and variables `?ev`, `?vn1` and `?vn2`;
- the set E contains four edges, including `(?ev, type, Food Festival)`;
- the set L contains the constants `type` and `venue`.

A basic property graph pattern is also defined by introducing variables.

Definition 2.6 (Basic property graph pattern)

We define a basic property graph pattern as a tuple $Q = (V, E, L, P, U, e, l, p)$, where $V \subseteq \mathbf{Term}$ is a set of node id terms, $E \subseteq \mathbf{Term}$ is a set of edge id terms, $L \subseteq \mathbf{Term}$ is a set of label terms, $P \subseteq \mathbf{Term}$ is a set of property terms, $U \subseteq \mathbf{Term}$ is a set of value terms, $e : E \rightarrow V \times V$ maps an edge id term to a pair of node id terms, $l : V \cup E \rightarrow 2^L$ maps a node or edge id term to a set of label terms, and $p : V \cup E \rightarrow 2^{P \times U}$ maps a node or edge id term to a set of pairs of property-value terms.

Towards defining the results of evaluating a basic graph pattern over a data graph (following the same model), we first define a partial mapping $\mu : \mathbf{Var} \rightarrow \mathbf{Con}$ from variables to constants, whose *domain* (the set of variables for which it is defined) is denoted by $\mathbf{dom}(\mu)$. Given a basic graph pattern Q , let $\mathbf{Var}(Q)$ denote the set of all variables appearing in (some recursively nested element of) Q . We further denote by $\mu(Q)$ the image of Q under μ , meaning that any variable $v \in \mathbf{Var}(Q) \cap \mathbf{dom}(\mu)$ is replaced in Q by $\mu(v)$. Observe that when $\mathbf{Var}(Q) \subseteq \mathbf{dom}(\mu)$, then $\mu(Q)$ is a data graph (in the corresponding model of Q).

Next, we define the notion of containment between data graphs. For two directed edge-labelled graphs $G_1 = (V_1, E_1, L_1)$ and $G_2 = (V_2, E_2, L_2)$, we say that G_1 is a *sub-graph* of G_2 , denoted $G_1 \subseteq$

Give us feedback!

G_2 , if and only if $V_1 \subseteq V_2$, $E_1 \subseteq E_2$, and $L_1 \subseteq L_2$.³ For two property graphs $G_1 = (V_1, E_1, L_1, P_1, U_1, e_1, l_1, p_1)$ and $G_2 = (V_2, E_2, L_2, P_2, U_2, e_2, l_2, p_2)$, we say that G_1 is a *sub-graph* of G_2 , denoted $G_1 \subseteq G_2$, if and only if $V_1 \subseteq V_2$, $E_1 \subseteq E_2$, $L_1 \subseteq L_2$, $P_1 \subseteq P_2$, $U_1 \subseteq U_2$, for all $x \in E_1$ it holds that $e_1(x) = e_2(x)$, and for all $y \in E_1 \cup V_1$ it holds that $l_1(y) \subseteq l_2(y)$ and $p_1(y) \subseteq p_2(y)$.

We are now ready to define the evaluation of a basic graph pattern.

Definition 2.7 (Evaluation of a basic graph pattern)

Let Q be a basic graph pattern and let G be a data graph (in the same model). We then define the evaluation of the basic graph pattern Q over the data graph G , denoted $Q(G)$, to be the set of mappings $Q(G) = \{\mu \mid \mu(Q) \subseteq G \text{ and } \text{dom}(\mu) = \text{Var}(Q)\}$.

Example 2.6 — Figure 2.5 enumerates all of the mappings given by the evaluation of the depicted basic graph pattern over the data graph of Figure 2.1. Each non-header row indicates a mapping μ .

The final results of evaluating a basic graph pattern may vary depending on the choice of semantics: the results under *homomorphism-based semantics* are defined as $Q(G)$. Conversely, under *isomorphism-based semantics*, mappings that send two edge variables to the same constant and/or mappings that send two node variables to the same constant may be excluded from the results. Henceforth we assume the more general *homomorphism-based semantics*.

2.2.2 COMPLEX GRAPH PATTERNS

A (basic) graph pattern transforms an input graph into a table of results (as shown in Figure 2.5). We may then consider using the relational algebra to combine and/or transform such tables, thus forming more complex queries from one or more graph patterns. Recall that the relational algebra consists of unary operators that accept one input table, and binary operators that accept two input tables. Unary operators include projection (π) to output a subset of columns, selection (σ) to output a subset of rows matching a given condition, and renaming of columns (ρ). Binary operators include union ($\cup$) to merge the rows of two tables into one table, difference ($-$) to remove the rows from the first table present in the second table, and joins ($\bowtie$) to extend the rows of one table with rows from the other table that satisfy a join condition. Selection and join conditions typically include equalities ($=$), inequalities ($\leq$), negation ($\neg$), disjunction ($\vee$), etc. From these operators, we can further define other (syntactic) operators, such as intersection ($\cap$) to output rows in both tables, anti-join ($\bar{\bowtie}$, aka *minus*) to output rows from the first table for which there are no join-compatible rows in the second table, left-join ($\ltimes$, aka *optional*) to perform a join but keeping rows from the first table without a compatible row in the second table, etc.

Basic graph patterns can then be expressed in a subset of relational algebra (namely $\pi, \sigma, \rho, \bowtie$). Assuming, for example, a single ternary relation $G(s, p, o)$ representing a graph – i.e., a table G with three columns s, p, o – the query of Figure 2.5 can be expressed in relational algebra as:

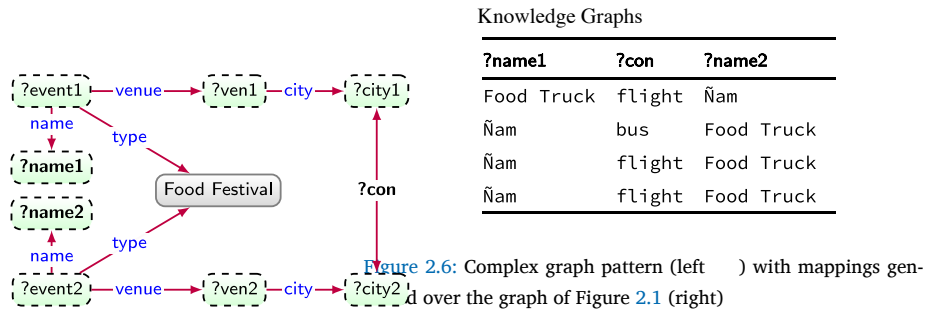
$$\pi_{ev, vn1, vn2}(\sigma_{p=\text{type}\wedge o=\text{Food Festival}\wedge p_1=p_2=\text{venue}}(\rho_{s/ev}(G \bowtie \rho_{p/p_1, o/vn1}(G) \bowtie \rho_{p/p_2, o/vn2}(G))))$$

where $\bowtie$ denotes a *natural join*, meaning that equality is checked across pairs of columns with the same name in both tables (here, the join is thus performed on the subject column s). The result of this query is a table with a column for each variable: $ev, vn1, vn2$. However, not all queries using π, σ, ρ and $\bowtie$ on G can be expressed as basic graph patterns; for example, we cannot choose which variables to project in a basic graph pattern, but rather must project all variables not fixed to a constant.

Graph query languages such as SPARQL [Harris et al., 2013] and Cypher [Francis et al., 2018] allow the full use of relational operators over the results of graph patterns, giving rise to *complex graph patterns* [Angles et al., 2017]. Figure 2.6 presents an example of a complex graph pattern with projected variables in bold, choosing particular variables to appear in the final results. In Figure 2.7, we give another example of a complex graph pattern looking for food festivals or drinks festivals not held in Santiago, optionally returning their start date and name (where available).

?name1	?con	?name2
Food Truck	bus	Food Truck
Food Truck	bus	Food Truck
Food Truck	bus	Ñam
Food Truck	flight	Ñam

note 3 Given, for exam remark that $G_1 \not\subseteq G_2$ latter has a node with cases of nodes or labe if and only if $E_1 \subseteq E_2$ without edges.



Complex graph patterns can give rise to duplicate results; for example, the first result in Figure 2.6 appears twice since ?city1 matches Arica and ?city2 matches Viña del Mar in one result, and vice-versa in the other. Query languages then offer two semantics: *bag semantics* preserves duplicates according to the multiplicity of the underlying mappings, while *set semantics* (typically invoked with a DISTINCT keyword) removes duplicates from the results.

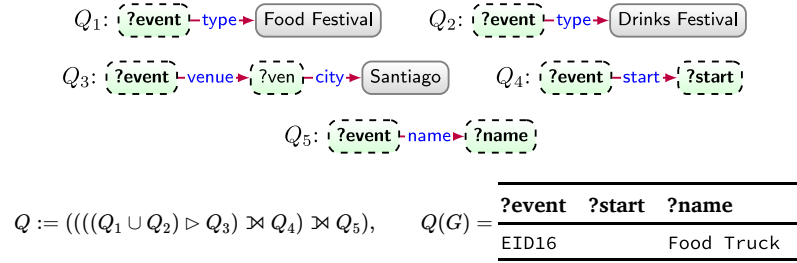


Figure 2.7: Complex graph pattern (Q) with mappings generated ($Q(G)$) over the graph of Figure 2.1 (G)

We now formally define complex graph patterns.

Definition 2.8 (Complex graph pattern)

Complex graph patterns are defined recursively, as follows:

- If Q is a basic graph pattern, then Q is a complex graph pattern.
- If Q is a complex graph pattern, and $\mathcal{V} \subseteq \text{Var}(Q)$, then $\pi_{\mathcal{V}}(Q)$ is a complex graph pattern.
- If Q is a complex graph pattern, and R is a selection condition with Boolean and equality connectives ($\wedge, \vee, \neg, =$), then $\sigma_R(Q)$ is a complex graph pattern.
- If both Q_1 and Q_2 are complex graph patterns, then $Q_1 \bowtie Q_2$, $Q_1 \cup Q_2$, $Q_1 - Q_2$ and $Q_1 \triangleright Q_2$ are also complex graph patterns.

We now define the evaluation of complex graph patterns. Given a mapping μ , for a set of variables $\mathcal{V} \subseteq \text{Var}$ let $\mu[\mathcal{V}]$ denote the mapping μ' such that $\text{dom}(\mu') = \text{dom}(\mu) \cap \mathcal{V}$ and $\mu'(v) = \mu(v)$ for all $v \in \text{dom}(\mu')$ (in other words, $\mu[\mathcal{V}]$ projects the variables $\mathcal{V}$ from μ). Letting R denote a Boolean selection condition and μ a mapping, we denote by $\mu \models R$ that μ satisfies the Boolean condition. Finally, we define two mappings μ_1 and μ_2 to be *compatible*, denoted $\mu_1 \sim \mu_2$, if and only if $\mu_1(v) = \mu_2(v)$ for all $v \in \text{dom}(\mu_1) \cap \text{dom}(\mu_2)$ (i.e., they map common variables to the same constant). We are now ready to provide the definition.

Definition 2.9 (Complex graph pattern evaluation)

Given a complex graph pattern Q , if Q is a basic graph pattern, then $Q(G)$ is defined per Definition 2.7. Otherwise, $Q(G)$ is defined as follows:

$$\begin{aligned}
 \pi_{\mathcal{V}}(Q)(G) &= \{\mu[\mathcal{V}] \mid \mu \in Q(G)\} \\
 \sigma_R(Q)(G) &= \{\mu \mid \mu \in Q(G) \text{ and } \mu \models R\} \\
 Q_1 \bowtie Q_2(G) &= \{\mu_1 \cup \mu_2 \mid \mu_1 \in Q_2(G), \mu_2 \in Q_1(G) \text{ and } \mu_1 \sim \mu_2\} \\
 Q_1 \cup Q_2(G) &= \{\mu \mid \mu \in Q_1(G) \text{ or } \mu \in Q_2(G)\} \\
 Q_1 - Q_2(G) &= \{\mu \mid \mu \in Q_1(G) \text{ and } \mu \notin Q_2(G)\} \\
 Q_1 \triangleright Q_2(G) &= \{\mu \mid \mu \in Q_1(G) \text{ and } \nexists \mu_2 \in Q_2(G) \text{ such that } \mu \sim \mu_2\}
 \end{aligned}$$

Based on these operators, we can define some additional syntactic operators, such as the *left-join* ($\bowtie$, aka *optional*):

$$Q_1 \bowtie Q_2(G) = (Q_1(G) \bowtie Q_2(G)) \cup (Q_1(G) \triangleright Q_2(G))$$

We call such operators *syntactic* as they do not add expressivity.

Example 2.7 — Figure 2.7 illustrates a complex graph pattern and its evaluation.

2.2.3 NAVIGATIONAL GRAPH PATTERNS

A key feature that distinguishes graph query languages is the ability to include *path expressions* in queries. A path expression r is a regular expression that allows for matching arbitrary-length paths between two nodes using a *regular path query* (x, r, y) , where x and y can be variables or constants (or even the same term). The base path expression is where r is a constant (an edge label). Furthermore if r is a path expression, then r^* (*Kleene star*: zero-or-more) is also a path expression. Finally, if r_1 and r_2 are path expressions, then $r_1 \mid r_2$ (*disjunction*) and $r_1 \cdot r_2$ (*concatenation*) are also path expressions. A related notion is that of *2-way regular path queries*, which also allow for querying inverse paths; specifically, if r is path expression, then it is a *2-way path expression*, and if r is a *2-way path expression*, then r^- (*inverse*) is a *2-way path expression*. Henceforth we will refer generically to both the 1-way and 2-way variants as path expressions and regular path queries.

Regular path queries can be evaluated under a number of different semantics. For example, $(Arica, bus^*, ?city)$ evaluated against the graph of Figure 2.1 may match the paths shown in Figure 2.8. In fact, since a cycle is present, an infinite number of paths are potentially matched. For this reason, restricted semantics are often applied, returning only the shortest paths, or paths without repeated nodes or edges (as in the case of Cypher).⁴ Rather than returning paths, another option is to instead return the (finite) set of pairs of nodes connected by a matching path (as in the case of SPARQL 1.1).

note 4 Mapping vari
Cypher [Francis et al
functions such as `len`
hand, allows for retu
projecting them as gra

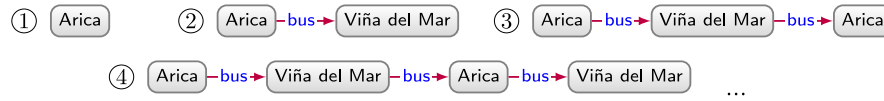


Figure 2.8: Example paths matching $(Arica, bus^*, ?city)$ over the graph of Figure 2.1

Regular path queries can then be used in basic graph patterns to express *navigational graph patterns* [Angles et al., 2017], as shown in Figure 2.9, which illustrates a query searching for food festivals in cities reachable (recursively) from Arica by bus or flight. Furthermore, when regular path queries and graph patterns are combined with operators such as projection, selection, union, difference, and optional, the result is known as *complex navigational graph patterns* [Angles et al., 2017].

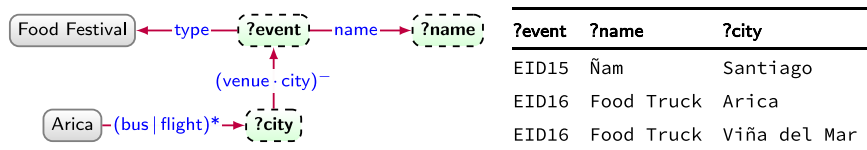


Figure 2.9: Navigational graph pattern (left) with mappings generated over the graph of Figure 2.1 (right)

We first define path expressions and regular path queries.

Definition 2.10 (Path expression)

A constant (edge label) c is a path expression. Furthermore, if r , r_1 and r_2 are path expressions, then:

- r^- (inverse) and r^* (Kleene star) are path expressions.
- $r_1 \cdot r_2$ (concatenation) and $r_1 \mid r_2$ (disjunction) are path expressions.

We now define the evaluation of a path expression on a directed-edge labelled graph under the SPARQL 1.1-style semantics whereby the endpoints (pairs of start and end nodes) of the path are

Give us feedback!

returned [Harris et al., 2013].

Definition 2.11 (Path evaluation (directed edge-labelled graph))

Given a directed edge-labelled graph $G = (V, E, L)$ and a path expression r , we define the evaluation of r over G , denoted $r[G]$, as follows:

$$\begin{aligned} r[G] &= \{(u, v) \mid (u, r, v) \in E\} \text{ (for } r \in \mathbf{Con}) \\ r^-[G] &= \{(u, v) \mid (v, u) \in r[G]\} \\ r_1 \mid r_2[G] &= r_1[G] \cup r_2[G] \\ r_1 \cdot r_2[G] &= \{(u, v) \mid \exists w \in V : (u, w) \in r_1[G] \text{ and } (w, v) \in r_2[G]\} \\ r^*[G] &= \{(u, u) \mid u \in V\} \cup \bigcup_{n \in \mathbb{N}^+} r^n[G] \end{aligned}$$

where by r^n we denote the n^{th} -concatenation of r (e.g., $r^3 = r \cdot r \cdot r$).

The inclusion of the reflexive pairs (u, u) in the definition of $r^*[G]$ captures zero-length paths. For example, in the query $(\text{Arica}, \text{bus}^*, ?\text{city})$, the reflexive pair $(\text{Arica}, \text{Arica})$ ensures that the variable $?\text{city}$ will also match Arica via the zero-length path.

The evaluation of a path expression on a property graph $G = (V, E, L, P, U, e, l, p)$ can be defined analogously by adapting the first definition (in the case that $r \in \mathbf{Con}$) as follows:

$$r[G] = \{(u, v) \mid \exists x \in E : e(x) = (u, v) \text{ and } l(e) = r\}.$$

The rest of the definitions then remain unchanged.

Query languages may support additional operators, some of which are syntactic (e.g., r^+ is sometimes used for one-or-more, but can be rewritten as $r \cdot r^*$), while others may add expressivity such as the case of SPARQL [Harris et al., 2013], which allows a limited form of negation in expressions (e.g., $!r$, with r being a constant or the inverse of a constant, matching any path not labelled r).

Next we define a regular path query and its evaluation.

Definition 2.12 (Regular path query)

A regular path query is a triple (x, r, y) where $x, y \in \mathbf{Con} \cup \mathbf{Var}$ and r is a path expression.

Definition 2.13 (Regular path query evaluation)

Let G denote a directed edge-labelled graph, $c, c_1, c_2 \in \mathbf{Con}$ denote constants and $z, z_1, z_2 \in \mathbf{Var}$ denote variables. Then the evaluation of a regular path query is defined as follows:

$$\begin{aligned} (c_1, r, c_2)(G) &= \{\mu_0 \mid (c_1, c_2) \in r[G]\} \\ (c, r, z)(G) &= \{\mu \mid \mathbf{dom}(\mu) = \{z\} \text{ and } (c, \mu(z)) \in r[G]\} \\ (z, r, c)(G) &= \{\mu \mid \mathbf{dom}(\mu) = \{z\} \text{ and } (\mu(z), c) \in r[G]\} \\ (z_1, r, z_2)(G) &= \{\mu \mid \mathbf{dom}(\mu) = \{z_1, z_2\} \text{ and } (\mu(z_1), \mu(z_2)) \in r[G]\} \end{aligned}$$

where μ_0 denotes the empty mapping such that $\mathbf{dom}(\mu) = \emptyset$ (the join identity).

Definition 2.14 (Navigational graph pattern)

If Q is a basic graph pattern, then Q is a navigational graph pattern. If Q is a navigational graph pattern and (x, r, y) is a regular path query, then $Q \bowtie (x, r, y)$ is a navigational graph pattern.

The definition of the evaluation of a navigational graph pattern then follows from the previous definition of a join and the definition of the evaluation of a regular path query (for a directed edge-labelled graph or a property graph, respectively). Likewise, *complex navigational graph patterns* – and their evaluation – are defined by extending this definition in the natural way with the same operators from Definition 2.8 following the same semantics seen in Definition 2.9.

2.2.4 OTHER FEATURES

Thus far, we have discussed features that form the practical and theoretical foundation of any query language for graphs [Angles et al., 2017]. However, specific query languages for graphs may support other features, such as aggregation (GROUP BY, COUNT, etc.), more complex filters and datatype operators (e.g., range queries on years extracted from a date), federation for querying remotely host

Give us feedback!

graphs over the Web, languages for updating graphs, support for entailment, etc. For more information, we refer to the documentation of the respective query languages (e.g., [Harris et al., 2013, Angles et al., 2018]) and to the survey by Angles et al. [2017].

2.2.5 QUERY INTERFACES

Knowledge graphs are often queried by non-expert users who may not be able to express their information needs in terms of a particular graph query language. Different types of interfaces have thus been proposed in order to assist users in querying data graphs. Such interfaces may support, for example:

Faceted browsing:

Users start by specifying a simple search, such as a keyword search, a type of node like Food Festival, or possibly other kinds of search. They are then presented with a set of matching results, and a set of facets, which are typically attributes (e.g., venue) and values (e.g., Santa Lucía) present in the current results set. Selecting a value for a facet restricts the current results set to include only results with the indicated value; this selection process can be applied iteratively to restrict results per multiple facets. Often the faceted criteria are translated into and evaluated as graph queries. Though relatively intuitive for users, such systems typically support acyclic queries that generate lists of results (analogous to graph queries that project a single variable), and rarely support more expressive queries. Examples of faceted browsing systems for graphs include VisiNav [Harth, 2010], Broccoli [Bast and Buchhold, 2013], SemFacet [Arenas et al., 2016], GraFa [Moreno-Vega and Hogan, 2018], etc.

Query building:

Users are provided with a form or graphical interface that can be used to specify a graph query without needing to understand the syntax of a specific query language. Such query builders allow for incrementally adding nodes or edges to the query, assisted by features such as auto-completion, previewing intermediate results, and graph navigation. Query builders typically allow for expressing queries equivalent to (cyclic) basic graph patterns, but may not support more expressive features of query languages as described herein. Graph query builder systems include Smeagol [Clemmer and Davies, 2011], QueryVOWL [Haag et al., 2015], VIIQ [Jayaram et al., 2015a], Sparklis [Ferré, 2017], RDF Explorer [Vargas et al., 2019], and more besides.

Query-by-example:

Users provide examples of positive and sometimes negative answers to their queries. For example, they may provide as positive examples the nodes (Arica), (Santiago), (Viña del Mar), and as negative examples the nodes (Chile), (Lima), where the system will then “reverse engineer” a query that returns positive examples but not negative examples (in this case, the query proposed may return nodes of type City whose country is Chile). Query-by-example systems typically support basic graph patterns, and may not support more expressive querying features. They are useful in cases where users have examples of what they are looking for, but are not necessarily sure of the query they need to retrieve similar examples. Query-by-example systems for graphs include GQBE [Jayaram et al., 2015b] and SPARQLByE [Díaz et al., 2016].

Question answering:

Users express their queries as questions in natural language; for example, they might ask “*What food festivals will be held in Arica?*”. The question answering system will then generate answers from the graph based on its best interpretation of the question. We identify three types of question answering system. *Navigation-based systems* identify entities/nodes from the graph that are mentioned in the query, and then attempt to navigate edges from those nodes whose labels best match the question; for example, they may match the nodes (Food Festival) and (Arica) in the graph based on the question, and from there, try to navigate edges in the graph whose labels match the question in order to find answers. *Template-based systems* rather pre-suppose a fixed list of question templates expressed in the query language, with placeholder variables that will be replaced with entities/nodes detected in the question; a template matched for the previous example may be of the form “*What X will be held in Y?*”. *Translation-based systems* attempt to translate the question into a query in the structured query language, using (typically neural) machine translation techniques. The latter two types of question answering systems can ad

tionally return a graph query that explains the answers generated. Question answering systems are often very intuitive to use, but may not always return correct results, particularly when considering complex questions/queries. Examples of question answering systems for knowledge graphs include Treo [Freitas et al., 2011], NFF [Hu et al., 2018], TemplateQA [Zheng et al., 2018], WDAqua-core1 [Diefenbach et al., 2020], and more besides.

Such query interfaces enable non-expert users to formulate queries over graphs, which in turn broadens the potential impact of knowledge graphs.

Chapter 3

Schema, Identity, Context

In this chapter we describe extensions of the data graph – relating to schema, identity and context – that provide additional structures for accumulating knowledge. Henceforth, we refer to a *data graph* as a collection of data represented as nodes and edges using one of the models discussed in Chapter 2. We refer to a *knowledge graph* as a data graph potentially enhanced with representations of schema, identity, context, ontologies and/or rules. These additional representations may be embedded in the data graph, or layered above. Representations for schema, identity and context are discussed now, while ontologies and rules will be discussed in Chapter 4.

3.1 SCHEMA

One of the benefits of modelling data as graphs – versus, for example, the relational model – is the option to forgo or postpone the definition of a schema. However, when modelling data as graphs, schemata *can* be used to prescribe a high-level structure and/or semantics that the graph follows or should follow. We discuss three types of graph schemata: *semantic*, *validating*, and *emergent*.

3.1.1 SEMANTIC SCHEMA

A semantic schema allows for defining the meaning of high-level terms (aka *vocabulary* or *terminology*) used in the graph, which facilitates reasoning over graphs using those terms. Looking at Figure 2.1, for example, we may notice some natural groupings of nodes based on the types of entities to which they refer. We may thus decide to define *classes*, such as Event, City, etc., to denote these groupings. In fact, Figure 2.1 already illustrates three low-level classes – Open Market, Food Market, Drinks Festival – grouping similar entities with an edge labelled *type*. We may subsequently wish to capture some relations between some of these classes. In Figure 3.1, we present a class hierarchy for events where children are defined to be *sub-classes* of their parents such that if we find an edge (EID15)-*type*→(Food Festival) in our graph, we may also *infer* that (EID15)-*type*→(Festival) and (EID15)-*type*→(Event) hold in the graph.

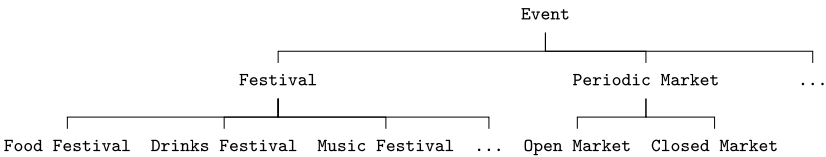


Figure 3.1: Example class hierarchy for Event

Give us feedback!

Aside from classes, we may also wish to define the semantics of edge labels, aka *properties*. Returning to Figure 2.1, we may consider that the properties *city* and *venue* are *sub-properties* of a more general property *location*, such that given an edge (Santa Lucía-city→Santiago), for example, we may also infer that (Santa Lucía-location→Santiago) must hold as an edge in the graph. We may also consider, for example, that *bus* and *flight* are both sub-properties of a more general property *connects to*. Along these lines, properties may also form a hierarchy similar to what we saw for classes. We may further define the *domain* of properties, indicating the class(es) of entities for nodes from which edges with that property extend; for example, we may define that the domain of *connects to* is a class *Place*, such that given the previous sub-property relations, we infer (Arica-type→Place). Conversely, we may define the *range* of properties, indicating the class(es) of entities for nodes to which edges with that property extend; for example, we may define that the range of *city* is a class *City*, inferring that (Arica-type→City).

A prominent standard for defining a semantic schema for (RDF) graphs is the *RDF Schema (RDFS)* standard [Brickley and Guha, 2014], which allows for defining sub-classes, sub-properties, domains, and ranges amongst the classes and properties used in an RDF graph, where such definitions can be serialised as a graph. We illustrate the semantics of these features in Table 3.1 and provide a concrete example of definitions in Figure 3.2 for a sample of terms used in the running example. These definitions can then be embedded into a data graph. More generally, the semantics of terms used in a graph can be defined in much more depth than seen here, as is supported by the *Web Ontology Language (OWL)* standard [Hitzler et al., 2012] for RDF graphs. We will return to such semantics later in Chapter 4.

Table 3.1: Definitions for sub-class, sub-property, domain and range

Feature	Definition	Condition	Example
SUB-CLASS	$c \text{ --subc. of-- } d$	$x \text{ --type-- } c \text{ implies } x \text{ --type-- } d$	$\text{City} \text{ --subc. of-- } \text{Place}$
SUB-PROPERTY	$p \text{ --subp. of-- } q$	$x \text{ --p-- } y \text{ implies } x \text{ --q-- } y$	$\text{venue} \text{ --subp. of-- } \text{location}$
DOMAIN	$p \text{ --domain-- } c$	$x \text{ --p-- } y \text{ implies } x \text{ --type-- } c$	$\text{venue} \text{ --domain-- } \text{Event}$
RANGE	$p \text{ --range-- } c$	$x \text{ --p-- } y \text{ implies } y \text{ --type-- } c$	$\text{venue} \text{ --range-- } \text{Venue}$

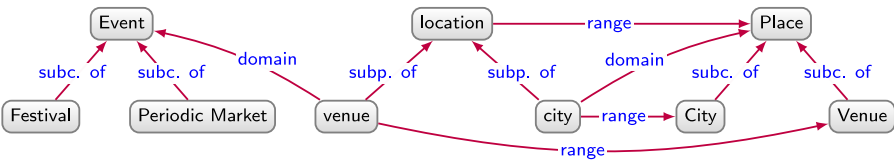


Figure 3.2: Example schema with sub-classes, sub-properties, domains, and ranges

Semantic schemata are typically defined for incomplete graph data, where the absence of an edge between two nodes, such as (Viña del Mar-flight→Arica), does not mean that the relation does not hold in the real world. Therefore, from the graph of Figure 2.1, we cannot assume that there is no flight between Viña del Mar and Arica. In contrast, if the *Closed World Assumption (CWA)* were adopted – as is the case in many classical database systems – it would be assumed that the data graph is a complete description of the world, thus allowing to assert with certainty that no flight exists between the two cities. Systems that do not adopt the CWA are said to adopt the *Open World Assumption (OWA)*. Considering our running example, it would be unreasonable to assume that the tourism organisation has complete knowledge of everything describable in its knowledge graph, and hence adopting the OWA appears more appropriate. However, it can be inconvenient if a system is unable to definitely answer “yes” or “no” to questions such as “is there a flight between Arica and Viña del Mar?”, especially when the organisation is certain that it has complete knowledge of the flights. A compromise between OWA and CWA is the *Local Closed World Assumption (LCWA)*, where portions of the data graph are assumed to be complete.

3.1.2 VALIDATING SCHEMA

Give us feedback!

When graphs are used to represent diverse, incomplete data at large scale, the OWA is the most appropriate choice for a *default* semantics. But in some scenarios, we may wish to guarantee that our data graph – or specific parts thereof – are in some sense “complete”. Returning to Figure 2.1, for example, we may wish to ensure that all events have at least a name, a venue, a start date, and an end date, such that applications using the data – e.g., one that sends event notifications to users – can ensure that they have the minimal information required. Furthermore, we may wish to ensure that the city of an event is *stated to be* a city (rather than *inferring* that it is a city). We can define such constraints in a validating schema and validate the data graph with respect to the resulting schema, listing constraint violations (if any). Thus while semantic schemata allow for inferring new graph data, validating schemata allow for validating a given data graph with respect to some constraints.

A standard way to define a validating schema for graphs is using *shapes* [Knublauch and Kontokostas, 2017, Prud'hommeaux et al., 2014, Labra Gayo et al., 2018]. A shape *targets* a set of nodes in a data graph and specifies *constraints* on those nodes. The shape's target can be defined in many ways, such as targeting all instances of a class, the domain or range of a property, the result of a query, nodes connected to the target of another shape by a given property, etc. Constraints can then be defined on the targeted nodes, such as to restrict the number or types of values taken on a given property, the shapes that such values must satisfy, etc.

A *shapes graph* is formed from a set of interrelated shapes. Shapes graphs can be depicted as UML-like class diagrams, where Figure 3.3 illustrates an example of a shapes graph based on Figure 2.1, defining constraints on four interrelated shapes. Each shape – denoted with a box like `PLACE`, `EVENT`, etc. – is associated with a set of constraints. Nodes conform to a shape if and only if they satisfy all constraints defined on the shape. Inside each shape box are placed constraints on the number (e.g., `[1..*]` denotes one-to-many, `[1..1]` denotes precisely one, etc.) and types (e.g., string, date-Time, etc.) of nodes that conforming nodes can relate to with a property (e.g., `name`, `start`, etc.). Another option is to place constraints on the number of nodes conforming to a particular shape that the conforming node can relate to with a property (thus generating edges between shapes); for example, `EVENT` `venue` `1..*` `VENUE` denotes that conforming nodes for `EVENT` must relate to at least one node with the property `venue` that conforms to the `VENUE` shape. Shapes can inherit the constraints of parent shapes – with inheritance denoted with an $\triangle$ connector – as in the case of `CITY` and `VENUE`, whose conforming nodes must also conform to the `PLACE` shape.

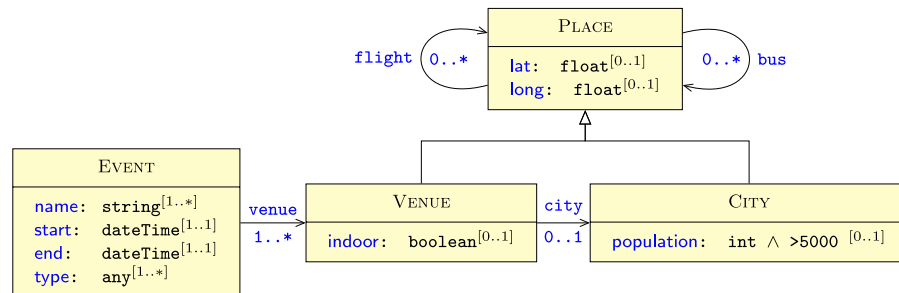


Figure 3.3: Example shapes graph depicted as a UML-like diagram

Given a shape and a targeted node, it is possible to check if the node conforms to that shape or not, which may require checking conformance of other nodes; for example, the node `(EID15)` conforms to the `EVENT` shape not only based on its local properties, but also based on conformance of `(Santa Lucía)` to `VENUE` and `(Santiago)` to `CITY`. Conformance dependencies may also be recursive, where the conformance of `(Santiago)` to `CITY` requires that it conforms to `PLACE`, which requires that `(Viña del Mar)` and `(Arica)` conform to `PLACE`, and so on. Conversely, `(EID16)` does not conform to `EVENT`, as it does not have the `start` and `end` properties required by the example shapes graph.

When declaring shapes, the data modeller may not know in advance the entire set of properties that some nodes can have (now or in the future). An *open shape* allows the node to have additional properties not specified by the shape, while a *closed shape* does not. For example, if we add the edge `(Santiago)` `founder` `Pedro de Valdivia` to the graph represented in Figure 2.1, then `(Santiago)` only conforms to the `CITY` shape if the shape is defined as open (since the shape does not mention `founder`).

Practical languages for shapes often support additional Boolean features, such as conjunction (AND), disjunction (OR), and negation (NOT) of shapes; for example, we may say that all the values of `venue` should conform to the shape `VENUE AND (NOT CITY)`, making explicit that venues in the data graph should not be directly given as cities. However, shapes languages that freely combine recursion and negation may lead to semantic problems, depending on how their semantics are defined. To illustrate, consider the following case inspired by the barber paradox [Labra Gayo et al., 2018], involving a shape `BARBER` whose conforming nodes `shave` at least one node conforming to `PERSON AND (NOT BARBER)`. Now, given (only) `(Bob) -shave-> (Bob)` with `(Bob)` conforming to `PERSON`, does `(Bob)` conform to `BARBER`? If yes – if `(Bob)` conforms to `BARBER` – then `(Bob)` violates the constraint by not shaving at least one node conforming to `PERSON AND (NOT BARBER)`. If no – if `(Bob)` does not conform to `BARBER` – then `(Bob)` satisfies the `BARBER` constraint by shaving such a node. Semantics to avoid such paradoxical situations have been proposed based on stratification [Boneva et al., 2017], partial assignments [Corman et al., 2018], and stable models [Gelfond and Lifschitz, 1988].

Although validating schemata and semantic schemata serve different purposes, they can complement each other. In particular, a validating schema can take into consideration a semantic schema, such that, for example, validation is applied on the data graph including inferences. Taking the class hierarchy of Figure 3.1 and the shapes graph of Figure 3.3, for example, we may define the target of the `EVENT` shape as the nodes that are of type Event (the class). If we first apply inferencing with respect to the class hierarchy of the semantic schema, the `EVENT` shape would now target `(EID15)` and `(EID16)`. The presence of a semantic schema may, however, require adapting the validating schema. Taking into account, for example, the aforementioned class hierarchy would require defining a relaxed cardinality on the `type` property. Open shapes may also be preferred in such cases rather than enumerating constraints on all possible properties that may be inferred on a node.

Two shapes languages have recently emerged for RDF graphs: *Shape Expressions (ShEx)*, published as a W3C Community Group Report [Prud'hommeaux et al., 2014]; and *SHACL (Shapes Constraint Language)*, published as a W3C Recommendation [Knublauch and Kontokostas, 2017]. These languages support the discussed features (and more) and have been adopted for validating graphs in a number of domains relating to healthcare [Thornton et al., 2019], scientific literature [Hammond et al., 2017], spatial data [Car et al., 2019], amongst others. More details about ShEx and SHACL can be found in the book by Labra Gayo et al. [2018]. A recently proposed language that can be used as a common basis for both ShEx and SHACL reveals their similarities and differences [Labra Gayo et al., 2019]. A similar notion of schema has been proposed by Angles [2018] for property graphs.

We formally define shapes following the conventions of Labra Gayo et al. [2019].

Definition 3.1 (Shape)

A shape ϕ is defined as:

$\phi ::= \top$	<i>true</i>
$ \Delta_N$	<i>node belongs to the set of nodes N</i>
$ \Psi_{\text{cond}}$	<i>node satisfies the Boolean condition cond</i>
$ \phi_1 \wedge \phi_2$	<i>conjunction of shape ϕ_1 and shape ϕ_2</i>
$ \neg\phi$	<i>negation of shape ϕ</i>
$ @s$	<i>reference to shape with label s</i>
$ \xrightarrow{p} \phi\{\text{min}, \text{max}\}$	<i>between min and max outward edges (inclusive) with label p to nodes satisfying shape ϕ</i>

where $\text{min} \in \mathbb{N}_{(0)}$, $\text{max} \in \mathbb{N}_{(0)} \cup \{*\}$, with “*” indicating unbounded.

Definition 3.2 (Shapes schema)

A shapes schema is defined as a tuple $\Sigma = (\Phi, S, \lambda)$ where Φ is a set of shapes, S is a set of shape labels, and $\lambda : S \rightarrow \Phi$ is a total function from labels to shapes.

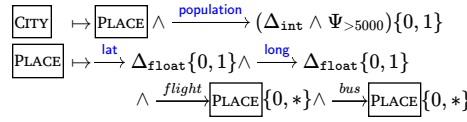
Example 3.1 — The shapes schema from Figure 3.3 can be expressed as:

```

EVENT  $\mapsto$   $\xrightarrow{\text{name}} \Delta_{\text{string}}\{1, *\} \wedge \xrightarrow{\text{start}} \Delta_{\text{dateTime}}\{1, 1\} \wedge \xrightarrow{\text{end}} \Delta_{\text{dateTime}}\{1, 1\}$ 
 $\wedge \xrightarrow{\text{type}} \top\{1, *\} \wedge \xrightarrow{\text{venue}} \text{VENUE}\{1, *\}$ 
VENUE  $\mapsto$   $\text{PLACE} \wedge \xrightarrow{\text{indoor}} \Delta_{\text{boolean}}\{0, 1\} \wedge \xrightarrow{\text{city}} \text{CITY}\{0, 1\}$ 

```

Give us feedback!



For example, $\boxed{\text{EVENT}}$ is a shape label (an element of S) that maps to a shape (an element of ϕ). This mapping is defined by λ .

In a shapes schema, shapes may refer to other shapes, giving rise to a graph that is sometimes known as the *shapes graph* [Knublauch and Kontokostas, 2017]. Figure 3.3 illustrates a shapes graph of this form.

The semantics of a shape is defined in terms of the evaluation of that shape over each node of a given data graph. The semantics of a shapes schema, in turn, is the result of evaluating each shape of the schema over each node of a given data graph; the result of this evaluation is a *shapes map*.

Definition 3.3 (Shapes map)

Given a directed edge-labelled graph $G = (V, E, L)$ and a shapes schema $\Sigma = (\Phi, S, \lambda)$, a *shapes map* is a (partial) mapping $\sigma : V \times S \rightarrow \{0, 1\}$.

The shapes map σ is a way of labelling the nodes of G with the labels of shapes from S . If $\sigma(v, s) = 1$, then node v is labelled s (possibly amongst other labels); otherwise if $\sigma(v, s) = 0$, then node v is not labelled s . The precise semantics depends on whether or not σ is a total or partial mapping: whether or not it is defined for every pair in $V \times S$. Herein we present the semantics for the more straightforward case wherein σ is assumed to be a total shapes map.

Definition 3.4 (Shape evaluation)

Given a shapes schema $\Sigma := (\Phi, S, \lambda)$, a directed edge-labelled graph $G = (V, E, L)$, a node $v \in V$ and a total shapes map σ , the *shape evaluation function* $[\phi]^{G,v,\sigma} \in \{0, 1\}$ is defined as follows:

$$\begin{aligned}
[\top]^{G,v,\sigma} &= 1 \\
[\Delta_N]^{G,v,\sigma} &= 1 \text{ iff } v \in N \\
[\Psi_{\text{cond}}]^{G,v,\sigma} &= 1 \text{ iff } \text{cond}(v) \text{ is true} \\
[\phi_1 \wedge \phi_2]^{G,v,\sigma} &= \min\{[\phi_1]^{G,v,\sigma}, [\phi_2]^{G,v,\sigma}\} \\
[\neg\phi]^{G,v,\sigma} &= 1 - [\phi]^{G,v,\sigma} \\
[@s]^{G,v,\sigma} &= 1 \text{ iff } \sigma(v, s) = 1 \\
[\xrightarrow{p} \phi\{\min, \max\}]^{G,v,\sigma} &= 1 \text{ iff } \min \leq |\{(v, p, u) \in E \mid [\phi]^{G,u,\sigma} = 1\}| \leq \max
\end{aligned}$$

If $[\phi]^{G,v,\sigma} = 1$, then v is said to satisfy ϕ in G under σ .

Typically for the purposes of validating a graph with respect to a shapes schema, a *target* is defined that requires certain nodes to satisfy certain shapes.

Definition 3.5 (Shapes target)

Given a directed edge-labelled graph $G = (V, E, L)$ and a shapes schema $\Sigma = (\Phi, S, \lambda)$, a *shapes target* $T \subseteq V \times S$ is a set of pairs of nodes and shape labels from G and Σ , respectively.

The nodes that a shape targets can be selected a manual selection, based on the type(s) of the nodes, based on the results of a graph query, etc. [Corman et al., 2018, Labra Gayo et al., 2019]. Lastly, we define the notion of a valid graph under a given shapes schema and target based on the existence of a shapes map satisfying certain conditions.

Definition 3.6 (Valid graph)

Given a shapes schema $\Sigma = (\Phi, S, \lambda)$, a directed edge-labelled graph $G = (V, E, L)$, and a shapes target T , we say that G is *valid under Σ and T* if and only if there exists a shapes map σ such that, for all $s \in S$ and $v \in V$ it holds that $\sigma(v, s) = [\lambda(s)]^{G,v,\sigma}$, and $(v, s) \in T$ implies $\sigma(v, s) = 1$.

Example 3.2 — Taking the graph G from Figure 2.1 and the shapes schema Σ from Figure 3.3, first assume an empty shapes target $T = \{\}$. If we consider a shapes map where (e.g.) $\sigma(\boxed{\text{EID15}}, \boxed{\text{EVENT}}) = 1$, $\sigma(\boxed{\text{Santa Lucía}}, \boxed{\text{VENUE}}) = 1$, $\sigma(\boxed{\text{Santa Lucía}}, \boxed{\text{PLACE}}) = 1$, etc., but where $\sigma(\boxed{\text{EID16}}, \boxed{\text{EVENT}}) = 0$ (as it

Give us feedback!

does not have the required values for **start** and **end**), etc., then we see that G is valid under Σ and T . However, if we were to define a shapes target T to ensure that the $\boxed{\text{EVENT}}$ shape targets $\boxed{\text{EID15}}$ and $\boxed{\text{EID16}}$ – i.e., to define T such that $\{(\boxed{\text{EID15}}, \boxed{\text{EVENT}}), (\boxed{\text{EID16}}, \boxed{\text{EVENT}})\} \subseteq T$ – then the graph would no longer be valid under Σ and T since $\boxed{\text{EID16}}$ does not satisfy $\boxed{\text{EVENT}}$.

The semantics we present here assumes that each node in the graph either satisfies or does not satisfy each shape labelled by the schema. More complex semantics – for example, based on Kleene’s three-valued logic [Corman et al., 2018, Labra Gayo et al., 2019] – have been proposed that support partial shapes maps, where the satisfaction of some nodes for some shapes can be left as undefined. Shapes languages in practice may support other more advanced forms of constraints, such as counting on paths [Knublauch and Kontokostas, 2017]. In terms of implementing validation with respect to shapes, work has been done on translating constraints into sets of graph queries, whose results are input to a SAT solver for recursive cases [Corman et al., 2019].

3.1.3 EMERGENT SCHEMA

Both semantic and validating schemata require a domain expert to explicitly specify definitions and constraints. However, a data graph will often exhibit latent structures that can be automatically extracted as an *emergent schema* [Pham et al., 2015] (aka *graph summary* [Liu et al., 2018, Čebirić et al., 2019, Spahiu et al., 2016]).

A framework often used for defining emergent schema is that of *quotient graphs*, which partition groups of nodes in the data graph according to some equivalence relation while preserving some structural properties of the graph. Taking Figure 2.1, we can intuitively distinguish different *types* of nodes based on their context, such as event nodes, which link to venue nodes, which in turn link to city nodes, and so forth. In order to describe the structure of the graph, we could consider six partitions of nodes: *event*, *name*, *venue*, *class*, *date-time*, *city*. In practice, these partitions may be computed based on the class or shape of the node. Merging the nodes of each partition into one node while preserving edges leads to the quotient graph shown in Figure 3.4: the nodes of this quotient graph are the partitions of nodes from the data graph and an edge $\boxed{X} \xrightarrow{y} \boxed{Z}$ is included in the quotient graph if and only if there exists $x \in X$ and $z \in Z$ such that $\boxed{x} \xrightarrow{y} \boxed{z}$ is in the original data graph.

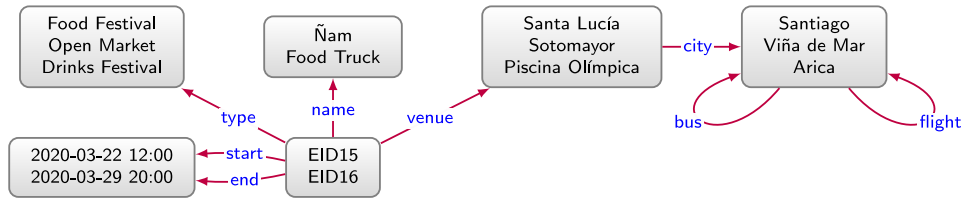


Figure 3.4: Example quotient graph simulating the data graph in Figure 2.1

There are many ways in which quotient graphs may be defined, depending not only on how nodes are partitioned, but also how the edges are defined. Different quotient graphs may provide different guarantees with respect to the structure they preserve. Formally, we can say that every quotient graph *simulates* its input graph (based on the *simulation relation* of set membership between data nodes and quotient nodes), meaning that for all $x \in X$ with x an input node and X a quotient node, if $\boxed{x} \xrightarrow{y} \boxed{z}$ is an edge in the data graph, then there must exist an edge $\boxed{X} \xrightarrow{y} \boxed{Z}$ in the quotient graph such that $z \in Z$; for example, the quotient graph of Figure 3.4 simulates the data graph of Figure 2.1. However, this quotient graph seems to suggest (for instance) that $\boxed{\text{EID16}}$ would have a start and end date in the data graph when this is not the case. A stronger notion of structural preservation is given by *bisimilarity*, which in this case would further require that if $\boxed{X} \xrightarrow{y} \boxed{Z}$ is an edge in the quotient graph, then for all $x \in X$, there must exist a $z \in Z$ such that $\boxed{x} \xrightarrow{y} \boxed{z}$ is in the data graph; this is not satisfied by $\boxed{\text{EID16}}$ in the quotient graph of Figure 3.4, which does not have an outgoing edge labelled **start** or **end** in the original data graph. Figure 3.5 illustrates a bisimilar version of the quotient graph, splitting the *event* partition into two nodes reflecting their different outgoing edges. An interesting property of bisimilarity is that it preserves forward-directed paths: given a path expression r without inverses and two bisimilar graphs, r will match a path in one graph if and or

if it matches a corresponding path in the other bisimilar graph. One can verify, for example, that a path matches $x \text{--city--(flight--bus)*--} z$ in Figure 2.1 if and only if there is a path matching $x \text{--city--(flight--bus)*--} z$ in Figure 3.5 such that $x \in X$ and $z \in Z$.

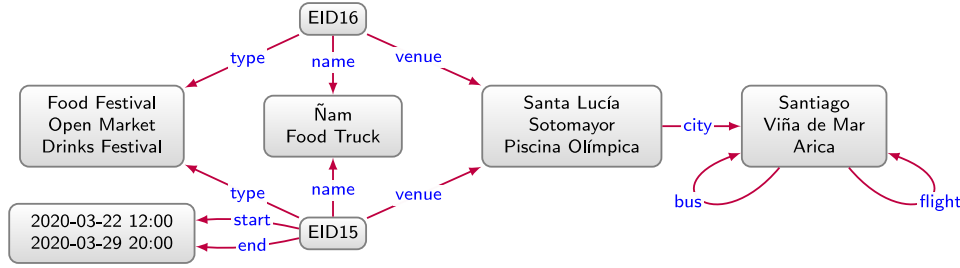


Figure 3.5: Example quotient graph bisimilar with the data graph in Figure 2.1

There are many ways in which quotient graphs may be defined, depending on the equivalence relation that partitions nodes. Furthermore, there are many ways in which other similar or bisimilar graphs can be defined, depending on the (bi)simulation relation that preserves the data graph's structure [Čebirić et al., 2019]. Such techniques aim to *summarise* the data graph into a higher-level topology. In order to reduce the memory overhead of the quotient graph, in practice, nodes may rather be labelled with the cardinality of the partition and/or a high-level label (e.g., *event*, *city*) for the partition rather than storing the labels of all nodes in the partition.

Various other forms of emergent schema not directly based on a quotient graph framework have also been proposed; examples include emergent schemata based on relational tables [Pham et al., 2015], and based on formal concept analysis [González and Hogan, 2018]. Emergent schemata may be used to provide a human-understandable overview of the data graph, to aid with the definition of a semantic or validating schema, to optimise the indexing and querying of the graph, to guide the integration of data graphs, and so forth. We refer to the survey by Čebirić et al. [2019] dedicated to the topic for further details.

Emergent schemata are often based on the notion of a quotient graph.

Definition 3.7 (Quotient graph)

Given a directed edge-labelled graph $G = (V, E, L)$, a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, L)$ is a quotient graph of G if and only if:

- $\mathcal{V}$ is a partition of V without the empty set, i.e., $\mathcal{V} \subseteq (2^V - \emptyset)$, $V = \bigcup_{U \in \mathcal{V}} U$, and for all $U \in \mathcal{V}$, $W \in \mathcal{V}$, it holds that $U = W$ or $U \cap W = \emptyset$; and
- $\mathcal{E} = \{(U, l, W) \mid U \in \mathcal{V}, W \in \mathcal{V} \text{ and } \exists u \in U, \exists w \in W : (u, l, w) \in E\}$.

A quotient graph can “merge” multiple nodes into one node, keeping the edges of its constituent nodes. For an input graph $G = (V, E, L)$, there is an exponential number of possible quotient graphs based on partitions of the input nodes. On one extreme, the input graph is a quotient graph of itself (turning nodes like $\{u\}$ into singleton nodes like $\{\{u\}\}$). On the other extreme, a single node $\{V\}$, with all input nodes, and loops (V, l, V) for each edge-label l used in the set of input edges E , is also a quotient graph. Quotient graphs typically fall somewhere in between, where the partition $\mathcal{V}$ of V is often defined in terms of an *equivalence relation* $\sim$ on the set V such that $\mathcal{V} := \sim/V$; i.e., $\mathcal{V}$ is defined as the *quotient set* of V with respect to $\sim$; for example, we might define an equivalence relation on nodes such that $u \sim v$ if and only if they have the same set of defined types, where $\sim/V$ is then a partition whose parts contain all nodes with the same types. Another way to induce a quotient graph is to define the partition in a way that preserves some of the topology (i.e., connectivity) of the input graph. One way to formally define this idea is through *simulation* and *bisimulation*.

Definition 3.8 (Simulation)

Given two directed edge-labelled graph $G = (V, E, L)$ and $G' = (V', E', L')$, let $R \subseteq V \times V'$ be a relation between the nodes of G and G' , respectively. We call R a simulation on G and G' if, for all $(v, v') \in R$, the following holds:

- if $(v, p, w) \in E$ then there exists w' such that $(v', p, w') \in E'$ and $(w, w') \in R$.

If a simulation exists on G and G' , we say that G' simulates G , denoted $G \rightsquigarrow G'$.

Definition 3.9 (Bisimulation)

If R is a simulation on G and G' , we call it a bisimulation if, for all $(v, v') \in R$, the following condition holds:

- if $(v', p, w') \in E'$ then there exists w such that $(v, p, w) \in E$ and $(w, w') \in R$.

If a bisimulation exists on G and G' , we call them bisimilar, denoted $G \approx G'$.

Bisimulation ($\approx$) is then an equivalence relation on graphs. By defining the (bi)simulation relation R in terms of set membership $\in$, every quotient graph simulates its input graph, but does not necessarily bisimulate its input graph. This gives rise to the notion of *bisimilar quotient graphs*.

Example 3.3 — Figures 3.4 and 3.5 exemplify quotient graphs for the graph of Figure 2.1. Figure 3.4 simulates but is not bisimilar to the data graph. Figure 3.5 is bisimilar to the data graph. Often the goal will be to compute the most concise quotient graph that satisfies a given condition; for example, the nodes without outgoing edges in Figure 3.5 could be merged while preserving bisimilarity.

3.2 IDENTITY

Figure 2.1 uses nodes like Santiago, but to which Santiago does this node refer? Do we refer to Santiago de Chile, Santiago de Cuba, Santiago de Compostela, or do we perhaps refer to the indie rock band Santiago? Based on edges such as Santa Lucía—city→Santiago, we may deduce that it is one of the three cities mentioned (not the rock band), and based on the fact that the graph describes tourist attractions in Chile, we may further deduce that it refers to Santiago de Chile. Without further details, however, *disambiguating* nodes of this form may rely on heuristics prone to error in more difficult cases. To help avoid such ambiguity, first we may use globally-unique identifiers to avoid naming clashes when the knowledge graph is extended with external data, and second we may add external identity links to disambiguate a node with respect to an external source.

3.2.1 PERSISTENT IDENTIFIERS

Assume we wished to compare tourism in Chile and Cuba, and we have acquired an appropriate knowledge graph for Cuba similar to the one we have for Chile. We can merge two graphs by taking their union. However, as shown in Figure 3.6, using an ambiguous node like Santiago may yield a *naming clash*: the node is referring to two different real-world cities in both graphs, where the merged graph indicates that Santiago is a city in both Chile and Cuba (rather than two distinct cities).⁵ To avoid such clashes, long-lasting *persistent identifiers* (PIDs) [Hakala, 2010] can be created in order to uniquely identify an entity; examples of PID schemes include *Digital Object Identifiers* (DOIs) for papers, *ORCID iDs* for authors, *International Standard Book Numbers* (ISBNs) for books, *Alpha-2 codes* for counties, and more besides.

note 5 Such a naming clash occurs with trees, etc.

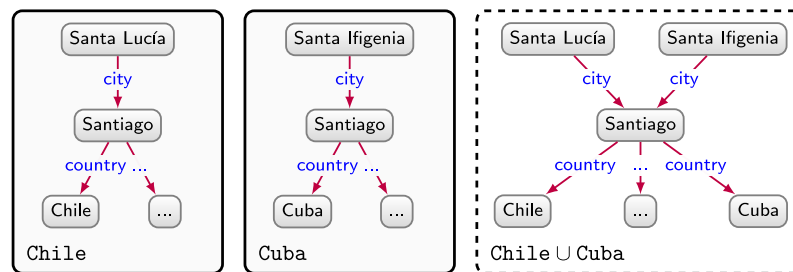


Figure 3.6: Result of merging two graphs with ambiguous local identifiers

In the context of the Semantic Web, the RDF data model goes one step further and recommends that global Web identifiers be used for nodes and edge labels. However, rather than adopt the *Uniform Resource Locators (URLs)* used to identify the location of *information resources* such as webpages, RDF 1.1 proposes to use *Internationalised Resource Identifiers (IRIs)* to identify *non-information resources* such as cities or events.⁶ Hence, for example, in the RDF representation of the Wikidata [Vrandečić and Krötzsch, 2014] – a knowledge graph proposed to complement Wikipedia, discussed in more detail in Chapter 10 – while the URL <https://www.wikidata.org/wiki/Q2887> refers to a webpage that can be loaded in a browser providing human-readable metadata about Santiago, the IRI <http://www.wikidata.org/entity/Q2887> refers to the city itself. Distinguishing the identifiers for the webpage and the city itself avoids naming clashes; for example, if we use the URL to identify both the webpage and the city, we may end up with an edge in our graph, such as (with readable labels below the edge):

<https://www.wikidata.org/wiki/Q2887> – <https://www.wikidata.org/wiki/Property:P112> – <https://www.wikidata.org/wiki/Q203534>
 [Santiago (URL)] [founded by (URL)] [Pedro de Valdivia (URL)]

Such an edge leaves ambiguity: was Pedro de Valdivia the founder of the webpage, or the city? Using IRIs for entities distinct from the URLs for the webpages that describe them avoids such ambiguous cases, where Wikidata thus rather defines the previous edge using less ambiguous identifiers, as follows:

<http://www.wikidata.org/entity/Q2887> – <http://www.wikidata.org/prop/direct/P112> – <http://www.wikidata.org/entity/Q203534>
 [Santiago (IRI)] [founded by (IRI)] [Pedro de Valdivia (IRI)]

using IRIs for the city, person, and founder of, distinct from the webpages describing them. These Wikidata identifiers use the prefix <http://www.wikidata.org/entity/> for entities and the prefix <http://www.wikidata.org/prop/direct/> for relations. Such prefixes are known as *namespaces*, and are often abbreviated with prefix strings, such as `wd:` or `wdt:`, where the latter edge can then be written more concisely using such abbreviations as the edge `(wd:Q2887) – (wdt:P112) – (wd:Q203534)`.

If HTTP IRIs are used to identify the graph's entities, when the IRI is looked up (via HTTP), the web-server can return (or redirect to) a description of that entity in formats such as RDF. This further enables RDF graphs to link to related entities described in external RDF graphs over the Web, giving rise to *Linked Data* [Berners-Lee, 2006, Heath and Bizer, 2011] (discussed in Chapter 9). Though HTTP IRIs offer a flexible and powerful mechanism for issuing global identifiers on the Web, they are not necessarily persistent: websites may go offline, the resources described at a given location may change, etc. In order to enhance the persistence of such identifiers, *Persistent URL (PURL)* services offer redirects from a central server to a particular location, where the PURL can be redirected to a new location if necessary, changing the address of a document without changing its identifier. The persistence of HTTP IRIs can then be improved by using namespaces defined through PURL services.

3.2.2 EXTERNAL IDENTITY LINKS

Assume that the tourist board opts to define the `chile:` namespace with an IRI such as <http://turismo.cl/entity/> on a web-server that they control, allowing nodes such as `(chile:Santiago)` – a shortcut for the IRI <http://turismo.cl/entity/Santiago> – to be looked up over the Web. While using such a naming scheme helps to avoid naming clashes, the use of IRIs does not necessarily help ground the identity of a resource. For example, an external geographic knowledge graph may assign the same city the IRI `(geo:SantiagoDeChile)` in their own namespace, where we have no direct w

note 6 Uniform Resource Locators (URLs) used to locate information resources. International Resource Identifiers (IRIs) used to identify non-information resources. International

Give us feedback!

of knowing that the two identifiers refer to the same city. If we merge the two knowledge graphs, we will end up with two distinct nodes for the same city, and thus not integrate their data.

There are a number of ways to ground the identity of an entity. The first is to associate the entity with uniquely-identifying information in the graph, such as its geo-coordinates, its postal code, the year it was founded, etc. Each additional piece of information removes ambiguity regarding which city is being referred to, providing (for example) more options for matching the city with its analogue in external sources. A second option is to use *identity links* to state that a local entity has the same identity as another *coreferent* entity found in an external source; an instantiation of this concept can be found in the OWL standard, which defines the `owl:sameAs` property relating coreferent entities. Using this property, we could state the edge `(chile:Santiago)-owl:sameAs->(geo:SantiagoDeChile)` in our RDF graph, thus establishing an identity link between the corresponding nodes in both graphs. Rather than specifying pairwise identity links between all knowledge graphs, it suffices if two knowledge graphs provide corresponding identity links to the same external knowledge graph, such as DBpedia or Wikidata; for example, if the local knowledge graph provides an identity link to Wikidata indicating `(chile:Santiago)-owl:sameAs->(wd:Q2887)`, while the remote knowledge graph has the identity link `(geo:SantiagoDeChile)-owl:sameAs->(wd:Q2887)`, then we can infer `(chile:Santiago)-owl:sameAs->(geo:SantiagoDeChile)`. The semantics of `owl:sameAs` defined by the OWL standard then allows us to combine the data for both nodes. Such semantics will be discussed later in Chapter 4. Ways in which identity links can be computed will also be discussed later in Chapter 8.

3.2.3 DATATYPES

Consider the two date-times on the left of Figure 2.1: how should we assign these nodes persistent/global identifiers? Intuitively it would not make much sense, for example, to assign IRIs to these nodes since their syntactic form tells us what they refer to: specific dates and times in March 2020. This syntactic form is further recognisable by machine, meaning that with appropriate software, we could order such values in ascending or descending order, extract the year, etc.

Most practical data models for graphs allow for defining nodes that are datatype values. RDF utilises *XML Schema Datatypes (XSD)* [Peterson et al., 2012], amongst others, where a datatype node is given as a pair (l, d) where l is a lexical string, such as “2020-03-29T20:00:00”, and d is an IRI denoting the datatype, such as `xsd:dateTime`. The node is then denoted `"2020-03-29T20:00:00"^^xsd:dateTime`. Datatype nodes in RDF are called *literals* and are not allowed to have outgoing edges. Other datatypes commonly used in RDF data include `xsd:string`, `xsd:integer`, `xsd:decimal`, `xsd:boolean`, etc. If the datatype is omitted, the value is assumed to be of type `xsd:string`. Applications built on top of RDF can then recognise these datatypes, parse them into datatype objects, and apply equality checks, normalisation, ordering, transformations, etc., according to their standard definition. In the context of property graphs, Neo4j [Miller, 2013] also defines a set of internal datatypes on property values that includes numbers, strings, Booleans, spatial points, and temporal values.

3.2.4 LEXICALISATION

Global identifiers for entities will sometimes have a human-interpretable form, such as `(chile:Santiago)`, but the identifier strings themselves do not carry any formal semantic significance. In other cases, the identifiers used may not be human-interpretable by design. In Wikidata, for instance, Santiago de Chile is identified as `(wd:Q2887)`, where such a scheme has the advantage of providing better persistence and of not being biased to a particular human language. As a real-world example, the Wikidata identifier for Eswatini (`(wd:Q1050)`) was not affected when the country changed its name from Swaziland, and does not necessitate choosing between languages for creating (more readable) IRIs such as `(wd:Eswatini)` (English), `(wd:eSwatini)` (Swazi), `(wd:Esuatini)` (Spanish), etc.

Since identifiers can be arbitrary, it is common to add edges that provide a human-interpretable label for nodes, such as `(wd:Q2887)-rdfs:label->"Santiago"`, indicating how people may refer to the subject node linguistically. Linguistic information of this form plays an important role in grounding knowledge such that users can more clearly identify which real-world entity a particular node in a knowledge graph actually references [de Melo, 2015]; it further permits cross-referencing entity labels with text corpora to find, for example, documents that potentially speak of a given

[Give us feedback!](#)

entity [Martínez-Rodríguez et al., 2020]. Labels can be complemented with aliases (e.g., `wd:Q2887` `skos:altLabel` `"Santiago de Chile"`) or comments (e.g., `wd:Q2887` `rdfs:comment` `"Santiago is the capital of Chile"`) to further help ground the node's identity.

Nodes such as `"Santiago"` denote string literals, rather than an identifier. Depending on the specific graph model, such literal nodes may also be defined as a pair (s, l) , where s denotes the string and l a language code; in RDF, for example we may state `chile:City` `rdfs:label` `"City"@en`, `chile:City` `rdfs:label` `"Ciudad"@es`, etc., indicating labels for the node in different languages. In other models, the pertinent language can rather be specified, e.g., via metadata on the edge. Knowledge graphs with human-interpretable labels, aliases, comments, etc., (in various languages) are sometimes called (*multilingual*) *lexicalised knowledge graphs* [Bonatti et al., 2018].

3.2.5 EXISTENTIAL NODES

When modelling incomplete information, we may in some cases know that there must exist a particular node in the graph with particular relationships to other nodes, but without being able to identify the node in question. For example, we may have two co-located events `chile:EID42` and `chile:EID43` whose venue has yet to be announced. One option is to simply omit the venue edges, in which case we lose the information that these events have a venue and that both events have the same venue. Another option might be to create a fresh IRI representing the venue, but semantically this becomes indistinguishable from there being a known venue. Hence some graph models permit the use of existential nodes, represented here as a blank circle:



These edges denote that there exists a common venue for `chile:EID42` and `chile:EID43` without identifying it. Existential nodes are supported in RDF as blank nodes [Cyganiak et al., 2014], which are also commonly used to support modelling complex elements in graphs, such as *RDF lists* [Cyganiak et al., 2014, Hogan et al., 2014]. Figure 3.7 exemplifies an RDF list, which uses blank nodes in a linked-list structure to encode order. Though existential nodes can be convenient, their presence can complicate operations on graphs, such as deciding if two data graphs have the same structure modulo existential nodes [Cyganiak et al., 2014, Hogan, 2017]. Hence methods for *skolemising* existential nodes in graphs – replacing them with canonical labels – have been proposed [Longley and Sporny, 2019, Hogan, 2017]. Other authors rather call to minimise the use of such nodes in graph data [Heath and Bizer, 2011].

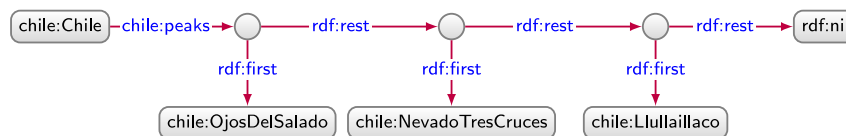


Figure 3.7: RDF list representing the three largest peaks of Chile, in order

3.3 CONTEXT

Many (arguably *all*) facts presented in the data graph of Figure 2.1 can be considered true with respect to a certain *context*. With respect to *temporal context*, `Santiago` has existed as a city since 1541, flights from `Arica` to `Santiago` began in 1956, etc. With respect to *geographic context*, the graph describes events in Chile. With respect to *provenance*, data relating to `EID15` were taken from – and are thus said to be true with respect to – the Ñam webpage on January 4th, 2020. Other forms of context may also be used. We may further combine contexts, such as to indicate that `Arica` is a Chilean city (*geographic*) since 1883 (*temporal*) per the Treaty of Ancón (*provenance*).

By context we herein refer to the *scope of truth*, i.e., the context in which some data are held to be true [McCarthy, 1993, Guha et al., 2004]. The graph of Figure 2.1 leaves much of its context implicit. However, making context explicit can allow for interpreting the data from different perspectives, such as to understand what held true in 2016, what holds true excluding webpages later found to have spurious data, etc. As seen previously, context for graph data may be considered at different

Give us feedback!

levels: on individual nodes, individual edges, or sets of edges (sub-graphs). We now discuss various representations by which context can be made explicit at different levels.

3.3.1 DIRECT REPRESENTATION

The first way to represent context is to consider it as data no different from other data. For example, the dates for the event (EID15) in Figure 2.1 can be seen as representing a form of temporal context, indicating the temporal scope within which edges such as (EID15)–venue→(Santa Lucia) are held true. Another option is to change a relation represented as an edge, such as (Santiago)–flight→(Arica), into a node, such as seen in Figure 2.3a, allowing us to assign additional context to the relation. While in these examples context is represented in an ad hoc manner, a number of specifications have been proposed to represent context as data in a more standard way. One example is the *Time Ontology* [Cox et al., 2017], which specifies how temporal entities, intervals, time instants, etc. – and relations between them such as *before*, *overlaps*, etc. – can be described in RDF graphs in an interoperable manner. Another example is the *PROV Data Model* [Gil et al., 2013], which specifies how provenance can be described in RDF graphs, where entities (e.g., graphs, nodes, physical document) are derived from other entities, are generated and/or used by activities (e.g., extraction, authorship), and are attributed to agents (e.g., people, software, organisations).

3.3.2 REIFICATION

Often we may wish to directly define the context of edges themselves; for example, we may wish to state that the edge (Santiago)–flight→(Arica) is valid from 1956. While we could use the pattern of turning the edge into a node – as illustrated in Figure 2.3a – to directly represent such context, another option is to use *reification*, which allows for making statements about statements in a generic manner (or in the case of a graph, for defining edges about edges). In Figure 3.8 we present three forms of reification that can be used for modelling temporal context on the aforementioned edge within a directed edge-labelled graph [Hernández et al., 2015]. We use e to denote an arbitrary identifier representing the edge itself to which the context can be associated. Unlike in a direct representation, e represents an edge, not a flight. RDF reification [Brickley and Guha, 2014] (Figure 3.8a) defines a new node e to represent the edge and connects it to the source node (via *subject*), target node (via *object*), and edge label (via *predicate*) of the edge. In contrast, n -ary relations [Brickley and Guha, 2014] (Figure 3.8b) connect the source node of the edge directly to the edge node e with the label of the edge; the target node of the edge is then connected to e (via *value*). Finally, singleton properties [Nguyen et al., 2014] (Figure 3.8c) rather use e as an edge label, connecting it to a node indicating the original edge label (via *singleton*). Other forms of reification have been proposed in the literature, including, for example, NdFluents [Giménez-García et al., 2017]. In general, a reified edge does not assert the edge it reifies; for example, we may reify an edge to state that it is no longer valid. We refer to Hernández et al. [2015] for further comparison of reification alternatives.

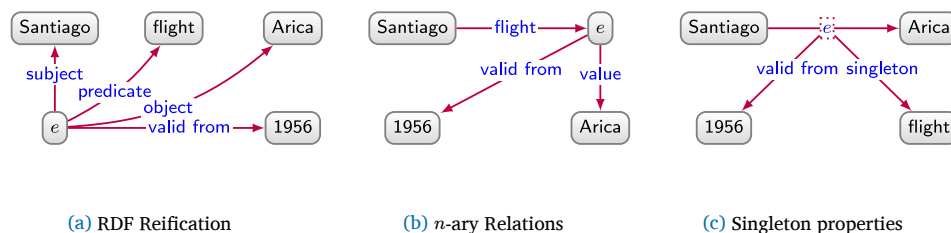


Figure 3.8: Three representations of temporal context on a directed labelled edge

3.3.3 HIGHER-ARITY REPRESENTATION

As an alternative to reification, we can rather use higher-arity representations for modelling context. Taking again the edge (Santiago)–flight→(Arica), Figure 3.9 illustrates three higher-arity representations of temporal context. First, we can use a named graph (Figure 3.9a) to contain the edge and then define the temporal context on the graph name. Second, we can use a property graph (Figure 3.9b) to contain the edge and then define the temporal context on the property name. Third, we can use a higher-arity representation (Figure 3.9c) to contain the edge and then define the temporal context on the higher-arity representation name.

[Give us feedback!](#)

where the temporal context is defined as a property on the edge. Third, we can use *RDF** [Hartig, 2017] (Figure 3.9c): an extension of RDF that allows edges to be defined as nodes. Amongst these options, the most flexible is the named graph representation, where we can assign context to multiple edges at once by placing them in one named graph; for example, we can add more edges to the named graph of Figure 3.9a that are also valid from 1956. The least flexible option is *RDF**, which, in the absence of an edge id, does not permit different groups of contextual values to be assigned to an edge; for example, if we add four contextual values to the edge (Chile)–*president*→(M. Bachelet), to state that it was valid from 2006 until 2010 and valid from 2014 until 2018, we cannot pair the values, but may rather have to create a node to represent different presidencies (in the other models, we could have used two named graphs or edge ids).

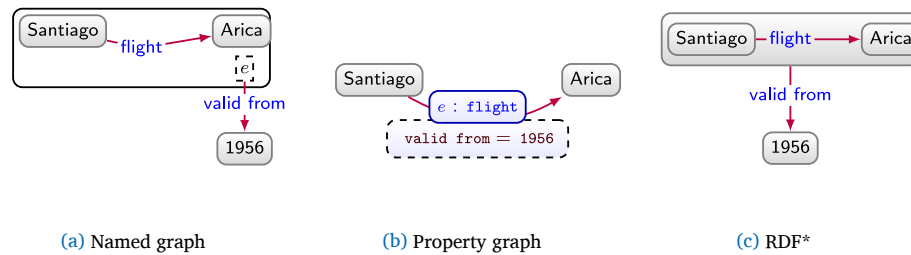


Figure 3.9: Three higher-arity representations of temporal context on an edge

3.3.4 ANNOTATIONS

Thus far, we have discussed representing context in a graph, but we have not spoken about automated mechanisms for reasoning about context; for example, if there are only seasonal summer flights from (Santiago) to (Arica), we may wish to find other routes from Santiago for winter events taking place in (Arica). While the dates for buses, flights, etc., can be represented directly in the graph, or using reification, writing a query to manually intersect the corresponding temporal contexts will be difficult. An alternative is to consider *annotations* that provide mathematical definitions of a contextual domain and key operations over that domain that can be applied automatically.

Some annotations model a particular contextual domain; for example, *Temporal RDF* [Gutiérrez et al., 2007] allows for annotating edges with time intervals, such as (Chile)–*president*_[2006,2010]→(M. Bachelet), while *Fuzzy RDF* [Straccia, 2009] allows for annotating edges with a degree of truth such as (Santiago)–*climate*_{0.8}→(Semi-Arid), indicating that it is more-or-less true – with a degree of 0.8 – that Santiago has a semi-arid climate.

Other forms of annotation are domain-independent; for example, *Annotated RDF* [Dividino et al., 2009, Udre et al., 2010, Zimmermann et al., 2012] allows for representing context modelled as *semi-rings*: algebraic structures consisting of domain values (e.g., temporal intervals, fuzzy values, etc.) and two operators to combine domain values: *meet* and *join*.⁷ We provide an example in Figure 3.10, where G is annotated with values from a temporal domain using sets of integers (1–365 to represent days of the year). For brevity we use intervals, where, e.g., $\{[150, 152]\}$ denotes the set $\{150, 151, 152\}$. Query Q then asks for flights from Santiago to cities with events; this query will check and return an annotation reflecting the temporal validity of each answer. To derive these answers, we require a conjunction of annotations on compatible *flight* and *city* edges, using the *meet operator* to compute the annotation for which both edges hold. The natural way to define *meet* here is as the intersection of sets of days, where, for example, applying *meet* on the event annotation $\{[150, 152]\}$ and the flight annotation $\{[1, 120], [220, 365]\}$ for (Punta Arenas) leads to the empty time interval $\{\}$, which may thus lead to the city being filtered from the results (depending on the query evaluation semantics). However, for (Arica), we find two different non-empty intersections: $\{[123, 125]\}$ for (EID16) and $\{[276, 279]\}$ for (EID17). Given that we are interested in just the city (a projected variable), we can combine the two annotations for (Arica) using the *join operator*, returning the annotation in which either result holds true. The natural way to define *join* is as the union of the sets of days, giving $\{[123, 125], [276, 279]\}$.

note 7 The *join* operator algebra.

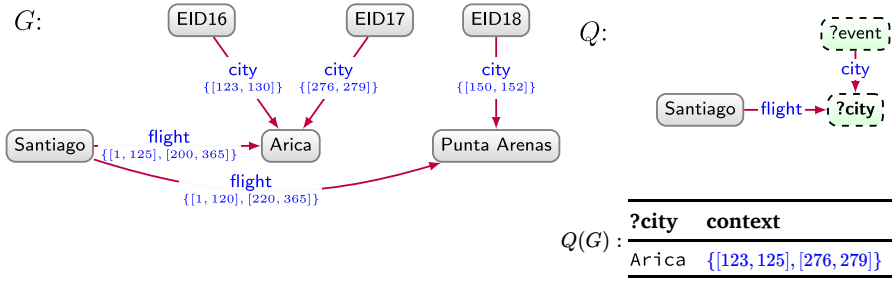


Figure 3.10: Example query on a temporally annotated graph

We define an annotation domain per Zimmermann et al. [2012].

Definition 3.10 (Annotation domain)

Let A be a set of annotation values. An annotation domain is an idempotent, commutative semi-ring $D = \langle A, \oplus, \otimes, \perp, \top \rangle$.

This definition can then instantiate specific domains of context.

Letting D be a semi-ring imposes that, for any values a, a_1, a_2, a_3 in A , the following hold:

- $(a_1 \oplus a_2) \oplus a_3 = a_1 \oplus (a_2 \oplus a_3)$
- $(\perp \oplus a) = (a \oplus \perp) = a$
- $(a_1 \oplus a_2) = (a_2 \oplus a_1)$
- $(a_1 \otimes a_2) = (a_2 \otimes a_1)$
- $(a_1 \otimes a_2) \otimes a_3 = a_1 \otimes (a_2 \otimes a_3)$
- $(\top \otimes a) = (a \otimes \top) = a$
- $a_1 \otimes (a_2 \oplus a_3) = (a_1 \otimes a_2) \oplus (a_1 \otimes a_3)$
- $(a_1 \oplus a_2) \otimes a_3 = (a_1 \otimes a_3) \oplus (a_2 \otimes a_3)$
- $(\perp \otimes a) = (a \otimes \perp) = \perp$

The requirement that it be idempotent further imposes the following:

- $(a \oplus a) = a$

Finally, the requirement that it be commutative imposes the following:

- $(a_1 \otimes a_2) = (a_2 \otimes a_1)$

Idempotence induces a partial order: $a_1 \leq a_2$ if and only if $a_1 \oplus a_2 = a_2$. Imposing these conditions on the annotation domain allow for reasoning and querying to be conducted over the annotation domain in a well-defined manner. Annotated graphs can then be defined in the natural way:

Definition 3.11 (Annotated directed edge-labelled graph)

Letting $D = \langle A, \oplus, \otimes, \perp, \top \rangle$ denote an idempotent, commutative semi-ring, we define an annotated directed edge-labelled graph (or annotated directed edge-labelled graph) as $G = (V, E_A, L)$ where $V \subseteq \mathbf{Con}$ is a set of nodes, $L \subseteq \mathbf{Con}$ is a set of edge labels, and $E_A \subseteq V \times L \times V \times A$ is a set of edges annotated with values from A .

Example 3.4 — Figure 3.10 exemplifies query answering on a graph annotated with days of the year. Formally this domain can be defined as follows: $A := 2^{\mathbb{N}_{[1,365]}}$, $\oplus := \cup$, $\otimes := \cap$, $\top := \mathbb{N}_{[1,365]}$, $\perp := \emptyset$, where one may verify that $D = \langle 2^{\mathbb{N}_{[1,365]}}, \cup, \cap, \mathbb{N}_{[1,365]}, \emptyset \rangle$ is indeed an idempotent, commutative semi-ring.

3.3.5 OTHER CONTEXTUAL FRAMEWORKS

Other frameworks have been proposed for modelling and reasoning about context in graphs. A notable example is that of *contextual knowledge repositories* [Serafini and Homola, 2012], which allow for assigning individual (sub-)graphs to their own context. Unlike in the case of named graphs, cc

Give us feedback!

text is explicitly modelled along one or more dimensions, where each (sub-)graph takes a value for each dimension. Each dimension is associated with a partial order over its values – e.g., $(2020-03-22) \preceq (2020-03) \preceq (2020)$ – enabling the selection and combination of sub-graphs that are valid within contexts at different granularities. Schuetz et al. [2021] similarly propose a form of contextual OnLine Analytic Processing (OLAP), based on a data cube formed by dimensions where each cell contains a knowledge graph. Operations such as “slice-and-dice” (selecting knowledge according to given dimensions), as well as “roll-up” (aggregating knowledge at a higher level) are supported. We refer the reader to the respective papers for more details [Serafini and Homola, 2012, Schuetz et al., 2021].

Chapter 4

Deductive Knowledge

As humans, we can *deduce* more from the data graph of Figure 2.1 than what the edges explicitly indicate. We may deduce, for example, that the Ñam festival (EID15) will be located in Santiago, even though the graph does not contain an edge $(EID15) \text{--} \text{location} \rightarrow (Santiago)$. We may further deduce that the cities connected by flights must have some airport nearby, even though the graph does not contain nodes referring to these airports. In these cases, given the data as premises, and some general rules about the world that we may know *a priori*, we can use a deductive process to derive new data, allowing us to know more than what is explicitly given by the data. These types of general premises and rules, when shared by many people, form part of “commonsense knowledge” [McCarthy, 1990]; conversely, when rather shared by a few experts in an area, they form part of “domain knowledge”, where, for example, an expert in biology may know that *hemocyanin* is a protein containing copper that carries oxygen in the blood of some species of *Mollusca* and *Arthropoda*.

Machines, in contrast, do not have *a priori* access to such deductive faculties; rather they need to be given formal instructions, in terms of premises and *entailment regimes*, facilitating similar deductions to what a human can make. In this way, we will be making more of the meaning (i.e., *semantics*) of the graph explicit in a machine-readable format. These entailment regimes formalise the conclusions that logically follow as a consequence of a given set of premises. Once instructed in this manner, machines can (often) apply deductions with a precision, efficiency, and scale beyond human performance. These deductions may serve a range of applications, such as improving query answering, (deductive) classification, finding inconsistencies, etc. As a concrete example involving query answering, assume we are interested in knowing *the festivals located in Santiago*; we may straightforwardly express such a query as per the graph pattern shown in Figure 4.1. This query returns no results for the graph in Figure 2.1: there is no node named (Festival), and nothing has (directly) the *location* (Santiago). However, an answer (Ñam) could be automatically entailed were we to state that *x* being a Food Festival *entails* that *x* is a Festival, or that *x* having venue *y* in city *z* *entails* that *x* has location *z*. How, then, should such entailments be captured? In Section 3.1.1 we already discussed how the former entailment can be captured with sub-class relations in a semantic schema; the second entailment, however, requires a more expressive entailment regime than seen thus far.

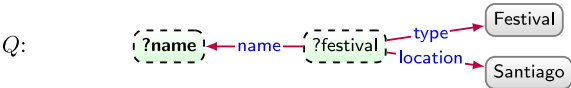


Figure 4.1: Graph pattern querying for names of festivals in Santiago

In this chapter, we discuss ways in which more complex entailments can be expressed and automated. Though we could leverage a number of logical frameworks for these purposes – such as First-Order Logic, Datalog, Prolog, Answer Set Programming, etc. – we focus on *ontologies*, which consist of

Give us feedback!

tute a formal representation of knowledge that, importantly for us, can be represented as a graph. We then discuss how these ontologies can be formally defined, how they relate to existing logical frameworks, and how reasoning can be conducted with respect to such ontologies.

4.1 ONTOLOGIES

To enable entailment, we must be precise about what the terms we use mean. Returning to Figure 2.1, for example, and examining the node (EID16) more closely, we may begin to question how it is modelled, particularly in comparison with (EID15). Both nodes – according to the class hierarchy of Figure 3.1 – are considered to be events. But what if, for example, we wish to define two pairs of start and end dates for (EID16) corresponding to the different venues? Should we rather consider what takes place in each venue as a different event? What then if an event has various start and end dates in a single venue: would these also be considered as one (recurring) event, or many events? These questions are facets of a more general question: *what precisely do we mean by an “event”*? Does it happen in one contiguous time interval or can it happen many times? Does it happen in one place or can it happen in multiple? There are no “correct” answers to such questions – we may understand the term “event” in a variety of ways, and thus the answers are a matter of *convention*.

In the context of computing, an *ontology*⁸ is then a concrete, formal representation of what terms mean within the scope in which they are used (e.g., a given domain). For example, one event ontology may formally define that if an entity is an “event”, then it has precisely one venue and precisely one time instant in which it begins. Conversely, a different event ontology may define that an “event” can have multiple venues and multiple start times, etc. Each such ontology formally captures a particular perspective – a particular *convention*. Under the first ontology, for example, we could not call the Olympics an “event”, while under the second ontology we could. Likewise ontologies can guide how graph data are modelled. Under the first ontology we may split (EID16) into two events. Under the second, we may elect to keep (EID16) as one event with two venues. Ultimately, given that ontologies are formal representations, they can be used to automate entailment.

Like all conventions, the usefulness of an ontology depends on the level of agreement on what that ontology defines, how detailed it is, and how broadly and consistently it is adopted. Adoption of an ontology by the parties involved in one knowledge graph may lead to a consistent use of terms and consistent modelling in that knowledge graph. Agreement over multiple knowledge graphs will, in turn, enhance the interoperability of those knowledge graphs.

Amongst the most popular ontology languages used in practice are the *Web Ontology Language* (OWL) [Hitzler et al., 2012]⁹, recommended by the W3C and compatible with RDF graphs; and the *Open Biomedical Ontologies Format* (OBOF) [Mungall et al., 2012], used mostly in the biomedical domain. Since OWL is the more widely adopted, we focus on its features, though many similar features are found in both [Mungall et al., 2012]. Before introducing such features, however, we must discuss how graphs are to be *interpreted*.

note 8 The term stems from entities that exist, the way they may be identified.

note 9 We could include OWL, which extends it.

4.1.1 INTERPRETATIONS AND MODELS

We as humans may *interpret* the node (Santiago) in the data graph of Figure 2.1 as referring to the real-world city that is the capital of Chile. We may further *interpret* an edge (Arica)–flight→(Santiago) as stating that there are flights from the city of Arica to this city. We thus interpret the data graph as another graph – what we here call the *domain graph* – composed of real-world entities connected by real-world relations. The process of interpretation, here, involves *mapping* the nodes and edges in the data graph to nodes and edges of the domain graph.

Along these lines, we can abstractly define an *interpretation* of a data graph as being composed of two elements: a domain graph, and a mapping from the *terms* (nodes and edge-labels) of the data graph to those of the domain graph. The domain graph follows the same model as the data graph; for example, if the data graph is a directed edge-labelled graph, then so too will be the domain graph. For simplicity, we will speak of directed edge-labelled graphs and refer to the nodes of the domain graph as *entities*, and to its edges as *relations*. Given a data graph and an interpretation, while we denote nodes in the data graph by (Santiago), we will denote the entity it refers to in the domain graph by (Santiago) (per the mapping of the given interpretation). Likewise, while we denote an edge by (Arica

Give us feedback!

$\text{flight} \rightarrow \text{Santiago}$, we will denote the relation by $\text{Arica} \rightarrow \text{flight} \rightarrow \text{Santiago}$ (again, per the mapping of the given interpretation). In this abstract notion of an interpretation, we do not require that Santiago or Arica be the real-world cities, nor even that the domain graph contain real-world entities and relations: an interpretation can have any domain graph and mapping.

Why is such an abstract notion of interpretation useful? The distinction between nodes/edges and entities/relations becomes important when we define the meaning of ontology features and entailment. To illustrate this distinction, if we ask whether there is an edge labelled flight between Arica and Viña del Mar for the data graph in Figure 2.1, the answer is *no*. However, if we ask if the entities Arica and Viña del Mar are connected by the relation flight , then the answer depends on what assumptions we make when interpreting the graph. Under the Closed World Assumption (CWA), if we do not have additional knowledge, then the answer is a definite *no* – since what is not known is assumed to be false. Conversely, under the Open World Assumption (OWA), we cannot be certain that this relation does not exist as this could be part of some knowledge not (yet) described by the graph. Likewise under the Unique Name Assumption (UNA), the data graph describes *at least two* flights to Santiago (since Viña del Mar and Arica are assumed to be different entities and, therefore, $\text{Arica} \rightarrow \text{flight} \rightarrow \text{Santiago}$ and $\text{Viña del Mar} \rightarrow \text{flight} \rightarrow \text{Santiago}$ must be different edges). Conversely, under No Unique Name Assumption (NUNA), we can only say that there is *at least one* such flight since Viña del Mar and Arica may be the same entity with two “names”.

These assumptions (or lack thereof) define which interpretations are valid, and which interpretations *satisfy* which data graphs. We call an interpretation that satisfies a data graph a *model* of that data graph. The UNA forbids interpretations that map two data terms to the same domain term. The NUNA allows such interpretations. Under the CWA, an interpretation that contains an edge $x \rightarrow p \rightarrow y$ in its domain graph can only satisfy a data graph from which we can entail $x \rightarrow p \rightarrow y$. Under the OWA, an interpretation containing the edge $x \rightarrow p \rightarrow y$ can satisfy a data graph not entailing $x \rightarrow p \rightarrow y$ so long it does not explicitly contradict that edge. OWL adopts the NUNA and OWA, which is the most general case: multiple nodes/edge-labels in the graph may refer to the same entity/relation-type (per the NUNA), and anything not entailed by the data graph is *not* assumed to be false as a consequence (per the OWA).

A graph interpretation – or simply interpretation – captures the assumptions under which the semantics of a graph can be defined. We define interpretations for directed edge-labelled graphs, though the notion extends naturally to other graph models (assuming the data and domain graphs follow the same model).

Definition 4.1 (Graph interpretation)

A (graph) interpretation I is defined as a pair $I := (\Gamma, \cdot^I)$ where $\Gamma = (V_\Gamma, E_\Gamma, L_\Gamma)$ is a (directed edge-labelled) graph called the domain graph and $\cdot^I : \text{Con} \rightarrow V_\Gamma \cup L_\Gamma$ is a partial mapping from constants to terms in the domain graph.

We denote the domain of the mapping $\cdot^I$ by $\text{dom}(\cdot^I)$. For interpretations under the UNA, the mapping $\cdot^I$ is required to be injective, while with no UNA (NUNA), no such requirement is necessary.

Interpretations that *satisfy* a graph are then said to be *models* of that graph.

Definition 4.2 (Graph models)

Let $G := (V, E, L)$ be a directed edge-labelled graph. An interpretation $I := (\Gamma, \cdot^I)$ satisfies G if and only if the following hold:

- $V \cup L \subseteq \text{dom}(\cdot^I)$;
- for all $v \in V$, it holds that $v^I \in V_\Gamma$;
- for all $l \in L$, it holds that $l^I \in L_\Gamma$; and
- for all $(u, l, v) \in E$, it holds that $(u^I, l^I, v^I) \in E_\Gamma$.

If I satisfies G we call I a (graph) model of G .

4.1.2 ONTOLOGY FEATURES

Beyond our base assumptions, we can associate certain patterns in the data graph with *semantic conditions* that define which interpretations satisfy it; for example, we can add a semantic condition

[Give us feedback!](#)

enforce that if our data graph contains the edge $(p \text{--} \text{subp. of} \rightarrow q)$, then any edge $(x \text{--} p \rightarrow y)$ in the domain graph of the interpretation must also have a corresponding edge $(x \text{--} q \rightarrow y)$ to satisfy the data graph. These semantic conditions then form the features of an ontology language. In what follows, to aid readability, we will introduce the features of OWL using an abstract graphical notation with abbreviated terms. For details of concrete syntaxes, we rather refer to the OWL and OBOF standards [Hitzler et al., 2012, Mungall et al., 2012]. Likewise we present semantic conditions over interpretations for each feature in the same graphical format;¹⁰ further details of these conditions will be described later in Section 4.1.3.

note 10 We abbreviate “if ϕ then ψ ” and “if ψ

Individuals

In Table 4.1, we list the main features supported by OWL for describing *individuals* (e.g., Santiago, EID16), sometimes distinguished from classes and properties. First, we can *assert* (binary) relations between individuals using edges such as $(\text{Santa Lucía} \text{--} \text{city} \rightarrow \text{Santiago})$. In the condition column, when we write $(x \text{--} y \rightarrow z)$, for example, we refer to the condition that the relation is given in the domain graph of the interpretation; if so, the interpretation satisfies the axiom. OWL further allows for defining relations to explicitly state that two terms refer to the *same* entity, where, e.g., $(\text{Región V} \text{--} \text{same as} \rightarrow \text{Región de Valparaíso})$ states that both refer to the same region (per Section 3.2); or that two terms refer to *different* entities, where, e.g., $(\text{Valparaíso} \text{--} \text{diff. from} \rightarrow \text{Región de Valparaíso})$ distinguishes the city from the region of the same name. We may also state that a relation does not hold using *negation*, which can be serialised as a graph using a form of reification (see Figure 3.8a).

Table 4.1: Ontology features for individuals

Feature	Axiom	Condition	Example
ASSERTION	$(x \text{--} y \rightarrow z)$	$(x \text{--} y \rightarrow z)$	$(\text{Chile} \text{--} \text{capital} \rightarrow \text{Santiago})$
NEGATION	$(x \text{--} y \rightarrow z)$	$\text{not } (x \text{--} y \rightarrow z)$	$(\text{Chile} \text{--} \text{capital} \rightarrow \text{Arica})$
SAME AS	$(x_1 \text{--} \text{same as} \rightarrow x_2)$	$(x_1 = x_2)$	$(\text{Región V} \text{--} \text{same as} \rightarrow \text{Región de Valparaíso})$
DIFFERENT FROM	$(x_1 \text{--} \text{diff. from} \rightarrow x_2)$	$(x_1 \neq x_2)$	$(\text{Valparaíso} \text{--} \text{diff. from} \rightarrow \text{Región de Valparaíso})$

Properties

In Section 3.1.1, we already discussed how *sub-properties*, *domains* and *ranges* may be defined for properties. OWL allows such definitions, and further includes other features, as listed in Table 4.2. We may define a pair of properties to be *equivalent*, *inverses*, or *disjoint*. We can further define a particular property to denote a *transitive*, *symmetric*, *asymmetric*, *reflexive*, or *irreflexive* relation. We can also define the multiplicity of the relation denoted by properties, based on being *functional* (many-to-one) or *inverse-functional* (one-to-many). We may further define a *key* for a class, denoting the set of properties whose values uniquely identify the entities of that class. Without adopting a Unique Name Assumption (UNA), from these latter three features we may conclude that two or more terms refer to the same entity. Finally, we can relate a property to a *chain* (a path expression only allowing concatenation of properties) such that pairs of entities related by the chain are also related by the given property. Note that for the latter two features in Table 4.2 we require representing a list, denoted with a vertical notation $(\cdot)$; while such a list may be serialised as a graph in a number of concrete ways, OWL uses RDF lists (see Figure 3.7).

Table 4.2: Ontology features for property axioms

Feature	Axiom	Condition (for all x_*, y_*, z_*)	Example
SUB-PROPERTY	$(p \text{--} \text{subp. of} \rightarrow q)$	$(x \text{--} p \rightarrow y) \text{ implies } (x \text{--} q \rightarrow y)$	$(\text{venue} \text{--} \text{subp. of} \rightarrow \text{location})$
DOMAIN	$(p \text{--} \text{domain} \rightarrow c)$	$(x \text{--} p \rightarrow y) \text{ implies } (x \text{--} \text{type} \rightarrow c)$	$(\text{venue} \text{--} \text{domain} \rightarrow \text{Event})$
RANGE	$(p \text{--} \text{range} \rightarrow c)$	$(x \text{--} p \rightarrow y) \text{ implies } (y \text{--} \text{type} \rightarrow c)$	$(\text{venue} \text{--} \text{range} \rightarrow \text{Venue})$
EQUIVALENCE	$(p \text{--} \text{equiv. p.} \rightarrow q)$	$(x \text{--} p \rightarrow y) \text{ iff } (x \text{--} q \rightarrow y)$	$(\text{start} \text{--} \text{equiv. p.} \rightarrow \text{begins})$
INVERSE	$(p \text{--} \text{inv. of} \rightarrow q)$	$(x \text{--} p \rightarrow y) \text{ iff } (y \text{--} q \rightarrow x)$	$(\text{venue} \text{--} \text{inv. of} \rightarrow \text{hosts})$

Give us feedback!

Feature	Axiom	Condition (for all x, y, z)	Example
DISJOINT	$p \text{ --disj. } p \rightarrow q$	not	$\text{venue --disj. } p \rightarrow \text{hosts}$
TRANSITIVE	$p \text{ --type } \rightarrow \text{Transitive}$		$\text{part of --type } \rightarrow \text{Transitive}$
SYMMETRIC	$p \text{ --type } \rightarrow \text{Symmetric}$	$x \rightarrow p \rightarrow y$ iff $y \rightarrow p \rightarrow x$	$\text{nearby --type } \rightarrow \text{Symmetric}$
ASYMMETRIC	$p \text{ --type } \rightarrow \text{Asymmetric}$	not	$\text{capital --type } \rightarrow \text{Asymmetric}$
REFLEXIVE	$p \text{ --type } \rightarrow \text{Reflexive}$		$\text{part of --type } \rightarrow \text{Reflexive}$
IRREFLEXIVE	$p \text{ --type } \rightarrow \text{Irreflexive}$	not	$\text{flight --type } \rightarrow \text{Irreflexive}$
FUNCTIONAL	$p \text{ --type } \rightarrow \text{Functional}$	$y_1 \rightarrow p \rightarrow x \rightarrow p \rightarrow y_2$ implies $y_1 = y_2$	$\text{population --type } \rightarrow \text{Functional}$
INV. FUNCTIONAL	$p \text{ --type } \rightarrow \text{Inv. Functional}$	$x_1 \rightarrow p \rightarrow y \leftarrow p \rightarrow x_2$ implies $x_1 = x_2$	$\text{capital --type } \rightarrow \text{Inv. Functional}$
KEY	$c \text{ --key } \rightarrow \begin{matrix} p_1 \\ \vdots \\ p_n \end{matrix}$		$\text{City --key } \rightarrow \begin{matrix} \text{lat} \\ \text{long} \end{matrix}$
CHAIN	$p \text{ --chain } \rightarrow \begin{matrix} q_1 \\ \vdots \\ q_n \end{matrix}$	$x \rightarrow q_1 \rightarrow y_1 \rightarrow \dots \rightarrow y_{n-1} \rightarrow q_n \rightarrow z$ implies $x \rightarrow p \rightarrow z$	$\text{location --chain } \rightarrow \text{location part of}$

Classes

In Section 3.1.1, we discussed how class hierarchies can be modelled using a *sub-class* relation. OWL supports sub-classes, and many additional features, for defining and making claims about classes; these additional features are summarised in Table 4.3. Given a pair of classes, OWL allows for defining that they are *equivalent*, or *disjoint*. Thereafter, OWL provides a variety of features for defining novel classes by applying set operators on other classes, or based on conditions that the properties of its instances satisfy. First, using set operators, one can define a novel class as the *complement* of another class, the *union* or *intersection* of a list (of arbitrary length) of other classes, or as an *enumeration* of all of its instances. Second, by placing restrictions on a particular property p , one can define classes whose instances are all of the entities that have: *some value* from a given class on p ; *all values* from a given class on p ; ¹¹ have a specific individual as a value on p (*has value*); have themselves as a reflexive value on p (*has self*); have at least, at most or exactly some number of values on p (*cardinality*); and have at least, at most or exactly some number of values on p from a given class (*qualified cardinality*). For the latter two cases, in Table 4.3, we use the notation “ $\# \{ \phi \}$ ” to count distinct entities satisfying ϕ in the interpretation. These features can then be combined to create more complex classes, where combining the examples for INTERSECTION and HAS SELF in Table 4.3 gives the definition: *self-driving taxis are taxis having themselves as a driver*.

note 11 While something can be a more natural example for *all* condition is known not to have an intersection of such a

Table 4.3: Ontology features for class axioms and definitions

Feature	Axiom	Condition (for all x, y, z)	Example
SUB-CLASS	$c \text{ --subc. of } \rightarrow d$	$x \text{ --type } \rightarrow c$ implies $x \text{ --type } \rightarrow d$	$\text{City --subc. of } \rightarrow \text{Place}$
EQUIVALENCE	$c \text{ --equiv. } c \rightarrow d$	$x \text{ --type } \rightarrow c$ iff $x \text{ --type } \rightarrow d$	$\text{Human --equiv. of } \rightarrow \text{Person}$
DISJOINT	$c \text{ --disj. } c \rightarrow d$	not $c \text{ --type } \rightarrow x \text{ --type } \rightarrow d$	$\text{City --disj. } c \rightarrow \text{Region}$
COMPLEMENT	$c \text{ --comp. } \rightarrow d$	$x \text{ --type } \rightarrow c$ iff not $x \text{ --type } \rightarrow d$	$\text{Dead --comp. } \rightarrow \text{Alive}$
UNION	$c \text{ --union } \rightarrow \begin{matrix} d_1 \\ \vdots \\ d_n \end{matrix}$	$x \text{ --type } \rightarrow c$ iff $x \text{ --type } \rightarrow d_1$ OR $x \text{ --type } \rightarrow \dots$ OR $x \text{ --type } \rightarrow d_n$	$\text{Flight --union } \rightarrow \begin{matrix} \text{DomesticFlight} \\ \text{InternationalFlight} \end{matrix}$

Feature	Axiom	Condition (for all x_*, y_*, z_*)	Example
INTERSECTION			
ENUMERATION			
SOME VALUES			
ALL VALUES			
HAS VALUE			
HAS SELF			
CARDINALITY $\star \in \{=, \leq, \geq\}$			
QUALIFIED CARDINALITY $\star \in \{=, \leq, \geq\}$			

Other features

OWL supports other language features not previously discussed, including: *annotation properties*, which provide metadata about ontologies, such as versioning info; *datatype vs. object properties*, which distinguish properties that take datatype values from those that do not; and *datatype facets*, which allow for defining new datatypes by applying restrictions to existing datatypes, such as to define that places in Chile must have a *float between* -66.0 and -110.0 as their value for the (datatype) property *latitude*. For more details we refer to the OWL 2 standard [Hitzler et al., 2012]. We will further discuss methodologies for the creation of ontologies in Section 6.5.

Models under semantic conditions

Each axiom described by the previous tables, when added to a graph, enforces some condition(s) on the models the graph. If we were to consider only the base condition of the ASSERTION feature in Table 4.1, for example, then the models of a graph would be any interpretation such that for every edge $(x-y \rightarrow z)$ in the graph, there exists a relation $(x-y \rightarrow z)$ in the model. Given that there may be other relations in the model (under the OWA), the number of models of any such graph is infinite. Furthermore, given that we can map multiple nodes in the graph to one entity in the model (under the NUNA), any interpretation with (for example) the relation $(a-a \rightarrow a)$ is a model of any graph so long as for every edge $(x-y \rightarrow z)$ in the graph, it holds that $(x = y = z = a)$ in the interpretation (in other words, the interpretation maps everything to a). As we add axioms with their associated conditions to the graph, we restrict models for the graph; for example, considering a graph with two edges $(x-y \rightarrow z)$ and $(y-type \rightarrow Irreflexive)$ – the interpretation with $(a-a \rightarrow a)$, $x = y = \dots = a$ is no longer a model as it breaks the condition for the irreflexive axiom. In this way, we can define a precise model-theoretic semantics for graphs based on how the aforementioned ontological features used in the graph restrict the models of that graph.

We define models under semantics conditions.

Definition 4.3 (Semantic condition)

Let 2^G denote the set of all (directed edge-labelled) graphs. A semantic condition is a mapping $\phi : 2^G \rightarrow \{\text{true}, \text{false}\}$. An interpretation $I := (\Gamma, \cdot^I)$ is a model of G under ϕ if and only if I is a model of G and $\phi(I)$. Given a set of semantic conditions Φ , we say that I is a model of G if and only if I is a model of G and for all $\phi \in \Phi$, $\phi(I)$ is true.

Give us feedback!

We do not restrict the language used to define semantic conditions, but, for example, we can define the HAS VALUE semantic condition of Table 4.3 in FOL as:

$$\forall c, p, y \left((\Gamma(c, \text{prop}, p) \wedge \Gamma(c, \text{value}, y)) \leftrightarrow \forall x (\Gamma(x, \text{type}, c) \leftrightarrow \Gamma(x, p, y)) \right)$$

Here we overload Γ as a ternary predicate to capture the edges of Γ . The other semantic conditions enumerated in Tables 4.1–4.3 can be defined in a similar way [Schneider and Sutcliffe, 2011].¹² This FOL formula defines an if-and-only-if version of the semantic condition for HAS VALUE (described in Section 4.1.4).

note 12 Although these check their image in the axioms are reflect the definitions consid

4.1.3 ENTAILMENT

The conditions listed in the previous tables give rise to *entailments*, where, for example, in reference to the SYMMETRIC feature of Table 4.2, the definition $\text{nearby} \xrightarrow{\text{type}} \text{Symmetric}$ and edge $\text{Santiago} \xrightarrow{\text{nearby}} \text{Santiago Airport}$ entail the edge $\text{Santiago Airport} \xrightarrow{\text{nearby}} \text{Santiago}$ according to the condition given for that feature. We now describe how these conditions lead to entailments.

We say that one graph *entails* another if and only if any model of the former graph is also a model of the latter graph. Intuitively this means that the latter graph says nothing new over the former graph and thus holds as a logical consequence of the former graph. For example, consider the graph $\text{Santiago} \xrightarrow{\text{type}} \text{City} \xrightarrow{\text{subc. of}} \text{Place}$ and the graph $\text{Santiago} \xrightarrow{\text{type}} \text{Place}$. All models of the latter must have that $\text{Santiago} \xrightarrow{\text{type}} \text{Place}$, but so must all models of the former, which must have $\text{Santiago} \xrightarrow{\text{type}} \text{City} \xrightarrow{\text{subc. of}} \text{Place}$ and further must satisfy the condition for SUB-CLASS, which requires that $\text{Santiago} \xrightarrow{\text{type}} \text{Place}$ also hold. Hence we conclude that any model of the former graph must be a model of the latter graph, or, in other words, the former graph entails the latter graph.

We now formally define entailment under semantic conditions.

Definition 4.4 (Graph entailment)

Letting G_1 and G_2 denote two (directed edge-labelled) graphs, and Φ a set of semantic conditions, we say that G_1 entails G_2 under Φ – denoted $G_1 \models_{\Phi} G_2$ – if and only if any model of G_1 under Φ is also a model of G_2 under Φ .

An example of entailment is discussed in Section 4.1.3.¹³

4.1.4 IF-THEN VS. IF-AND-ONLY-IF SEMANTICS

Consider the graph $\text{nearby} \xrightarrow{\text{type}} \text{Symmetric}$ and the graph $\text{nearby} \xrightarrow{\text{inv. of}} \text{nearby}$. Both of these graphs result in the same semantic conditions being applied in the domain graph, but does one entail the other? The answer depends on the semantics applied. Considering the axioms and conditions of Tables 4.1, we can consider two semantics. Under *if-then* semantics – if **Axiom** matches the data graph *then* **Condition** holds in domain graph – the graphs do not entail each other: though both graphs give rise to the same condition, this condition is not translated back into the axioms that describe it.¹⁴ Conversely, under *if-and-only-if* semantics – **Axiom** matches data graph *if-and-only-if* **Condition** holds in domain graph – the graphs entail each other: both graphs give rise to the same condition, which is translated back into all possible axioms that describe it. Hence if-and-only-if semantics allows for entailing more axioms in the ontology language than if-then semantics. OWL generally applies an if-and-only-if semantics in order to enable richer entailments [Hitzler et al., 2012].

note 13 Here we have let $G \models_{\Phi} (s, p, o)$ denotation). Under CWA, we (strictly speaking, a p that any (positive) edge to G and Φ . However edges in a graph can FOL-compatibility un needed.

note 14 Here, $\text{nearby} \xrightarrow{\text{type}} \text{Symmetric}$ while $\text{nearby} \xrightarrow{\text{inv. of}} \text{nearby}$ the graph entails the

4.2 REASONING

Unfortunately, given two graphs, deciding if the first entails the second – per the notion of entailment we have defined and for all of the ontological features listed in Tables 4.1–4.3 – is *undecidable*: no (finite) algorithm for such entailment can exist that halts on all inputs with the correct true/false answer [Hitzler et al., 2010]. However, we can provide practical reasoning algorithms for ontologies that (1) halt on any pair of input ontologies but may miss entailments, returning false instead of true in some cases, (2) always halt with the correct answer but only accept input ontologies with restricted features, or (3) only return correct answers for any pair of input ontologies but may never halt on certain inputs. Though option (3) has been explored using, e.g., theorem

Give us feedback!

provers for First Order Logic (FOL) [Schneider and Sutcliffe, 2011], options (1) and (2) are more commonly pursued using rules and/or Description Logics. Option (1) generally allows for more efficient and scalable reasoning algorithms and is useful where data are incomplete and having some entailments is valuable. Option (2) may be a better choice in domains – such as medical ontologies – where missing entailments may have undesirable outcomes.

4.2.1 RULES

A straightforward way to provide automated access to the knowledge that can be deduced through (ontological or other forms of) entailments is through *inference rules* (or simply *rules*) encoding IF-THEN-style consequences. A rule is composed of a *body* (IF) and a *head* (THEN). Both the body and head are given as graph patterns. A rule indicates that if we can replace the variables of the body with terms from the data graph and form a sub-graph of a given data graph, then using the same replacement of variables in the head will yield a valid entailment. The head must typically use a subset of the variables appearing in the body to ensure that the conclusion leaves no variables unreplaced. Rules of this form correspond to (positive) Datalog [Ceri et al., 1989] in Databases, Horn clauses [Lloyd, 1984] in Logic Programming, etc.

Rules can capture entailments under ontological conditions. In Table 4.4, we list some example rules for sub-class, sub-property, domain and range features [Muñoz et al., 2009]; these rules may be considered incomplete, not capturing, for example, that every class is a sub-class of itself, that every property is a sub-property of itself, etc. A more comprehensive set of rules for the OWL features of Tables 4.1–4.3 have been defined as OWL 2 RL/RDF [Motik et al., 2012]; these rules are likewise incomplete as such rules cannot fully capture negation (e.g., COMPLEMENT), existentials (e.g., SOME VALUES), universals (e.g., ALL VALUES), or counting (e.g., CARDINALITY and QUALIFIED CARDINALITY). Other rule languages have, however, been proposed to support additional such features, including existentials (see, e.g., Datalog[±] [Bellomarini et al., 2018]), disjunction (see, e.g., Disjunctive Datalog [Rudolph et al., 2008]), etc.

Table 4.4: Example rules for sub-class, sub-property, domain, and range features

Feature	Body	⇒ Head
SUB-CLASS (I)	$\langle \bar{x} \rangle\text{-type} \rightarrow \langle \bar{c} \rangle\text{-subc. of} \rightarrow \langle \bar{d} \rangle$	$\Rightarrow \langle \bar{x} \rangle\text{-type} \rightarrow \langle \bar{d} \rangle$
SUB-CLASS (II)	$\langle \bar{c} \rangle\text{-subc. of} \rightarrow \langle \bar{d} \rangle\text{-subc. of} \rightarrow \langle \bar{e} \rangle$	$\Rightarrow \langle \bar{c} \rangle\text{-subc. of} \rightarrow \langle \bar{e} \rangle$
SUB-PROPERTY (I)	$\langle \bar{x} \rangle\text{-} \xrightarrow{\text{subp. of}} \langle \bar{p} \rangle \rightarrow \langle \bar{y} \rangle$	$\Rightarrow \langle \bar{x} \rangle\text{-} \xrightarrow{\text{subp. of}} \langle \bar{q} \rangle \rightarrow \langle \bar{y} \rangle$
SUB-PROPERTY (II)	$\langle \bar{p} \rangle\text{-subp. of} \rightarrow \langle \bar{q} \rangle\text{-subp. of} \rightarrow \langle \bar{r} \rangle$	$\Rightarrow \langle \bar{p} \rangle\text{-subp. of} \rightarrow \langle \bar{r} \rangle$
DOMAIN	$\langle \bar{x} \rangle\text{-} \xrightarrow{\text{domain}} \langle \bar{p} \rangle \rightarrow \langle \bar{y} \rangle$	$\Rightarrow \langle \bar{x} \rangle\text{-type} \rightarrow \langle \bar{c} \rangle$
RANGE	$\langle \bar{x} \rangle\text{-} \xrightarrow{\text{range}} \langle \bar{p} \rangle \rightarrow \langle \bar{y} \rangle$	$\Rightarrow \langle \bar{y} \rangle\text{-type} \rightarrow \langle \bar{c} \rangle$

Rules can be leveraged for reasoning in a number of ways. *Materialisation* refers to the idea of applying rules recursively to a graph, adding the conclusions generated back to the graph until a fixpoint is reached and nothing more can be added. The materialised graph can then be treated as any other graph. Although the efficiency and scalability of materialisation can be enhanced through optimisations like Rete networks [Forgy, 1982], or using distributed frameworks like MapReduce [Urbani et al., 2012], depending on the rules and the data, the materialised graph may become unfeasibly large to manage. Another strategy is to use rules for *query rewriting*, which given a query, will automatically extend the query in order to find solutions entailed by a set of rules; for example, taking the schema graph in Figure 3.2 and the rules in Table 4.4, the (sub-)pattern $\langle \bar{x} \rangle\text{-type} \rightarrow \langle \text{Event} \rangle$ in a given input query would be rewritten to the following disjunctive pattern evaluated on the original graph:

$$\langle \bar{x} \rangle\text{-type} \rightarrow \langle \text{Event} \rangle \cup \langle \bar{x} \rangle\text{-type} \rightarrow \langle \text{Type} \rangle \cup \langle \bar{x} \rangle\text{-type} \rightarrow \langle \text{Periodic Market} \rangle \cup \langle \bar{x} \rangle\text{-venue} \rightarrow \langle \bar{y} \rangle$$

Figure 4.2 provides a more complete example of an ontology that is used to rewrite the query of Figure 4.1; if evaluated over the graph of Figure 2.1, $\langle \text{Nam} \rangle$ will be returned as a solution. However, not all of the aforementioned features of OWL can be supported in this manner. The OWL 2 QL p

[Give us feedback!](#)

file [Motik et al., 2012] is a subset of OWL designed specifically for query rewriting of this form [Artale et al., 2009].

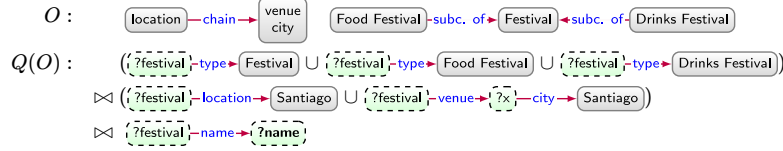
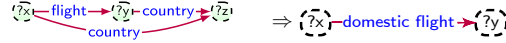


Figure 4.2: Query rewriting example for the query Q of Figure 4.1

While rules can be used to (partially) capture ontological entailments, they can also be defined independently of an ontology language, capturing entailments for a given domain. In fact, some rules – such as the following – cannot be captured by the ontology features previously seen, as they do not support ways to infer relations from cyclical graph patterns (for computability reasons):



Various languages allow for expressing rules over graphs – independently or alongside of an ontology language – including: Notation3 (N3) [Berners-Lee and Connolly, 2011], Rule Interchange Format (RIF) [Kifer and Boley, 2013], Semantic Web Rule Language (SWRL) [Horrocks et al., 2004], and SPARQL Inferencing Notation (SPIN) [Knublauch et al., 2011], amongst others.

Given a graph pattern Q – be it a directed edge-labelled graph pattern per Definition 2.5 or a property graph pattern per Definition 2.6 – recall that $\text{Var}(Q)$ denotes the variables appearing in Q . We now define rules for graphs.

Definition 4.5 (Rule)

A rule is a pair $R := (B, H)$ such that B and H are graph patterns and $\text{Var}(H) \subseteq \text{Var}(B)$. The graph pattern B is called the body of the rule while H is called the head of the rule.

This definition of a rule applies for directed edge-labelled graphs and property graphs by considering the corresponding type of graph pattern. The head is considered to be a conjunction of edges. Given a graph G , a rule is *applied* by computing the mappings from the body to the graph and then using those mappings to substitute the variables in H . The restriction $\text{Var}(H) \subseteq \text{Var}(B)$ ensures that the results of this substitution is a graph, with no variables in H left unsubstituted.

Definition 4.6 (Rule application)

Given a rule $R = (B, H)$ and a graph G , we define the application of R over G as the graph $R(G) := \bigcup_{\mu \in B(G)} \mu(H)$.

Given a set of rules $\mathcal{R} := \{R_1, \dots, R_n\}$ and a knowledge graph G , towards defining the set of inferences given by the rules over the graph, we denote by $\mathcal{R}(G) := \bigcup_{R \in \mathcal{R}} R(G)$ the union of the application of all rules of $\mathcal{R}$ over G , and we denote by $\mathcal{R}^+(G) := \mathcal{R}(G) \cup G$ the extension of G with respect to the application of $\mathcal{R}$. Finally, we denote by $\mathcal{R}^k(G)$ (for $k \in \mathbb{N}^+$) the recursive application of $\mathcal{R}^+(G)$, where $\mathcal{R}^1(G) := \mathcal{R}^+(G)$, and $\mathcal{R}^{i+1}(G) := \mathcal{R}^+(\mathcal{R}^i(G))$. We are now ready to define the *least model*, which captures the inferences possible for $\mathcal{R}$ over G .

Definition 4.7 (Least model)

The least model of $\mathcal{R}$ over G is defined as $\mathcal{R}^*(G) := \bigcup_{k \in \mathbb{N}} (\mathcal{R}^k(G))$.

At some point $\mathcal{R}^{k'}(G) = \mathcal{R}^{k'+1}(G)$: the rule applications reach a fixpoint and we have the least model. Once the least model $\mathcal{R}^*(G)$ is computed, the entailed data can be treated as any other data.

Rules can support graph entailments of the form $G_1 \models_{\Phi} G_2$. We say that a set of rules $\mathcal{R}$ is *correct* for Φ if, for any graph G , $G \models_{\Phi} \mathcal{R}^*(G)$. We say that $\mathcal{R}$ is *complete* for Φ if, for any graph G , there does not exist a graph $G' \not\models_{\Phi} \mathcal{R}^*(G)$ such that $G \models_{\Phi} G'$. Table 4.4 exemplifies a correct but incomplete set of rules for the semantic conditions of the RDFS standard [Brickley and Guha, 2014].

Alternatively, rather than supporting ontology-based graph entailments, rules can be directly specified in a rule language such as Notation3 (N3) [Berners-Lee and Connolly, 2011], Rule Interchange Format (RIF) [Kifer and Boley, 2013], Semantic Web Rule Language

(SWRL) [Horrocks et al., 2004], or SPARQL Inferencing Notation (SPIN) [Knublauch et al., 2011]. Languages such as SPIN represent rules as graphs, allowing the rules of a knowledge graph to be embedded in the data graph. Taking advantage of this fact, we can then consider a form of graph entailment $G_1 \cup \gamma(\mathcal{R}) \models_{\Phi} G_2$, where by $\gamma(\mathcal{R})$ we denote the graph representation of rules $\mathcal{R}$. If the set of rules $\mathcal{R}$ is correct and complete for Φ , we may simply write $G_1 \cup \gamma(\mathcal{R}) \models G_2$, indicating that Φ captures the same semantics for $\gamma(\mathcal{R})$ as applying the rules in $\mathcal{R}$. Rules thus offer another form of graph entailment.

4.2.2 DESCRIPTION LOGICS

Description Logics (DLs) were initially introduced as a way to formalise the meaning of *frames* and *semantic networks*. Since semantic networks are an early version of knowledge graphs, and DLs have heavily influenced the Web Ontology Language, DLs thus hold an important place in the logical formalisation of knowledge graphs. DLs form a family of logics rather than a particular logic. Initially, DLs were restricted fragments of FOL that permit decidable reasoning tasks, such as entailment checking [Baader et al., 2017]. Different DLs strike different balances between expressive power and computational complexity of reasoning. DLs were later extended with features beyond FOL that are useful in the context of modelling graph data, such as transitive closure, datatypes, etc.

DLs are based on three types of elements: *individuals*, such as Santiago; *classes* (aka *concepts*) such as City; and *properties* (aka *roles*) such as flight. DLs then allow for making claims, known as *axioms*, about these elements. *Assertional axioms* can be either unary class relations on individuals, such as `City(Santiago)`, or binary property relations on individuals, such as `flight(Santiago, Arica)`. Such axioms form the *Assertional Box* (A-Box). DLs further introduce logical symbols to allow for defining *class axioms* (forming the *Terminology Box*, or T-Box for short), and *property axioms* (forming the *Role Box*, R-Box); for example, the class axiom `City  $\sqsubseteq$  Place` states that the former class is a sub-class of the latter one, while the property axiom `flight  $\sqsubseteq$  connectsTo` states that the former property is a sub-property of the latter one. DLs may then introduce a rich set of logical symbols, not only for defining class and property axioms, but also defining new classes based on existing terms; as an example of the latter, we can define a class `$\exists$ nearby.Airport` as the class of individuals that have some airport nearby. Noting that the symbol $\top$ is used in DLs to denote the class of all individuals, we can then add a class axiom `$\exists$ flight. $\top$   $\sqsubseteq$   $\exists$ nearby.Airport` to state that individuals with an outgoing flight must have some airport nearby. Noting that the symbol $\sqcup$ can be used in DL to define that a class is the union of other classes, we can further define, for example, that `Airport  $\sqsubseteq$  DomesticAirport  $\sqcup$  InternationalAirport`, i.e., that an airport is either a domestic airport or an international airport (or both).

The similarities between DL features and the OWL features seen previously are not coincidental: the OWL standard was heavily influenced by DLs, where, for example, the OWL 2 DL language is a fragment of OWL restricted so that entailment becomes decidable, where the restrictions are inspired by those defined for DLs. To exemplify a restriction, `DomesticAirport  $\sqsubseteq$  = 1 destination  $\circ$  country. $\top$` defines in DL syntax that domestic airports have flights destined to precisely one country (where $p \circ q$ denotes a chain of properties). However, counting chains (in this case with `= 1 destination  $\circ$  country`) is often disallowed in DLs to ensure decidability.

Expressive DLs support complex entailments involving existentials, universals, counting, etc. A common strategy for deciding such entailments is to reduce entailment to *satisfiability*, which decides if an ontology is consistent or not [Horrocks and Patel-Schneider, 2004].¹⁵ Thereafter methods such as *tableau* can be used to check satisfiability, cautiously constructing models by completing them along similar lines to the materialisation strategy previously described, but additionally branching models in the case of disjunction, introducing new elements to represent existentials, etc. If any model is successfully “completed”, the process concludes that the original definitions are satisfiable (see, e.g., [Motik et al., 2009]). Due to their prohibitive computational complexity [Motik et al., 2012] – where for example, disjunction may lead to an exponential number of branching possibilities – such reasoning strategies are not typically applied in the case of large-scale data, though they may be useful when modelling complex domains for knowledge graphs.

A DL knowledge base consists of an A-Box, a T-Box, and an R-Box.

note 15 G entails G' if :

Give us feedback!

Definition 4.8 (DL knowledge base)

DL knowledge base K is defined as a tuple (A, T, R) , where A is the *A-Box*: a set of assertional axioms; T is the *T-Box*: a set of class (aka concept/terminological) axioms; and R is the *R-Box*: a set of relation (aka property/role) axioms.

Table 4.5 provides definitions for all of the constructs typically found in Description Logics. The syntax column denotes how the construct is expressed in DL. The semantics column defines the meaning of axioms using *interpretations*, which are defined in a slightly different way to those seen previously for graphs.

Definition 4.9 (DL interpretation)

A DL interpretation I is defined as a pair $(\Delta^I, \cdot^I)$, where Δ^I is the interpretation domain, and $\cdot^I$ is the interpretation function. The interpretation domain is a set of individuals. The interpretation function accepts a definition of either an individual a , a class C , or a relation R , mapping them, respectively, to an element of the domain ($a^I \in \Delta^I$), a subset of the domain ($C^I \subseteq \Delta^I$), or a set of pairs from the domain ($R^I \subseteq \Delta^I \times \Delta^I$).

An interpretation I satisfies a knowledge-base K if and only if, for all of the syntactic axioms in K , the corresponding semantic conditions in Table 4.5 hold for I . In this case, we call I a *model* of K .

Example 4.1 — For $K := (A, T, R)$, let:

- $A := \{\text{City}(\text{Arica}), \text{City}(\text{Santiago}), \text{flight}(\text{Arica}, \text{Santiago})\};$
- $T := \{\text{City} \sqsubseteq \text{Place}, \exists \text{flight}. T \sqsubseteq \exists \text{nearby}. \text{Airport}\};$
- $R := \{\text{flight} \sqsubseteq \text{connectsTo}\}.$

For $I = (\Delta^I, \cdot^I)$, let:

- $\Delta^I := \{\text{✂}, \text{🌴}, \text{✈}\};$
- $\text{Arica}^I := \text{✂}, \text{Santiago}^I := \text{🌴}, \text{AricaAirport}^I := \text{✈};$
- $\text{City}^I := \{\text{✂}, \text{🌴}\}, \text{Airport}^I := \{\text{✈}\};$
- $\text{flight}^I := \{(\text{✂}, \text{🌴})\}, \text{connectsTo}^I := \{(\text{✂}, \text{🌴})\}, \text{sells}^I := \{(\text{✈}, \text{☕})\}.$

The interpretation I is not a model of K since it does not have that ✂ is nearby some Airport, nor that ✂ and 🌴 are in the class Place. However, if we extend the interpretation I with the following:

- $\text{Place}^I := \{\text{✂}, \text{🌴}\};$
- $\text{nearby} := \{(\text{✂}, \text{✈})\}.$

Now I is a model of K . Note that although K does not imply that $\text{sells}(\text{AricaAirport}, \text{coffee})$ while I indicates that ✈ does indeed sell ☕ , I is still a model of K since K is not assumed to be a complete description, per the OWA.

Finally, the notion of a model gives rise to the notion of entailment, which tells us which knowledge bases hold as a logical consequence of which others.

Definition 4.10 (Entailment)

Given two DL knowledge bases K_1 and K_2 , we define that K_1 entails K_2 , denoted $K_1 \models K_2$, if and only if any model of K_1 is a model of K_2 .

Example 4.2 — Let K_1 denote the knowledge base K from the Example 4.1, and define a second knowledge base with one assertion: $K_2 := (\{\text{connectsTo}(\text{Arica}, \text{Santiago})\}, \{\}, \{\})$ with one assertion. Though K_1 does not assert this axiom, it does entail K_2 : to be a model of K_2 , an interpretation must have that $(\text{Arica}^I, \text{Santiago}) \in \text{connectsTo}^I$, but this must also be the case for any interpretation that satisfies K_1 since it must have that $(\text{Arica}^I, \text{Santiago}^I) \in \text{flight}$ and $\text{flight} \subseteq \text{connectsTo}$. Hence any model of K_1 must also be a model of K_2 , and $K_1 \models K_2$ holds.

Unfortunately, the problem of deciding entailment for knowledge bases expressed in the DL composed of the unrestricted use of all of the axioms of Table 4.5 is undecidable since we could reduce instances of the Halting Problem to such entailment. Hence DLs in practice restrict use of

[Give us feedback!](#)

the features listed in Table 4.5. Different DLs apply different restrictions, implying different trade-offs for expressivity and the complexity of entailment. Most DLs are founded on one of the following base DLs (we use indentation to denote derivation):

- $[ALC]$ (Attributive Language with Complement) [Schmidt-Schauß and Smolka, 1991]), supports atomic classes, the top and bottom classes, class intersection, class union, class negation, universal restrictions and existential restrictions. Relation and class assertions are also supported.
 - $[S]$ extends ALC with transitive closure.

These base languages can be extended as follows:

- $[H]$ adds relation inclusion.
 - $[R]$ adds (limited) complex relation inclusion, relation reflexivity, relation irreflexivity, relation disjointness and the universal relation.
- $[O]$ adds (limited) nominals.
- $[I]$ adds inverse relations.
- $[F]$ adds (limited) functional properties.
 - $[N]$ adds (limited) number restrictions (covering F with $\top$).
 - $[Q]$ adds (limited) qualified number restrictions (covering N with $\top$).

We use “(limited)” to indicate that such features are often only allowed under certain restrictions to ensure decidability; for example, complex relations (chains) typically cannot be combined with cardinality restrictions. DLs are then typically named per the following scheme, where $[a|b]$ denotes an alternative between a and b and $[c][d]$ denotes a concatenation cd :

$$[ALC][S][H][R][O][I][F][N][Q]$$

Examples include $ALCO$, $ALCHI$, $SHIF$, $SRIOQ$, etc. These languages often apply additional restrictions on class and property axioms to ensure decidability, which we do not discuss here. For further details on DLs, we refer to the recent book by Baader et al. [2017].

As mentioned in the body of the survey, DLs have been very influential in the definition of OWL, where the OWL 2 DL fragment (roughly) corresponds to the DL $SRIOQ$. For example, the axiom $\text{venue} \text{--domain} \rightarrow \text{Event}$ in OWL can be translated to $\exists \text{venue}.\top \sqsubseteq \text{Event}$, meaning that the class of individuals with some value for venue (in any class) is a sub-class of the class Event. We leave other translations from the OWL axioms of Tables 4.1–4.3 to DL as an exercise.¹⁶ Note, however, that axioms like $\text{sub-taxon of} \text{--subp. of} \rightarrow \text{subc. of}$ – which given a graph such as $\text{Fred} \text{--type} \rightarrow \text{Homo sapiens} \text{--sub-taxon of} \rightarrow \text{Hominini}$ entails the edge $\text{Fred} \text{--type} \rightarrow \text{Hominini}$ – cannot be expressed in DL: “ $\text{subTaxonOf} \sqsubseteq \text{--}$ ” is not syntactically valid. Hence only a subset of graphs can be translated into well-formed DL ontologies; we refer to the OWL standard for details [Hitzler et al., 2012].

note 16 Though not p and Nothing that coi

Table 4.5: Description Logic semantics (such that $x, y, z, a^I, \dots, a_n^I, b^I$ are in Δ^I)

Name	Syntax	Semantics ($\cdot^I$)
CLASS DEFINITIONS		
Atomic Class	A	A^I (a subset of Δ^I)
Top Class	$\top$	Δ^I
Bottom Class	$\perp$	$\emptyset$
Class Negation	$\neg C$	$\Delta^I \setminus C^I$
Class Intersection	$C \sqcap D$	$C^I \cap D^I$
Class Union	$C \sqcup D$	$C^I \cup D^I$
Nominal	$\{a_1, \dots, a_n\}$	$\{a_1^I, \dots, a_n^I\}$
Existential Restriction	$\exists R.C$	$\{x \mid \exists y : (x, y) \in R^I \text{ and } y \in C^I\}$
Universal Restriction	$\forall R.C$	$\{x \mid \forall y : (x, y) \in R^I \text{ implies } y \in C^I\}$
Self Restriction	$\exists R.\text{Self}$	$\{x \mid (x, x) \in R^I\}$
Number Restriction	$\star n R$ (where $\star \in \{\geq, \leq, =\}$)	$\{x \mid \#\{y : (x, y) \in R^I\} \star n\}$
Qualified Number Restriction	$\star n R.C$ (where $\star \in \{\geq, \leq, =\}$)	$\{x \mid \#\{y : (x, y) \in R^I \text{ and } y \in C^I\} \star n\}$
CLASS AXIOMS (T-BOX)		
Class Inclusion	$C \sqsubseteq D$	$C^I \subseteq D^I$
RELATION DEFINITIONS		

Give us feedback!

Name	Syntax	Semantics ($\cdot^I$)
Relation	R	R^I (a subset of $\Delta^I \times \Delta^I$)
Inverse Relation	R^-	$\{(y, x) \mid (x, y) \in R^I\}$
Universal Relation	U	$\Delta^I \times \Delta^I$
RELATION AXIOMS (R-Box)		
Relation Inclusion	$R \sqsubseteq S$	$R^I \subseteq S^I$
Complex Relation Inclusion	$R_1 \circ \dots \circ R_n \sqsubseteq S$	$R_1^I \circ \dots \circ R_n^I \subseteq S^I$
Transitive Relations	$\text{Trans}(R)$	$R^I \circ R^I \subseteq R^I$
Functional Relations	$\text{Func}(R)$	$\{(x, y), (x, z)\} \subseteq R^I \text{ implies } y = z$
Reflexive Relations	$\text{Ref}(R)$	for all $x : (x, x) \in R^I$
Irreflexive Relations	$\text{Irref}(R)$	for all $x : (x, x) \notin R^I$
Symmetric Relations	$\text{Sym}(R)$	$R^I = (R^-)^I$
Asymmetric Relations	$\text{Asym}(R)$	$R^I \cap (R^-)^I = \emptyset$
Disjoint Relations	$\text{Disj}(R, S)$	$R^I \cap S^I = \emptyset$
ASSERTIONAL DEFINITIONS		
Individual	a	a^I
ASSERTIONAL AXIOMS (A-Box)		
Relation Assertion	$R(a, b)$	$(a^I, b^I) \in R^I$
Negative Relation Assertion	$\neg R(a, b)$	$(a^I, b^I) \notin R^I$
Class Assertion	$C(a)$	$a^I \in C^I$
Equality	$a = b$	$a^I = b^I$
Inequality	$a \neq b$	$a^I \neq b^I$

Chapter 5

Inductive Knowledge

While deductive knowledge is characterised by precise logical consequences, inductively acquiring knowledge involves generalising patterns from a given set of input observations, which can then be used to generate novel but potentially imprecise predictions. For example, from a large data graph with geographical and flight information, we may observe the pattern that almost all capital cities of countries have international airports serving them, and hence predict that if Santiago is a capital city, it *likely* has an international airport serving it; however, the predictions drawn from this pattern do not hold for certain, where (e.g.) Vaduz, the capital city of Liechtenstein, has no (international) airport serving it. Hence predictions will often be associated with a level of confidence; for example, we may say that a capital has an international airport in $\frac{187}{195}$ of cases, offering a confidence of 0.959 for predictions made with that pattern. We then refer to knowledge acquired inductively as *inductive knowledge*, which includes both the models used to encode patterns, as well as the predictions made by those models. Though fallible, inductive knowledge can be highly valuable.

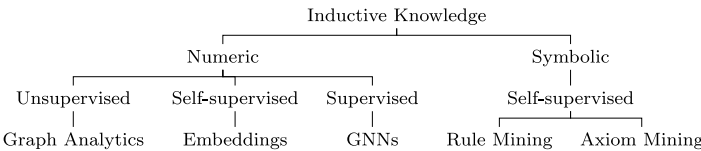


Figure 5.1: Conceptual overview of popular inductive techniques for knowledge graphs in terms of type of representation generated (Numeric/Symbolic) and type of paradigm used (Unsupervised/Self-supervised/Supervised)

In Figure 5.1 we provide an overview of the inductive techniques typically applied to knowledge graphs. In the case of unsupervised methods, there is a rich body of work on *graph analytics*, which uses well-known functions/algorithms to detect communities or clusters, find central nodes and edges, etc., in a graph. Alternatively, *knowledge graph embeddings* can use self-supervision to learn a low-dimensional numeric model of a knowledge graph that (typically) maps input edges to an output *plausibility score* indicating the likelihood of the edge being true. The structure of graphs can also be directly leveraged for supervised learning, as explored in the context of *graph neural networks*. Finally, while the aforementioned techniques learn numerical models, *symbolic learning* can learn symbolic models – i.e., logical formulae in the form of rules or axioms – from a graph in a self-supervised manner. We now discuss each of the aforementioned techniques in turn.

5.1 GRAPH ANALYTICS

Analytics is the process of discovering, interpreting, and communicating meaningful patterns inherent to (typically large) data collections. Graph analytics is then the application of analytical processes to (typically large) graph data. The nature of graphs naturally lends itself to certain types of analytics that derive conclusions about nodes and edges based on the *topology* of the graph, i.e., how the nodes of the graph are connected. Graph analytics draws upon techniques from related areas, such as graph theory and network analysis, which have been used to study graphs representing social networks, the Web, internet routing, transport networks, ecosystems, protein–protein interactions, linguistic cooccurrences, and more besides [Estrada, 2011].

Returning to the domain of our running example, the tourism board could use graph analytics to extract knowledge about, for instance: key transport hubs that serve many tourist attractions (centrality); groupings of attractions visited by the same tourists (community detection); attractions that may become unreachable in the event of strikes or other route failures (connectivity), or pairs of attractions that are similar to each other (node similarity). Given that such analytics will require a complex, large-scale graph, for the purposes of illustration, in Figure 5.2 we present a more concise example of some transportation connections in Chile directed towards popular tourist destinations. We first introduce a selection of key techniques that can be applied for graph analytics. We then discuss frameworks and languages that can be used to compute such analytics in practice. Given that many traditional graph algorithms are defined for unlabelled graphs, we then describe ways in which analytics can be applied over directed edge-labelled graphs. Finally we discuss the potential connections between graph analytics and querying and reasoning.

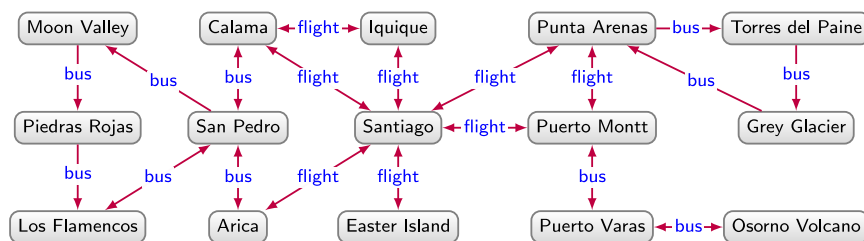


Figure 5.2: Data graph representing transport routes in Chile

5.1.1 TECHNIQUES

A wide variety of techniques can be applied for graph analytics. In the following we will enumerate some of the main techniques – as recognised, for example, by the survey of Iosup et al. [2016] – that can be invoked in this setting.

- **Centrality:** aims to identify the most important (aka *central*) nodes or edges of a graph. Specific node centrality measures include *degree*, *betweenness*, *closeness*, *Eigenvector*, *PageRank*, *HITS*, *Katz*, amongst others. Betweenness centrality can also be applied to edges. For example, a node centrality measure might predict the transport hubs in Figure 5.2, while edge centrality might predict traffic by finding connections on which many shortest routes depend.

- *Community detection*: aims to identify *communities* in a graph, i.e., sub-graphs that are more densely connected internally than to the rest of the graph. Community detection algorithms, such as *minimum-cut algorithms*, *label propagation*, *Louvain modularity*, amongst others, can discover such communities. Community detection applied to Figure 5.2 may, for example, detect a community to the left (the north of Chile), to the right (the south of Chile), and perhaps also the centre (Chilean cities with airports).
- *Connectivity*: aims to estimate how well-connected the graph is, revealing, for instance, the resilience and (un)reachability of elements of the graph. Specific techniques include measuring *graph density* or *k-connectivity*, detecting *strongly connected components* and *weakly connected components*, computing *spanning trees* or *minimum cuts*, etc. In the context of Figure 5.2, such analysis may tell us that routes to Grey Glacier, Osorno Volcano and Piedras Rojas are the most “brittle”, becoming disconnected if one of two bus routes fails.
- *Node (or vertex) similarity*: aims to find nodes that are similar to other nodes by virtue of how they are connected within their neighbourhood. Node similarity metrics may be computed using *structural equivalence*, *random walks*, *diffusion kernels*, etc. These methods provide an understanding of what connects nodes, and, thereafter, in what ways they are similar. In the context of Figure 5.2, such analysis may tell us that Calama and Arica are similar nodes based on both having return flights to Santiago and return buses to San Pedro.

While the previous techniques accept a graph alone as input,¹⁷ other forms of graph analytics may further accept a node, a pair of nodes, etc., along with the graph.

note 17 Node similarity for each node, or can be also measures for (Bobed, 2018)] that ac

- *Path finding*: aims to find paths in a graph, typically between pairs of nodes given as input. Various technical definitions exist that restrict the set of valid paths between such nodes, including simple paths that do not visit the same node twice, shortest paths that visit the fewest number of edges, or – as previously discussed in Section 4.3.2 – regular path queries that restrict the labels of edges that can be traversed by the path according to a regular expression [Angles et al., 2017]. We could use such algorithms to find, for example, the shortest path(s) in Figure 5.2 from Torres del Paine to Moon Valley.

Most of the aforementioned techniques for graph analytics were originally proposed and studied for simple graphs or directed graphs without edge labels. We will discuss their application to more complex graph models – and how they can be combined with other techniques such as reasoning and querying – later in Section 5.1.3.

5.1.2 FRAMEWORKS

Various frameworks have been proposed for large-scale graph analytics, often in a distributed (cluster) setting. Amongst these we can mention Apache Spark (GraphX) [Xin et al., 2013a, Dave et al., 2016], GraphLab [Low et al., 2012], Pregel [Malewicz et al., 2010], Signal-Collect [Stutz et al., 2016], Shark [Xin et al., 2013b], etc. These *graph parallel frameworks* apply a *systolic abstraction* [Kung, 1982] based on a directed graph, where nodes are seen as processors that can send messages to other nodes along edges. Computation is then iterative, where in each iteration, each node reads messages received through inward edges (and possibly its own previous state), performs a computation, and then sends messages through outward edges based on the result. These frameworks then define the systolic computational abstraction on top of the data graph being processed: nodes and edges in the data graph become nodes and edges in the systolic graph.

To take an example, assume we wish to compute the places that are most (or least) easily reached by the routes shown in the graph of Figure 5.2. A good way to measure this is using centrality, where we choose PageRank [Page et al., 1999], which computes the probability of a tourist randomly following the routes shown in the graph being at a particular place after a given number of “hops”. We can implement PageRank on large graphs using a graph parallel framework. In Figure 5.3, we provide an example of an iteration of PageRank for an illustrative sub-graph of Figure 5.2. The nodes are initialised with a score of $\frac{1}{|V|} = \frac{1}{6}$, where we assume the tourist to have an equal chance of starting at any point. In the *message phase* (MSG), each node v passes a score of $\frac{dR_i(v)}{|E(v)|}$ on each of its outgoing edges, where we denote by d a constant damping factor used to ensure convergence (typically $d = 0.85$, indicating the probability that a tourist randomly “jumps” to any place), by $R_i(v)$ the score

Give us feedback!

of node v in iteration i (the probability of the tourist being at node v after i hops), and by $|E(v)|$ the number of outgoing edges of v . The aggregation phase (AGG) for v then sums all incoming messages received along with its constant share of the damping factor ($\frac{1-d}{|V|}$) to compute $R_{i+1}(v)$. We then proceed to the message phase of the next iteration, continuing until some termination criterion is reached (e.g., iteration count or residual threshold, etc.) and final scores are output.

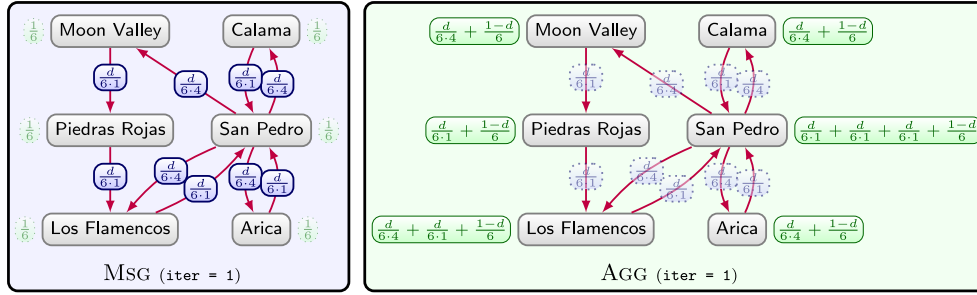


Figure 5.3: Example of a systolic iteration of PageRank on a sub-graph of Figure 5.2

While the given example is for PageRank, the systolic abstraction is general enough to support a wide variety of graph analytics, including those previously mentioned. An algorithm in this framework consists of the functions to compute message values in the *message phase* (MSG), and to accumulate the messages in the aggregation phase (AGG). The framework will take care of distribution, message passing, fault tolerance, etc. However, such frameworks – based on message passing between neighbours – have limitations: not all types of analytics can be expressed in such frameworks [Xu et al., 2019].¹⁸ Hence frameworks may allow additional features, such as a *global step* that performs a global computation on all nodes, making the result available to each node [Malewicz et al., 2010]; or a *mutation step* that allows for adding or removing nodes and edges during processing [Malewicz et al., 2010].

note 18 Formally, Xu et al. (the (incomplete) We graphs. This test invo received from neighb

Before defining a graph parallel framework, in the interest of generality, we first define a directed graph labelled with feature vectors, which captures the type of input that such a framework can accept, with vectors on both nodes and edges.

Definition 5.1 (Directed vector-labelled graph)

We define a directed vector-labelled graph $G = (V, E, F, \lambda)$, where V is a set of nodes, $E \subseteq V \times V$ is a set of edges, F is a set of feature vectors, and $\lambda : V \cup E \rightarrow F$ labels each node and edge with a feature vector.

A directed-edge labelled graph or property graph may be encoded as a directed vector-labelled graph in a number of ways. The type of node and/or a selection of its attributes may be encoded in the node feature vectors, while the label of an edge and/or a selection of its attributes may be encoded in the edge feature vector (including, for example, weights applied to edges). Typically node feature vectors will all have the same dimensionality, as will edge feature vectors.

Example 5.1 — We define a directed vector-labelled graph in preparation for later computing PageRank using a graph parallel framework. Let $G = (V, E, L)$ denote a directed edge-labelled graph. Let $|E(u)|$ denote the outdegree of node $u \in V$. We then initialise a directed vector-labelled graph $G' = (V, E', F, \lambda)$ such that $E' = \{(x, z) \mid \exists y : (x, y, z) \in E\}$, and for all $u \in V$, we define $\lambda(u) := \begin{bmatrix} \frac{1}{|V|} \\ |E'(u)| \end{bmatrix}$, and $\lambda(u, v) := []$, with $F := \{\lambda(u) \mid u \in V\} \cup \{\lambda(u, v) \mid (u, v) \in E'\}$, assigning each node a vector containing its initial PageRank score, the outdegree of the node, and the number of nodes in the graph. Conversely, edge-vectors are not used in this case.

We now define a graph parallel framework, where we use $\{\cdot\}$ to denote a multiset, $2^{S \rightarrow \mathbb{N}}$ to denote the set of all multisets containing (only) elements from the set S , and $\mathbb{R}^a$ to denote the set of all vectors of dimension a (i.e., the set of all vectors containing a real-valued elements).

Definition 5.2 (Graph parallel framework)

A graph parallel framework (GPF) is a triple of functions $\mathfrak{G} := (\text{MSG}, \text{AGG}, \text{END})$ such that (with $a, b, c \in \mathbb{N}$):

- $\text{MSG}: \mathbb{R}^a \times \mathbb{R}^b \rightarrow \mathbb{R}^c$
- $\text{AGG}: \mathbb{R}^a \times 2^{\mathbb{R}^c \rightarrow \mathbb{N}} \rightarrow \mathbb{R}^a$
- $\text{END}: 2^{\mathbb{R}^a \rightarrow \mathbb{N}} \rightarrow \{\text{true}, \text{false}\}$

The function MSG defines what message (i.e., vector) must be passed from a node to a neighbouring node along a particular edge, given the current feature vectors of the node and the edge; the function AGG is used to compute a new feature vector for a node, given its previous feature vector and incoming messages; the function END defines a condition for termination of vector computation. The integers a , b and c denote the dimensions of node feature vectors, edge feature vectors, and message vectors, respectively; we assume that a and b correspond with the dimensions of input feature vectors for nodes and edges. Given a GPF $\mathfrak{G} = (\text{MSG}, \text{AGG}, \text{END})$, a directed vector-labelled graph $G = (V, E, F, \lambda)$, and a node $u \in V$, we define the output vector assigned to node u in G by $\mathfrak{G}$ (written $\mathfrak{G}(G, u)$) as follows. First let $\mathbf{n}_u^{(0)} := \lambda(u)$. For all $i \geq 1$, let:

$$\begin{aligned} M_u^{(i)} &:= \left\{ \text{MSG} \left(\mathbf{n}_v^{(i-1)}, \lambda(v, u) \right) \mid (v, u) \in E \right\} \\ \mathbf{n}_u^{(i)} &:= \text{AGG} \left(\mathbf{n}_u^{(i-1)}, M_u^{(i)} \right) \end{aligned}$$

where $M_u^{(i)}$ is the multiset of messages received by node u during iteration i , and $\mathbf{n}_u^{(i)}$ is the state (vector) of node u at the end of iteration i . If j is the smallest integer for which $\text{END}(\{\mathbf{n}_u^{(j)} \mid u \in V\})$ is true, then $\mathfrak{G}(G, u) := \mathbf{n}_u^{(j)}$.

This particular definition assumes that vectors are dynamically computed for nodes, and that messages are passed only to outgoing neighbours, but the definitions can be readily adapted to consider dynamic vectors for edges, or messages being passed to incoming neighbours, etc. We now provide an example instantiating a GPF to compute PageRank over a directed graph.

Example 5.2 — We take as input the directed vector labelled graph $G' = (V, E, F, \lambda)$ from Example 5.1 for a PageRank GPF. First we define the messages passed from u to v :

$$\text{MSG}(\mathbf{n}_v, \lambda(v, u)) := \left[\frac{d(\mathbf{n}_v)_1}{(\mathbf{n}_v)_2} \right]$$

where d denotes PageRank's constant dampening factor (typically $d := 0.85$) and $(\mathbf{n}_v)_k$ denotes the k^{th} element of the $\mathbf{n}_v$ vector. In other words, v will pass to u its PageRank score multiplied by the dampening factor and divided by its out-degree (we do not require $\lambda(v, u)$ in this particular example). Next we define the function for u to aggregate the messages it receives from other nodes:

$$\text{AGG}(\mathbf{n}_u, M_u) := \left[\frac{1-d}{(\mathbf{n}_u)_3} + \sum_{\mathbf{m} \in M_u} \frac{(\mathbf{m})_1}{(\mathbf{n}_u)_3} \right]$$

Here, we sum the scores received from other nodes along with its share of rank from the dampening factor, copying over the node's degree and the total number of nodes for future use. Finally, there are a number of ways that we could define the termination condition; here we simply define:

$$\text{END}(\{\mathbf{n}_u^{(i)} \mid u \in V\}) := (i \geq z)$$

where z is a fixed number of iterations, at which point the process stops.

We may note in this example that the total number of nodes is duplicated in the vector for each node of the graph. Part of the benefit of GPFs is that only local information in the neighbourhood of the node is required for each computation step. In practice, such frameworks may allow additional features, such as global computation steps whose results are made available to all nodes [Malewicz et al., 2010], operations that dynamically modify the graph [Malewicz et al., 2010], etc.

5.1.3 ANALYTICS ON DATA GRAPHS

As aforementioned, most analytics presented thus far are, in their “native” form, applicable for undirected or directed graphs without the *edge metadata* – i.e., edge labels or property–value pairs – typical of graph data models.¹⁹ A number of strategies can be applied to make data graphs subject to analytics of this form:

^{note 19} We remark that this has been converted by map

- *Projection* involves simply “projecting” an undirected or directed graph by optionally selecting a sub-graph from the data graph from which all edge meta-data are dropped; for example, the graph of Figure 5.3 may be the result of extracting the sub-graph induced by the edge labels **bus** and **flight** from a larger data graph, where the labels are then dropped to create a directed graph.
- *Weighting* involves converting edge meta-data into numerical values according to some function. Many of the aforementioned techniques are easily adapted to the case of weighted (directed) graphs; for example, we could consider weights on the graph of Figure 5.3 denoting trip duration (or price, traffic, etc.), and then compute the shortest (weighted) paths considering time by adding the duration of each leg of the respective journey.²⁰ In the absence of external weights, we may rather map edge labels to weights, assigning the same weight to all **flight** edges, to all **bus** edges, etc., based on some criteria.
- *Transformation* involves transforming the graph to a lower arity model. A transformation may be *lossy*, meaning that the original graph cannot be recovered; or *lossless*, meaning that the original graph can be recovered. Figure 5.4 provides an example of a lossy and lossless transformation from a directed edge-labelled graph to directed graphs. In the lossy transformation, we cannot tell, for example, if the original graph contained the edge (Iquique)–**flight**→(Santiago), or rather the edge (Iquique)–**flight**→(Arica), or both. The lossless transformation must introduce new nodes (similar to reification) to maintain information about directed labelled edges. Both transformed graphs further attempt to preserve the directionality of the original graph.
- *Customisation* involves changing the analytical procedure to incorporate edge meta-data, such as was the case for path finding based on path expressions. Other examples might include structural measures for node similarity that not only consider common neighbours, but also common neighbours connected by edges with the same label, or aggregate centrality measures that capture the importance of edges grouped by label, etc.

note 20 Other forms of example, if we annotate one place to the next, reducibility, periodic routes [Dynkin, 1965]

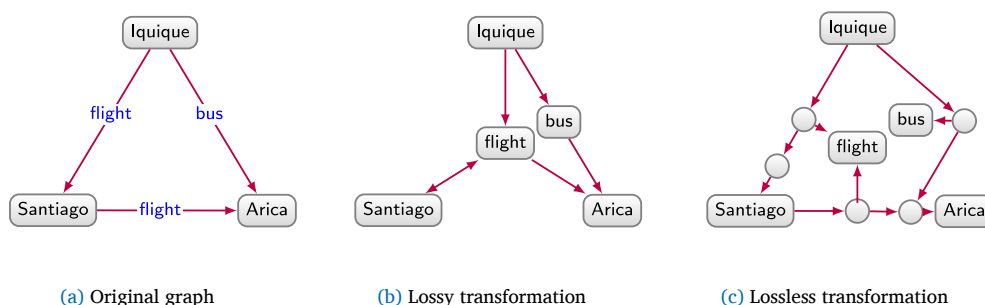


Figure 5.4: Transformations from a directed edge-labelled graph to a directed graph

The results of an analytical process may change drastically depending on which of the previous strategies are chosen to prepare the graph for analysis. The choice of strategy may be a non-trivial one to make *a priori* and may require empirical validation. More study is required to more generally understand the effects of such strategies on the results of different analytical techniques over different graph models.

5.1.4 ANALYTICS WITH QUERIES

As discussed in Section 2.2, various languages for querying graphs have been proposed down through the years [Angles et al., 2017]. One may consider a variety of ways in which query languages and analytics can complement each other. First, we may consider using query languages to project or transform a graph suitable for a particular analytical task, such as to extract the graph of Figure 5.2 from a larger data graph. Query languages such as SPARQL [Harris et al., 2013], Cypher [Francis et al., 2018], and G-CORE [Angles et al., 2018] allow for outputting graphs, where such queries can be used to select sub-graphs for analysis. These languages can also express some limited (non-recursive) analytics, where aggregations can be used to compute degree centrality, for example; they may also have some built-in analytical support, where, for example, Cypher [Francis et al., 2018] allows for finding shortest paths. In the other direction, analytics can contribute to t

Give us feedback!

querying process in terms of *optimisations*, where, for example, analysis of connectivity may suggest how to better distribute a large data graph over multiple machines for querying using, e.g., *minimum cuts* [Akhter et al., 2018, Janke et al., 2018]. Analytics have also been used to *rank* query results over large graphs [Wagner et al., 2012, Fan et al., 2013], selecting the most important results for presentation to the user.

In some use-cases we may further wish to interleave querying and analytical processes. For example, from the full data graph collected by the tourist board, consider an upcoming airline strike where the board wishes to find *the events during the strike with venues in cities unreachable from Santiago by public transport due to the strike*. Hypothetically, we could use a query to extract the transport network excluding the airline's routes (assuming, per Figure 2.3a that the airline information is available), use analytics to extract the strongly connected component containing Santiago, and finally use a query to find events in cities not in the Santiago component on the given dates.²¹ While one could solve this task using an imperative language such as Gremlin [Rodriguez, 2015], GraphX [Xin et al., 2013a], or R [The R Foundation, 1992], more declarative languages are also being explored to express such tasks, with proposals including the extension of graph query languages with recursive capabilities [Bischof et al., 2012, Reutter et al., 2015, Hogan et al., 2020],²² combining linear algebra with relational (query) algebra [Hutchison et al., 2017], and so forth.

note 21 Such a task could be expressed using such expressions would be a very long airline.

note 22 Recursive query operations on binary relations.

5.1.5 ANALYTICS WITH ENTAILMENT

Knowledge graphs are often associated with a semantic schema or ontology that defines the semantics of domain terms, giving rise to entailments (per Chapter 4). Applying analytics with or without such entailments – e.g., before or after materialisation – may yield radically different results. For example, observe that an edge $(\text{Santa Lucia} \xrightarrow{\text{hosts}} \text{EID15})$ is semantically equivalent to an edge $(\text{EID15} \xrightarrow{\text{venue}} \text{Santa Lucia})$ once the inverse axiom $(\text{hosts} \xrightarrow{\text{inv. of}} \text{venue})$ is invoked; however, these edges are far from equivalent from the perspective of analytical techniques that consider edge direction, for which including one type of edge, or the other, or both, may have a major bearing on the final results. To the best of our knowledge, the combination of analytics and entailment has not been well-explored, leaving open interesting research questions. Along these lines, it may be of interest to explore *semantically-invariant analytics* that yield the same results over semantically-equivalent graphs (i.e., graphs that entail one another), thus analysing the semantic content of the knowledge graph rather than simply the topological features of the data graph; for example, semantically-invariant analytics would yield the same results over a graph containing the inverse axiom $(\text{hosts} \xrightarrow{\text{inv. of}} \text{venue})$ and a number of *hosts* edges, the same graph but where every *hosts* edge is replaced by an inverse *venue* edge, and the union of both graphs.

5.2 KNOWLEDGE GRAPH EMBEDDINGS

Methods for machine learning have gained significant attention in recent years. In the context of knowledge graphs, machine learning can either be used for directly *refining* a knowledge graph [Paulheim, 2017] (discussed further in Chapter 8); or for *downstream tasks* using the knowledge graph, such as recommendation [Zhang et al., 2016], information extraction [Vashishth et al., 2018], question answering [Huang et al., 2019], query relaxation [Wang et al., 2018], query approximation [Hamilton et al., 2018], etc. (discussed further in Chapter 10). However, many traditional machine learning techniques assume dense numeric input representations in the form of vectors, which is quite distinct from how graphs are usually expressed. So how can graphs – or nodes, edges, etc., thereof – be encoded as numeric vectors?

A first attempt to represent a graph using vectors would be to use a *one-hot encoding*, generating a vector for each node of length $|L| \cdot |V|$ – with $|V|$ the number of nodes in the input graph and $|L|$ the number of edge labels – placing a one at the corresponding index to indicate the existence of the respective edge in the graph, or zero otherwise. Such a representation will, however, typically result in large and sparse vectors, which will be detrimental for most machine learning models.

The main goal of knowledge graph embedding techniques is to create a dense representation of the graph (i.e., *embed* the graph) in a continuous, low-dimensional vector space that can then be used for machine learning tasks. The dimensionality d of the embedding is fixed and usually low (often, e.

Give us feedback!

$50 \leq d \leq 1000$). Typically the graph embedding is composed of an *entity embedding* for each node: a vector with d dimensions that we denote by $\mathbf{e}$; and a *relation embedding* for each edge label: (typically) a vector with d dimensions that we denote by $\mathbf{r}$. The overall goal of these vectors is to abstract and preserve latent structures in the graph. There are many ways in which this notion of an embedding can be instantiated. Most commonly, given an edge $(s \xrightarrow{p} o)$, a specific embedding approach defines a *scoring function* that accepts $\mathbf{e}_s$ (the entity embedding of node (s)), $\mathbf{r}_p$ (the entity embedding of edge label p) and $\mathbf{e}_o$ (the entity embedding of node (o)) and computes the *plausibility* of the edge, which estimates how likely it is to be true. Given a data graph, the goal is then to compute the embeddings of dimension d that maximise the plausibility of positive edges (typically edges in the graph) and minimise the plausibility of negative examples (typically edges in the graph with a node or edge label changed such that they are no longer in the graph) according to the given scoring function. The resulting embeddings can then be seen as models learnt through self-supervision that encode (latent) features of the graph, mapping input edges to output plausibility scores.

Embeddings can then be used for a number of low-level tasks involving the nodes and edge-labels of the graph from which they were computed. First, we can use the plausibility scoring function to assign a confidence to edges that may, for example, have been extracted from an external source (discussed later in Chapter 6). Second, the plausibility scoring function can be used to complete edges with missing nodes/edge labels for the purposes of link prediction (discussed later in Chapter 8); for example, in Figure 5.2, we might ask which nodes in the graph are likely to complete the edge $(\text{Grey Glacier} \xrightarrow{\text{bus}} ?)$, where – aside from (Punta Arenas) , which is already given – we might intuitively expect $(\text{Torres del Paine})$ to be a plausible candidate. Third, embedding models will typically assign similar vectors to similar nodes and similar edge-labels, and thus they can be used as the basis of similarity measures, which may be useful for finding duplicate nodes that refer to the same entity, or for the purposes of providing recommendations (discussed later in Chapter 10).

A wide range of knowledge graph embedding techniques have been proposed [Wang et al., 2017]. Our goal here is to provide a high-level introduction to some of the most popular techniques proposed thus far. We first discuss *tensor-based approaches* that include three different sub-approaches using linear/tensor algebra to compute embeddings. We then discuss *language models* that leverage existing word embedding techniques, proposing ways of generating graph-like analogues for their expected (textual) inputs. Finally we discuss *entailment-aware models* that can take into account the semantics of the graph, when available.

5.2.1 TENSOR-BASED MODELS

We first discuss tensor-based models, which we sub-divide into three categories: *translational models* that adopt a geometric perspective whereby relation embeddings translate subject entities to object entities, *tensor decomposition models* that extract latent factors approximating the graph's structure, and *neural models* that use neural networks to train embeddings that provide accurate plausibility scores.

Translational models

Translational models interpret edge labels as transformations from subject nodes (aka the *source* or *head*) to object nodes (aka the *target* or *tail*); for example, in the edge $(\text{San Pedro} \xrightarrow{\text{bus}} \text{Moon Valley})$, the edge label *bus* is seen as transforming (San Pedro) to (Moon Valley) , and likewise for other *bus* edges. The most elementary approach in this family is TransE [Bordes et al., 2013]. Over all positive edges $(s \xrightarrow{p} o)$, TransE learns vectors $\mathbf{e}_s$, $\mathbf{r}_p$, and $\mathbf{e}_{os}$ aiming to make $\mathbf{e}_s + \mathbf{r}_p$ as close as possible to $\mathbf{e}_o$. Conversely, if the edge is a negative example, TransE attempts to learn a representation that keeps $\mathbf{e}_s + \mathbf{r}_p$ away from $\mathbf{e}_o$. To illustrate, Figure 5.5 provides a toy example of two-dimensional ($d = 2$) entity and relation embeddings computed by TransE. We keep the orientation of the vectors similar to the original graph for clarity. For any edge $(s \xrightarrow{p} o)$ in the original graph, adding the vectors $\mathbf{e}_s + \mathbf{r}_p$ should approximate $\mathbf{e}_o$. In this toy example, the vectors correspond precisely where, for instance, adding the vectors for (Licantén) ($\mathbf{e}_L$) and *west of* ($\mathbf{r}_{wo}$) gives a vector corresponding to (Curico) ($\mathbf{e}_C$). We can use these embeddings to predict edges (amongst other tasks); for example, in order to predict which node in the graph is most likely to be *west of* (Antofagasta) (A.), by computing

Give us feedback!

$e_{A.} + r_{wo.}$ we find that the resulting vector (dotted in Figure 5.5c) is closest to $e_{T.}$, thus predicting Toconao (T.) to be the most *plausible* such node.

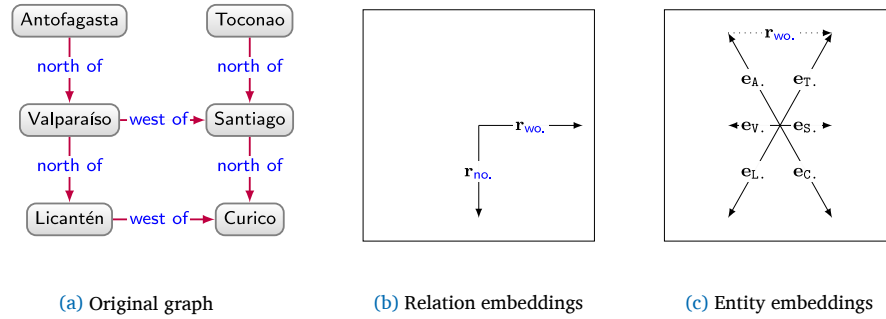


Figure 5.5: Toy example of two-dimensional relation and entity embeddings learnt by TransE; the entity embeddings use abbreviations and include an example of vector addition to predict what is west of Antofagasta

Aside from this toy example, TransE can be too simplistic; for example, in Figure 5.2, *bus* not only transforms San Pedro to Moon Valley, but also to Arica, Calama, and so forth. TransE will, in this case, aim to give similar vectors to all such target locations, which may not be feasible given other edges. TransE will also tend to assign cyclical relations a zero vector, as the directional components will tend to cancel each other out. To resolve such issues, many variants of TransE have been investigated. Amongst these, for example, TransH [Wang et al., 2014] represents different relations using distinct hyperplanes, where for the edge $s \xrightarrow{p} o$, s is first projected onto the hyperplane of p before the translation to o is learnt (uninfluenced by edges with other labels for s and for o). TransR [Lin et al., 2015] generalises this approach by projecting s and o into a vector space specific to p , which involves multiplying the entity embeddings for s and o by a projection matrix specific to p . TransD [Ji et al., 2015] simplifies TransR by associating entities and relations with a second vector, where these secondary vectors are used to project the entity into a relation-specific vector space. Recently, RotatE [Sun et al., 2019] proposes translational embeddings in complex space, which allows to capture more characteristics of relations, such as direction, symmetry, inversion, antisymmetry, and composition. Embeddings have also been proposed in non-Euclidean space; for example, MuRP [Balazevic et al., 2019a] uses relation embeddings that transform entity embeddings in the hyperbolic space of the Poincaré ball mode, whose curvature provides more “space” to separate entities with respect to the dimensionality. For discussion of other translational models, we refer to surveys by Cai et al. [2018], Wang et al. [2017].

Tensor decomposition models

A second approach to derive graph embeddings is to apply methods based on *tensor decomposition*. A *tensor* is a multidimensional numeric field that generalises scalars (0-order tensors), vectors (1-order tensors) and matrices (2-order tensors) towards arbitrary dimension/order. Tensors have become a widely used abstraction for machine learning [Rabanser et al., 2017]. Tensor decomposition involves decomposing a tensor into more “elemental” tensors (e.g., of lower order) from which the original tensor can be recomposed (or approximated) by a fixed sequence of basic operations over the output tensors. These elemental tensors can be viewed as capturing *latent factors* underlying the information contained in the original tensor. There are many approaches to tensor decomposition, where we will now briefly introduce the main ideas behind *rank decompositions* [Rabanser et al., 2017].

Leaving aside graphs momentarily, consider an (a, b) -matrix (i.e., a 2-order tensor) C , where a is the number of cities in Chile, b is the number of months in a year, and each element $(C)_{ij}$ denotes the average temperature of the i^{th} city in the j^{th} month. Noting that Chile is a long, thin country – ranging from subpolar climates in the south, to a desert climate in the north – we may find a decomposition of C into two vectors representing latent factors – specifically x (with a elements) giving lower values for cities with lower latitude, and y (with b elements), giving lower values for months with lower temperatures – such that computing the outer product²³ of the two vectors approximates C reasonably well: $x \otimes y \approx C$. In the (unlikely) case that there exist vectors x and y such that C is precisely the outer product of two vectors ($x \otimes y = C$) we call C a rank-1 matrix; we can then precisely

note 23 The outer prod

encode $\mathbf{C}$ using $a + b$ values rather than $a \times b$ values. Most times, however, to get precisely $\mathbf{C}$, we need to sum multiple rank-1 matrices, where the rank r of $\mathbf{C}$ is the minimum number of rank-1 matrices that need to be summed to derive precisely $\mathbf{C}$, such that $\mathbf{x}_1 \otimes \mathbf{y}_1 + \dots + \mathbf{x}_r \otimes \mathbf{y}_r = \mathbf{C}$. In the temperature example, $\mathbf{x}_2 \otimes \mathbf{y}_2$ might correspond to a correction for altitude, $\mathbf{x}_3 \otimes \mathbf{y}_3$ for higher temperature variance further south, etc. A (low) rank decomposition of a matrix then sets a limit d on the rank and computes the vectors $(\mathbf{x}_1, \mathbf{y}_1, \dots, \mathbf{x}_d, \mathbf{y}_d)$ such that $\mathbf{x}_1 \otimes \mathbf{y}_1 + \dots + \mathbf{x}_d \otimes \mathbf{y}_d$ gives the best d -rank approximation of $\mathbf{C}$. Noting that to generate n -order tensors we need to compute the outer product of n vectors, we can generalise this idea towards low-rank decomposition of tensors; this method is called Canonical Polyadic (CP) decomposition [Hitchcock, 1927]. For example, a 3-order tensor $\mathcal{G}$ containing monthly temperatures for Chilean cities *at four different times of day* could be approximated with $\mathbf{x}_1 \otimes \mathbf{y}_1 \otimes \mathbf{z}_1 + \dots + \mathbf{x}_d \otimes \mathbf{y}_d \otimes \mathbf{z}_d$ (e.g., $\mathbf{x}_1$ might be a latitude factor, $\mathbf{y}_1$ a monthly variation factor, and $\mathbf{z}_1$ a daily variation factor, and so on). Various algorithms exist to compute (approximate) CP decompositions, including Alternating Least Squares, Jennrich's Algorithm, and the Tensor Power method [Rabanser et al., 2017].

Returning to graphs, similar principles can be used to decompose a graph into vectors, thus yielding embeddings. In particular, a graph can be encoded as a one-hot 3-order tensor $\mathcal{G}$ with $|V| \times |L| \times |V|$ elements, where the element $(\mathcal{G})_{ijk}$ is set to one if the i^{th} node links to the k^{th} node with an edge having the j^{th} label, or zero otherwise. As previously mentioned, such a tensor will typically be very large and sparse, where rank decompositions are thus applicable. A CP decomposition [Hitchcock, 1927] would compute a sequence of vectors $(\mathbf{x}_1, \mathbf{y}_1, \mathbf{z}_1, \dots, \mathbf{x}_d, \mathbf{y}_d, \mathbf{z}_d)$ such that $\mathbf{x}_1 \otimes \mathbf{y}_1 \otimes \mathbf{z}_1 + \dots + \mathbf{x}_d \otimes \mathbf{y}_d \otimes \mathbf{z}_d \approx \mathcal{G}$. We illustrate this scheme in Figure 5.6. Letting $\mathbf{X}, \mathbf{Y}, \mathbf{Z}$ denote the matrices formed by $[\mathbf{x}_1 \dots \mathbf{x}_d]$, $[\mathbf{y}_1 \dots \mathbf{y}_d]$, $[\mathbf{z}_1 \dots \mathbf{z}_d]$, respectively, with each vector forming a column of the corresponding matrix, we could then extract the i^{th} row of $\mathbf{Y}$ as an embedding for the i^{th} relation, and the j^{th} rows of $\mathbf{X}$ and $\mathbf{Z}$ as *two* embeddings for the j^{th} entity. However, knowledge graph embeddings typically aim to assign *one* vector to each entity.

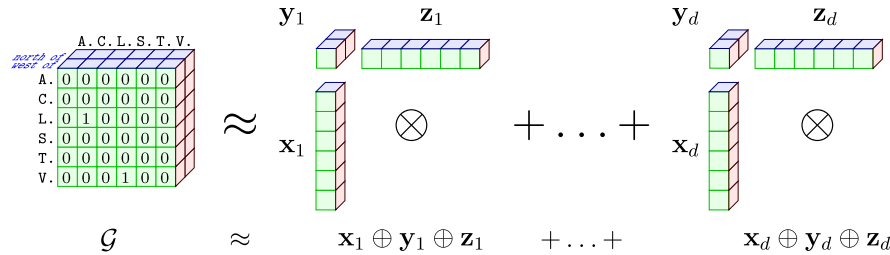


Figure 5.6: Abstract illustration of a CP d -rank decomposition of a tensor representing the graph of Figure 5.5a

DistMult [Yang et al., 2015] is a seminal method for computing knowledge graph embeddings based on rank decompositions, where each entity and relation is associated with a vector of dimension d , such that for an edge $(\textcircled{s} - \textcircled{p} \rightarrow \textcircled{o})$, a plausibility scoring function $\sum_{i=1}^d (\mathbf{e}_s)_i (\mathbf{r}_p)_i (\mathbf{e}_o)_i$ is defined, where $(\mathbf{e}_s)_i$, $(\mathbf{r}_p)_i$ and $(\mathbf{e}_o)_i$ denote the i^{th} elements of vectors $\mathbf{e}_s$, $\mathbf{r}_p$, $\mathbf{e}_o$, respectively. The goal, then, is to learn vectors for each node and edge label that maximise the plausibility of positive edges and minimise the plausibility of negative edges. This approach equates to a CP decomposition of the graph tensor $\mathcal{G}$, but where entities have one vector that is used twice: $\mathbf{x}_1 \otimes \mathbf{y}_1 \otimes \mathbf{x}_1 + \dots + \mathbf{x}_d \otimes \mathbf{y}_d \otimes \mathbf{x}_d \approx \mathcal{G}$. A weakness of this approach is that per the scoring function, the plausibility of $(\textcircled{s} - \textcircled{p} \rightarrow \textcircled{o})$ will always be equal to that of $(\textcircled{o} - \textcircled{p} \rightarrow \textcircled{s})$; in other words, DistMult does not consider edge direction.

Rather than use a vector as a relation embedding, RESCAL [Nickel and Tresp, 2013] uses a matrix, which allows for combining values from $\mathbf{e}_s$ and $\mathbf{e}_o$ across all dimensions, and thus can capture (e.g.) edge direction. However, RESCAL incurs a higher cost in terms of space and time than DistMult. HoLE [Nickel et al., 2016b] uses vectors for relation and entity embeddings, but proposes to use the *circular correlation operator* – which takes sums along the diagonals of the outer product of two vectors – to combine them. This operator is not commutative, and can thus consider edge direction. ComplEx [Trouillon et al., 2016], on the other hand, uses a complex vector (i.e., a vector containing complex numbers) as a relational embedding, which similarly allows for breaking the aforementioned symmetry of DistMult's scoring function while keeping the number of parameters lo

SimpleE [Kazemi and Poole, 2018] rather proposes to compute a standard CP decomposition computing two initial vectors for entities from $\mathbf{X}$ and $\mathbf{Z}$ and then averaging terms across $\mathbf{X}$, $\mathbf{Y}$, $\mathbf{Z}$ to compute the final plausibility scores. TuckER [Balazevic et al., 2019b] employs a different type of decomposition – called a Tucker Decomposition [Tucker, 1964], which computes a smaller “core” tensor $\mathcal{T}$ and a sequence of three matrices $\mathbf{A}$, $\mathbf{B}$ and $\mathbf{C}$, such that $\mathcal{G} \approx \mathcal{T} \otimes \mathbf{A} \otimes \mathbf{B} \otimes \mathbf{C}$ – where entity embeddings are taken from $\mathbf{A}$ and $\mathbf{C}$, while relation embeddings are taken from $\mathbf{B}$. Of these approaches, TuckER [Balazevic et al., 2019b] currently provides state-of-the-art results on standard benchmarks.

Neural models

A limitation of the aforementioned approaches is that they assume either linear (preserving addition and scalar multiplication) or bilinear (e.g., matrix multiplication) operations over embeddings to compute plausibility scores. Other approaches rather use neural networks to learn embeddings with non-linear scoring functions for plausibility.

One of the earliest proposals of a neural model was Semantic Matching Energy (SME) [Glorot et al., 2013], which learns parameters (aka weights: $\mathbf{w}$, $\mathbf{w}'$) for two functions – $f_{\mathbf{w}}(\mathbf{e}_s, \mathbf{r}_p)$ and $g_{\mathbf{w}'}(\mathbf{e}_o, \mathbf{r}_p)$ – such that the dot product of the result of both functions – $f_{\mathbf{w}}(\mathbf{e}_s, \mathbf{r}_p) \cdot g_{\mathbf{w}'}(\mathbf{e}_o, \mathbf{r}_p)$ – gives the plausibility score. Both linear and bilinear variants of $f_{\mathbf{w}}$ and $g_{\mathbf{w}'}$ are proposed. Another early proposal was Neural Tensor Networks (NTN) [Socher et al., 2013], which proposes to maintain a tensor $\mathcal{W}$ of internal weights, such that the plausibility score is computed by a complex function that combines the outer product $\mathbf{e}_s \otimes \mathcal{W} \otimes \mathbf{e}_o$ with a standard neural layer over $\mathbf{e}_s$ and $\mathbf{e}_o$, which in turn is combined with $\mathbf{r}_p$, to produce a plausibility score. The tensor $\mathcal{W}$ results in a high number of parameters, limiting scalability [Wang et al., 2017]. Multi-Layer Perceptron (MLP) [Dong et al., 2014] is a simpler model, where $\mathbf{e}_s$, $\mathbf{r}_p$ and $\mathbf{e}_o$ are concatenated and fed into a hidden layer to compute plausibility scores.

A number of more recent approaches have proposed using convolutional kernels in their models. ConvE [Dettmers et al., 2018] proposes to generate a matrix from $\mathbf{e}_s$ and $\mathbf{r}_p$ by “wrapping” each vector over several rows and concatenating both matrices. The concatenated matrix serves as the input for a set of (2D) convolutional layers, which returns a feature map tensor. The feature map tensor is vectorised and projected into d dimensions using a parameterised linear transformation. The plausibility score is then computed based on the dot product of this vector and $\mathbf{e}_o$. A disadvantage of ConvE is that by wrapping vectors into matrices, it imposes an artificial two-dimensional structure on the embeddings. HypER [Balazevic et al., 2019c] is a similar model using convolutions, but avoids the need to wrap vectors into matrices. Instead, a fully connected layer (called the “hypernetwork”) is applied to $\mathbf{r}_p$ and used to generate a matrix of relation-specific convolutional filters. These filters are applied directly to $\mathbf{e}_s$ to give a feature map, which is vectorised. The same process is then applied as in ConvE: the resulting vector is projected into d dimensions, and a dot product applied with $\mathbf{e}_o$ to produce the plausibility score. The resulting model is shown to outperform ConvE on standard benchmarks [Balazevic et al., 2019c].

The presented approaches strike different balances in terms of expressivity and the number of parameters than need to be trained. While more expressive models, such as NTN, may better fit more complex plausibility functions over lower dimensional embeddings by using more hidden parameters, simpler models, such as that proposed by Dong et al. [Dong et al., 2014], and convolutional networks [Dettmers et al., 2018, Balazevic et al., 2019c] that enable parameter sharing by applying the same (typically small) kernels over different regions of a matrix, require handling fewer parameters overall and are more scalable.

Survey and definition of tensor-based approaches

We now formally define and survey the aforementioned tensor-based approaches. For simplicity, we will consider directed edge-labelled graphs.

Before defining embeddings, we first introduce tensors.

Definition 5.3 (Vector, matrix, tensor, order, mode)

For any positive integer a , a vector of dimension a is a family of real numbers indexed by integers in $\{1, \dots, a\}$. For a and b positive integers, an (a, b) -matrix is a family of real numbers indexed by pairs of integers in $\{1, \dots, a\} \times \{1, \dots, b\}$. A tensor is a family of real numbers indexed by a finite sequence of integers such that there exist positive numbers $a_1, \dots, a_n$ such that the indices are all the tuples of numbers in $\{1, \dots, a_1\} \times \dots \times \{1, \dots, a_n\}$. The number n is called the order of the tensor, the subindices $i \in \{1, \dots, n\}$ indicate the mode of a tensor, and each a_i defines the dimension of the i^{th} mode. A 1-order tensor is a vector and a 2-order tensor is a matrix. We denote the set of all tensors as $\mathbb{T}$.

For specific dimensions $a_1, \dots, a_n$ of modes, a tensor is an element of $(\dots(\mathbb{R}^{a_1})\dots)^{a_n}$ but we write $\mathbb{R}^{a_1, \dots, a_n}$ to simplify the notation. We use lower-case bold font to denote vectors ($\mathbf{x} \in \mathbb{R}^a$), upper-case bold font to denote matrices ($\mathbf{X} \in \mathbb{R}^{a,b}$) and calligraphic font to denote tensors ($\mathcal{X} \in \mathbb{R}^{a_1, \dots, a_n}$).

Now we are ready to abstractly define knowledge graph embeddings.

Definition 5.4 (Knowledge graph embedding)

Given a directed edge-labelled graph $G = (V, E, L)$, a knowledge graph embedding of G is a pair of mappings (ε, ρ) such that $\varepsilon : V \rightarrow \mathbb{T}$ and $\rho : L \rightarrow \mathbb{T}$.

In the most typical case, ε and ρ map nodes and edge-labels, respectively, to vectors of fixed dimension. In some cases, however, they may map to matrices. Given this abstract notion of a knowledge graph embedding, we can then define a plausibility scoring function.

Definition 5.5 (Plausibility scores)

A plausibility scoring function is a partial function $\phi : \mathbb{T} \times \mathbb{T} \times \mathbb{T} \rightarrow \mathbb{R}$. Given a directed edge-labelled graph $G = (V, E, L)$, an edge $(s, p, o) \in V \times L \times V$, and a knowledge graph embedding (ε, ρ) of G , the plausibility of (s, p, o) is given as $\phi(\varepsilon(s), \rho(p), \varepsilon(o))$.

Edges with higher scores are considered more plausible. Given a graph $G = (V, E, L)$, we assume a set of positive edges E^+ and a set of negative edges E^- . Positive edges are often simply the edges in the graph: $E^+ := E$. Negative edges use the vocabulary of G (i.e., $E^- \subseteq V \times L \times V$) and are typically defined by taking edges (s, p, o) from E and changing one term of each edge – often one of the nodes – such that the edge is no longer in E . Given sets of positive and negative edges, and a plausibility scoring function, the objective is then to find the embedding that maximises the plausibility of edges in E^+ while minimising the plausibility of edges in E^- . Specific knowledge graph embeddings then instantiate the type of embedding considered and the plausibility scoring function in various ways.

In Table 5.1, we define the plausibility scoring function and types of embeddings used by different knowledge graph embeddings. To simplify the definitions, we use $\mathbf{e}_x$ to denote $\varepsilon(x)$ when it is a vector, $\mathbf{r}_y$ to denote $\rho(y)$ when it is a vector, and $\mathbf{R}_y$ to denote $\rho(y)$ when it is a matrix. Some models involve learnt parameters (aka weights) for computing plausibility. We denote these as $\mathbf{v}$, $\mathbf{V}$, $\mathcal{V}$, $\mathbf{w}$, $\mathbf{W}$, $\mathcal{W}$ (for vectors, matrices or tensors). We use d_e and d_r to denote the dimensionality chosen for entity embeddings and relation embeddings, respectively. Often it is assumed that $d_e = d_r$, in which case we will write d . Weights may have their own dimensionality, which we denote w . The embeddings in Table 5.1 use a variety of operators on vectors, matrices and tensors, which will be defined later.

The embeddings defined in Table 5.1 vary in complexity, where a trade-off exists between the number of parameters used, and the expressiveness of the model in terms of its capability to capture latent features of the graph. To increase expressivity, many of the models in Table 5.1 use additional parameters beyond the embeddings themselves. A possible formal guarantee of such models is *full expressiveness*, which, given any disjoint sets of positive edges E^+ and negative edges E^- , asserts that the model can always correctly partition those edges. On the one hand, for example, DistMult [Yang et al., 2015] cannot distinguish an edge $\textcircled{s} \text{--} p \text{--} \textcircled{o}$ from its inverse $\textcircled{o} \text{--} p \text{--} \textcircled{s}$, so by adding an inverse of an edge in E^+ to E^- , we can show that it is *not* fully expressive. On the other hand, models such as ComplEx [Trouillon et al., 2016], Simple [Kazemi and Poole, 2018], and TuckER [Balazevic et al., 2019b] have been proven to be fully expressive

Give us feedback!

given sufficient dimensionality; for example, Tucker [Balazevic et al., 2019b] with dimensions $d_r = |L|$ and $d_e = |V|$ trivially satisfies full expressivity since its core tensor $\mathcal{W}$ then has sufficient capacity to store the full one-hot encoding of any graph. This formal property is useful to show that the model does not have built-in limitations for numerically representing a graph, though of course in practice the dimensions needed to reach full expressivity are often impractical/undesirable.

We continue by first defining the conventions used in Table 5.1.

- We use $(\mathbf{x})_i$, $(\mathbf{X})_{ij}$, and $(\mathcal{X})_{i_1 \dots i_n}$ to denote elements of vectors, matrices, and tensors, respectively. If a vector $\mathbf{x} \in \mathbb{R}^a$ is used in a context that requires a matrix, the vector is interpreted as an $(a, 1)$ -matrix (i.e., a column vector) and can be turned into a row vector (i.e., a $(1, a)$ -matrix) using the transpose operation $\mathbf{x}^T$. We use $\mathbf{x}^D \in \mathbb{R}^{a,a}$ to denote the diagonal matrix with the values of the vector $\mathbf{x} \in \mathbb{R}^a$ on its diagonal. We denote the identity matrix by $\mathbf{I}$ such that if $j = k$, then $(\mathbf{I})_{jk} = 1$; otherwise $(\mathbf{I})_{jk} = 0$.
- We denote by $\begin{bmatrix} \mathbf{X}_1 \\ \vdots \\ \mathbf{X}_n \end{bmatrix}$ the vertical stacking of matrices $\mathbf{X}_1, \dots, \mathbf{X}_n$ with the same number of columns. Given a vector $\mathbf{x} \in \mathbb{R}^b$, we denote by $\mathbf{x}^{[a,b]} \in \mathbb{R}^{a,b}$ the “reshaping” of $\mathbf{x}$ into an (a, b) -matrix such that $(\mathbf{x}^{[a,b]})_{ij} = (\mathbf{x})_{(i+a)(j-1)}$. Conversely, given a matrix $\mathbf{X} \in \mathbb{R}^{a,b}$, we denote by $\text{vec}(\mathbf{X}) \in \mathbb{R}^{ab}$ the *vectorisation* of $\mathbf{X}$ such that $\text{vec}(\mathbf{X})_k = (\mathbf{X})_{ij}$ where $i = ((k-1) \bmod m) + 1$ and $j = \frac{k-i}{m} + 1$ (observe that $\text{vec}(\mathbf{x}^{[a,b]}) = \mathbf{x}$).
- Given a tensor $\mathcal{X} \in \mathbb{R}^{a,b,c}$, we denote by $\mathcal{X}^{[i:\cdot:\cdot]} \in \mathbb{R}^{b,c}$, the i^{th} slice of tensor $\mathcal{X}$ along the first mode; for example, given $\mathcal{X} \in \mathbb{R}^{5,2,3}$, then $\mathcal{X}^{[4:\cdot:\cdot]}$ returns the $(2, 3)$ -matrix consisting of the elements $\begin{bmatrix} (\mathcal{X})_{411} & (\mathcal{X})_{412} & (\mathcal{X})_{413} \\ (\mathcal{X})_{421} & (\mathcal{X})_{422} & (\mathcal{X})_{423} \end{bmatrix}$. Analogously, we use $\mathcal{X}^{[\cdot:i:\cdot]} \in \mathbb{R}^{a,c}$ and $\mathcal{X}^{[\cdot:\cdot:i]} \in \mathbb{R}^{b,c}$ to indicate the i^{th} slice along the second and third modes of $\mathcal{X}$, respectively.
- We denote by $\psi(\mathcal{X})$ the element-wise application of a function ψ to the tensor $\mathcal{X}$, such that $(\psi(\mathcal{X}))_{i_1 \dots i_n} = \psi(\mathcal{X}_{i_1 \dots i_n})$. Common choices for ψ include a sigmoid function (e.g., the logistic function $\psi(x) = \frac{1}{1+e^{-x}}$ or the hyperbolic tangent function $\psi(x) = \tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}$), the rectifier ($\psi(x) = \max(0, x)$), softplus ($\psi(x) = \ln(1 + e^x)$), etc.

We now define the operators used in Table 5.1, where the first and most elemental operation we consider is that of matrix multiplication.

Definition 5.6 (Matrix multiplication)

The multiplication of matrices $\mathbf{X} \in \mathbb{R}^{a,b}$ and $\mathbf{Y} \in \mathbb{R}^{b,c}$ is a matrix $\mathbf{XY} \in \mathbb{R}^{a,c}$ such that $(\mathbf{XY})_{ij} = \sum_{k=1}^b (\mathbf{X})_{ik} (\mathbf{Y})_{kj}$. The matrix multiplication of two tensors $\mathcal{X} \in \mathbb{R}^{a_1, \dots, a_m, c}$ and $\mathcal{Y} \in \mathbb{R}^{c, b_1, \dots, b_n}$ is a tensor $\mathcal{XY} \in \mathbb{R}^{a_1, \dots, a_m, b_1, \dots, b_n}$ such that $(\mathcal{XY})_{i_1 \dots i_m i_{m+1} \dots i_{m+n}} = \sum_{k=1}^c (\mathcal{X})_{i_1 \dots i_m k} (\mathcal{Y})_{k i_{m+1} \dots i_{m+n}}$.

For convenience, we may implicitly add or remove modes with dimension 1 for the purposes of matrix multiplication and other operators; for example, given two vectors $\mathbf{x} \in \mathbb{R}^a$ and $\mathbf{y} \in \mathbb{R}^a$, we denote by $\mathbf{x}^T \mathbf{y}$ (aka the dot or inner product) the multiplication of matrix $\mathbf{x}^T \in \mathbb{R}^{1,a}$ with $\mathbf{y} \in \mathbb{R}^{a,1}$ such that $\mathbf{x}^T \mathbf{y} \in \mathbb{R}^{1,1}$ (i.e., a scalar in $\mathbb{R}$); conversely, $\mathbf{xy}^T \in \mathbb{R}^{a,a}$ (the outer product).

Constraints on embeddings are sometimes given as norms, defined next.

Definition 5.7 (L^p -norm, $L^{p,q}$ -norm)

For $p \in \mathbb{R}$, the L^p -norm of a vector $\mathbf{x} \in \mathbb{R}^a$ is the scalar $\|\mathbf{x}\|_p := (|\mathbf{x}_1|^p + \dots + |\mathbf{x}_a|^p)^{\frac{1}{p}}$, where $|\mathbf{x}_i|$ denotes the absolute value of the i^{th} element of $\mathbf{x}$. For $p, q \in \mathbb{R}$, the $L^{p,q}$ -norm of a matrix $\mathbf{X} \in \mathbb{R}^{a,b}$ is the scalar $\|\mathbf{X}\|_{p,q} := \left(\sum_{j=1}^b \left(\sum_{i=1}^a |(\mathbf{X})_{ij}|^p \right)^{\frac{1}{p}} \right)^{\frac{1}{q}}$.

The L^1 norm (i.e., $\|\mathbf{x}\|_1$) is thus simply the sum of the absolute values of $\mathbf{x}$, while the L^2 norm (i.e., $\|\mathbf{x}\|_2$) is the (Euclidean) length of the vector. The Frobenius norm of the matrix $\mathbf{X}$ then equates to $\|\mathbf{X}\|_{2,2} = \left(\sum_{j=1}^b \left(\sum_{i=1}^a |(\mathbf{X})_{ij}|^2 \right) \right)^{\frac{1}{2}}$; i.e., the square root of the sum of the squares of all elements.

Another type of product used by embedding techniques is the Hadamard product, which multiplies tensors of the same dimension and computes their product in an element-wise manner.

Definition 5.8 (Hadamard product)

Given two tensors $\mathcal{X} \in \mathbb{R}^{a_1 \dots a_n}$ and $\mathcal{Y} \in \mathbb{R}^{a_1 \dots a_n}$, the Hadamard product $\mathcal{X} \odot \mathcal{Y}$ is defined as a tensor in $\mathbb{R}^{a_1 \dots a_n}$, with each element computed as $(\mathcal{X} \odot \mathcal{Y})_{i_1 \dots i_n} := (\mathcal{X})_{i_1 \dots i_n} (\mathcal{Y})_{i_1 \dots i_n}$.

Other embedding techniques – namely RotatE [Sun et al., 2019] and ComplEx [Trouillon et al., 2016] – uses *complex space* based on complex numbers. With a slight abuse of notation, the definitions of vectors, matrices and tensors can be modified by replacing the set of real numbers $\mathbb{R}$ by the set of complex numbers $\mathbb{C}$, giving rise to complex vectors, complex matrices, and complex tensors. In this case, we denote by $\text{Re}(\cdot)$ the real part of a complex number. Given a complex vector $\mathbf{x} \in \mathbb{C}^I$, we denote by $\bar{\mathbf{x}}$ its complex conjugate (swapping the sign of the imaginary part of each element). Complex analogues of the aforementioned operators can then be defined by replacing the multiplication and addition of real numbers with the analogous operators for complex numbers, where RotatE [Sun et al., 2019] uses the complex Hadamard product, and ComplEx [Trouillon et al., 2016] uses complex matrix multiplication.

One embedding technique – MuRP [Balazevic et al., 2019a] – uses hyperbolic space, specifically based on the Poincaré ball. As this is the only embedding we cover that uses this space, and the formalisms are lengthy (covering the Poincaré ball, Möbius addition, Möbius matrix–vector multiplication, logarithmic maps, exponential maps, etc.), we rather refer the reader to the paper for further details [Balazevic et al., 2019a].

As discussed in Section 5.2, tensor decompositions are used for many embeddings, and at the heart of such decompositions is the tensor product, which is often used to reconstruct (an approximation of) the original tensor.

Definition 5.9 (Tensor product)

Given two tensors $\mathcal{X} \in \mathbb{R}^{a_1 \dots a_m}$ and $\mathcal{Y} \in \mathbb{R}^{b_1 \dots b_n}$, the tensor product $\mathcal{X} \otimes \mathcal{Y}$ is defined as a tensor in $\mathbb{R}^{a_1 \dots a_m b_1 \dots b_n}$, with each element computed as $(\mathcal{X} \otimes \mathcal{Y})_{i_1 \dots i_m j_1 \dots j_n} := (\mathcal{X})_{i_1 \dots i_m} (\mathcal{Y})_{j_1 \dots j_n}$.²⁴

note 24 Please note Definition 3.10.

Example 5.3 — Assume that $\mathcal{X} \in \mathbb{R}^{2,3}$ and $\mathcal{Y} \in \mathbb{R}^{3,4,5}$. Then $\mathcal{X} \otimes \mathcal{Y}$ will be a tensor in $\mathbb{R}^{2,3,3,4,5}$. Element $(\mathcal{X} \otimes \mathcal{Y})_{12345}$ will be the product of $(\mathcal{X})_{12}$ and $(\mathcal{Y})_{345}$.

An n -mode product is used by other embeddings to transform elements along a given mode of a tensor by computing a product with a given matrix along that particular mode of the tensor.

Definition 5.10 (n -mode product)

For a positive integer n , a tensor $\mathcal{X} \in \mathbb{R}^{a_1 \dots a_{n-1}, a_n, a_{n+1}, \dots, a_m}$ and matrix $\mathbf{Y} \in \mathbb{R}^{b, a_n}$, the n -mode product of $\mathcal{X}$ and $\mathbf{Y}$ is the tensor $\mathcal{X} \otimes_n \mathbf{Y} \in \mathbb{R}^{a_1 \dots a_{n-1}, b, a_{n+1}, \dots, a_m}$ such that $(\mathcal{X} \otimes_n \mathbf{Y})_{i_1 \dots i_{n-1} j i_{n+1} \dots i_m} := \sum_{k=1}^{a_n} (\mathcal{X})_{i_1 \dots i_{n-1} k i_{n+1} \dots i_m} (\mathbf{Y})_{jk}$.

Example 5.4 — Let us assume that $\mathcal{X} \in \mathbb{R}^{2,3,4}$ and $\mathbf{Y} \in \mathbb{R}^{5,3}$. The result of $\mathcal{X} \otimes_2 \mathbf{Y}$ will be a tensor in $\mathbb{R}^{2,5,4}$, where, for example, $(\mathcal{X} \otimes_2 \mathbf{Y})_{142}$ will be given as $(\mathcal{X})_{112}(\mathbf{Y})_{41} + (\mathcal{X})_{122}(\mathbf{Y})_{42} + (\mathcal{X})_{132}(\mathbf{Y})_{43}$. Observe that if $\mathbf{y} \in \mathbb{R}^{a_n}$ – i.e., if $\mathbf{y}$ is a (column) vector – then the n -mode tensor product $\mathcal{X} \otimes_n \mathbf{y}^T$ “flattens” the n^{th} mode of $\mathcal{X}$ to one dimension, effectively reducing the order of $\mathcal{X}$ by one.

One embedding technique – HoLE [Nickel et al., 2016b] – uses the circular correlation operator $\mathbf{x} \star \mathbf{y}$, where each element is the sum of elements along a diagonal of the outer product $\mathbf{x} \otimes \mathbf{y}$ that “wraps” if not the primary diagonal.

Definition 5.11 (Circular correlation)

The circular correlation of vector $\mathbf{x} \in \mathbb{R}^a$ with $\mathbf{y} \in \mathbb{R}^a$ is the vector $\mathbf{x} \star \mathbf{y} \in \mathbb{R}^a$ such that $(\mathbf{x} \star \mathbf{y})_k := \sum_{i=1}^a (\mathbf{x})_i (\mathbf{y})_{((k+i-2) \bmod a)+1}$.

Example 5.5 — Assuming $a = 5$, then $(\mathbf{x} \star \mathbf{y})_1 = (\mathbf{x})_1(\mathbf{y})_1 + (\mathbf{x})_2(\mathbf{y})_2 + (\mathbf{x})_3(\mathbf{y})_3 + (\mathbf{x})_4(\mathbf{y})_4 + (\mathbf{x})_5(\mathbf{y})_5$, or a case that wraps: $(\mathbf{x} \star \mathbf{y})_4 = (\mathbf{x})_1(\mathbf{y})_4 + (\mathbf{x})_2(\mathbf{y})_5 + (\mathbf{x})_3(\mathbf{y})_1 + (\mathbf{x})_4(\mathbf{y})_2 + (\mathbf{x})_5(\mathbf{y})_3$.

Finally, a couple of neural models that we include – namely ConvE [Dettmers et al., 2018] and Hyper [Balazevic et al., 2019c] – are based on convolutional architectures using the convolution operator.

Give us feedback!

Definition 5.12 (Convolution)

Given two matrices $\mathbf{X} \in \mathbb{R}^{a,b}$ and $\mathbf{Y} \in \mathbb{R}^{e,f}$, the convolution of $\mathbf{X}$ and $\mathbf{Y}$ is the matrix $\mathbf{X} * \mathbf{Y} \in \mathbb{R}^{(a+e-1),(b+f-1)}$ such that $(\mathbf{X} * \mathbf{Y})_{ij} = \sum_{k=1}^a \sum_{l=1}^b (\mathbf{X})_{ki} (\mathbf{Y})_{(i+k-a)(j+l-b)}$.²⁵ In cases where $(i+k-a) < 1$, $(j+l-b) < 1$, $(i+k-a) > e$ or $(j+l-b) > f$ (i.e., where $(\mathbf{Y})_{(i+k-a)(j+l-b)}$ lies outside the bounds of $\mathbf{Y}$), we say that $(\mathbf{Y})_{(i+k-a)(j+l-b)} = 0$.

Intuitively speaking, the convolution operator overlays $\mathbf{X}$ in every possible way over $\mathbf{Y}$ such that at least one pair of elements $(\mathbf{X})_{ij}, (\mathbf{Y})_{lk}$ overlaps, summing the products of pairs of overlapping elements to generate an element of the result. Elements of $\mathbf{X}$ extending beyond $\mathbf{Y}$ are ignored (equivalently we can consider $\mathbf{Y}$ to be “zero-padded” outside its borders).

Example 5.6 — Given $\mathbf{X} \in \mathbb{R}^{3,3}$ and $\mathbf{Y} \in \mathbb{R}^{4,5}$, then $\mathbf{X} * \mathbf{Y} \in \mathbb{R}^{6,7}$, where, for example, $(\mathbf{X} * \mathbf{Y})_{11} = (\mathbf{X})_{33}(\mathbf{Y})_{11}$ (with the bottom right corner of $\mathbf{X}$ overlapping the top left corner of $\mathbf{Y}$), while $(\mathbf{X} * \mathbf{Y})_{34} = (\mathbf{X})_{11}(\mathbf{Y})_{12} + (\mathbf{X})_{12}(\mathbf{Y})_{13} + (\mathbf{X})_{13}(\mathbf{Y})_{14} + (\mathbf{X})_{21}(\mathbf{Y})_{22} + (\mathbf{X})_{22}(\mathbf{Y})_{23} + (\mathbf{X})_{23}(\mathbf{Y})_{24} + (\mathbf{X})_{31}(\mathbf{Y})_{32} + (\mathbf{X})_{32}(\mathbf{Y})_{33} + (\mathbf{X})_{33}(\mathbf{Y})_{34}$ (with $(\mathbf{X})_{22}$ — the centre of $\mathbf{X}$ — overlapping $(\mathbf{Y})_{23}$).²⁶

In a convolution $\mathbf{X} * \mathbf{Y}$, the matrix $\mathbf{X}$ is often called the “kernel” (or “filter”). Often several kernels are used in order to apply multiple convolutions. Given a tensor $\mathcal{X} \in \mathbb{R}^{c,a,b}$ (representing c (a,b) -kernels) and a matrix $\mathbf{Y} \in \mathbb{R}^{e,f}$, we denote by $\mathcal{X} * \mathbf{Y} \in \mathbb{R}^{c,(a+e-1),(b+f-1)}$ the result of the convolutions of the c first-mode slices of $\mathcal{X}$ over $\mathbf{Y}$ such that $(\mathcal{X} * \mathbf{Y})^{[i:\cdot:\cdot]} = \mathcal{X}^{[i:\cdot:\cdot]} * \mathbf{Y}$ for $1 \leq i \leq c$, yielding a tensor of results for c convolutions.

note 25 We define the traditional neural network where traditional convolution is used. Since in our settings it is not used and hence the distinct

note 26 Models applying on the “stride” of the kernel would see the kernel $j \bmod 3 = 0$, reducing such details here.

Table 5.1: Details for selected knowledge graph embeddings, including the plausibility scoring function $\phi(\varepsilon(s), \rho(p), \varepsilon(o))$ for edge $(s \xrightarrow{p} o)$, and other conditions

Model	$\phi(\varepsilon(s), \rho(p), \varepsilon(o))$	Conditions (for all $x \in V, y \in L$)
TransE	$-\ \mathbf{e}_s + \mathbf{r}_p - \mathbf{e}_o\ _q$	$\mathbf{e}_x \in \mathbb{R}^d, \mathbf{r}_y \in \mathbb{R}^d, q \in \{1, 2\}, \ \mathbf{e}_x\ _2 = 1$
TransH	$-\ (\mathbf{e}_s - (\mathbf{e}_s^T \mathbf{w}_p) \mathbf{w}_p) + \mathbf{r}_p - (\mathbf{e}_o - (\mathbf{e}_o^T \mathbf{w}_p) \mathbf{w}_p)\ _2^2$	$\mathbf{e}_x \in \mathbb{R}^d, \mathbf{r}_y \in \mathbb{R}^d, \mathbf{w}_y \in \mathbb{R}^d, \ \mathbf{w}_y\ _2 = 1, \frac{\mathbf{w}_y^T \mathbf{r}_y}{\ \mathbf{r}_y\ _2} \approx 0, \ \mathbf{e}_x\ _2 \leq 1$
TransR	$-\ \mathbf{W}_p \mathbf{e}_s + \mathbf{r}_p - \mathbf{W}_p \mathbf{e}_o\ _2^2$	$\mathbf{e}_x \in \mathbb{R}^{d_e}, \mathbf{r}_y \in \mathbb{R}^{d_r}, \mathbf{W}_y \in \mathbb{R}^{d_r, d_e}, \ \mathbf{e}_x\ _2 \leq 1, \ \mathbf{r}_y\ _2 \leq 1, \ \mathbf{W}_y \mathbf{e}_x\ _2 \leq 1$
TransD	$-\ (\mathbf{w}_p \otimes \mathbf{w}_s + \mathbf{I}) \mathbf{e}_s + \mathbf{r}_p - (\mathbf{w}_p \otimes \mathbf{w}_o + \mathbf{I}) \mathbf{e}_o\ _2^2$	$\mathbf{e}_x \in \mathbb{R}^{d_e}, \mathbf{r}_y \in \mathbb{R}^{d_r}, \mathbf{w}_x \in \mathbb{R}^{d_e}, \mathbf{w}_y \in \mathbb{R}^{d_r}, \ \mathbf{e}_x\ _2 \leq 1, \ \mathbf{r}_y\ _2 \leq 1, \ (\mathbf{w}_y \otimes \mathbf{w}_x + \mathbf{I}) \mathbf{e}_x\ _2 \leq 1$
RotatE	$-\ \mathbf{e}_s \odot \mathbf{r}_p - \mathbf{e}_o\ _2$	$\mathbf{e}_x \in \mathbb{C}^d, \mathbf{r}_y \in \mathbb{C}^d, \ \mathbf{r}_y\ _2 = 1$
RESCAL	$\mathbf{e}_s^T \mathbf{R}_p \mathbf{e}_o$	$\mathbf{e}_x \in \mathbb{R}^d, \mathbf{R}_y \in \mathbb{R}^{d,d}, \ \mathbf{e}_x\ _2 \leq 1, \ \mathbf{R}_y\ _{2,2} \leq 1$
DistMult	$\mathbf{e}_s^T \mathbf{r}_p \mathbf{e}_o$	$\mathbf{e}_x \in \mathbb{R}^d, \mathbf{r}_y \in \mathbb{R}^d, \ \mathbf{e}_x\ _2 = 1, \ \mathbf{r}_y\ _2 \leq 1$
HolE	$\mathbf{r}_p^T (\mathbf{e}_s * \mathbf{e}_o)$	$\mathbf{e}_x \in \mathbb{R}^d, \mathbf{r}_y \in \mathbb{R}^d, \ \mathbf{e}_x\ _2 \leq 1, \ \mathbf{r}_y\ _2 \leq 1$
ComplEx	$\text{Re}(\mathbf{e}_s^T \mathbf{r}_p \mathbf{e}_o)$	$\mathbf{e}_x \in \mathbb{C}^d, \mathbf{r}_y \in \mathbb{C}^d, \ \mathbf{e}_x\ _2 \leq 1, \ \mathbf{r}_y\ _2 \leq 1$
Simple	$\frac{\mathbf{e}_s^T \mathbf{r}_p \mathbf{w}_o + \mathbf{e}_o^T \mathbf{r}_p \mathbf{w}_s}{2}$	$\mathbf{e}_x \in \mathbb{R}^d, \mathbf{r}_y \in \mathbb{R}^d, \mathbf{w}_x \in \mathbb{R}^d, \mathbf{w}_y \in \mathbb{R}^d, \ \mathbf{e}_x\ _2 \leq 1, \ \mathbf{r}_y\ _2 \leq 1, \ \mathbf{w}_x\ _2 \leq 1, \ \mathbf{w}_y\ _2 \leq 1$
TuckER	$\mathcal{W} \otimes_1 \mathbf{e}_s^T \otimes_2 \mathbf{r}_p^T \otimes_3 \mathbf{e}_o^T$	$\mathbf{e}_x \in \mathbb{R}^{d_e}, \mathbf{r}_y \in \mathbb{R}^{d_r}, \mathcal{W} \in \mathbb{R}^{d_e, d_r, d_e}$
SME L.	$(\mathbf{V} \mathbf{e}_s + \mathbf{V}' \mathbf{r}_p + \mathbf{v})^T (\mathbf{W} \mathbf{e}_o + \mathbf{W}' \mathbf{r}_p + \mathbf{w})$	$\mathbf{e}_x \in \mathbb{R}^d, \mathbf{r}_y \in \mathbb{R}^d, \mathbf{v} \in \mathbb{R}^w, \mathbf{w} \in \mathbb{R}^w, \ \mathbf{e}_x\ _2 = 1, \mathbf{V} \in \mathbb{R}^{w,d}, \mathbf{V}' \in \mathbb{R}^{w,d}, \mathbf{W} \in \mathbb{R}^{w,d}, \mathbf{W}' \in \mathbb{R}^{w,d}$
SME Bi.	$((\mathcal{V} \otimes_3 \mathbf{r}_p^T) \mathbf{e}_s + \mathbf{v})^T ((\mathcal{W} \otimes_3 \mathbf{r}_p^T) \mathbf{e}_o + \mathbf{w})$	$\mathbf{e}_x \in \mathbb{R}^d, \mathbf{r}_y \in \mathbb{R}^d, \mathbf{v} \in \mathbb{R}^w, \mathbf{w} \in \mathbb{R}^w, \ \mathbf{e}_x\ _2 = 1, \mathcal{V} \in \mathbb{R}^{w,d,d}, \mathcal{W} \in \mathbb{R}^{w,d,d}$
NTN	$\mathbf{r}_p^T \psi \left(\mathbf{e}_s^T \mathcal{W} \mathbf{e}_o + \mathbf{W} \begin{bmatrix} \mathbf{e}_s \\ \mathbf{r}_p \\ \mathbf{e}_o \end{bmatrix} + \mathbf{w} \right)$	$\mathbf{e}_x \in \mathbb{R}^d, \mathbf{r}_y \in \mathbb{R}^d, \mathbf{w} \in \mathbb{R}^w, \mathbf{W} \in \mathbb{R}^{w,2d}, \mathcal{W} \in \mathbb{R}^{d,w,d}, \ \mathbf{e}_x\ _2 \leq 1, \ \mathbf{r}_y\ _2 \leq 1, \ \mathbf{w}\ _2 \leq 1, \ \mathbf{W}\ _{2,2} \leq 1, \ \mathcal{W}\ _{1 \leq i \leq w}^{[i:\cdot:\cdot]} \ _{2,2} \leq 1$
MLP	$\mathbf{v}^T \psi \left(\mathbf{W} \begin{bmatrix} \mathbf{e}_s \\ \mathbf{r}_p \\ \mathbf{e}_o \end{bmatrix} + \mathbf{w} \right)$	$\mathbf{e}_x \in \mathbb{R}^d, \mathbf{r}_y \in \mathbb{R}^d, \mathbf{v} \in \mathbb{R}^w, \mathbf{w} \in \mathbb{R}^w, \mathbf{W} \in \mathbb{R}^{w,3d}, \ \mathbf{e}_x\ _2 \leq 1, \ \mathbf{r}_y\ _2 \leq 1$
ConvE	$\psi \left(\text{vec} \left(\psi \left(\mathcal{W} * \begin{bmatrix} \mathbf{e}_s^{[a,b]} \\ \mathbf{r}_p^{[a,b]} \end{bmatrix} \right) \right)^T \mathbf{W} \right) \mathbf{e}_o$	$\mathbf{e}_x \in \mathbb{R}^d, \mathbf{r}_y \in \mathbb{R}^d, d = ab, \mathcal{W} \in \mathbb{R}^{w_1(w_2+2a-1)(w_3+b-1)d}, \mathbf{W} \in \mathbb{R}^{w_1, w_2, w_3}$
Hyper	$\psi \left(\text{vec} (\mathbf{r}_p^T \mathcal{W} * \mathbf{e}_s)^T \mathbf{W} \right) \mathbf{e}_o$	$\mathbf{e}_x \in \mathbb{R}^{d_e}, \mathbf{r}_y \in \mathbb{R}^{d_r}, \mathcal{W} \in \mathbb{R}^{w_2(w_1+d_e-1)d_e}, \mathbf{W} \in \mathbb{R}^{d_r, w_1, w_2}$

5.2.2 LANGUAGE MODELS

Embedding techniques were first explored as a way to represent natural language within machine learning frameworks, with word2vec [Mikolov et al., 2013] and GloVe [Pennington et al., 2014] being two seminal approaches. Both approaches compute embeddings for words based on large corpora of text such that words used in similar contexts (e.g., “frog”, “toad”) have similar vector representations. Word2vec uses neural networks trained either to predict the current word from surrounding words

Give us feedback!

(*continuous bag of words*), or to predict the surrounding words given the current word (*continuous skip-gram*). GloVe rather applies a regression model over a matrix of co-occurrence probabilities of word pairs. Embeddings generated by both approaches have become widely used in natural language processing tasks.

Another approach for graph embeddings is thus to leverage proven approaches for language embeddings. However, while a graph consists of an unordered set of sequences of three terms (i.e., a set of edges), text in natural language consists of arbitrary-length sequences of terms (i.e., sentences of words). RDF2Vec [Ristoski and Paulheim, 2016] thus performs (biased [Cochez et al., 2017a]) random walks on the graph and records the paths (the sequence of nodes and edge labels traversed) as “sentences”, which are then fed as input into the word2vec [Mikolov et al., 2013] model. An example of such a path extracted from Figure 5.2 might be, for example, (San Pedro) $\xrightarrow{\text{bus}}$ (Calama) $\xrightarrow{\text{flight}}$ (Iquique) $\xrightarrow{\text{flight}}$ (Santiago), where the paper experiments with 500 paths of length 8 per entity. RDF2Vec also proposes a second mode where sequences are generated for nodes from canonically-labelled sub-trees of which they are a root node, where sub-trees of depth 1 and 2 are used for experiments. KGloVe [Cochez et al., 2017b] is rather based on GloVe. Given that the original GloVe model [Pennington et al., 2014] considers words that co-occur frequently in windows of text to be more related, KGloVe uses personalised PageRank²⁷ to determine the most related nodes to a given node, which are fed into the GloVe model.

note 27 Intuitively speaking, it mines the probability of steps. A higher number indicates global node centrality relative to the starting node.

5.2.3 ENTAILMENT-AWARE MODELS

The embeddings thus far consider the data graph alone. But what if an ontology or set of rules is provided? Such deductive knowledge could be used to improve the embeddings. One approach is to use constraint rules to refine the predictions made by embeddings; for example, Wang et al. [2015] use functional and inverse-functional definitions as constraints (under UNA) such that, for example, if we define that an event can have at most one value for *venue*, this is used to lower the plausibility of edges that would assign multiple venues to an event.

More recent approaches rather propose joint embeddings that consider both the data graph and rules when computing embeddings. KALE [Guo et al., 2016] computes entity and relation embeddings using a translational model (specifically TransE) that is adapted to further consider rules using *t-norm fuzzy logics*. With reference to Figure 5.2, consider a simple rule $\{?x\} \xrightarrow{\text{bus}} \{?y\} \Rightarrow \{?x\} \xrightarrow{\text{connects to}} \{?y\}$. We can use embeddings to assign plausibility scores to new edges, such as $e_1: \text{(Piedras Rojas)} \xrightarrow{\text{bus}} \text{(Moon Valley)}$. We can further apply the previous rule to generate a new edge $e_2: \text{(Piedras Rojas)} \xrightarrow{\text{connects to}} \text{(Moon Valley)}$ from the predicted edge e_1 . But what plausibility should we assign to this second edge? Letting p_1 and p_2 be the current plausibility scores of e_1 and e_2 (initialised using the standard embedding), then *t-norm fuzzy logics* suggests that the plausibility be updated as $p_1 p_2 - p_1 + 1$. Embeddings are then trained to jointly assign larger plausibility scores to positive examples versus negative examples of both edges and *ground rules*. An example of a positive ground rule based on Figure 5.2 would be $\text{(Arica)} \xrightarrow{\text{bus}} \text{(San Pedro)} \Rightarrow \text{(Arica)} \xrightarrow{\text{connects to}} \text{(San Pedro)}$. Negative ground rules randomly replace the relation in the head of the rule; for example, $\text{(Arica)} \xrightarrow{\text{bus}} \text{(San Pedro)} \not\Rightarrow \text{(Arica)} \xrightarrow{\text{flight}} \text{(San Pedro)}$. Guo et al. [2018] later propose RUGE, which uses a joint model over ground rules (possibly soft rules with confidence scores) and plausibility scores to align both forms of scoring for unseen edges.

Generating ground rules can be costly. An alternative approach, called FSL [Demeester et al., 2016], observes that in the case of a simple rule, such as $\{?x\} \xrightarrow{\text{bus}} \{?y\} \Rightarrow \{?x\} \xrightarrow{\text{connects to}} \{?y\}$, the relation embedding *bus* should always return a lower plausibility than *connects to*. Thus, for all such rules, FSL proposes to train relation embeddings while avoiding violations of such inequalities. While relatively straightforward, FSL only supports simple rules, while KALE also supports more complex rules. These works exemplify how deductive and inductive forms of knowledge – in this case rules and embeddings – can interplay and complement each other.

5.3 GRAPH NEURAL NETWORKS

While embeddings aim to provide a dense numerical representation of graphs suitable for use within existing machine learning models, another approach is to build custom machine learning models.

Give us feedback!

adapted for graph-structured data. Most custom learning models for graphs are based on (artificial) neural networks [Wu et al., 2019], exploiting a natural correspondence between both: a neural network already corresponds to a weighted, directed graph, where nodes serve as artificial neurons, and edges serve as weighted connections (axons). However, the typical topology of a traditional neural network – more specifically, a fully-connected feed-forward neural network – is quite homogeneous, being defined in terms of sequential layers of nodes where each node in one layer is connected to all nodes in the next layer. Conversely, the topology of a data graph is quite heterogeneous, being determined by the relations between entities that its edges represent.

A *graph neural network* (GNN) [Scarselli et al., 2009] builds a neural network based on the topology of the data graph; i.e., nodes are connected to their neighbours per the data graph. Typically a model is then learnt to map input features for nodes to output features in a supervised manner; output features of the example nodes used for training may be manually labelled, or may be taken from the knowledge graph. Unlike knowledge graph embeddings, GNNs support end-to-end supervised learning for specific tasks: given a set of labelled examples, GNNs can be used to classify elements of the graph or the graph itself. GNNs have been used to perform classification over graphs encoding compounds, objects in images, documents, etc.; as well as to predict traffic, build recommender systems, verify software, etc. [Wu et al., 2019]. Given labelled examples, GNNs can even replace graph algorithms; for example, GNNs have been used to find central nodes in knowledge graphs in a supervised manner [Scarselli et al., 2009, Park et al., 2019, Park et al., 2020].

We now discuss the ideas underlying two main flavours of GNN, specifically, *recursive GNNs* and *non-recursive GNNs*.

5.3.1 RECURSIVE GRAPH NEURAL NETWORKS

Recursive graph neural networks (RecGNNs) are the seminal approach to graph neural networks [Sperduti and Starita, 1997, Scarselli et al., 2009]. The approach is conceptually similar to the systolic abstraction illustrated in Figure 5.3, where messages are passed between neighbours towards recursively computing some result. However, rather than define the functions used to decide the messages to pass, we rather label the output of a training set of nodes and let the framework learn the functions that generate the expected output, thereafter applying them to label other examples.

In a seminal paper, Scarselli et al. [2009] proposed what they generically call a graph neural network (GNN), which takes as input a directed graph where nodes and edges are associated with *feature vectors* that can capture node and edge labels, weights, etc. These feature vectors remain fixed throughout the process. Each node in the graph is also associated with a *state vector*, which is recursively updated based on information from the node's neighbours – i.e., the feature and state vectors of the neighbouring nodes and the feature vectors of the edges extending to/from them – using a parametric function, called the *transition function*. A second parametric function, called the *output function*, is used to compute the final output for a node based on its own feature and state vector. These functions are applied recursively up to a fixpoint. Both parametric functions can be implemented using neural networks where, given a partial set of *supervised nodes* in the graph – i.e., nodes labelled with their desired output – parameters for the transition and output functions can be learnt that best approximate the supervised outputs. The result can thus be seen as a recursive neural network architecture.²⁸ To ensure convergence up to a fixpoint, certain restrictions are applied, namely that the transition function be a *contractor*, meaning that upon each application of the function, points in the numeric space are brought closer together (intuitively, in this case, the numeric space “shrinks” upon each application, ensuring convergence to a unique fixpoint).

To illustrate, consider, for example, that we wish to find priority locations for creating new tourist information offices. A good strategy would be to install them in hubs from which many tourists visit popular destinations. Along these lines, in Figure 5.7 we illustrate the GNN architecture proposed by Scarselli et al. [2009] for a sub-graph of Figure 5.2, where we highlight the neighbourhood of Punta Arenas. In this graph, nodes are annotated with feature vectors ($\mathbf{n}_x$) and hidden states at step t ($\mathbf{h}_x^{(t)}$), while edges are annotated with feature vectors ($\mathbf{a}_{xy}$). Feature vectors for nodes may, for example, one-hot encode the type of node (*City*, *Attraction*, etc.), directly encode statistics such as the number of tourists visiting per year, etc. Feature vectors for edges may, for example, one-hot encode the edge label (the type of transport), directly encode statistics such as the distance or number

note 28 Some authors indicate that the internal over a sequence of tra

Give us feedback!

tickets sold per year, etc. Hidden states can be randomly initialised. The right-hand side of Figure 5.7 provides the GNN transition and output functions, where $N(x)$ denotes the neighbouring nodes of x , $f_w(\cdot)$ denotes the transition function with parameters w , and $g_{w'}(\cdot)$ denotes the output function with parameters w' . An example is also provided for Punta Arenas ($x = 1$). These functions will be recursively applied until a fixpoint is reached. To train the network, we can label examples of places that already have (or should have) tourist offices and places that do (or should) not have tourist offices. These labels may be taken from the knowledge graph, or may be added manually. The GNN can then learn parameters w and w' that give the expected output for the labelled examples, which can subsequently be used to label other nodes.

This GNN model is flexible and can be adapted in various ways [Scarselli et al., 2009]: we may define neighbouring nodes differently, for example to include nodes for outgoing edges, or nodes one or two hops away; we may allow pairs of nodes to be connected by multiple edges with different vectors; we may consider transition and output functions with distinct parameters for each node; we may add states and outputs for edges; we may change the sum to another aggregation function; etc.

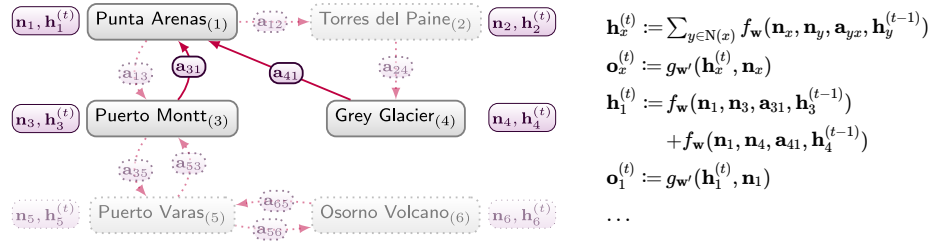


Figure 5.7: On the left a sub-graph of Figure 5.2 highlighting the neighbourhood of Punta Arenas, where nodes are annotated with feature vectors ($\mathbf{n}_x$) and hidden states at step t ($\mathbf{h}_x^{(t)}$), and edges are annotated with feature vectors ($\mathbf{a}_{xy}$); on the right, the GNN transition and output functions proposed by Scarselli et al. [2009] and an example for Punta Arenas ($x = 1$), where $N(x)$ denotes the neighbouring nodes of x , $f_w(\cdot)$ denotes the transition function with parameters w and $g_{w'}(\cdot)$ denotes the output function with parameters w'

We now define a recursive graph neural network. We assume that the GNN accepts a directed vector-labelled graph as input (see Definition 5.1).

Definition 5.13 (Recursive graph neural network)

A recursive graph neural network (RecGNN) is a pair of functions $\mathfrak{R} := (\text{AGG}, \text{OUT})$, such that (with $a, b, c \in \mathbb{N}$):

- $\text{AGG}: \mathbb{R}^a \times 2^{(\mathbb{R}^a \times \mathbb{R}^b) \rightarrow \mathbb{N}} \rightarrow \mathbb{R}^a$
- $\text{OUT}: \mathbb{R}^a \rightarrow \mathbb{R}^c$

The function AGG computes a new feature vector for a node, given its previous feature vector and the feature vectors of the nodes and edges forming its neighbourhood; the function OUT transforms the final feature vector computed by AGG for a node to the output vector for that node. We assume that a and b correspond to the dimensions of the input node and edge vectors, respectively, while c denotes the dimension of the output vector for each node. Given a RecGNN $\mathfrak{R} = (\text{AGG}, \text{OUT})$, a directed vector-labelled graph $G = (V, E, F, \lambda)$, and a node $u \in V$, we define the output vector assigned to node u in G by $\mathfrak{R}$ (written $\mathfrak{R}(G, u)$) as follows. First let $\mathbf{n}_u^{(0)} := \lambda(u)$. For all $i \geq 1$, let:

$$\mathbf{n}_u^{(i)} := \text{AGG} \left(\mathbf{n}_u^{(i-1)}, \{ \{ \mathbf{n}_v^{(i-1)}, \lambda(v, u) \} \mid (v, u) \in E \} \right)$$

If $j \geq 1$ is an integer such that $\mathbf{n}_u^{(j)} = \mathbf{n}_u^{(j-1)}$ for all $u \in V$, then $\mathfrak{R}(G, u) := \text{OUT}(\mathbf{n}_u^{(j)})$.

In a RecGNN, the same aggregation function (AGG) is applied recursively until a fixpoint is reached, at which point an output function (OUT) creates the final output vector for each node. While in practice RecGNNs will often consider a static feature vector and a dynamic state vector [Scarselli et al., 2009], we can more concisely encode this as one vector, where part may remain static throughout the aggregation process representing input features, and part may be dynamically computed representing the state. In practice, AGG and OUT are often based on para-

metric combinations of vectors, with the parameters learnt based on a sample of output vectors for labelled nodes.

Example 5.7 — The aggregation function for the GNN of Scarselli et al. [2009] is given as:

$$\text{AGG}(\mathbf{n}_u, N) := \sum_{(\mathbf{n}_v, \mathbf{a}_{vu}) \in N} f_{\mathbf{w}}(\mathbf{n}_u, \mathbf{n}_v, \mathbf{a}_{vu})$$

where $f_{\mathbf{w}}(\cdot)$ is a contraction function with parameters $\mathbf{w}$. The output function is defined as:

$$\text{OUT}(\mathbf{n}_u) := g_{\mathbf{w}'}(\mathbf{n}_u)$$

where again $g_{\mathbf{w}'}(\cdot)$ is a function with parameters $\mathbf{w}'$. Given a set of nodes labelled with their expected output vectors, the parameters $\mathbf{w}$ and $\mathbf{w}'$ are learnt.

There are notable similarities between graph parallel frameworks (GPFs; see Definition 5.2) and RecGNNs. While we defined GPFs using separate MSG and AGG functions, this is not essential: conceptually they could be defined in a similar way to RecGNN, with a single AGG function that “pulls” information from its neighbours (we maintain MSG to more closely reflect how GPFs are defined/implemented in practice). The key difference between GPFs and GNNs is that in the former, the functions are defined by the user, while in the latter, the functions are generally learnt from labelled examples. Another difference arises from the termination condition present in GPFs, though often the GPF’s termination condition will – like in RecGNNs – reflect convergence to a fixpoint.

5.3.2 NON-RECURSIVE GRAPH NEURAL NETWORKS

GNNs can also be defined in a non-recursive manner, where a fixed number of layers are applied over the input in order to generate the output. A benefit of this approach is that we do not need to worry about convergence since the process is non-recursive. Also, each layer will often have independent parameters, representing different transformation steps. Naively, a downside is that adding many layers could give rise to a high number of parameters. Addressing this problem, a popular approach for non-recursive GNNs is to use convolutional neural networks.

Convolutional neural networks (CNNs) have gained a lot of attention, in particular, for machine learning tasks involving images [Krizhevsky et al., 2017]. The core idea in the image setting is to train and apply small kernels (aka filters) over localised regions of an image using a convolution operator to extract features from that local region. When applied to all local regions, the convolution outputs a feature map of the image. Since the kernels are small, and are applied multiple times to different regions of the input, the number of parameters to train is reduced. Typically multiple kernels can thus be applied, forming multiple convolutional layers.

One may note that in GNNs and CNNs, operators are applied over local regions of the input data. In the case of GNNs, the transition function is applied over a node and its neighbours in the graph. In the case of CNNs, the convolution is applied on a pixel and its neighbours in the image. Following this intuition, a number of *convolutional graph neural networks (ConvGNNs)* [Bruna et al., 2014, Kipf and Welling, 2017, Wu et al., 2019] have been proposed, where the transition function is implemented by means of convolutions. A key consideration for ConvGNNs is how regions of a graph are defined. Unlike the pixels of an image, nodes in a graph may have varying numbers of neighbours. This creates a challenge: a benefit of CNNs is that the same kernel can be applied over all the regions of an image, but this requires more careful consideration in the case of ConvGNNs since neighbourhoods of different nodes can be diverse. Approaches to address these challenges involve working with spectral (e.g. [Bruna et al., 2014, Kipf and Welling, 2017]) or spatial (e.g., [Monti et al., 2017]) representations of graphs that induce a more regular structure from the graph. An alternative is to use an attention mechanism [Velickovic et al., 2018] to learn the nodes whose features are most important to the current node.

Next we abstractly define a non-recursive graph neural network.

Definition 5.14 (Non-recursive graph neural network)

A non-recursive graph neural network (NRecGNN) with l layers is an l -tuple of functions $\mathcal{N} := (\text{AGG}^{(1)}, \dots, \text{AGG}^{(l)})$, such that, for $1 \leq k \leq l$ (with $a_0, \dots, a_l, b \in \mathbb{N}$), $\text{AGG}^{(k)} : \mathbb{R}^{a_{k-1}} \times 2^{(\mathbb{R}^{a_{k-1}} \times \mathbb{R}^b) \rightarrow \mathbb{N}} \rightarrow \mathbb{R}^{a_k}$.

Each function $\text{AGG}^{(k)}$ (as before) computes a new feature vector for a node, given its previous feature vector and the feature vectors of the nodes and edges forming its neighbourhood. We as-

Give us feedback!

sume that a_0 and b correspond to the dimensions of the input node and edge vectors, respectively, where each function $\text{AGG}^{(k)}$ for $2 \leq k \leq l$ accepts as input node vectors of the same dimension as the output of the function $\text{AGG}^{(k-1)}$. Given an NRecGNN $\mathfrak{N} = (\text{AGG}^{(1)}, \dots, \text{AGG}^{(l)})$, a directed vector-labelled graph $G = (V, E, F, \lambda)$, and a node $u \in V$, we define the output vector assigned to node u in G by $\mathfrak{N}$ (written $\mathfrak{N}(G, u)$) as follows. First let $\mathbf{n}_u^{(0)} := \lambda(u)$. For all $i \geq 1$, let:

$$\mathbf{n}_u^{(i)} := \text{AGG}^{(i)} \left(\mathbf{n}_u^{(i-1)}, \{ \{ \mathbf{n}_v^{(i-1)}, \lambda(v, u) \} \mid (v, u) \in E \} \right)$$

Then $\mathfrak{N}(G, u) := \mathbf{n}_u^{(l)}$.

In an l -layer NRecGNN, a different aggregation function can be applied at each step (i.e., in each layer), up to a fixed number of steps l . We do not consider a separate OUT function as it can be combined with the final aggregation function $\text{AGG}^{(l)}$. When the aggregation functions use a convolutional operator based on kernels learned from labelled examples, we call the result a *convolutional graph neural network* (ConvGNN). We refer to the survey by Wu et al. [2019] for discussion of ConvGNNs proposed in the literature.

We have considered GNNs that define the neighbourhood of a node based on its incoming edges. These definitions can be adapted to also consider outgoing neighbours by either adding inverse edges to the directed vector-labelled graph in pre-processing, or by adding outgoing neighbours as arguments to the $\text{AGG}(\cdot)$ function. More generally, GNNs (and indeed GPFs) relying solely on the neighbourhood of each node have limited expressivity in terms of their ability to distinguish nodes and graphs [Xu et al., 2019]; for example, Barceló et al. [2020] show that such NRecGNNs have a similar expressiveness for classifying nodes as the $\mathcal{ALCQ}$ Description Logic discussed in Section 4.3.2. More expressive GNN variants have been proposed that allow the aggregation functions to access and update a globally shared vector [Barceló et al., 2020]. We refer to the papers by Xu et al. [2019] and Barceló et al. [2020] for further discussion.

5.4 SYMBOLIC LEARNING

The supervised techniques discussed thus far – namely knowledge graph embeddings and graph neural networks – learn numerical models over graphs. However, such models are often difficult to explain or understand. For example, taking the graph of Figure 5.8, knowledge graph embeddings might predict the edge $(\text{SCL} \text{--} \text{flight} \rightarrow \text{ARI})$ as being highly plausible, but they will not provide an interpretable model to help understand why this is the case: the reason for the result may lie in a matrix of parameters learnt to fit a plausibility score on training data. Such approaches also suffer from the *out-of-vocabulary* problem, where they are unable to provide results for edges involving previously unseen nodes or edges; for example, if we add an edge $(\text{SCL} \text{--} \text{flight} \rightarrow \text{CDG})$, where (CDG) is new to the graph, a knowledge graph embedding will not have the entity embedding for (CDG) and would need to be retrained in order to estimate the plausibility of an edge $(\text{CDG} \text{--} \text{flight} \rightarrow \text{SCL})$.

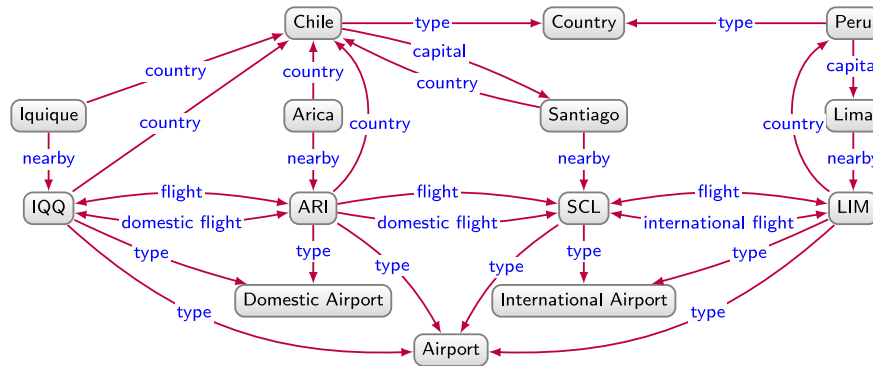


Figure 5.8: A directed edge-labelled graph describing flights between airports

An alternative (sometimes complementary) approach is to adopt *symbolic learning* in order to learn *hypotheses* in a symbolic (logical) language that “explain” a given set of positive and negative edges. These edges are typically generated from the knowledge graph in an automatic manner (similar to the case of knowledge graph embeddings). The hypotheses then serve as interpretable models that can be used for further deductive reasoning. Given the graph of Figure 5.8, we may, for exam

learn the rule $\langle \tilde{x} \rangle \text{--flight--} \langle \tilde{y} \rangle \Rightarrow \langle \tilde{y} \rangle \text{--flight--} \langle \tilde{x} \rangle$ from observing that *flight* routes tend to be return routes. Alternatively, rather than learn rules, we might learn a DL axiom from the graph stating that airports are either domestic, international, or both: $\text{Airport} \sqsubseteq \text{DomesticAirport} \sqcup \text{InternationalAirport}$. Such rules and axioms can then be used for deductive reasoning, and offer an interpretable model for new knowledge that is entailed/predicted; for example, from the aforementioned rule for return flights, one can interpret why a novel edge $\langle \text{SCL} \rangle \text{--flight--} \langle \text{ARI} \rangle$ is predicted. This further offers domain experts the opportunity to verify the models – e.g., the rules and axioms – derived by such processes. Finally, rules/axioms are quantified (*all* flights have a return flight, *all* airports are domestic or international, etc.), so they can be applied to unseen examples (e.g., with the aforementioned rule, we can derive $\langle \text{CDG} \rangle \text{--flight--} \langle \text{SCL} \rangle$ from a new edge $\langle \text{SCL} \rangle \text{--flight--} \langle \text{CDG} \rangle$ with the unseen node $\langle \text{CDG} \rangle$).

In this section, we discuss two forms of symbolic learning: *rule mining*, which learns rules, and *axiom mining*, which learns other forms of logical axioms.

5.4.1 RULE MINING

Rule mining, in the general sense, refers to discovering meaningful patterns in the form of rules from large collections of background knowledge. In the context of knowledge graphs, we assume a set of positive and negative edges as given. Typically positive edges are observed edges (i.e., those given or entailed by a knowledge graph) while negative edges are defined according to a given assumption of completeness (discussed later). The goal of rule mining is to identify new rules that entail a high ratio of positive edges from other positive edges, but entail a low ratio of negative edges from positive edges. The types of rules considered may vary from more simple cases, such as $\langle \tilde{x} \rangle \text{--flight--} \langle \tilde{y} \rangle \Rightarrow \langle \tilde{y} \rangle \text{--flight--} \langle \tilde{x} \rangle$ mentioned previously, to more complex rules, such as $\langle \tilde{x} \rangle \text{--capital--} \langle \tilde{y} \rangle \text{--nearby--} \langle \tilde{z} \rangle \text{--type--} \langle \text{Airport} \rangle \Rightarrow \langle \tilde{z} \rangle \text{--type--} \langle \text{International Airport} \rangle$, based on observing in the graph that airports near capitals tend to be international airports; or $\langle \tilde{x} \rangle \text{--flight--} \langle \tilde{y} \rangle \text{--country--} \langle \tilde{z} \rangle \Rightarrow \langle \tilde{x} \rangle \text{--domestic flight--} \langle \tilde{y} \rangle$, indicating that flights within the same country denote domestic flights (as seen previously in Section 4.3.1).

Per the example inferring that airports near capital cities are international airports, rules are not assumed to hold in all cases, but rather are associated with measures of how well they conform to the positive and negative edges. In more detail, we call the edges entailed by a rule and the set of positive edges (not including the entailed edge itself), the *positive entailments* of that rule. The number of entailments that are positive is called the *support* for the rule, while the ratio of a rule's entailments that are positive is called the *confidence* for the rule [Suchanek et al., 2019]. Support and confidence indicate, respectively, the number and ratio of entailments “confirmed” to be true for the rule, where the goal is to identify rules that have both high support and high confidence. Techniques for rule mining in relational settings have long been explored in the context of *Inductive Logic Programming (ILP)* [De Raedt, 2008]. However, knowledge graphs present novel challenges due to the scale of the data and the frequent assumption of incomplete data (OWA), where dedicated techniques have been proposed to address these issues [Galárraga et al., 2013].

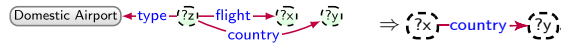
When dealing with an incomplete knowledge graph, it is not immediately clear how to define negative edges. A common heuristic – also used for knowledge graph embeddings – is to adopt a Partial Completeness Assumption (PCA) [Galárraga et al., 2013], which considers the set of positive edges to be those contained in the data graph, and the set of negative examples to be the set of all edges $\langle x \rangle \text{--}p\text{--} \langle y \rangle$ not in the graph but where there exists a node $\langle y' \rangle$ such that $\langle x \rangle \text{--}p\text{--} \langle y' \rangle$ is in the graph. Taking Figure 5.8, an example of a negative edge under PCA would be $\langle \text{SCL} \rangle \text{--flight--} \langle \text{ARI} \rangle$ (given the presence of $\langle \text{SCL} \rangle \text{--flight--} \langle \text{LIM} \rangle$); conversely, $\langle \text{SCL} \rangle \text{--domestic flight--} \langle \text{ARI} \rangle$ is neither positive nor negative. The PCA confidence measure is then the ratio of the support to the number of entailments in the positive or negative set [Galárraga et al., 2013]. For example, the support for the rule $\langle \tilde{x} \rangle \text{--domestic flight--} \langle \tilde{y} \rangle \Rightarrow \langle \tilde{y} \rangle \text{--domestic flight--} \langle \tilde{x} \rangle$ is 2 (since it entails $\langle \text{IQQ} \rangle \text{--domestic flight--} \langle \text{ARI} \rangle$ and $\langle \text{ARI} \rangle \text{--domestic flight--} \langle \text{IQQ} \rangle$ in the graph, which are thus positive edges), while the confidence is $\frac{2}{2} = 1$ (noting that $\langle \text{SCL} \rangle \text{--domestic flight--} \langle \text{ARI} \rangle$, though entailed, is neither positive nor negative, and is thus ignored by the measure). The support for the rule $\langle \tilde{x} \rangle \text{--flight--} \langle \tilde{y} \rangle \Rightarrow \langle \tilde{y} \rangle \text{--flight--} \langle \tilde{x} \rangle$ is analogously 4, while the confidence is $\frac{4}{5} = 0.8$ (noting that $\langle \text{SCL} \rangle \text{--flight--} \langle \text{ARI} \rangle$ is a negative edge).

The goal then, is to find rules satisfying given support and confidence thresholds. An influential rule-mining system for graphs is AMIE [Galárraga et al., 2013, Galárraga et al., 2015], which adopts the PCA measure of confidence, and builds rules in a top-down fashion [Suchanek et al., 2019] starting

Give us feedback!

with rule heads of the form $\Rightarrow \{?x\} \text{--country--} \{?y\}$. For each such rule head (one for each edge label), three types of *refinements* are considered, each of which adds a new edge to the body of the rule. This new edge takes an edge label from the graph and may otherwise use *fresh variables* not appearing previously in the rule, *existing variables* that already appear in the rule, or nodes from the graph. The three refinements may then:

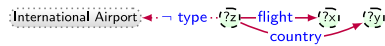
1. add an edge with one existing variable and one fresh variable; for example, refining the aforementioned rule head might give: $\{?z\} \text{--flight--} \{?x\} \Rightarrow \{?x\} \text{--country--} \{?y\}$;
2. add an edge with an existing variable and a graph node; for example, refining the above rule might give: $\text{Domestic Airport} \text{--type--} \{?z\} \text{--flight--} \{?x\} \Rightarrow \{?x\} \text{--country--} \{?y\}$;
3. add an edge with two existing variables; for example, refining the above rule might give:



These refinements can be combined arbitrarily, which gives rise to a potentially exponential search space, where rules meeting given thresholds for support and confidence are maintained. To improve efficiency, the search space can be pruned; for example, these three refinements always decrease support, so if a rule does not meet the support threshold, there is no need to explore its refinements. Further restrictions are imposed on the types of rules generated. First, only rules up to a certain fixed size are considered. Second, a rule must be *closed*, meaning that each variable appears in at least two edges of the rule, which ensures that rules are *safe*, meaning that each variable in the head appears in the body; for example, the rules produced by the first and second refinements in the example are neither closed (variable $\{?y\}$ appears once) nor safe (variable $\{?y\}$ appears only in the head).²⁹ The third refinement is thus applied until a rule is closed. For further discussion of possible optimisations based on pruning and indexing, we refer to the paper [Galárraga et al., 2015].

note 29 Safe rule: International Airport are closed is strictly st

Later works have built on these techniques for mining rules from knowledge graphs. Gad-Elrab et al. [2016] propose a method to learn non-monotonic rules – rules with negated edges in the body – in order to capture exceptions to base rules; for example, the rule



$\Rightarrow \{?x\} \text{--country--} \{?y\}$ may be learnt, indicating that flights

are within the same country *except* when the (departure) airport is international, where the exception is shown dotted and we use $\neg$ to negate an edge (representing an exception). The RuLES system [Ho et al., 2018] – which is also capable of learning non-monotonic rules – proposes to mitigate the limitations of the PCA heuristic by extending the confidence measure to consider the plausibility scores of knowledge graph embeddings for entailed edges not appearing in the graph. Where available, explicit statements about the completeness of the knowledge graph (such as expressed in shapes; see Section 3.1.2) can be used in lieu of PCA for identifying negative edges. Along these lines, CARL [Pellissier Tanon et al., 2017] exploits additional knowledge about the cardinalities of relations to refine the set of negative examples and the confidence measure for candidate rules. Alternatively, where available, ontologies can be used to derive logically-certain negative edges under OWA through, for example, disjointness axioms. The system proposed by d'Amato et al. [d'Amato et al., 2016b, d'Amato et al., 2016a] leverages ontologically-entailed negative edges for determining the confidence of rules generated through an evolutionary algorithm.

While the previous works involve discrete expansions of candidate rules for which a fixed confidence scoring function is applied, another line of research is on a technique called *differentiable rule mining* [Rocktäschel and Riedel, 2017, Yang et al., 2017, Sadeghian et al., 2019], which allows end-to-end learning of rules. The core idea is that the joins in rule bodies can be represented as matrix multiplication. More specifically, we can represent the relations of an edge label p by the adjacency matrix $\mathbf{A}_p$ (of size $|V| \times |V|$) such that the value on the i^{th} row of the j^{th} column is 1 if there is an edge labelled p from the i^{th} entity to the j^{th} entity; otherwise the value is 0. Now we can represent a join in a rule body as matrix multiplication; for example, given $\{?x\} \text{--domestic flight--} \{?y\} \text{--country--} \{?z\} \Rightarrow \{?x\} \text{--country--} \{?z\}$, we can denote the body by the matrix multiplication $\mathbf{A}_{\text{df}} \mathbf{A}_{\text{c}}$, which gives an adjacency matrix representing entailed *country* edges, where we should expect the 1's in $\mathbf{A}_{\text{df}} \mathbf{A}_{\text{c}}$ to be covered by the head's adjacency matrix $\mathbf{A}_{\text{c}}$. Since we are given adjacency matrices for all edge labels, we are left to learn confidence scores for individual rules, and to learn rules (of varying length) with a threshold confidence. Along these lines, NeuralLP [Yang et al., 2017] uses an *attention mechanism* to select a variable-length sequence of edge labels for path-like rules of the form $\{?x\} \text{--} p_1 \text{--} \{?y_1\} \text{--} p_2 \text{--} \dots$

Give us feedback!

$\rightarrow(y_n) \rightarrow p_{n+1} \rightarrow(z_z) \Rightarrow (x) \rightarrow p \rightarrow(z_z)$, for which confidences are likewise learnt. DRUM [Sadeghian et al., 2019] also learns path-like rules, where, observing that some edge labels are more/less likely to follow others in the rules – for example, *flight* will not be followed by *capital* in the graph of Figure 5.2 as the join will be empty – the system uses bidirectional recurrent neural networks (a popular technique for learning over sequential data) to learn sequences of relations for rules, and their confidences. These differentiable rule mining techniques are, however, currently limited to learning path-like rules.

5.4.2 AXIOM MINING

More general forms of axioms beyond rules – expressed in logical languages such as DLs (see Section 4.3.2) – can be mined from knowledge graphs. We can divide these approaches into two: those mining specific axioms and more general axioms.

Among systems mining specific types of axioms, disjointness axioms are a popular target; for example, $\text{DomesticAirport} \sqcap \text{InternationalAirport} \equiv \perp$ states that the two classes are disjoint by equivalently stating that the intersection of the two classes is equivalent to the empty class, or in simpler terms, no node can be simultaneously of type `Domestic Airport` and `International Airport`. The system proposed by Völker et al. [2015] extracts disjointness axioms based on (negative) *association rule mining* [Agrawal et al., 1993], which finds pairs of classes where each has many instances in the knowledge graph but there are relatively few (or no) instances of both classes. Töpper et al. [2012] rather extract disjointness for pairs of classes that have a cosine similarity below a fixed threshold. For computing this cosine similarity, class vectors are computed using a TF-IDF analogy, where the “document” of each class is constructed from all of its instances, and the “terms” of this document are the properties used on the class instances (preserving multiplicities). While the previous two approaches find disjointness constraints between named classes (e.g., *city* is disjoint with *airport*), Rizzo et al. [2017], Rizzo et al. [2021] propose an approach that can capture disjointness constraints between class descriptions (e.g., *city without an airport nearby* is disjoint with *city that is the capital of a country*). The approach first clusters similar nodes of the knowledge base. Next, a *terminological cluster tree* is extracted, where each leaf node indicates a cluster extracted previously, and each internal (non-leaf) node is a class definition (e.g., *cities*) where the left child is either a cluster having all nodes in that class or a sub-class description (e.g., *cities without airports*) and the right child is either a cluster having no nodes in that class or a disjoint-class description (e.g., *non-cities with events*). Finally, candidate disjointness axioms are proposed for pairs of class descriptions in the tree that are not entailed to have a sub-class relation.

Other systems propose methods to learn more general axioms. One of the first proposals in this direction is the DL-FOIL system [Fanizzi et al., 2008, Rizzo et al., 2020], which is based on algorithms for *class learning* (aka *concept learning*), whereby given a set of positive nodes and negative nodes, the goal is to find a logical class description that divides the positive and negative sets. For example, given $\{\text{Iquique}, \text{Arica}\}$ as the positive set and $\{\text{Santiago}\}$ as the negative set, we may learn a (DL) class description $\exists \text{nearby.Airport} \sqcap \neg(\exists \text{capital} \sqcap \top)$, denoting entities near to an airport that are not capitals, of which all positive nodes are instances and no negative nodes are instances. Such class descriptions are learnt in an analogous manner to how aforementioned systems like AMIE learn rules, with a refinement operator used to move from more general classes to more specific classes (and vice-versa), a confidence scoring function, and a search strategy. Another prominent such system is DL-Learner [Bühmann et al., 2016], which system further supports learning more general axioms through a scoring function that uses count queries to determine what ratio of expected edges – edges that would be entailed were the axiom true – are indeed found in the graph; for example, to score the axiom $\exists \text{flight} \sqcap \text{DomesticAirport} \sqsubseteq \text{InternationalAirport}$ over Figure 5.8, we can use a graph query to count how many nodes have incoming flights from a domestic airport (there are 3), and how many nodes have incoming flights from a domestic airport *and* are international airports (there is 1), where the greater the difference between both counts, the weaker the evidence for the axiom.

5.4.3 HYPOTHESIS MINING

We now provide some abstract formal definitions for the tasks of *rule mining* and *axiom mining* over graphs, which we generically refer to as *hypothesis mining*.

First we introduce *hypothesis induction*: a task that captures a more abstract (ideal) case for hypothesis mining. For simplicity, we focus on directed edge-labelled graphs. With a slight abuse of notation, we may interpret a set of edges E as the graph with precisely those edges and with no nodes or labels without edges. We may also interpret an edge e as the graph formed by $\{e\}$.

Definition 5.15 (Hypothesis induction)

The task of hypothesis induction assumes a particular graph entailment relation $\models_\Phi$ (see Definition 4.4; hereafter simply $\models$). Given background knowledge in the form of a knowledge graph G (a directed edge-labelled graph, possibly extended with rules or ontologies), a set of positive edges E^+ such that G does not entail any edge in E^+ (i.e., for all $e^+ \in E^+$, $G \not\models e^+$) and E^+ does not contradict G (i.e., there is a model of $G \cup E^+$), and a set of negative edges E^- such that G does not entail any edge in E^- (i.e., for all $e^- \in E^-$, $G \not\models e^-$), the task is to find a set of hypotheses (i.e., a set of directed edge-labelled graphs) Ψ such that:

- $G \not\models \psi$ for all $\psi \in \Psi$ (the background knowledge does not entail any hypothesis directly);
- $G \cup \Psi^* \models E^+$ (the background knowledge and hypotheses together entail all positive edges);
- for all $e^- \in E^-$, $G \cup \Psi^* \not\models e^-$ (the background knowledge and hypotheses together do not entail any negative edge);
- $G \cup \Psi^* \cup E^+$ has a model (the background knowledge, hypotheses and positive edges taken together do not contain a contradiction);
- for all $e^+ \in E^+$, $\Psi^* \not\models e^+$ (the hypotheses alone do not entail a positive edge).

where by $\Psi^* := \bigcup_{\psi \in \Psi} \psi$ we denote the union of all graphs in Ψ .

Example 5.8 — Let us assume ontological entailment $\models$ with semantic conditions Φ as defined in Tables 4.1–4.3. Given the graph of Figure 5.8 as the background knowledge G , along with:

- a set of positive edges $E^+ = \{(\text{SCL} \text{--} \text{flight} \rightarrow \text{ARI}), (\text{SCL} \text{--} \text{domestic flight} \rightarrow \text{ARI})\}$, and
- a set of negative edges $E^- = \{(\text{ARI} \text{--} \text{flight} \rightarrow \text{LIM}), (\text{SCL} \text{--} \text{domestic flight} \rightarrow \text{LIM})\}$,

then a set of hypotheses $\Psi = \{(\text{flight} \text{--} \text{type} \rightarrow \text{Symmetric}), (\text{domestic flight} \text{--} \text{type} \rightarrow \text{Symmetric})\}$ are not entailed by G , entail all positive edges in E^+ and no negative edges in E^- when combined with G , do not contradict $G \cup E^+$, and do not entail a positive edge without G . Thus Ψ satisfies the conditions for hypothesis induction.

This task represents a somewhat idealised case. Often there is no set of positive edges distinct from the background knowledge itself. Furthermore, hypotheses not entailing a few positive edges, or entailing a few negative edges, may still be useful. The task of *hypothesis mining* rather accepts as input the background knowledge G and a set of negative edges E^- (such that for all $e^- \in E^-$, $G \not\models e^-$), and attempts to score individual hypotheses ψ (such that $G \not\models \psi$) per their ability to “explain” G while minimising the number of elements of E^- entailed by G and ψ . We can now abstractly define the task of hypothesis mining.

Definition 5.16 (Hypothesis mining)

Given a knowledge graph G , a set of negative edges E^- , a scoring function σ , and a threshold $\min_\sigma$, the goal of hypothesis mining is to identify a set of hypotheses $\{\psi \mid G \not\models \psi \text{ and } \sigma(\psi, G, E^-) \geq \min_\sigma\}$.

There are two scoring functions that are frequently used for σ in the literature: *support* and *confidence*.

Definition 5.17 (Hypothesis support and confidence)

Given a knowledge graph $G = (V, E, L)$ and a hypothesis ψ , the positive support of ψ is defined as:

$$\sigma^+(\psi, G) := |\{e \in E \mid G' \not\models e \text{ and } G' \cup \psi \models e\}|$$

where G' denotes G with the edge e removed. Further given a set of negative edges E^- , the negative support of ψ is defined as:

$$\sigma^-(\psi, G, E^-) := |\{e^- \in E^- \mid G \cup \psi \models e^-\}|$$

Finally, the confidence of ψ is defined as $\sigma^+(\psi, G, E^-) := \frac{\sigma^+(\psi, G)}{\sigma^+(\psi, G) + \sigma^-(\psi, G, E^-)}$.

We have yet to define how the set of negative edges are defined, which, in the context of a knowledge graph G , depends on which assumption is applied:

- *Closed world assumption (CWA)*: For any (positive) edge e , $G \not\models e$ if and only if $G \models \neg e$. Under CWA, any edge e not entailed by G can be considered a negative edge.
- *Open world assumption*: For a (positive) edge e , $G \not\models e$ does not necessarily imply $G \models \neg e$. Under OWA, the negation of an edge must be entailed by G for it to be considered negative.
- *Partial completeness assumption (PCA)*: If there exists an edge (s, p, o) such that $G \models (s, p, o)$, then for all o' such that $G \not\models (s, p, o')$, it is assumed that $G \models \neg(s, p, o')$. Under PCA, if G entails some outgoing edge(s) labelled p from a node s , then such edges are assumed to be complete, and any edge (s, p, o') not entailed by G can be considered a negative edge.

Knowledge graphs are generally incomplete – in fact, one of the main applications of hypothesis mining is to try to improve the completeness of the knowledge graph – and thus it would appear unwise to assume that any edge that is not currently entailed is false/negative. We can thus rule out CWA. Conversely, under OWA, potentially few (or no) negative edges might be entailed by the given ontologies/rules, and thus hypotheses may end up having low negative support despite entailing many edges that do not make sense in practice. Hence the PCA can be adopted as a heuristic to increase the number of negative edges and apply more sensible scoring of hypotheses. We remark that one can adapt PCA to define negative triples by changing the subject or predicate instead of the object.

Different implementations of hypothesis mining may consider different logical languages. Rule mining, for example, mines hypotheses expressed either as monotonic rules (with positive edges) or non-monotonic edges (possibly with negated edges). On the other hand, axiom mining considers hypotheses expressed in a logical language such as Description Logics. Particular implementations may, for practical reasons, impose further syntactic restrictions on the hypotheses generated, such as to impose thresholds on their length, on the symbols they use, or on other structural properties (such as “closed rules” in the case of the AMIE rule mining system [Galárraga et al., 2013]; see Section 5.4). Systems may further implement different search strategies for hypotheses. Systems such as DL-FOIL [Fanizzi et al., 2008, Rizzo et al., 2020], AMIE [Galárraga et al., 2013], RuLES [Ho et al., 2018], CARL [Pellissier Tanon et al., 2017], DL-Learner [Bühmann et al., 2016], etc., propose *discrete mining* that recursively generates candidate formulae through refinement/genetic operators that are then scored and checked for threshold criteria. On the other hand, systems such as NeuralLP [Yang et al., 2017] and DRUM [Sadeghian et al., 2019] apply *differentiable mining* that allows for learning (path-like) rules and their scores in a more continuous fashion (e.g., using gradient descent). We refer to Section 5.4 for further discussion and examples of such techniques for mining hypotheses.

Creation and Enrichment

In this chapter, we discuss the principal techniques by which knowledge graphs can be created and subsequently enriched from diverse sources of legacy data that may range from plain text to structured formats (and anything in between). The appropriate methodology to follow when creating a knowledge graph depends on the actors involved, the domain, the envisaged applications, the available data sources, etc. Generally speaking, however, the flexibility of knowledge graphs lends itself to starting with an initial core that can be incrementally enriched from other sources as required (typically following an Agile [Hunt and Thomas, 2003] or “pay-as-you-go” [Sequeda et al., 2019] methodology). For our running example, we assume that the tourism board decides to build a knowledge graph from scratch, aiming to initially describe the main tourist attractions – places, events, etc. – in Chile in order to help visiting tourists identify those that most interest them. The board decides to postpone adding further data, like transport routes, reports of crime, etc., for a later date.

6.1 HUMAN COLLABORATION

One approach for creating and enriching knowledge graphs is to solicit direct contributions from human editors. Such editors may be found in-house (e.g., employees of the tourist board), using crowd-sourcing platforms, through feedback mechanisms (e.g., tourists adding comments on attractions), through collaborative-editing platforms (e.g., an attractions wiki open to public edits), etc. Though human involvement incurs high costs [Paulheim, 2018], some prominent knowledge graphs have been primarily based on direct contributions from human editors [Vrandečić and Krötzsch, 2014, He et al., 2016]. Depending on how the contributions are solicited, however, the approach has a number of key drawbacks, due primarily to human error [Pellissier Tanon et al., 2016], disagreement [Yasseri et al., 2012], bias [Janowicz et al., 2018], vandalism [Heindorf et al., 2016], etc. Successful collaborative creation further raises challenges concerning licensing, tooling, and culture [Pellissier Tanon et al., 2016]. Humans are sometimes rather employed to verify and curate additions to a knowledge graph extracted by other means [Pellissier Tanon et al., 2016] (through, e.g., video games with a purpose [Jurgens and Navigli, 2014]), to define high-quality mappings from other sources [Das et al., 2012], to define appropriate high-level schema [Keet, 2018, Labra Gayo et al., 2018], and so forth.

6.2 TEXT SOURCES

Text corpora – such as sourced from newspapers, books, scientific articles, social media, emails, web crawls, etc. – are an abundant source of rich information [Hellmann et al., 2013, Rospocher et al., 2016]. However, extracting such information with high precision and recall for the purposes of creating or enriching a knowledge graph is a non-trivial challenge. To address this, techniques from Natural Language Processing (NLP) [Maynard et al., 2016, Jurafsky and Martin, 2019] and Information Extraction (IE) [Weikum and Theobald, 2010, Grishman, 2012, Martínez-Rodríguez et al., 2020] can be applied. Though processes vary considerably across text extraction frameworks, in Figure 6.1 we illustrate four core tasks for text extraction on a sample sentence. We will discuss these tasks in turn.

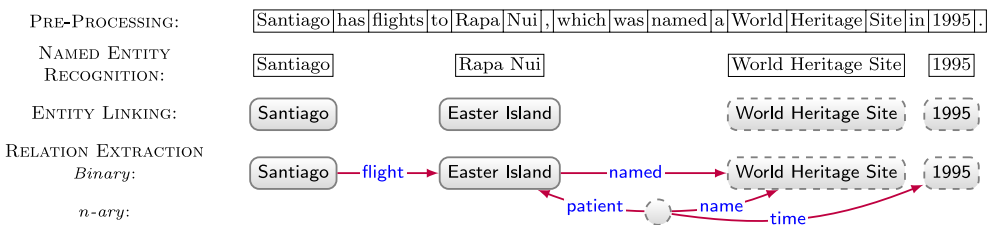


Figure 6.1: Text extraction example; dashed nodes are new to the knowledge graph

Give us feedback!

6.2.1 PRE-PROCESSING

The pre-processing task may involve applying various techniques to the input text, where Figure 6.1 illustrates *Tokenisation*, which parses the text into atomic terms and symbols. Other pre-processing tasks applied to a text corpus may include: *Part-of-Speech (POS) tagging* [Maynard et al., 2016, Jurafsky and Martin, 2019] to identify terms representing verbs, nouns, adjectives, etc.; *Dependency Parsing*, which extracts a grammatical tree structure for a sentence where leaf nodes indicate individual words that together form phrases (e.g., noun phrases, verb phrases) and eventually clauses and sentences [Maynard et al., 2016, Jurafsky and Martin, 2019]; and *Word Sense Disambiguation (WSD)* [Navigli, 2009] to identify the meaning (aka *sense*) in which a word is used, linking words with a lexicon of senses (e.g., WordNet [Miller and Fellbaum, 2007] or BabelNet [Navigli and Ponzetto, 2012]), where, for instance, the term `flights` may be linked with the WordNet sense “an instance of travelling by air” rather than “a stairway between one floor and the next”. The appropriate type of pre-processing to apply often depends on the requirements of later tasks in the pipeline.

6.2.2 NAMED ENTITY RECOGNITION (NER)

The NER task identifies mentions of named entities in a text [Nadeau and Sekine, 2007, Ratinov and Roth, 2009], typically targeting mentions of people, organisations, locations, and potentially other types [Ling and Weld, 2012, Nakashole et al., 2013, Yogatama et al., 2015]. A variety of NER techniques exist, with many modern approaches based on learning frameworks that leverage lexical features (e.g., POS tags, dependency parse trees, etc.) and gazetteers (e.g., lists of common first names, last names, countries, prominent businesses, etc.). Supervised methods [Bikel et al., 1999, Finkel et al., 2005, Lample et al., 2016] require manually labelling all entity mentions in a training corpus, whereas *bootstrapping*-based approaches [Collins and Singer, 1999, Etzioni et al., 2004, Nakashole et al., 2013, Gupta and Manning, 2014] rather require a small set of *seed examples* of entity mentions from which patterns can be learnt and applied to unlabelled text. *Distant supervision* [Ling and Weld, 2012, Ren et al., 2015, Yogatama et al., 2015] uses known entities in a knowledge graph as seed examples through which similar entities can be detected. Aside from learning-based frameworks, traditional approaches based on manually-crafted rules [Kluegl et al., 2009, Chiticariu et al., 2018] are still sometimes used due to their more controllable and predictable behaviour [Chiticariu et al., 2013]. The named entities identified by NER may be used to generate new candidate nodes for the knowledge graph (known as *emerging entities*, shown dashed in Figure 6.1), or may be linked to existing nodes per the Entity Linking task described in the following.

6.2.3 ENTITY LINKING (EL)

The EL task associates mentions of entities in a text with the existing nodes of a target knowledge graph, which may be the nucleus of a knowledge graph under creation, or an external knowledge graph [Wu et al., 2018]. In Figure 6.1, we assume that the nodes `(Santiago)` and `(Easter Island)` already exist in the knowledge graph (possibly extracted from other sources). EL may then link the given mentions to these nodes. The EL task presents two main challenges. First, there may be multiple ways to mention the same entity, as in the case of `[Rapa Nui]` and `[Easter Island]`; if we created a node `(Rapa Nui)` to represent that mention, we would split the information available under both mentions across different nodes, where it is thus important for the target knowledge graph to capture the various aliases and multilingual labels by which one can refer to an entity [Moro et al., 2014]. Second, the same mention in different contexts can refer to distinct entities; for instance, `[Santiago]` can refer to cities in Chile, Cuba, Spain, amongst others. The EL task thus considers a *disambiguation phase* where in mentions are associated to candidate nodes in the knowledge graph, the candidates are ranked, and the most likely node being mentioned is chosen [Wu et al., 2018]. Context can be used in this phase; for example, if `(Easter Island)` is a likely candidate for the corresponding mention alongside `[Santiago]`, we may boost the probability that this mention refers to the Chilean capital as both candidates are located in Chile. Other heuristics for disambiguation consider a prior probability, where for example, `[Santiago]` most often refers to the Chilean capital (being, e.g., the largest city with that name); centrality measures on the knowledge graph can be used for such purposes [Wu et al., 2018].

6.2.4 RELATION EXTRACTION (RE)

The RE task extracts relations between entities in the text [Zhou et al., 2005, Bach and Badaskar, 2007]. The simplest case is that of extracting binary relations in a *closed setting* wherein a fixed set of relation types are considered. While traditional approaches often relied on manually-crafted patterns [Hearst, 1992], modern approaches rather tend to use learning-based frameworks [Roller et al., 2018], including supervised methods over manually-labelled examples [Bunescu and Mooney, 2005, Zhou et al., 2005]. Other learning-based approaches again use bootstrapping [Etzioni et al., 2004, Bunescu and Mooney, 2007] and distant supervision [Mintz et al., 2009, Riedel et al., 2010, Hoffmann et al., 2011, Surdeanu et al., 2012, Xu et al., 2013, Smirnova and Cudré-Mauroux, 2019] to forgo the need for manual labelling; the former requires a subset of manually-labelled seed examples, while the latter finds sentences in a large corpus of text mentioning pairs of entities with a known relation/edge, which are used to learn patterns for that relation. Binary RE can also be applied using unsupervised methods in an open setting – often referred to as *Open Information Extraction (OIE)* [Banko et al., 2007, Etzioni et al., 2011, Fader et al., 2011, Mausam et al., 2012, Mausam, 2016, Mitchell et al., 2018] – whereby the set of target relations is not pre-defined but rather extracted from text based on, for example, dependency parse trees from which relations are taken.

A variety of RE methods have been proposed to extract n -ary relations that capture further context for how entities are related. In Figure 6.1, we see how an n -ary relation captures additional temporal context, denoting when Rapa Nui was named a World Heritage site; in this case, an anonymous node is created to represent the higher-arity relation in the directed-labelled graph. Various methods for n -ary RE are based on *frame semantics* [Fillmore, 1976], which, for a given verb (e.g., “named”), captures the entities involved and how they may be interrelated. Resources such as FrameNet [Baker et al., 1998] then define frames for words, which, for example, may identify that the semantic frame for “named” includes a *speaker* (the person naming something), an *entity* (the thing named) and a *name*. Optional frame elements are an *explanation*, a *purpose*, a *place*, a *time*, etc., that may add context to the relation. Other RE methods are rather based on *Discourse Representation Theory (DRT)* [Kamp, 1981], which considers a logical representation of text based on existential events. Under this theory, for example, the naming of Easter Island as a World Heritage Site is considered to be an (existential) event where Easter Island is the *patient* (the entity affected), leading to the logical (neo-Davidsonian) formula:

$$\exists e : (\text{naming}(e), \text{patient}(e, \text{Easter Island}), \text{name}(e, \text{World Heritage Site}))$$

Such a formula is analogous to reification, as discussed previously in Section 3.3, where e is an existential term that refers to the n -ary relation being extracted.

Finally, while relations extracted in a closed setting are typically mapped directly to a knowledge graph, relations that are extracted in an open setting may need to be aligned with the knowledge graph; for example, if an OIE process extracts a binary relation (Santiago) *has flights to* (Easter Island), it may be the case that the knowledge graph does not have other edges labelled *has flights to*, where alignment may rather map such a relation to the edge (Santiago) *flight* (Easter Island) assuming *flight* is used in the knowledge graph. A variety of methods have been applied for performing such alignments, including mappings [Corcoglioniti et al., 2016, Gangemi et al., 2017] and rules [Rouces et al., 2015] for aligning n -ary relations; distributional and dependency-based similarities [Moro and Navigli, 2013], association rule mining [Dutta et al., 2014], Markov clustering [Dutta et al., 2015] and linguistic techniques [Martínez-Rodríguez et al., 2018] for aligning OIE relations; amongst others.

6.2.5 JOINT TASKS

Having presented the four main tasks for building knowledge graphs from text, it is important to note that frameworks do not always follow this particular sequence of tasks. A common trend, for example, is to combine interdependent tasks, jointly performing WSD and EL [Moro et al., 2014], or NER and EL [Luo et al., 2015, Nguyen et al., 2016], or NER and RE [Ren et al., 2017, Zheng et al., 2017], etc., in order to mutually improve the performance of multiple tasks. For further details on extracting knowledge graphs from text we refer to the book by Maynard et al. [2016] and the recent survey by Martínez-Rodríguez et al. [2020].

6.3 MARKUP SOURCES

The Web was founded on interlinking *markup documents* wherein markers (aka *tags*) are used to separate elements of the document (typically for formatting purposes). Most documents on the Web use the HyperText Markup Language (HTML). Figure 6.2 presents an example HTML webpage about World Heritage Sites in Chile. Other formats of markup include Wikitext used by Wikipedia, TeX for typesetting, Markdown used by Content Management Systems, etc. One approach for extracting information from markup documents – in order to create and/or enrich a knowledge graph – is to strip the markers (e.g., HTML tags), leaving only plain text upon which the techniques from the previous section can be applied. However, markup can be useful for extraction purposes, where variations of the aforementioned tasks for text extraction have been adapted to exploit such markup [Lu et al., 2013, Lockard et al., 2018, Martínez-Rodríguez et al., 2020]. We can divide extraction techniques for markup documents into three main categories: general approaches that work independently of the markup used in a particular format, often based on *wrappers* that map elements of the document to the output; focussed approaches that target specific forms of markup in a document, most typically *web tables* (but sometimes also lists, links, etc.); and form-based approaches that extract the data underlying a webpage, per the notion of the *Deep Web*. These approaches can often benefit from the regularities shared by webpages of a given website; for example, intuitively speaking, while the webpage of Figure 6.2 is about Chile, we will likely find pages for other countries following the same structure on the same website.



Figure 6.2: Example markup document (HTML) with source-code (left) and formatted document (right)

6.3.1 WRAPPER-BASED EXTRACTION

Many general approaches are based on *wrappers* that locate and extract the useful information directly from the markup document. While the traditional approach was to define such wrappers manually – a task for which a variety of declarative languages and tools have been defined – such approaches are brittle to changes in a website’s layout [Ferrara et al., 2014]. Hence other approaches allow for (semi-)automatically *inducing* wrappers [Flesca et al., 2004]. A modern such approach – used to enrich knowledge graphs in systems such as LODIE [Gentile et al., 2014] – is to apply distant supervision, whereby EL is used to identify and link entities in the webpage to nodes in the knowledge graph such that paths in the markup that connect pairs of nodes for known edges can be extracted, ranked, and applied to other examples. Taking Figure 6.2, for example, distant supervision may link Rapa Nui and World Heritage Sites to the nodes Easter Island and World Heritage Site in the knowledge graph using EL, and given the edge Easter Island *named* World Heritage Site in the knowledge graph (extracted per Figure 6.1), identify the candidate path $(x, td[1]^- \cdot tr^- \cdot table^- \cdot h1, y)$ as reflecting edges of the form $\boxed{x}$ *named* $\boxed{y}$, where $t[n]$ indicates the n^{th} child of tag t , t^- its inverse, and $t_1 \cdot t_2$ concatenation. Finally, paths with high confidence (e.g., ones “witnessed” by many known edges in the knowledge graph) can then be used to extract novel edges, such as Qhapaq Ñan *named* World Heritage Site, both on this page and on related pages of the website with similar structure (e.g., for other countries).

6.3.2 WEB TABLE EXTRACTION

Other approaches target specific types of markup, most commonly *web tables* embedded in HTML webpages. However, web tables are designed to enhance human rather than machine readability. Many web tables are used for layout and page structure (e.g., navigation bars). Those that contain data may follow different formats, such as relational tables, listings, attribute-value tables, and matrices [Cafarella et al., 2008, Crestan and Pantel, 2011]. A first step is to classify tables to find ones appropriate for the given extraction mechanism(s) [Crestan and Pantel, 2011, Eberius et al., 2015]. Next, web tables may contain column spans, row spans, inner tables, or may be split vertically to improve human aesthetics. Table normalisation merges split tables, un-nests tables, transposes tables, etc. [Pivk et al., 2007, Cafarella et al., 2008, Crestan and Pantel, 2011, Deng et al., 2013, Ermilov and Ngonga Ngomo, 2016, Lehmborg et al., 2016]. Some approaches then identify the table *protagonist* [Crestan and Pantel, 2011, Muñoz et al., 2014] – the main entity that the table describes – often found elsewhere in the webpages; for example, though not mentioned by the table of Figure 6.1, World Heritage Sites is its protagonist. Finally, extraction processes may associate cells with entities [Limaye et al., 2010, Mulwad et al., 2013], columns with types [Deng et al., 2013, Limaye et al., 2010, Mulwad et al., 2013], and column pairs with relations [Limaye et al., 2010, Muñoz et al., 2014]. When enriching knowledge graphs, recent approaches apply distant supervision, linking cells to knowledge graph nodes in order to generate candidates for type and relation extraction [Limaye et al., 2010, Mulwad et al., 2013, Muñoz et al., 2014]. Statistical distributions can also help to link numerical columns [Neumaier et al., 2016]. Specialised table extraction frameworks have also been proposed for specific websites, where prominent knowledge graphs, such as DBpedia [Lehmann et al., 2015] and YAGO [Suchanek et al., 2008] focus on extraction from info-box tables in Wikipedia.

6.3.3 DEEP WEB CRAWLING

The *Deep Web* presents a rich source of information accessible only through searches on web forms, thus requiring *Deep Web crawling* techniques to access [Madhavan et al., 2008]. Systems have been proposed to extract knowledge graphs from Deep Web sources [Geller et al., 2008, Lehmann et al., 2012, Collarana et al., 2016]. Approaches typically attempt to generate sensible form inputs – which may be based on a user query or generated from reference knowledge – and then extract data from the generated responses (markup documents) using the aforementioned techniques [Geller et al., 2008, Lehmann et al., 2012, Collarana et al., 2016].

6.4 STRUCTURED SOURCES

Much of the legacy data available within organisations and on the Web is represented in structured formats, primarily tables – in the form of relational databases, CSV files, etc. – but also tree-structured formats such as JSON, XML etc. Unlike text and markup documents, structured sources can often be *mapped* to knowledge graphs whereby the structure is (precisely) transformed according to a mapping rather than (imprecisely) extracted. The mapping process involves two steps: 1) create a mapping from the source to a graph, and 2) use the mapping in order to materialise the source data as a graph or to virtualise the source (creating a graph view over the legacy data).

6.4.1 MAPPING FROM TABLES

Tabular sources of data are prevalent; for example, the structured content underlying many organisations and websites are housed in relational databases. In Figure 6.3 we present an example of a relational database instance that we wish to integrate into our knowledge graph. There are then two approaches for mapping content from tables to knowledge graphs: a *direct mapping*, and a *custom mapping*.

Report				Claimant		
crime	claimant	station	date	id	name	country
Pickpocketing	XY12SDA	Viña del Mar	2019-04-12	XY12SDA	John Smith	U.S.
Assault	AB9123N	Arica	2019-04-12	AB9123N	Jeanne Dubois	France
Pickpocketing	XY12SDA	Rapa Nui	2019-04-12	XI92HAS	Jorge Hernández	Chile
Fraud	FI92HAS	Arica	2019-04-13			

Give us feedback!

Figure 6.3: Relational database instance with two tables describing crime data

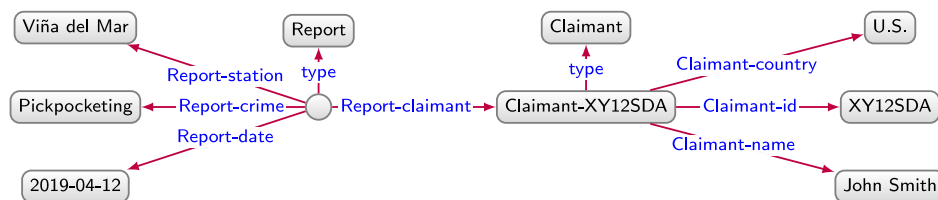


Figure 6.4: Direct mapping result for the first rows of both tables in Figure 6.3

A direct mapping automatically generates a graph from a table. We present in Figure 6.4 the result of a standard direct mapping [Arenas et al., 2012], which creates an edge $(x) \xrightarrow{y} (z)$ for each (non-header, non-empty, non-NULL) cell of the table, such that (x) represents the row of the cell, y the column name of the cell, and (z) the value of the cell. In particular, (x) typically encodes the values of the primary key for a row (e.g., **Claimant.id**); otherwise, if no primary key is defined (e.g., per the **Report** table), (x) can be an anonymous node or a node based on the row number. The node (x) and edge label y further encode the name of the table to avoid clashes across tables that have the same column names used with different meanings. For each row (x) , we may add a type edge based on the name of its table. The value (z) may be mapped to datatype values in the corresponding graph model based on the source domain (e.g., a value in an SQL column of type Date can be mapped to `xsd:date` in the RDF data model). If the value is NULL (or empty), typically the corresponding edge will be omitted.³⁰ With respect to Figure 6.4, we highlight the difference between the nodes `(Claimant-XY12SDA)` and `(XY12SDA)`, where the former denotes the row (or entity) identified by the latter primary key value. In case of a foreign key between two tables – such as **Report.claimant** referencing **Claimant.id** – we can link, for example, to `(Claimant-XY12SDA)` rather than `(XY12SDA)`, where the former node also has the name and country of the claimant. A direct mapping along these lines has been standardised for mapping relational databases to RDF [Arenas et al., 2012], where Stoica et al. [2019] have recently proposed an analogous direct mapping for property graphs. Another direct mapping has been defined for CSV and other tabular data [Tandy et al., 2015] that further allows for specifying column names, primary/foreign keys, and data types – which are often missing in such data formats – as part of the mapping itself.

note 30 One might consider NULLS in SQL can be used in intentional semantics of :

Although a direct mapping can be applied automatically on tabular sources of data and preserve the information of the original source – i.e., allowing a deterministic inverse mapping that reconstructs the tabular source from the output graph [Sequeda et al., 2012] – in many cases it is desirable to customise a mapping, such as to align edge labels or nodes with a knowledge graph under enrichment, etc. Along these lines, declarative mapping languages allow for manually defining custom mappings from tabular sources to graphs. A standard language along these lines is the RDB2RDF Mapping Language (R2RML) [Das et al., 2012], which allows for mapping from individual rows of a table to one or more custom edges, with nodes and edges defined either as constants, as individual cell values, or using templates that concatenate multiple cell values from a row and static substrings into a single term; for example, a template `{id}-{country}` may produce nodes such as `(XY12SDA-U.S.)` from the **Claimant** table. In case that the desired output edges cannot be defined from a single row, R2RML allows for (SQL) queries to generate tables from which edges can be extracted where, for example, edges such as `(U.S.)-crimes-(2)` can be generated by defining the mapping with respect to a query that joins the **Report** and **Claimant** tables on **claimant=id**, grouping by **country**, and applying a count for each country group. A mapping can then be defined on the results table such that the source node denotes the value of **country**, the edge label is the constant **crimes**, and the target node is the count value. An analogous standard also exists for mapping CSV and other tabular data to RDF graphs, again allowing keys, column names, and datatypes to be chosen as part of the mapping [Tennison and Kellogg, 2015].

Once the mappings have been defined, one option is to use them to *materialise* graph data following an *Extract-Transform-Load (ETL)* approach, whereby the tabular data are transformed and explicitly serialised as graph data using the mapping. A second option is to use *virtualisation* through a *Query Rewriting (QR)* approach, whereby queries on the graph (using, e.g., SPARQL, Cypher, etc.) are trans-

Give us feedback!

lated to queries over the tabular data (typically using SQL). Comparing these two options, ETL allows the graph data to be used as if they were any other data in the knowledge graph. However, ETL requires updates to the underlying tabular data to be explicitly propagated to the knowledge graph, whereas a QR approach only maintains one copy of data to be updated. The area of *Ontology-Based Data Access (OBDA)* [Xiao et al., 2018] is concerned with QR approaches that support ontological entailments as seen in Chapter 4. Although most QR approaches only support non-recursive entailments expressible as a single (non-recursive) query, some QR approaches support recursive entailments through rewritings to recursive queries [Sequeda et al., 2014].

6.4.2 MAPPING FROM TREES

A number of popular data formats are based on trees, including XML and JSON. While one could imagine – leaving aside issues such as the ordering of children in a tree – a trivial direct mapping from trees to graphs by simply creating edges of the form $(x) \xrightarrow{\text{child}} (y)$ for each node y that is a child of x in the source tree, such an approach is not typically used, as it represents the literal structure of the source data. Instead, the content of tree-structured data can be more naturally represented as a graph using a custom mapping. Along these lines, the GRDDL standard [Connolly, 2007] allows for mapping from XML to (RDF) graphs, while languages such as RML allow for mapping from a variety of formats, including XML and JSON, to (RDF) graphs [Dimou et al., 2014]. In contrast, hybrid query languages such as XSPARQL [Bischof et al., 2012] allow for querying XML and RDF in unison, thus supporting both materialisation and virtualisation of graphs over tree-structured sources of legacy data.

6.4.3 MAPPING FROM OTHER KNOWLEDGE GRAPHS

We may also leverage existing knowledge graphs in order to construct or enrich another knowledge graph. For example, a large number of points of interest for the Chilean tourist board may be available in existing knowledge graphs such as BabelNet [Navigli and Ponzetto, 2012], DBpedia [Lehmann et al., 2015], LinkedGeoData [Stadler et al., 2012], Wikidata [Vrandečić and Krötzsch, 2014], YAGO [Hoffart et al., 2011], etc. However, not all entities and/or relations may be of interest. A standard option to extract a relevant sub-graph of data is to use construct queries that generate graphs as output [Neumaier and Polleres, 2019]. Entity and schema alignment between the knowledge graphs may be further necessary to better integrate (parts of) external knowledge graphs, using linking tools for graph-external identifiers [Pellissier Tanon et al., 2016], or indeed may be done manually [Pellissier Tanon et al., 2016]. For instance, Wikidata [Vrandečić and Krötzsch, 2014] uses Freebase [Bollacker et al., 2007b, Pellissier Tanon et al., 2016] as a source; Gottschalk and Demidova [2018] extract an event-centric knowledge graph from Wikidata, DBpedia and YAGO; while Neumaier and Polleres [2019] construct a spatio-temporal knowledge graph from Geonames, Wikidata, and PeriodO [Golden and Shaw, 2016] (as well as tabular data).

6.5 SCHEMA/ONTOLOGY CREATION

The discussion thus far has focussed on extracting *data* from external sources in order to create and enrich a knowledge graph. In this section, we discuss some of the principal methods for generating a schema based on external sources of data, including human knowledge. For discussion on extracting a schema from the knowledge graph itself, we refer back to Section 3.1.3. In general, much of the work in this area has focussed on the creation of ontologies using either ontology engineering methodologies, and/or ontology learning. We discuss these two approaches in turn.

6.5.1 ONTOLOGY ENGINEERING

Ontology engineering refers to the development and application of methodologies for building ontologies, proposing principled processes by which better quality ontologies can be constructed and maintained with less effort. Early methodologies [Grüninger and Fox, 1995a, Fernández et al., 1997, Noy and McGuinness, 2001] were often based on a waterfall-like process, where requirements a

[Give us feedback!](#)

conceptualisation were fixed before starting to define the ontology, using, for example, an ontology engineering tool [Gómez-Pérez et al., 2006, Keet, 2018, Kendall and McGuinness, 2019]. However, for situations involving large or ever-evolving ontologies, more iterative and agile ways of building and maintaining ontologies have been proposed.

DILIGENT [Pinto et al., 2009] was an early example of an agile methodology, proposing a complete process for ontology life-cycle management and knowledge evolution, as well as separating local changes (local views on knowledge) from global updates of the core part of the ontology, using a review process to authorise the propagation of changes from the local to the global level. This methodology is similar to how, for instance, the large clinical reference terminology SNOMED CT [IHTSDO, 2019] (also available as an ontology) is maintained and evolved, where the (international) core terminology is maintained based on global requirements, while national or local extensions to SNOMED CT are maintained based on local requirements. A group of authors then decides which national or local extensions to propagate to the core terminology. More modern agile methodologies include eXtreme Design (XD) [Presutti et al., 2009, Blomqvist et al., 2016], Modular Ontology Modelling (MOM) [Krisnadhi and Hitzler, 2016b, Hitzler and Krisnadhi, 2018], Simplified Agile Methodology for Ontology Development (SAMOD) [Peroni, 2016], and more besides. Such methodologies typically include two key elements: *ontology requirements* and (more recently) *ontology design patterns*.

Ontology requirements specify the intended task of the resulting ontology, or of the knowledge graph itself in conjunction with the new ontology. A common way to express ontology requirements is through *Competency Questions (CQ)* [Grüninger and Fox, 1995b], which are natural language questions illustrating the typical information needs that one would require the ontology (or the knowledge graph) to respond to. Such CQs can then be complemented with additional restrictions, and reasoning requirements, in case that the ontology should also contain restrictions and general axioms for inferring new knowledge or checking data consistency. A common way of testing ontologies (or knowledge graphs based on them) is then to formalise the CQs as queries over some test set of data, and make sure the expected results are entailed [Blomqvist et al., 2012, Keet and Ławrynowicz, 2016]. We may, for example, consider the CQ “What are all the events happening in Santiago?”, which can be represented as a graph query $(\text{Event} \xrightarrow{\text{type}} ?\text{event} \xrightarrow{\text{location}} \text{Santiago})$. Taking the data graph of Figure 2.1 and the axioms of Figure 3.2, we can check to see if the expected result (EID15) is entailed by the ontology and the data, and since it is not, we may consider expanding the axioms to assert that $(\text{location} \xrightarrow{\text{type}} \text{Transitive})$.

Ontology Design Patterns (ODPs) are another common feature of modern methodologies [Gangemi, 2005, Blomqvist and Sandkuhl, 2005], specifying generalisable ontology modelling patterns that can be used as inspiration for modelling similar patterns, as modelling templates [Egaña et al., 2008, Skjæveland et al., 2018], or as directly reusable components [Daga et al., 2008, Shimizu et al., 2019]. Several pattern libraries have been made available online, ranging from carefully curated ones [Aranguren et al., 2008, Shimizu et al., 2019] to open and community moderated ones [Daga et al., 2008]. As an example, to model events in our scenario, we may adopt the Core Event ontology pattern proposed by Krisnadhi and Hitzler [2016a], which specifies a spatio-temporal extent, sub-events, and participants of an event, along with competency questions, formal definitions, etc., to support this pattern.

6.5.2 ONTOLOGY LEARNING

The previous methodologies outline methods by which ontologies can be built and maintained manually. Ontology learning, in contrast, can be used to (semi-)automatically extract information from text that is useful for the ontology engineering process [Buitelaar et al., 2005, Cimiano, 2006]. Early methods focussed on extracting terminology from text that may represent the relevant domain’s classes; for example, from a collection of text documents about tourism, a terminology extraction tool – using measures of *unitihood* that determine how cohesive an *n*-gram is as a unitary phrase, and *termhood* that determine how relevant the phrase is to a domain [Martínez-Rodríguez et al., 2018] – may identify *n*-grams such as “visitor visa”, “World Heritage Site”, “off-peak rate”, etc., as terminology of particular importance to the tourist domain that thus may merit inclusion in such an ontology. Ontological axioms may also be extracted from text. A common target is to extract sub-class axioms from text, leveraging patterns based on modifying nouns and adjectives that incrementally specialise concepts (e.g., extracting $(\text{Visitor Visa} \xrightarrow{\text{subc. of}} \text{Visa})$ from the noun phrase “visitor visa” and isolat

Give us feedback!

appearances of “visa” elsewhere), or using Hearst patterns [Hearst, 1992] (e.g., extracting (Off-Peak Rate)–subc. of→(Discount) from “many discounts, such as off-peak rates, are available” based on the pattern “X, such as Y”). Textual definitions can also be harvested from large texts to extract hypernym relations and induce a taxonomy from scratch [Velardi et al., 2013]. More recent works aim to extract more expressive axioms from text, including disjointness axioms [Völker et al., 2015]; and axioms involving the union and intersection of classes, along with existential, universal, and qualified-cardinality restrictions [Petrucci et al., 2016]. The results of an ontology learning process can then serve as input to a more general ontology engineering methodology, allowing us to validate the terminological coverage of an ontology, to identify new classes and axioms, etc.

Chapter 7

Quality Assessment

Independently of the (kinds of) source(s) from which a knowledge graph is created, the resulting initial knowledge graph will usually be incomplete, and will often contain duplicate, contradictory or even incorrect statements, especially when taken from multiple sources. After the initial creation and enrichment of a knowledge graph from external sources, a crucial step is thus to assess the *quality* of the resulting knowledge graph. By quality, we here refer to *fitness for purpose*. Quality assessment then helps to ascertain for which purposes a knowledge graph can be reliably used. Take, for instance, the sample of an initial knowledge graph created by the tourist board shown in Figure 7.1. Is this knowledge graph of good quality? Does it exhibit issues that might limit the applications for which it is fit for purpose? Can we define and detect such issues? These questions are crucial to address before the knowledge graph is deployed, but they are also challenging to address in a general way.

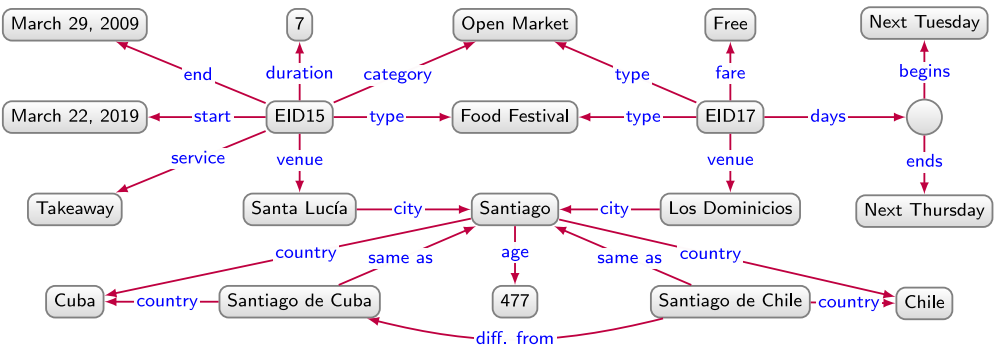


Figure 7.1: A newly created knowledge graph about events and their venues

This chapter discusses (sometimes overlapping) *quality dimensions* that capture qualitative aspects of the multifaceted notion of data quality; some of these dimensions apply more generally to databases [Batini et al., 2015], while others are more specific to knowledge graphs [Zaveri et al., 2016]. We further discuss *quality metrics* that provide ways to measure quantitative aspects of these dimensions. We group dimensions and metrics in a manner inspired by Batini and Scannapieco [2016].

7.1 ACCURACY

Accuracy refers to the extent to which entities and relations – encoded by nodes and edges in the graph – correctly represent real-life phenomena. Accuracy can be divided into three dimensions: s

Give us feedback!

tactic accuracy, semantic accuracy, and timeliness.

7.1.1 SYNTACTIC ACCURACY

Syntactic accuracy is the degree to which the data are accurate with respect to the grammatical rules defined for the domain and/or data model. A prevalent example of syntactic inaccuracy occurs with datatype nodes, which may be incompatible with a defined range or be malformed. For example, assuming that a property `start` is defined with the range `xsd:dateTime`, the value `(March 29, 2019)` in Figure 7.1 would be incompatible with the defined range, while a value `"March 29, 2019, 20:00"^^xsd:dateTime` would be malformed (a value such as `"2019-03-22T20:00:00"^^xsd:dateTime` is rather expected). A corresponding metric for syntactic accuracy is the ratio between the number of invalid values of a given property and the total number of values for the same property [Zaveri et al., 2016]. Such forms of syntactic accuracy can typically be assessed using validation tools [Fürber and Hepp, 2011, Hogan et al., 2010].

7.1.2 SEMANTIC ACCURACY

Semantic accuracy is the degree to which data values correctly represent real-world phenomena, which may be affected by imprecise extraction results, untrustworthy sources, vandalism, etc. For instance, in Figure 7.1, the start of the `(EID15)` event comes after the end of the event, possibly due to a typo in the year. While such a case could potentially be identified using, for example, shape-based validation, other cases might be more difficult to detect; for example, if we were to accidentally (and incorrectly) swap the venues for `(EID15)` and `(EID17)`, there might be no indication whatsoever in the knowledge graph that the venues are incorrect, even if we have additional schemata/ontologies/rules available. Assessing the level of semantic inaccuracy is challenging. While one option is to apply manual verification, an automatic option may be to check the stated relation against several sources [Lei et al., 2007, Esteves et al., 2018]. An alternative is to validate the quality of the processes used to generate the knowledge graph, based on measures such as precision, possibly with the help of human experts or gold standards [Martínez-Rodríguez et al., 2020].

7.1.3 TIMELINESS

Timeliness is the degree to which the knowledge graph is kept up-to-date with the real world state [Käfer et al., 2013]. A knowledge graph may be semantically accurate now, but may quickly become inaccurate (outdated) if no procedures are in place to keep it up-to-date in a timely manner. Considering Figure 7.1, the events appear to be from years ago, and if not updated, then the knowledge graph will not be suitable for applications that wish to recommend upcoming events to users. Additionally, the meaning of some values in the graph, such as `(Next Tuesday)` or `(Next Thursday)` (which may have been extracted from the text of a news article, for example), will change over time, and become semantically inaccurate in the future. Similarly, the age of Santiago will quickly become outdated, where instead representing the year that the city was founded would facilitate timeliness. Timeliness can be assessed based on how frequently the knowledge graph is updated with respect to underlying sources [Käfer et al., 2013, Rula et al., 2014], which can be done using temporal annotations of changes in the knowledge graph [Rula et al., 2012, Rula et al., 2019], as well as contextual representations that capture the temporal validity of data (see Section 3.3).

7.2 COVERAGE

Coverage refers to avoiding the omission of domain-relevant elements, which otherwise may yield incomplete query results or entailments, biased models, etc.

7.2.1 COMPLETENESS

Completeness refers to the degree to which all required information is present in a particular dataset. Completeness comprises the following aspects: (i) *schema completeness* refers to the degree to whi

[Give us feedback!](#)

the classes and properties of a schema are represented in the data graph, (ii) *property completeness* refers to the ratio of missing values for a specific property, (iii) *population completeness* refers to the percentage of all real-world entities of a particular type that are represented in the datasets, and (iv) *linkability completeness* refers to the degree to which instances in the data set are interlinked. Taking some examples from Figure 7.1, the lack of information about the fare for (EID15) might be seen as a form of property incompleteness, while missing events held in Chile around the same time might lead to population incompleteness. Measuring completeness is non-trivial as it assumes knowledge of a hypothetical *ideal knowledge graph* [Darari et al., 2018] that contains all the elements that the knowledge graph in question *should* have. Concrete strategies may involve comparison with gold standards that provide samples of the ideal knowledge graph (possibly based on *completeness statements* [Darari et al., 2018]), or measuring the recall of extraction methods from complete sources [Martínez-Rodríguez et al., 2020].

7.2.2 REPRESENTATIVENESS

Representativeness is a related dimension that, instead of focusing on the ratio of domain-relevant elements that are missing, rather focuses on assessing high-level *biases* in what is included/excluded from the knowledge graph [Baeza-Yates, 2018]. As such, this dimension assumes that the knowledge graph is incomplete – i.e., that it is a sample of the ideal knowledge graph – and asks how biased this sample is. Biases may occur in the data, in the schema, or during reasoning [Janowicz et al., 2018]. Examples of data biases include geographic biases that under-represent entities/relations from certain parts of the world [Janowicz et al., 2018], linguistic biases that under-represent multilingual resources (e.g., labels and descriptions) for certain languages [Kaffee et al., 2017], social biases that under-represent people of particular genders or races [Wagner et al., 2016], and so forth. In contrast, schema biases may result from high-level definitions extracted from biased data [Janowicz et al., 2018], semantic definitions that do not cover uncommon cases, etc. Unrecognised biases may lead to adverse effects; for example, if the knowledge graph of Figure 7.1 has a geographic bias towards events and attractions close to Santiago city – due perhaps to the sources used for creation, the employment of curators from the city, etc. – then this may lead to tourism in and around Santiago being disproportionately promoted to the detriment of tourism elsewhere in Chile. Measures of representativeness may involve comparing known statistical distributions with those of the knowledge graph, for example, comparing geolocated entities with known population densities [Janowicz et al., 2018], linguistic distributions with known distributions of speakers [Kaffee et al., 2017], etc. Another more general option is to compare the knowledge graph with general statistical laws, where Soulet et al. [2018] use (non-)conformance with Benford’s law³¹ to measure representativeness in knowledge graphs.

note 31 Benford’s law
bers is more likely to 1

7.3 COHERENCY

Coherency refers to how well the knowledge graph conforms to – or is coherent with – the formal semantics and constraints defined at the schema-level.

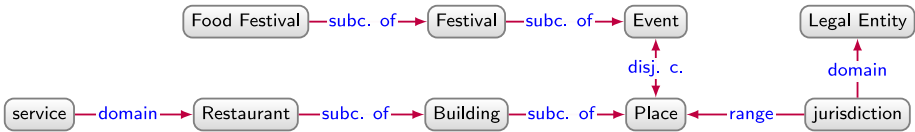


Figure 7.2: An ontology for the knowledge graph of Figure 7.1

7.3.1 CONSISTENCY

Consistency means that a knowledge graph is free of contradictions (i.e., inconsistencies) with respect to the particular logical entailment considered. For example, if we apply the entailments defined in Table 4.1 over the graph of Figure 7.1, we see that the edge (Santiago de Chile) –same as→ (Santiago de Cu

Give us feedback!

is inferred from both entities being the same as `(Santiago)`, which generates an inconsistency with the edge `(Santiago de Chile) -diff. from- (Santiago de Cuba)` as stated in the graph. While in this case it is evident that `(Santiago de Cuba) -same as- (Santiago)` is semantically inaccurate (considering that the venues connected to `(Santiago)` are in Chile), in other cases there may not be an obvious inaccuracy. Take, for example, the ontology defined in Figure 7.2, combined with the graph of Figure 7.1, and the ontological entailments of Tables 4.1–4.3. Noting that the food festival `(EID15)` offers a takeaway service, according to the ontology, this entails that `(EID15)` is a restaurant, a building, and a place, which is disjoint with event. However, `(EID15)` is also entailed to be a festival, and then an event, generating an inconsistency. In this case there is no clear individual “error” leading to an inconsistency. Possibly the graph of Figure 7.1 should not use the property `service` for a food event (though it seems a “good fit”), or perhaps the ontology of Figure 7.2 should not define the domain of the property `service` to be a restaurant. Any ontological features in Tables 4.1–4.3 with a “not” condition can give rise to inconsistencies if the negated condition is entailed. A measure of consistency can be the number of inconsistencies found in a knowledge graph, possibly sub-divided into the number of such inconsistencies identified by each semantic feature [Bonatti et al., 2011].

7.3.2 VALIDITY

Validity means that the knowledge graph is free of constraint violations, such as captured by shape expressions [Thornton et al., 2019] (see Section 3.1.2). We may, for example, specify a shape `[CITY]` whose target nodes have at most one country. Then, taking the edges `(Chile) -country- (Santiago) -country- (Cuba)` from Figure 7.1, and assuming that `(Santiago)` becomes a target of `[CITY]`, we have a constraint violation. Conversely, even if we defined analogous cardinality restrictions in an ontology (e.g., even if we defined that `country` was functional), this would not necessarily cause an inconsistency since, without UNA, we would first infer that `(Chile)` and `(Cuba)` refer to the same entity. Similarly, using shapes, we can more easily detect missing data; for example, we can define a shape `[EVENT]`, and require that it have at least one value for the property `fare`. Now, if `(EID15)` becomes targetted by `[EVENT]`, then we will have a constraint violation as the node has no value for `fare`. Conversely, even if we defined analogous cardinality restrictions in an ontology (e.g., we defined that events have a minimum cardinality of 1 for `fare`), this would not cause an inconsistency since, under the OWA, we would rather entail that the event `(EID15)` has some fair (that is not described in the graph). Consistency and validity can thus indicate different types of issues. A straightforward measure of validity is to count the number of violations per constraint.

7.4 SUCCINCTNESS

Succinctness refers to the inclusion only of relevant content (avoiding “information overload”) that is represented in a concise and intelligible manner.

7.4.1 CONCISENESS

Conciseness refers to avoiding schema and data elements that are irrelevant to the domain. Mendes et al. [2012b] distinguish *intensional conciseness* (schema level), which refers to the case when the data do not contain redundant schema elements (properties, classes, shapes, etc.), and *extensional conciseness* (data level), where the data do not describe redundant entities and relations. For example, the inclusion of a property and class for modelling jurisdictions and legal entities in the ontology of Figure 7.2 may affect the intensional conciseness of the ontology in the context of a knowledge graph about tourist events. Similarly, the inclusion of data about `(Santiago de Cuba)` in our knowledge graph dedicated to tourism in Chile may affect the extensional conciseness of the knowledge graph, potentially returning irrelevant results for the given domain. In general, conciseness can be measured in terms of the ratio of properties, classes, shapes, entities, relations, etc., of relevance to the domain, which may in turn require a gold standard, or measures of domain-relevance.

7.4.2 REPRESENTATIONAL CONCISENESS

Give us feedback!

Representational conciseness refers to the extent to which content is compactly represented in the knowledge graph, which may again be intensional or extensional [Zaveri et al., 2016]. For example, having two properties `category` and `type` serving the same purpose would negatively affect the intensional form of representational conciseness, while having two nodes `(Santiago)` and `(Santiago de Chile)` that split the data available about the capital of Chile would affect the extensional form of representational conciseness. Another example of poor representational conciseness is the unnecessary use of complex modelling constructs, such as using reification unnecessarily, or using linked lists when the order of elements is not important [Hogan et al., 2012a]. An example of this is the anonymous node used in Figure 7.1 to represent the days on which `(EID17)` starts and ends, which could rather be directly associated with the event (at least if we assume that events have one start and one end moment in time). A different example is the specification of the duration of `(EID15)`, which could be calculated from the start and end values (assuming the correct datatypes were used). Though representational conciseness is challenging to assess, measures such as the number of redundant nodes can be used [Fürber and Hepp, 2011].

7.4.3 UNDERSTANDABILITY

Understandability refers to the ease with which data can be interpreted without ambiguity by human users, which involves – at least – the provision of human-readable labels and descriptions (preferably in different languages [Kaffee et al., 2017]) that allow such beings to understand what is being spoken about [Hogan et al., 2012a]. Referring back to Figure 7.1, though the nodes `(EID15)` and `(EID17)` are used to ensure unique identifiers for events, they should also be associated with labels, such as `(Nam)`. Ideally the human readable information is sufficient to disambiguate a particular node, such as associating a description `(“Santiago, the capital of Chile”@en)` with `(Santiago)` to disambiguate the city from synonymous ones. Measures of understandability may include the ratio of nodes with human-readable labels and descriptions, the uniqueness of such labels and descriptions, the languages supported, etc.

7.5 OTHER QUALITY DIMENSIONS

The list of quality dimensions provided here should be considered illustrative rather than complete. Further dimensions may be pertinent in the context of specific domains, applications, or graph data models. For more discussion, we refer to the survey by Zaveri et al. [2016] and to the book by Batini and Scannapieco [2016].

Chapter 8

Refinement

Beyond assessing the quality of a knowledge graph, there exist techniques to *refine* the knowledge graph, in particular to (semi-)automatically complete and correct the knowledge graph [Paulheim, 2017], aka *knowledge graph completion* and *knowledge graph correction*, respectively. As distinguished from the creation and enrichment tasks outlined in Chapter 6, refinement typically does not involve applying extraction or mappings over external sources in order to ingest their content into a given knowledge graph (potentially using external sources to verify its content).

8.1 COMPLETION

Knowledge graphs are characterised by incompleteness [West et al., 2014]. As such, knowledge graph completion aims at filling in the *missing edges* (aka *missing links*) of a knowledge graph, i.e., edges that are deemed correct but are neither given nor entailed by the knowledge graph. This task is often addressed with *link prediction* techniques proposed in the area of *Statistical Relational Learning* [Getoor and Taskar, 2007], which predict the existence – or sometimes more generally, predict the probability of correctness – of missing edges. For instance, one might predict that the edge $(\text{Moon Valley}) \xrightarrow{\text{bus}} (\text{San Pedro})$ is a probable missing edge for the graph of Figure 5.2, given that most bus routes observed are return services (i.e., *bus* is typically symmetric). Link prediction may target three settings: *general links* involving edges with arbitrary labels, e.g., *bus*, *flight*, *type*, etc.; *type links* involving edges with label *type*, indicating the type of an entity; and *identity links* involving edges with label *same as*, indicating that two nodes refer to the same entity (cf. Section 3.2.2). While type and identity links can be addressed using general link prediction techniques, the particular semantics of type and identity links can be addressed with custom techniques. The related task of generating links across knowledge graphs – referred to as *link discovery* [Nentwig et al., 2017] – will be discussed later in Section 9.1.

8.1.1 GENERAL LINK PREDICTION

Link prediction, in the general case, is often addressed with inductive techniques as discussed in Chapter 5, and in particular, knowledge graph embeddings and rule/axiom mining. For example, given Figure 5.2, using knowledge graph embeddings, we may detect that given an edge of the form $(x) \xrightarrow{\text{bus}} (y)$, a (missing) edge $(y) \xrightarrow{\text{bus}} (x)$ has high plausibility, while using symbol-based approaches, we may learn the high-level rule $(?x) \xrightarrow{\text{bus}} (?y) \Rightarrow (?y) \xrightarrow{\text{bus}} (?x)$ that may infer/predict new *bus* links. Either approach would help us to predict the missing link $(\text{Moon Valley}) \xrightarrow{\text{bus}} (\text{San Pedro})$.

8.1.2 TYPE-LINK PREDICTION

Type links are of particular importance to a knowledge graph, where dedicated techniques can be leveraged taking into account the specific semantics of such links. In the case of type prediction, there is only one edge label (*type*) and typically fewer distinct values (classes) than in other cases, such that the task can be reduced to a traditional classification task [Paulheim, 2017], training models to identify each semantic class based on features such as outgoing and/or incoming edge labels on their instances in the knowledge graph [Paulheim and Bizer, 2013, Sleeman and Finin, 2013]. For example, assume that in Figure 5.2 we also know that (Arica) , (Calama) , (Puerto Montt) , (Punta Arenas) and (Santiago) are of *type* (City) . We may then predict that (Iquique) and (Easter Island) are also of *type* (City) based on the presence of edges labelled *flight* to/from these nodes, which (we assume) are learnt to be a good feature for prediction of that class (the former prediction is correct, while the latter is incorrect). Graph neural networks (see Section 5.3) can also be used for node classification/type prediction.

8.1.3 IDENTITY-LINK PREDICTION

Predicting identity links involves searching for nodes that refer to the same entity, but are not stated or entailed to be the same; this is analogous to the task of *entity matching* (aka record linkage, deduplication, etc.) considered in more general data integration settings [Köpcke and Rahm, 2010]. Such techniques are generally based on two types of *matchers*: *value matchers* determine how similar the values of two entities on a given property are, which may involve similarity metrics on strings, numbers, dates, etc.; while *context matchers* consider the similarity of entities based on various nodes and edges [Köpcke and Rahm, 2010]. An illustrative example is given in Figure 8.1, where value matchers will compute similarity between values such as (7400) and (7500) , while context matchers will compute similarity between (Easter Island) and (Rapa Nui) based on their surrounding information, such as similar latitudes, longitudes, populations, and the same seat (conversely, a value matcher on this pair of nodes would measure string similarity between “Easter Island” and “Rapa Nui”).

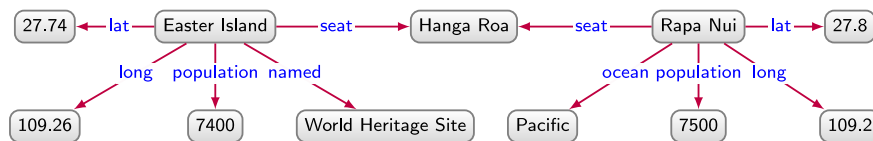


Figure 8.1: Identity linking example: (Easter Island) and (Rapa Nui) denote the same place

A major challenge in this setting is efficiency, where a pairwise matching would require $O(n^2)$ comparisons for n the number of nodes. To address this issue, *blocking* can be used to group similar entities into (possibly overlapping, possibly disjoint) “blocks” based on similarity-preserving keys, with matching performed within each block [Isele et al., 2011, Köpcke and Rahm, 2010, Draisbach and Naumann, 2011]; for example, if matching places based on latitude/longitude, blocks may represent geographic regions. An alternative to discrete blocking is to use *windowing* over entities in a similarity-preserving ordering [Draisbach and Naumann, 2011], or to consider searching for similar entities within *multi-dimensional spaces* (e.g., spacetime [Santipantakis et al., 2019], spaces with Minkowski distances [Ngonga Ngomo, 2012], orthodromic spaces [Ngonga Ngomo, 2013], etc. [Sherif and Ngonga Ngomo, 2018]). The results can either be pairs of nodes with a computed confidence of them referring to the same entity, or crisp identity links extracted based on a fixed threshold, or binary classification [Köpcke and Rahm, 2010]. For confident identity links, the nodes’ edges may then be *consolidated* [Hogan et al., 2012b]; for example, we may select (Easter Island) as the canonical node and merge the edges of (Rapa Nui) onto it, enabling us to find, e.g., *World Heritage Sites in the Pacific Ocean* from Figure 8.1 based on the (consolidated) sub-graph (World Heritage Site) $\leftarrow$ named (Easter Island) $\rightarrow$ ocean (Pacific).

8.2 CORRECTION

As opposed to completion – which finds new edges in a knowledge graph – correction identifies and removes existing incorrect edges in the knowledge graph. We here divide the principal approaches for knowledge graph correction into two main lines: *fact validation*, which assigns a plausibility score to a given edge, typically in reference to external sources; and *inconsistency repairs*, which aim to resolve inconsistencies found in the knowledge graph through ontological axioms.

8.2.1 FACT VALIDATION

The task of *fact validation* (aka *fact checking*) [Gerber et al., 2015, Syed et al., 2018, Yin et al., 2008, Syed et al., 2019, Esteves et al., 2018, Shiralkar et al., 2017, Shi and Weninger, 2016, Socher et al., 2013, Bordes et al., 2013] involves assigning plausibility or *veracity* scores to facts/edges, typically between 0 and 1. An ideal fact-checking function assumes a hypothetical reference universe (an ideal knowledge graph) and would return 1 for the fact (Santa Lucía) $\rightarrow$ city (Santiago) (being true) while returning 0 for (Sotomayor) $\rightarrow$ city (Santiago) (being false). There is a clear relation between fact validation and link prediction – with both relying on assessing the plausibility of edges/facts/links – and indeed the same numeric- and symbol-based techniques can be applied for both cases. However, fact validation often considers online assessment of edges given as input, whereas link prediction is often an offline task that generates novel candidate edges to be assessed from the knowledge graph. Furthermore, works on fact validation are characterised by their consideration of external reference sources, which may be *unstructured sources* [Gerber et al., 2015, Syed et al., 2018, Samadi et al., 2016, Yin et al., 2008] or *structured sources* [Syed et al., 2019, Shiralkar et al., 2017, Shi and Weninger, 2016, Socher et al., 2013, Bordes et al., 2013].

Approaches based on unstructured sources assume that they are given a *verbalisation function* – using, for example, rule-based approaches [Ngonga Ngomo et al., 2013, Ell et al., 2014], encoder–decoder architectures [Gardent et al., 2017], etc. – that is able to translate edges into natural language. Thereafter, approaches for computing the plausibility of facts in natural language – called *fact finders* [Pasternack and Roth, 2010, Pasternack and Roth, 2011] – can be directly employed. Many fact finding algorithms construct an n -partite (often bipartite) graph whose nodes are facts and sources

Give us feedback!

where a source is connected to a fact if the source “evidences” the fact, i.e., if it contains a text snippet that matches – with sufficient confidence – the verbalisation of the input edge. Two mutually-dependent scores, namely the trustworthiness of sources and the plausibility of facts, are then calculated based on this graph, where fact finders differ on how they compute these scores [Pasternack and Roth, 2011]. Here we mention three scores proposed by Pasternack and Roth [2010]:

- *Sums* [Pasternack and Roth, 2010] adapts the classical HITS centrality algorithm [Kleinberg, 1999] by defining sources as hubs (with 0 authority score) and facts as authorities (with 0 hub score).
- *Average Log* [Pasternack and Roth, 2010] extends HITS with a normalisation factor that prevents a single source from receiving a high trustworthiness score by evidencing many facts (that may be false).
- *Investment* [Pasternack and Roth, 2010] lets the scores of facts grow with a non-linear function based on “investments” coming from the connected sources. The score a source receives from a fact is based on the individual facts in this particular source compared to the other connected sources.

Pasternack and Roth [2011] then show that these three algorithms can be generalised into a single multi-layered graph-based framework within which (1) a source can support a fact with a weight expressing uncertainty, (2) similar facts can support each other, and (3) sources can be grouped together leading to an implicit support between sources of the same group. Other approaches for fact checking of knowledge graphs later extended this framework [Galland et al., 2010, Samadi et al., 2016]. Alternative approaches based on machine learning classifiers have also emerged, where commonly-used features include trust scores for information sources, co-occurrences of facts in sources, and so forth [Gerber et al., 2015, Syed et al., 2018].

Approaches for fact validation based on structured data typically assume external knowledge graphs as reference sources and are based on finding paths that support the edge being validated. Unsupervised approaches search for undirected [Shiralkar et al., 2017, Ciampaglia et al., 2015] or directed [Syed et al., 2019] paths up to a given threshold length that support the input edge. The relatedness between input edges and paths is computed using a mutual information function, such as normalised pointwise mutual information [Bouma, 2009]. Supervised approaches rather extract features for input edges from external knowledge graphs [Sun et al., 2011, Zhao et al., 2015, Lao and Cohen, 2010] and train a classification model to label the edges as true or false. An important set of features are *metapaths*, which encode sequences of predicates that correlate positively with the edge label of the input edge. Amongst such works, PredPath [Shi and Weninger, 2016] automatically extracts metapaths based on type information. Several approaches rather encode the reference nodes and edges using graph embeddings (see Section 5.2), which are then used to estimate the plausibility of the input edge being validated.

8.2.2 INCONSISTENCY REPAIRS

Ontologies can contain axioms – such as disjointness – that lead to inconsistencies. While such axioms can be provided by experts, they can also be derived through symbolic learning, as discussed in Section 5.4. Such axioms can then be used to detect inconsistencies. With respect to correcting a knowledge graph, however, detecting inconsistencies is not enough: techniques are also required to *repair* such inconsistencies, which itself is not a trivial task. In the simplest case, we may have an instance of two disjoint classes, such as that (Santiago) is of type (City) and (Airport), which are stated or found to be disjoint. To repair the inconsistency, it would be preferable to remove only the “incorrect” class, but which should we remove? This is not a trivial question, particularly if we consider that one edge can be involved in many inconsistencies, and one inconsistency can involve many edges. The issue of computing repairs becomes more complex when entailment is considered, where we not only need to remove the stated type, but also all of the ways in which it might be entailed; for example, removing the edge (Santiago) *type* (Airport) is insufficient if we further have an edge (Arica) *flight* (Santiago) combined with an axiom (flight) *range* (Airport). Töpper et al. [2012] suggest potential repairs for such violations – remove a domain/range constraint, remove a disjointness constraint, remove a type edge, or remove an edge with a domain/range constraint – where one is chosen manually. In contrast, Bonatti et al. [2011] propose an automated method to repair incons

Give us feedback!

tencies based on *minimal hitting sets* [Reiter, 1987], where each set is a minimal explanation for an inconsistency. The edges to remove are chosen based on scores of the trustworthiness of their sources and how many minimal hitting sets they are either elements of or help to entail an element of, where the knowledge graph is revised to avoid re-entailment of the removed edges. Rather than repairing the data, another option is to evaluate queries under inconsistency-aware semantics, such as returning *consistent answers* valid under every possible repair [Lukasiewicz et al., 2013].

8.3 OTHER REFINEMENT TASKS

In comparison to the quality clusters discussed in Chapter 7, the refinement methods discussed here-in address particular aspects of the accuracy, coverage, and coherency dimensions. Beyond these, one could conceive of further refinement methods to address further quality issues of knowledge graphs, such as succinctness. In general, however, the refinement tasks of *knowledge graph completion* and *knowledge graph correction* have received the majority of attention until now. For further details on knowledge graph refinement, we refer to the survey by Paulheim [2017].

Chapter 9

Publication

While it may not always be desirable to publish knowledge graphs (for example, those that offer a competitive advantage to a company [Noy et al., 2019]), it may be desirable or even required to publish other knowledge graphs, such as those produced by volunteers [Vrandečić and Krötzsch, 2014, Mahdisoltani et al., 2015, Lehmann et al., 2015], by publicly-funded research [Callahan et al., 2013, Groth et al., 2014, The UniProt Consortium, 2014], by governmental organisations [Hendler et al., 2012, Shadbolt and O'Hara, 2013]. Publishing refers to making the knowledge graph (or part thereof) accessible to the public, often on the Web. Knowledge graphs published as open data are called open knowledge graphs (discussed in Section 10.1).

In the following, we first discuss two sets of principles that have been proposed to guide the publication of data on the Web. We next discuss access protocols by which the public can interact with the content of a knowledge graph. Finally, we consider techniques to restrict the access or usage of (parts of) a knowledge graph.

9.1 BEST PRACTICES

We now discuss two key sets of publishing principles: the FAIR Principles [Wilkinson et al., 2016], and the Linked Data Principles [Berners-Lee, 2006].

9.1.1 FAIR PRINCIPLES

The FAIR Principles were originally proposed in the context of publishing scientific data [Wilkinson et al., 2016] – particularly motivated by maximising the impact of publicly-funded research – but the principles generally apply to other situations where data are to be published in a manner that facilitates their re-use by external agents, with particular emphasis on machine-readability.

FAIR itself is an acronym for four foundational principles, each with particular goals [Wilkinson et al., 2016], that may apply to *data*, *metadata*, or both – the latter being denoted *(meta)data*.³² We now describe the FAIR principles (slightly rephrasing the original wording in some cases for brevity [Wilkinson et al., 2016]).

note 32 Metadata are a part of knowledge graphs, where in a knowledge graph metadata may include

Give us feedback!

- *Findability* refers to the ease with which external agents who might benefit from the dataset can initially locate the dataset. Four sub-goals should be met:
 - F1: (meta)data are assigned a globally unique and persistent identifier.
 - F2: data are described with rich metadata (see R1).
 - F3: metadata explicitly include the identifier of the data they describe.
 - F4: (meta)data are registered or indexed in a searchable resource.
- *Accessibility* refers to the ease with which external agents can access the dataset (after locating it). Two goals are defined, the first with two sub-goals:
 - A1: (meta)data are retrievable by their identifier via a standard protocol.
 - A1.1: the protocol is open, free, and universally implementable.
 - A1.2: the protocol uses authentication and authorisation if suitable.
 - A2: metadata are accessible, even when the data are no longer available.
- *Interoperability* refers to the ease with which the dataset can be exploited (in unison with other datasets) using standard tools. Three goals are defined:
 - I1: (meta)data use an accessible, agreed-upon, and general knowledge representation formalism.
 - I2: (meta)data use vocabularies that follow FAIR principles.
 - I3: (meta)data include qualified references to other (meta)data.
- *Reusability* refers to the ease with which the dataset can be re-used in conjunction with other datasets. One goal is defined (with three sub-goals):
 - R1: meta(data) are richly described with accurate and relevant attributes.
 - R1.1: (meta)data are released with a clear and accessible license.
 - R1.2: (meta)data are associated with detailed provenance.
 - R1.3: (meta)data meet domain-relevant community standards.

In the context of knowledge graphs, a variety of vocabularies, tools, and services have been proposed that both directly and indirectly help to satisfy the FAIR principles. In terms of *Findability*, as discussed in Chapter 2, IRIs are built into the RDF model, providing a general schema for global identifiers. In addition, resources such as the Vocabulary of Interlinked Datasets (VoID) [Alexander et al., 2009] allow for representing metadata about graphs, while services such as DataHub [Bhardwaj et al., 2015] provide a central repository of such dataset descriptions. Access protocols that enable *Accessibility* will be discussed in Section 9.2, while mechanisms for authorisation will be discussed in Section 9.3. With respect to *Interoperability*, as discussed in Chapter 4, ontologies serve as a general knowledge representation formalism, and can in turn be used to describe vocabularies that follow FAIR principles. Regarding *Reusability*, licensing will be discussed in Section 9.3, while the *PROV Data Model* [Gil et al., 2013] discussed in Chapter 3, can encode provenance in detail.

Various knowledge graphs have been published using FAIR principles, where Wilkinson et al. [2016] explicitly mention Open PHACTS [Groth et al., 2014], a data integration platform for drug discovery, and UniProt [The UniProt Consortium, 2014], a large collection of protein sequence and annotation data, as conforming to FAIR principles. Both datasets offer graph views of their content through RDF.

9.1.2 LINKED DATA PRINCIPLES

Wilkinson et al. [2016] state that FAIR Principles “precede implementation choices”, meaning that the principles do not cover *how* they can or should be achieved. Preceding the FAIR Principles by almost a decade are the Linked Data Principles, proposed by Berners-Lee [2006], which provide a technical basis for one possible way in which these FAIR Principles can be achieved. Specifically the Linked Data Principles are as follows:

1. Use IRIs as names for things.
2. Use HTTP IRIs so those names can be looked up.
3. When a HTTP IRI is looked up, provide useful content about the entity that the IRI names using standard data formats.
4. Include links to the IRIs of related entities in the content returned.

These principles were proposed in a Semantic Web setting, where for principle (3), the standards based on RDF (including RDFS, OWL, etc.) are currently recommended for use, particularly because

Give us feedback!

they allow for naming entities using HTTP IRIs, which further paves the way for satisfying all four principles. As such, these principles outline a way in which (RDF) graph-structured data can be published on the Web such that these graphs are interlinked to form what [Berners-Lee \[2006\]](#) calls a “Web of Data”, whose goal is to increase automation on the Web by making content available not only in (HTML) documents intended for human consumption, but also as (RDF) structured data that machines can locate, retrieve, combine, validate, reason over, query over, etc., towards solving tasks automatically [[Hogan, 2020b](#)]. Conceptually, the Web of Data is then composed of graphs of data published on individual web-pages, where one can click on a node or edge-label – or more precisely perform a HTTP lookup on an IRI of the graph – to be transported to another graph elsewhere on the Web with relevant content for that node or edge-label, and so on recursively.

Figure 9.1 provides a small example with two Linked Data documents published on the Web, with each containing an RDF graph. As discussed in Section 3.2, terms such as `clv:Concert`, `wd:Q142701`, `rdfs:label`, etc., are abbreviations for IRIs, where, for example, `wd:Q142701` expands to <http://www.wikidata.org/entity/Q142701>. Prefixes beginning with `cl` are fictitious prefixes we assume to have been created by the Chilean tourist board. The IRIs prefixed with  indicate the document returned if the node is looked up. The leftmost document is published by the tourist board and describes Lollapalooza 2018 (identified by the node `cle:LP2018`), which links to the headlining act Pearl Jam (`wd:Q142701`) described by an external knowledge graph, namely Wikidata. By looking up the node `wd:Q142701` in the leftmost graph, the IRI *dereferences* (i.e., returns via HTTP) the document with the RDF graph on the right describing that entity in more detail. From the rightmost document, the node `wd:Q221535` can be looked up, in turn, to find a graph about Eddie Vedder (not shown in the example). The IRIs for entities and documents are distinguished to ensure that we do not confuse data about the entity and the document; for example, while `wd:Q221535` refers to Eddie Vedder, the IRI `wdd:Q221535` refers to the document about Eddie Vedder; if we were to assign a last-modified date to the document, we should use `wdd:Q221535` not `wd:Q221535`. In Figure 9.1, we can further observe that edge labels (which are also IRIs) and nodes representing classes (e.g., `clv:Concert`) can also be dereferenced, typically returning semantic definitions of the respective terms.

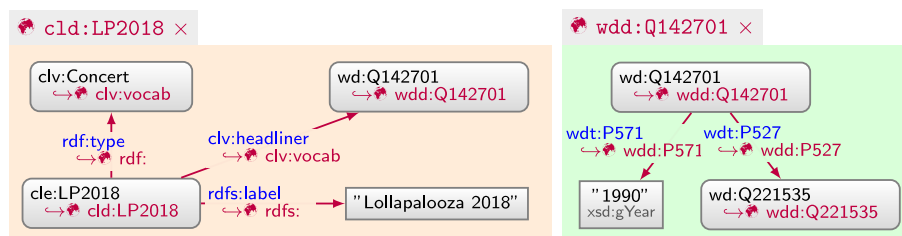


Figure 9.1: Two example Linked Data documents from two websites, each containing an RDF graph, where `wd:Q142701` refers to Pearl Jam in Wikidata while `wdd:Q142701` refers to the RDF graph about Pearl Jam, and where `wd:Q221535` refers to Eddie Vedder while `wdd:Q221535` refers to the RDF graph about Eddie Vedder; the edge-label `wdt:571` refers to “inception” in Wikidata, while `wdt:527` refers to “has part”

A key challenge is posed by the fourth principle – include links to related entities – as illustrated in Figure 9.1, where `wd:Q221535` in the leftmost graph constitutes a link to related content about Pearl Jam in an external knowledge graph. Specifically, the *link discovery* task considers adding such links from one knowledge graph to another, which may involve inclusion of IRIs that dereference to external graphs (per Figure 9.1), or links with special semantics such as identity links. In comparison with the link prediction task discussed in Section 8.1, which is used to complete links within a knowledge graph, link discovery aims to discover links across knowledge graphs, which involves unique aspects: first, link discovery typically considers disjoint sets of source (local) nodes and target (remote) nodes; second, the knowledge graphs may often use different vocabularies; third, while in link prediction there already exist local examples of the links to predict, in link discovery, there are often no existing links between knowledge graphs to learn from. A common technique is to define manually-crafted linkage rules (aka link specifications) that apply heuristics for defining links that potentially incorporate similarity measures [[Ngonga Ngomo and Auer, 2011](#), [Volz et al., 2009](#)]. Link discovery is greatly expedited by the provision of standard identifier schemes within knowledge graphs, such as ISBNs for books, alpha-2 and alpha-3 codes for countries (e.g., CL, CLP), or even links to comm

knowledge graphs such as DBpedia [Lehmann et al., 2015] or Wikidata [Vrandečić and Krötzsch, 2014] (that themselves include standard identifiers). We refer to the survey on link discovery by Nentwig et al. [2017] for more details.

More finer-grained recommendations for publishing Linked Data have also been proposed, relating to how best to implement dereferencing, what kinds of links to include, how to publish and interlink vocabularies, amongst other considerations [Heath and Bizer, 2011, Janowicz et al., 2014]. We refer to the book by Heath and Bizer [2011] for more discussion on how to publish Linked Data on the Web.

9.2 ACCESS PROTOCOLS

Publishing involves giving access to the public to interact with the knowledge graph, which implies the provision of *access protocols* that define the requests that agents can make and the response that they can expect as a result. Per the *Accessibility* principle of FAIR (specifically A1.1), this protocol should be open, free, and universally implementable. In the context of knowledge graphs, as shown in Figure 9.2, there are a number of access protocols to choose from, varying from simple protocols that allow users to simply download all content, towards protocols that accept and evaluate increasingly complex requests. While simpler protocols require less computation on the server that publishes the data, more complex protocols allow agents to request more specific data, thus reducing bandwidth. A knowledge graph may also offer a variety of access protocols catering to different agents with different requirements [Verborgh et al., 2014]. We now discuss such access protocols.

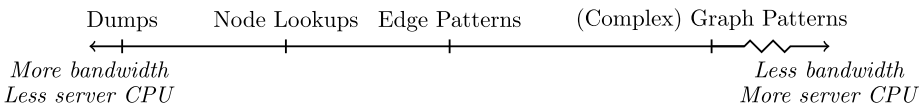


Figure 9.2: Access protocols for knowledge graphs, from simple protocols (left) to more complex protocols (right)

9.2.1 DUMPS

A dump is a file or collection of files containing the content of the knowledge graph available for download. The request in this case is for the file(s) and the response is the content of the file(s). In order to publish dumps, first of all, concrete – and ideally standard – syntaxes are required to serialise the graph. While for RDF graphs there are various standard syntaxes available based on XML [Gandon and Schreiber, 2014], JSON [Sporny et al., 2014], custom syntaxes [Prud'hommeaux and Carothers, 2014], and more besides, currently there are only non-standard syntaxes available for property graphs [Tomaszuk et al., 2019]. Second, to reduce bandwidth, compression methods can be applied. While standard compression such as GZIP or BZip2 can be straightforwardly applied on any file, custom compression methods have been proposed for graphs that not only offer better compression ratios than these standard methods, but also offer additional functionalities, such as compact indexes for performing efficient lookups once the file is downloaded [Fernández et al., 2013]. Finally, to further reduce bandwidth, when the knowledge graph is updated, “diffs” can be computed and published to obviate the need for agents to download all data from scratch (see [Tummarello et al., 2007, Papavasileiou et al., 2013, Ahn et al., 2015]). Still, however, dumps are only suited to certain use-cases, in particular for agents that wish to maintain a full local copy of a knowledge graph. If an agent were rather only interested in, for example, all food festivals in Santiago, downloading the entire dump may require transferring and processing a lot of irrelevant data.

9.2.2 NODE LOOKUPS

Protocols for performing node lookups accept a node (id) request (e.g., `cle:LP2018` in Figure 9.1) and return a (sub-)graph describing that node (e.g., the document `cld:LP2018`). Such a protocol is the basis for the Linked Data principles outlined previously, whereby node lookups are implement

Give us feedback!

through HTTP dereferencing, which further allows nodes in remote graphs to be referenced from across the Web. Although there are varying definitions on what content should be returned for a node [Stickler, 2005], a common convention is to return a sub-graph containing either all outgoing edges for that node or all incident edges (both outgoing and incoming) for that node [Hogan et al., 2012a]. Though simple, mechanisms for evaluating graph patterns can be implemented on top of a node lookup interface by traversing from node to node per the particular graph pattern [Hartig et al., 2009]; for example, to find all food festivals in Santiago – represented by the graph pattern $(\text{Food Festival}) \leftarrow \text{type} \text{---} \{?ff\} \text{---} \text{location} \rightarrow (\text{Santiago})$ – we may perform a node lookup for (Santiago) , subsequently performing a node lookup for each node connected by a `location` edge to (Santiago) , returning those nodes declared to be of type (Food Festival) . However, such an approach may not be feasible if no starting node is declared (e.g., if all nodes are variables), if the node lookup service does not return incoming edges, etc. The client agent may also need to request more data than necessary; for example, the document returned for (Santiago) may return a lot of data irrelevant to the query, and nodes with a `location` in (Santiago) that are not instances of (Food Festival) still need to be looked up to check their type. Node lookups are relatively inexpensive for servers to support in terms of CPU, but may again waste bandwidth due to transferring irrelevant data.

9.2.3 EDGE PATTERNS

Edge patterns – also known as *triple patterns* in the case of directed, edge-labelled graphs – are singleton graph patterns, i.e., graph patterns with a single edge. Examples of edge patterns are $\{?ff\} \text{---} \text{type} \rightarrow (\text{Food Festival})$ or $\{?ff\} \text{---} \text{location} \rightarrow (\text{Santiago})$, etc., where any term can be a variable or a constant. A protocol for edge patterns accepts such a pattern and returns all solutions for the pattern. Edge patterns provide more flexibility than node lookups, where graph patterns are more readily decomposed into edge patterns than node lookups. With respect to the agent interested in food festivals in Santiago, they can first, for example, request solutions for the edge pattern $\{?ff\} \text{---} \text{location} \rightarrow (\text{Santiago})$ and locally join/intersect these solutions with those of $\{?ff\} \text{---} \text{type} \rightarrow (\text{Food Festival})$. Given that some edge patterns (e.g., $\{?x\} \text{---} \text{type} \rightarrow (\text{Food Festival})$) can return many solutions, protocols for edge patterns may offer additional practical features such as iteration or pagination over results [Verborgh et al., 2016]. Much like node lookups, the server cost of responding to a request is relatively low and easy to predict. However, the server may often need to transfer irrelevant intermediate results to the client, which in the previous example may involve returning nodes located in Santiago that are not food festivals. This issue is further aggravated if the client does not have access to statistics about the knowledge graph in order to plan how to best perform the join; for example, if there are relatively few food festivals but many things located in Santiago, rather than intersecting the solutions of the two aforementioned edge patterns, it should be more efficient to send a request for each food festival to see if it is in Santiago, but deciding this requires statistics about the knowledge graph. Extensions to the edge-pattern protocol have thus been proposed to allow for more efficient joins [Hartig et al., 2017], such as allowing batches of solutions to be sent alongside the edge pattern to only return solutions compatible with the solutions in the request [Hartig and Buil Aranda, 2016] (e.g., sending a batch of solutions for $\{?ff\} \text{---} \text{type} \rightarrow (\text{Food Festival})$ to join with the solutions for the request $\{?ff\} \text{---} \text{location} \rightarrow (\text{Santiago})$).

9.2.4 (COMPLEX) GRAPH PATTERNS

Another alternative is to let client agents make requests based on (complex) graph patterns (see Section 2.2), with the server returning (only) the final solutions. In our running example, this involves the client issuing a request for $(\text{Food Festival}) \leftarrow \text{type} \text{---} \{?ff\} \text{---} \text{location} \rightarrow (\text{Santiago})$ and directly receiving the relevant results. Compared with the previous protocols, this protocol is much more efficient in terms of bandwidth: it allows clients to make more specific requests and the server to return more specific responses. However, this reduction in bandwidth use comes at the cost of the server having to evaluate much more complex requests, where, furthermore, the costs of a single request are much more difficult to anticipate. While a variety of optimised engines exist for evaluating (complex) graph patterns (e.g., [Erling, 2012, Miller, 2013, Thompson et al., 2014] amongst many others), the problem of evaluating such queries is known to be intractable [Angles et al., 2017]. Perhaps for this reason, public services offering such a protocol (most often supporting SPARQL queries [Harris et al., 2013]) have been found to often exhibit downtimes, timeouts, partial results, slow performance

Give us feedback!

etc. [Buil-Aranda et al., 2013b]. Even considering such issues, however, popular services continue to receive – and successfully evaluate – millions of requests/queries per day [Malyshev et al., 2018, Saleem et al., 2015], with difficult (worst-case) instances being rare in practice [Bonifati et al., 2017].

9.2.5 OTHER PROTOCOLS

While Figure 9.2 makes explicit reference to some of the most commonly-encountered access protocols found for knowledge graphs in practice, one may of course imagine other protocols lying almost anywhere on the spectrum from more simple to more complex interfaces. To the right of (Complex) Graph Patterns, one could consider supporting even more complex requests, such as queries with entailments [Glimm, 2011], queries that allow recursion [Reutter et al., 2015], federated queries that can join results from remote services [Buil-Aranda et al., 2013a], or even (hypothetically) supporting Turing-complete requests that allow running arbitrary procedural code on a knowledge graph. As mentioned at the outset, a server may also choose to support multiple, complementary protocols [Verborgh et al., 2014].

9.3 USAGE CONTROL

Considering our hypothetical tourism knowledge graph, at first glance, one might assume that the knowledge required to deliver the envisaged services is public and thus can be used both by the tourism board and the tourists. On closer inspection, however, we may see the need for usage control in various forms:

- both the tourist board and its partners should associate an appropriate license with knowledge that they contribute to the knowledge graph, such that the terms of use are clear to all interested parties;
- a tourist might opt to install an app on their mobile phone that could be used to recommend tourist attractions based on their location, bringing with it potential privacy concerns regarding who has access to their location;
- the tourist board may be required to report criminal activities to the police services and thus may need to encrypt personal information; and
- the tourist board could potentially share information relating to tourism demographics in an anonymous format to allow for other agencies and companies to anticipate demand and improve transport infrastructure on strategic routes.

Thus in this section, we examine the state of the art in terms of knowledge graph licensing, usage policies, encryption, and anonymisation.

9.3.1 LICENSING

When it comes to associating machine readable licenses with knowledge graphs, the W3C Open Digital Rights Language (ODRL) [Iannella and Villata, 2018] provides an information model and related vocabularies that can be used to specify permissions, duties, and prohibitions with respect to actions relating to assets. ODRL supports fine-grained descriptions of digital rights that are represented as – and thus can be embedded within – graphs. Figure 9.3 illustrates a license granting the assignee the permission to (Modify), (Distribute), and (Derive) work from the (Event Graph) (e.g., Figure 2.1); however the assignee is obliged to (Attribute) the copyright holder. From a modelling perspective, ODRL can be used to model several well-known license families, for instance Apache, Creative Commons (CC), and Berkeley Software Distribution (BSD), to name but a few [Cabrio et al., 2014, Panasiuk et al., 2018]. Additionally, Cabrio et al. [2014] propose methods to automatically extract machine-readable licenses from unstructured text. From a reasoning perspective, license compatibility validation and composition techniques [Villata and Gandon, 2012, Governatori et al., 2013, Moreau et al., 2019] can be used to combine knowledge graphs that are governed by different licenses. Such techniques are employed by the Data Licenses Clearance Center (DALICC), which includes a library of standard machine readable licenses, and tools that enable users both to comp

Give us feedback!

arbitrary custom licenses and also to verify the compatibility of different licenses [Pellegrini et al., 2019].

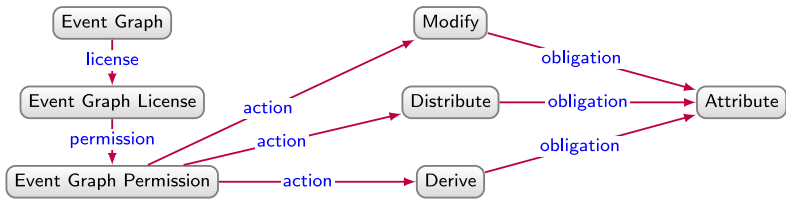


Figure 9.3: A license for event data, along with permissions, actions, and obligations

9.3.2 USAGE POLICIES

Access control policies based on edge patterns can be used to restrict access to parts of a knowledge graph [Reddivari et al., 2005, Flouris et al., 2010, Kirrane et al., 2013]. WebAccessControl (WAC)³³ is an access control framework for graphs that uses WebID for authentication and provides a vocabulary for specifying access control policies. Extensions of this WAC vocabulary have been proposed to capture privacy preferences [Sacco and Passant, 2011] and to cater for contextual constraints [Villata et al., 2011, Costabello et al., 2012]. Although ODRL is primarily used to specify licenses, profiles to additionally specify access policies [Steyskal and Polleres, 2014] and regulatory obligations [Agarwal et al., 2018, De Vos et al., 2019] have also been proposed in recent years, as discussed in the survey by Kirrane et al. [2017].

note 33 WAC, <http://www.wac-project.org/>

As a generalisation of access policies, usage policies specify how data can be used: what kinds of processing can be applied, by whom, for what purpose, etc. The example usage policy presented in Figure 9.4 states that the process (Analyse) of (Location Graph) can be performed on (Internal Servers) by members of (Company Staff) in order to provide (Event Recommendations). Vocabularies for usage policies have been proposed by the SPECIAL H2020 project [Bonatti et al., 2019] and the W3C Data Privacy Vocabularies and Controls Community Group (DPVCG) [Pandit et al., 2019, Bonatti and Kirrane, 2019]. Once specified in these vocabularies, usage policies can then be used to verify that data processing conforms to legal norms and to the consent provided by subjects [Delanaux et al., 2018, Bonatti and Kirrane, 2019].

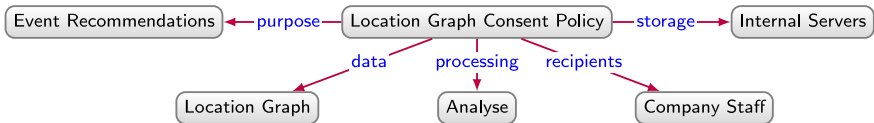


Figure 9.4: A policy for usage of a sub-graph of location data in the knowledge graph

9.3.3 ENCRYPTION

Rather than internally controlling usage, the tourist board could use encryption mechanisms on parts of the published knowledge graph, for example relating to reports of crimes, and provide keys to partners who should have access to the plaintext. While a straightforward approach is to encrypt the entire graph (or sub-graphs) with one key, more fine-grained encryption can be performed for individual nodes or edge-labels in a graph, potentially providing different clients access to different information through different keys [Giereth, 2005]. The CryptOntology [Gerbracht, 2008] can further be used to embed details about the encryption mechanism used within the knowledge graph. Figure 9.5 illustrates how this could be used to encrypt the names of claimants from Figure 6.4, storing the ciphertext (zhk...kjpg), as well as the key-length and encryption algorithm used. In order to grant access to the plaintext, one approach is to encrypt individual edges with symmetric keys so as to allow specific types of edge patterns to only be executed by clients with the appropriate key [Kasten et al., 2013]. This approach can be used, for example, to allow clients who know a claimant ID (e.g., (Claimant-XY12SDA)) and have the appropriate key to find (only) the name of t

Give us feedback!

claimant through an edge pattern $\langle \text{Claimant-XY12SDA} \rangle \text{--} \langle \text{Claimant-name} \rangle \text{--} \langle \text{?name} \rangle$. A key limitation of this approach, however, is that it requires attempting to decrypt all edges to find all possible solutions. A more efficient alternative is to combine functional encryption and specialised indexing to retrieve solutions from the encrypted graph without attempting to decrypt all edges [Fernández et al., 2017].

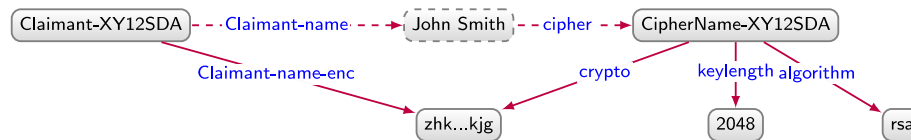


Figure 9.5: Directed edge-labelled graph with the name of the claimant encrypted; plaintext elements are dashed and may be omitted from published data (possibly along with encryption details)

9.3.4 ANONYMISATION

Consider that the tourist board acquires information on transport taken by individuals within the country, which can be used – not only by the board, but potentially other stakeholders, such as travel companies – to understand trajectories taken by tourists. However, from a data-protection perspective, it would be advisable to anonymise the knowledge graph to avoid leaking the personal travel history of individuals.

A first approach to anonymisation is to suppress and generalise knowledge in a graph such that individuals cannot be identified, based on k -anonymity [Samarati and Sweeney, 1998]³⁴, l -diversity [Li et al., 2007]³⁵, etc. Approaches that apply k -anonymity on graphs identify and suppress “quasi-identifiers” that would allow a given individual to be distinguished from fewer than $k - 1$ other individuals [Radulovic et al., 2015, Heitmann et al., 2017]. Figure 9.6 illustrates a possible result of k -anonymisation for a sub-graph describing a flight passenger, where quasi-identifiers (passport, plane ticket) have been converted into blank nodes, ensuring that the passenger (the dashed blank node) cannot be distinguished from $k - 1$ other individuals. In the context of a graph, however, *neighbourhood attacks* [Zhou and Pei, 2011] – using information about neighbours – can also break k -anonymity, where we also suppress the day and time of the flight, which, though not sensitive information per se, could otherwise break k -anonymity for passengers (if, for example, a particular flight had fewer than k males from the U.S. onboard). The graph shown in Figure 9.6 then offers k -anonymity for the particular individual assuming that at least k male passengers from the U.S. flew during December 2018 from Arica to Santiago.

note 34 k -anonymity g

note 35 l -diversity guar each group of individ of individuals) in the reveal sensitive inform

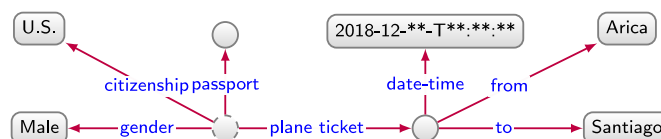


Figure 9.6: Anonymised sample of a directed edge-labelled graph describing a passenger (dashed) of a flight

More complex neighbourhood attacks may rely on more abstract graph patterns, observing that individuals can be deanonymised purely from knowledge of the graph structure, even if all nodes and edge labels are left blank; for example, if we know that a team of $k - 1$ players take flights together for a particular number of away games, we could use this information for a neighbourhood attack that reveals the set of players in the graph. Hence a number of guarantees specific to graphs have been proposed, including k -degree anonymity [Liu and Terzi, 2008], which ensures that individuals cannot be deanonymised by attackers with knowledge of the degree of particular individuals. The approach is based on minimally modifying the graph to ensure that each node has at least $k - 1$ other nodes with the same degree. A stronger guarantee, called k -isomorphic neighbour anonymity [Zhou and Pei, 2008], avoids neighbourhood attacks where an attacker knows how an individual is connected to nodes in their neighbourhood; this is done by modifying the graph to ensure

Give us feedback!

that for each node, there exist at least $k - 1$ nodes with isomorphic (i.e., identically structured) neighbourhoods elsewhere in the graph. Both approaches only protect against attackers with knowledge of bounded neighbourhoods. An even stronger notion is that of k -automorphism [Zou et al., 2009], which ensures that for every node, it is structurally indistinguishable from $k - 1$ other nodes, thus avoiding any attack based on structural information (as a trivial example, a k -clique or a k -cycle satisfy k -automorphism). Many of these techniques for anonymisation of graph data were motivated by social networks [Narayanan and Shmatikov, 2009], though they can also be applied to knowledge graphs, per the work of Lin and Tripunitara [2017], who adapt k -automorphism for directed edge-labelled graphs (specifically RDF graphs).

While the aforementioned approaches anonymise data, a second approach is to apply anonymisation when answering queries, such as adding noise to the solutions in a way that preserves privacy. One approach is to apply ϵ -differential privacy [Dwork, 2006]³⁶ for querying graphs [Silva et al., 2017]. Such mechanisms are typically used for aggregate (e.g., count) queries, where noise is added to avoid leaks about individuals. To illustrate, differential privacy may allow for counting the number of passengers of specified nationalities taking specified flights, adding (just enough) random noise to the count to ensure that we cannot tell, within a certain probability (controlled by ϵ), whether or not a particular individual took a flight, where, intuitively speaking, we would require (proportionally) less noise for nationalities with many passengers in the data, but more noise to “hide” passengers from more uncommon nationalities.

These approaches require information loss for stronger guarantees of privacy; which to choose is thus heavily application dependent. If the anonymised data are to be published in their entirety as a “dump”, then an approach based on k -anonymity can be used to protect individuals, while l -diversity can be used to protect groups. On the other hand, if the data are to be made available, in part, through a query interface, then ϵ -differential privacy is a more suitable framework.

note 36 ϵ -differential privacy (e.g., query) applied when the data include

Chapter 10

Knowledge Graphs in Practice

In this chapter, we discuss some of the most prominent knowledge graphs that have emerged in the past years. We begin by discussing open knowledge graphs, most of which have been published on the Web per the guidelines and protocols described in Chapter 9. We later discuss enterprise knowledge graphs that have been created by companies from diverse industries for a wide range of applications.

10.1 OPEN KNOWLEDGE GRAPHS

By *open knowledge graphs*, we refer to knowledge graphs published under the Open Data philosophy, namely that “*open means anyone can freely access, use, modify, and share for any purpose (subject, at most, to requirements that preserve provenance and openness)*”.³⁷ Many open knowledge graphs have been published in the form of *Linked Open Datasets* [Heath and Bizer, 2011], which are (RDF) graphs published under the Linked Data principles (see Section 9.1.2) following the Open Data philosophy. Many of the most prominent open knowledge graphs – including DBpedia [Lehmann et al., 2015], YAGO [Suchanek et al., 2007], Freebase [Bollacker et al., 2007b], and Wikidata [Vrandečić and Krötzsch, 2014] – cover multiple domains, representing a broad diversity of entities and relationships; we first discuss these in turn. Later we discuss some of the other (specific) domains for which open knowledge graphs are currently available. Most of the open knowledge graphs we discuss in this section are modelled in RDF, published following Linked Data principles, and offer access to their data through dumps (RDF), node lookups (Linked Data), graph patterns (SPARQL) and, in some cases, edge patterns (Triple Pattern Fragments).

note 37 See <http://open>

Give us feedback!

10.1.1 DBPEDIA

The DBpedia project was developed to extract a graph-structured representation of the semi-structured data embedded in Wikipedia articles [Auer et al., 2007], enabling the integration, processing, and querying of these data in a unified manner. The resulting knowledge graph is further enriched by linking to external open resources, including images, webpages, and external datasets such as DailyMed, DrugBank, GeoNames, MusicBrainz, New York Times, and WordNet [Lehmann et al., 2015]. The DBpedia extraction framework consists of several components, corresponding to abstractions of Wikipedia article sources, graph storage and serialisation destinations, wiki-markup extractors, parsers, and extraction managers [Bizer et al., 2009]. Specific extractors are designed to process labels, abstracts, interlanguage links, images, redirects, disambiguation pages, external links, internal pagelinks, homepages, categories, and geocoordinates. The content in the DBpedia knowledge graph is not only multidomain, but also multilingual: as of 2012, DBpedia contained labels and abstracts in up to 97 different languages [Mendes et al., 2012a]. Entities within DBpedia are classified using four different schemata in order to address varying requirements [Bizer et al., 2009]. These schemata include a Simple Knowledge Organization System (SKOS) representation of Wikipedia categories, a Yet Another Great Ontology (YAGO) classification schema (discussed presently), an Upper Mapping and Binding Exchange Layer (UMBEL) ontology categorisation schema, and a custom schema called the DBpedia ontology with classes such as Person, Place, Organisation, and Work [Lehmann et al., 2015]. DBpedia also supports live synchronisation in order to remain consistent with dynamic Wikipedia articles [Lehmann et al., 2015].

10.1.2 YET ANOTHER GREAT ONTOLOGY

YAGO likewise extracts graph-structured data from Wikipedia, which are then unified with the hierarchical structure of WordNet to create a “*light-weight and extensible ontology with high quality and coverage*” [Suchanek et al., 2007]. This knowledge graph aims to be applied for various information technology tasks, such as machine translation, word sense disambiguation, query expansion, document classification, data cleaning, information integration, etc. While earlier approaches automatically extracted structured knowledge from text using pattern matching, natural language processing (NLP), and statistical learning, the resulting content tended to lack in quality when compared with what was possible through manual construction [Suchanek et al., 2007]. However, manual construction is costly, making it challenging to achieve broad coverage and keep the data up-to-date. In order to extract data with high coverage and quality, YAGO (like DBpedia) mostly extracts data from Wikipedia infoboxes and category pages, which contain core entity information and lists of articles for a specific category, respectively. These, in turn, are unified with hierarchical concepts from WordNet [Suchanek et al., 2008]. A schema – called the YAGO model – provides a vocabulary defined in RDFS; this model allows for representing words as entities, capturing synonymy and ambiguity [Suchanek et al., 2007]. The model further supports reification, n -ary relations, and data types [Suchanek et al., 2008]. Refinement mechanisms employed within YAGO include canonicalisation, where each edge and node is mapped to a unique identifier and duplicate elements are removed, and type checking, where nodes that cannot be assigned to a class by deductive or inductive methods are eliminated [Suchanek et al., 2008]. YAGO would be extended in later years to support spatio-temporal context [Hoffart et al., 2011] and multilingual Wikipedias [Mahdisoltani et al., 2015].

10.1.3 FREEBASE

Freebase was a general-purpose, broad collection of human knowledge that aimed to address some of the large-scale information integration problems associated with the decentralised nature of the Semantic Web, such as uneven adoption, implementation challenges, and distributed query performance limitations [Bollacker et al., 2007a]. Unlike DBpedia and YAGO – which are mostly extracted from Wikipedia/WordNet – Freebase solicited contributions directly from human editors. Included in the Freebase platform were a scalable data store with versioning mechanisms; a large data object store (LOB) for the storage of text, image, and media files; an API that could be queried using the Metaweb Query Language (MQL); a Web user interface; and a lightweight typing system [Bollacker et al., 2007a].

[Give us feedback!](#)

[et al., 2007a](#)]. The latter typing system was designed to support collaborative processes. Rather than forcing ontological correctness or logical consistency, the system was implemented as a loose collection of structuring mechanisms – based on datatypes, semantic classes, properties, schema definitions, etc. – that allowed for incompatible types and properties to coexist simultaneously [[Bollacker et al., 2007a](#)]. Content could be added to Freebase interactively through the Web user interface or in an automated way by leveraging the API's write functionality. Freebase had been acquired by Google in 2010, where the content of Freebase formed an important part of the Google Knowledge Graph announced in 2012 [[Singhal, 2012](#)]. When Freebase became read-only as of March 2015, the knowledge graph contained over three billion edges. Much of this content was subsequently migrated to Wikidata [[Pellissier Tanon et al., 2016](#)].

10.1.4 WIKIDATA

Wikipedia contains a wealth of semi-structured data embedded in info-boxes, lists, tables, etc., as exploited by DBpedia and YAGO. However, these data have traditionally been curated and updated manually across different articles and languages; for example, a goal scored by a Chilean football player may require manual updates in the player's article, the tournament article, the team article, lists of top scorers, and so forth, across hundreds of language versions. Manual curation has led to a variety of data quality issues, including contradictory data in different articles, languages, etc. The Wikimedia Foundation uses Wikidata as a centralised, collaboratively-edited knowledge graph to supply Wikipedia – and arbitrary other clients – with data. Under this vision, a fact could be added to Wikidata once, triggering the automatic update of potentially multitudinous articles in Wikipedia across different languages [[Vrandečić and Krötzsch, 2014](#)]. Like Wikipedia, Wikidata is also considered a secondary source containing *claims* that should reference primary sources, though claims can also be initially added without reference [[Piscopo et al., 2017](#)]. Wikidata further allows for different viewpoints in terms of potentially contradictory (referenced) claims [[Vrandečić and Krötzsch, 2014](#)]. Wikidata is multilingual, where nodes and edges are assigned language-agnostic Qxx and Pxx codes (see Figure 9.1) and are subsequently associated with labels, aliases, and descriptions in various languages [[Kaffee et al., 2017](#)], allowing claims to be surfaced in these languages. Collaborative editing is not only permitted on the data level, but also on the schema level, allowing users to add or modify lightweight semantic axioms [[Piscopo and Simperl, 2018](#)] – including sub-classes, sub-properties, inverse properties, etc. – as well as shapes [[Boneva et al., 2019](#)]. Wikidata offers various access protocols [[Malyshev et al., 2018](#)] and has received broad adoption, being used by Wikipedia to generate infoboxes in certain domains [[Sáez and Hogan, 2018](#)], being supported by Google [[Pellissier Tanon et al., 2016](#)], and having been used as a data source for prominent end-user applications such as Apple's Siri, amongst others [[Malyshev et al., 2018](#)].

10.1.5 OTHER OPEN CROSS-DOMAIN KNOWLEDGE GRAPHS

Aside from DBpedia, YAGO, Freebase and Wikidata, a number of other cross-domain knowledge graphs have been developed down through the years. BabelNet [[Navigli and Ponzetto, 2012](#)], like YAGO, is based on unifying WordNet and Wikipedia, but with the integration of additional knowledge graphs such as Wikidata, and a focus on creating a knowledge graph of multilingual lexical forms (organised into multilingual synsets) by transforming lexicographic resources such as Wiktionary and OmegaWiki into knowledge graphs. Compared to other knowledge graphs, lexicalised knowledge graphs such as BabelNet bring together the encyclopedic information found in Wikipedia with the lexicographic information usually found in monolingual and bilingual dictionaries. The Cyc project [[Lenat, 1995](#)] aims to encode common-sense knowledge in a machine-readable way, where over 900 person-years of effort [[Matuszek et al., 2006](#)] have, since 1986, gone into the creation of 2.2 million facts and rules. Though Cyc is proprietary, an open subset called OpenCyc has been published, where we refer to the comparison by [Färber et al. \[2018\]](#) of DBpedia, Freebase, OpenCyc, and YAGO for further details. The Never Ending Language Learning (NELL) project [[Mitchell et al., 2018](#)] has, since 2010, extracted a graph of 120 million edges from the text of web pages using OIE methods (see Chapter 6). Each such open knowledge graph applies different combinations of the languages and techniques discussed in this book over different sources with differing results.

Give us feedback!

10.1.6 DOMAIN-SPECIFIC OPEN KNOWLEDGE GRAPHS

Open knowledge graphs have been published in a variety of specific domains. [Schmachtenberg et al. \[2014\]](#) identify the most prominent domains in the context of Linked Data as follows: *media*, relating to news, television, radio, etc. (e.g., the BBC World Service Archive [\[Raimond et al., 2014\]](#)); *government*, relating to the publication of data for transparency and development (e.g., by the U.S. [\[Hendler et al., 2012\]](#) and U.K. [\[Shadbolt and O'Hara, 2013\]](#) governments); *publications*, relating to academic literature in various disciplines (e.g., OpenCitations [\[Peroni et al., 2017\]](#), SciGraph [\[Iana et al., 2019\]](#), Microsoft Academic Knowledge Graph [\[Färber, 2019\]](#)); *geographic*, relating to places and regions of interest (e.g., LinkedGeoData [\[Stadler et al., 2012\]](#)); *life sciences*, relating to proteins, genes, drugs, diseases, etc. (e.g., Bio2RDF [\[Callahan et al., 2013\]](#)); and *user-generated content*, relating to reviews, open source projects, etc. (e.g., Revyu [\[Heath and Motta, 2008\]](#)). Open knowledge graphs have also been published in other domains, including *cultural heritage* [\[Hyvönen et al., 2009\]](#), *music* [\[Raimond et al., 2009\]](#), *law* [\[Montiel-Ponsoda et al., 2017\]](#), *theology* [\[Sherif and Ngonga Ngomo, 2015\]](#), and even *tourism* [\[Lu et al., 2016, Kärle et al., 2018, Maturana et al., 2018, Zhang et al., 2019\]](#). The envisaged applications for such knowledge graphs are as varied as the domains from which they emanate, but often relate to integration [\[Raimond et al., 2009, Callahan et al., 2013\]](#), recommendation [\[Raimond et al., 2009, Lu et al., 2016\]](#), transparency [\[Hendler et al., 2012, Shadbolt and O'Hara, 2013\]](#), archiving [\[Hyvönen et al., 2009, Raimond et al., 2014\]](#), decentralisation [\[Heath and Motta, 2008\]](#), multilingual support [\[Sherif and Ngonga Ngomo, 2015\]](#), regulatory compliance [\[Montiel-Ponsoda et al., 2017\]](#), etc.

10.2 ENTERPRISE KNOWLEDGE GRAPHS

A variety of companies have announced the creation of proprietary “enterprise knowledge graphs” with a variety of goals in mind, which include: improving search capabilities [\[Singhal, 2012, Shrivastava, 2017, Krishnan, 2018, Chang, 2018, Hamad et al., 2018\]](#), providing user recommendations [\[Chang, 2018, Hamad et al., 2018\]](#), implementing conversational/personal agents [\[Pittman et al., 2017\]](#), enhancing targeted advertising [\[He et al., 2016\]](#), empowering business analytics [\[He et al., 2016\]](#), connecting users [\[He et al., 2016, Noy et al., 2019\]](#), extending multilingual support [\[He et al., 2016\]](#), facilitating research and discovery [\[Bendtsen and Petrovski, 2019\]](#), assessing and mitigating risk [\[Tobin, 2017, Dalgliesh, 2016\]](#), tracking news events [\[Meij, 2019\]](#), and increasing transport automation [\[Henson et al., 2019\]](#), amongst (many) others. Though highly diverse, these enterprise knowledge graphs do follow some high-level trends, as reflected in the discussion by [Noy et al. \[2019\]](#): (1) data are typically integrated into the knowledge graph from a variety of both external and internal sources (often involving text); (2) the enterprise knowledge graph is often very large, with millions or even billions of nodes and edges, posing challenges in terms of scalability; (3) refinement of the initial knowledge graph – adding new links, consolidating duplicate entities, etc. – is important to improve quality; (4) techniques to keep the knowledge graph up-to-date with the domain are often crucial; (5) a mix of ontological and machine learning representations are often combined or used in different situations in order to draw conclusions from the enterprise knowledge graph; (6) the ontologies used tend to be lightweight, often simple taxonomies representing a hierarchy of classes or concepts. We now discuss the main industries in which enterprise knowledge graphs have been deployed.

10.2.1 WEB SEARCH

Web search engines have traditionally focused on matching a query string with sub-strings in web documents. The Google Knowledge Graph [\[Singhal, 2012, Noy et al., 2019\]](#) rather promoted a paradigm of “*things not strings*” – analogous to semantic search [\[Guha et al., 2003\]](#) – where the search engine would now try to identify the entities that a particular search may be expressing interest in. The knowledge graph itself describes these entities and how they interrelate. One of the main user-facing applications of the Google Knowledge Graph is the “Knowledge Panel”, which presents a pane on the right-hand side of (some) search results describing the principal entity that the search appears to be seeking, including some images, attribute–value pairs, and a list of related entities that use

Give us feedback!

also search for. The Google Knowledge Graph was key to popularising the modern usage of the phrase “knowledge graph” (see Appendix A). Other major search engines, such as Microsoft Bing³⁸ [Shrivastava, 2017], would later announce knowledge graphs along similar lines.

note 38 Microsoft’s Knowledge Graph in Japanese).

10.2.2 COMMERCE

Enterprise knowledge graphs have also been announced by companies that are principally concerned with selling or renting goods and services. A prominent example of such a knowledge graph is that used by Amazon [Krishnan, 2018, Dong, 2019], which describes the products on sale in their online marketplace. One of the main stated goals of this knowledge graph is to enable more advanced (semantic) search features for products, as well as to improve product recommendations to users of its online marketplace. Another knowledge graph for commerce was announced by eBay [Pittman et al., 2017], which encodes product descriptions and shopping behaviour patterns, and is used to power conversational agents that help users to find relevant products through a natural language interface. Airbnb [Chang, 2018] has also described a knowledge graph that encodes accommodation for rent, places, events, experiences, neighbourhoods, users, tags, etc., on top of which a taxonomic schema is defined. This knowledge graph is used to offer potential clients recommendations of attractions, events, and activities available in the neighbourhood of a particular home for rent. Uber [Hamad et al., 2018] has similarly announced a knowledge graph focused on food and restaurants for their “Uber Eats” delivery service. The goals are again to offer semantic search features and recommendations to users who are uncertain of precisely what kind of food they are looking for.

10.2.3 SOCIAL NETWORKS

Enterprise knowledge graphs have also emerged in the context of social networking services. Facebook [Noy et al., 2019] has gathered together a knowledge graph describing not only social data about users, but also the entities they are interested in, including celebrities, places, movies, music, etc., in order to connect people, understand their interests, and provide recommendations. LinkedIn [He et al., 2016] announced a knowledge graph containing users, jobs, skills, companies, places, schools, etc., on top of which a taxonomic schema is defined. The knowledge graph is used to provide multilingual translations of important concepts, to improve targeted advertising, to provide advanced features for job search and people search, and likewise to provide recommendations matching jobs to people (and vice versa). Another knowledge graph has been created by Pinterest [Gonçalves et al., 2019], describing users and their interests, the latter being organised into a taxonomy. The main use-cases for the knowledge graph are to help users to more easily find content of interest to them, as well as to enhance revenue through targeted advertisements.

10.2.4 FINANCE

The financial sector has also seen deployment of enterprise knowledge graphs. Amongst these, Bloomberg [Meij, 2019] has proposed a knowledge graph that powers financial data analytics, including sentiment analysis for companies based on current news reports and tweets, a question answering service, as well as detecting emerging events that may affect stock values. Thomson Reuters (Refinitiv) [Tobin, 2017] has likewise announced a knowledge graph encoding “the financial ecosystem” of people, organisations, equity instruments, industry classifications, joint ventures and alliances, supply chains, etc., using a taxonomic schema to organise these entities. Some of the applications they mention for the knowledge graph include supply chain monitoring, risk assessment, and investment research. Knowledge graphs have also been used for deductive reasoning, with Banca d’Italia [Bellomarini et al., 2019] using rule-based reasoning to determine, for example, the percentage of ownership of a company by various stakeholders. Other companies exploring financial knowledge graphs include Accenture [Okorafor and Ray, 2019], Capital One [Branum and Sehon, 2019], Wells Fargo [Newman, 2019], amongst various others.

10.2.5 OTHER INDUSTRIES

Give us feedback!

Enterprises have also been actively developing knowledge graphs to enable novel applications in a variety of other industries, including: *healthcare*, where IBM are exploring use-cases for drug discovery [Noy et al., 2019] and information extraction from package inserts [Gentile et al., 2019], while AstraZeneca [Bendtsen and Petrovski, 2019] are using a knowledge graph to advance genomics research and disease understanding; *transport*, where Bosch are exploring a knowledge graph of scenes and locations for driving automation [Henson et al., 2019]; *oil & gas*, where Maana [Dalglish, 2016] are using knowledge graphs to perform data integration for risk mitigation regarding oil wells and drilling; and more besides.

Chapter 11

Summary and Conclusion

We have provided a comprehensive introduction to knowledge graphs, which have been receiving more and more attention in recent years. Under the definition of a knowledge graph as *a graph of data intended to accumulate and convey knowledge of the real world, whose nodes represent entities of interest and whose edges represent relations between these entities*, we have discussed models by which data can be structured as graphs; representations of schema, identity and context; techniques for leveraging deductive and inductive knowledge; methods for the creation, enrichment, quality assessment and refinement of knowledge graphs; principles and standards for publishing knowledge graphs; and finally, we have discussed the adoption of both open and enterprise knowledge graphs in the real world.

In this final chapter, we provide some concluding remarks, and further offer some insights on potential future directions for research on knowledge graphs.

Concluding remarks. Knowledge graphs have garnered significant attention not only from diverse organisations and industries, but also diverse research communities. This attention is due, in no small part, to the ubiquitous nature of the problem that knowledge graphs address: integrating and extracting value from diverse sources of data at large scale, be it in the context of a particular organisation, community, or more general collections of human knowledge. The key insight of knowledge graphs is that graphs provide a simple, flexible, intuitive and yet powerful abstraction for representing and integrating diverse data at large scale. This insight is far from new (see Appendix A), but rather has finally come of age with the advent of knowledge graphs. Graphs have long been used to represent data and knowledge in areas such as Graph Algorithms and Theory, Graph Databases, Information Extraction, Knowledge Representation, Machine Learning, the Semantic Web, and more besides. The advances in these areas can now be unified and applied for knowledge graphs.

Thus, the decision to model data as a graph opens up a “tool-box” of languages, techniques and systems – stemming from diverse areas – that can be deployed in order to integrate and extract value from data at large scale, as follows:

- A variety of *graph query languages* are now available that (unlike other NoSQL alternatives) are fully-featured, supporting not only the relational algebra, but also novel features such as navigational queries that can match paths of arbitrary length. A broad selection of graph databases and user interfaces supporting these query languages are now also available.
- Though graphs do not depend on a detailed (relational-like) schema to represent data, various notions of *graph schemata* have been proposed in order to validate, summarise and define the semantics of graphs.
- *Contextual frameworks for graphs* can be used to represent and reason about the scope of truth of knowledge in the graph – relating to the time, space, provenance, confidence level, etc., for which something is held true – including various alternatives for reification, annotated graph frameworks, etc.

Give us feedback!

- Deductive forms of reasoning can be enabled over graphs using *ontologies* and/or *rules*, which can not only encode a machine-readable consensus about the meaning of the graph, but also provide automated access to implicit knowledge entailed by a graph through materialisation or query rewriting.
- *Graph algorithms*, such as centrality measures, community detection, clustering, etc., can be applied on the data to gain insights about influential entities or edges, close-knit sub-graphs of entities, and more besides, with *graph parallel frameworks* capable of applying such algorithms at large scale.
- Recent and continual advances in *knowledge graph embeddings* and *graph neural networks* have now opened up new possibilities for applying machine learning natively over graphs in the context of diverse tasks, including classification, question answering, recommendations, and more besides.
- *Rule and axiom mining* techniques allow for extracting formal, declarative hypotheses from a knowledge graph that encode high-level patterns and can be applied to derive new knowledge in a deductive, explainable manner.
- *Graph-based information extraction* can be applied to extract and/or enrich a knowledge graph from legacy sources of text and semi-structured data, while *graph-based mapping languages* facilitate integrating diverse sources of legacy structured data into the knowledge graph.
- Tools, techniques and methodologies for *ontology engineering* and *ontology learning* can further guide the – potentially collaborative – creation of an ontology for the knowledge graph, encoding a consensus about its semantics, and enabling access to implicit knowledge through deductive reasoning.
- *Quality dimensions and metrics for knowledge graphs* allow for systematically assessing the readiness of the knowledge graph for its envisaged applications, in both a qualitative and quantitative manner, where a variety of tools and frameworks are available to help perform such assessments.
- Knowledge graphs that have been integrated from diverse sources are likely to be incomplete, or to encode incorrect data, where techniques and tools for *knowledge graph refinement* facilitate the automated completion and correction of knowledge graphs, thus improving its overall quality and usefulness.
- For the purposes of publishing open knowledge graphs, *principles & best practices* and *access protocols*, as well as techniques for *linking*, *licensing*, *access & usage control*, *encryption* and *anonymisation*, can be leveraged to maximise their potential impact on society in an ethical way.

As we have discussed in Chapter 10, the various components of this “knowledge graph tool-box” can already be found deployed in practice, having been applied – to varying degrees – in the context of numerous open and enterprise knowledge graphs. As adoption of knowledge graphs continues, work will also continue on improving and combining these tools, as well as on developing novel tools that help to better integrate and extract value from diverse sources of data at large scale.

Future directions. Research on knowledge graphs involves a confluence of techniques from different research areas with the common objective of maximising the knowledge – and thus value – that can be distilled from diverse sources at large scale using a graph-based data abstraction [Hogan, 2020a]. In the intersection of data graphs and deductive knowledge, we emphasise emerging topics such as *formal semantics for property graphs*, with languages that can take into account the meaning of labels and property-value pairs on nodes and edges [Krötzsch et al., 2018]; and *reasoning and querying over contextual data*, in order to derive conclusions and results valid in a particular setting [Serafini and Homola, 2012, Zimmermann et al., 2012, Schuetz et al., 2021]. In the intersection of data graphs and inductive knowledge, we highlight topics such as *similarity-based query relaxation*, allowing to find approximate answers to exact queries based on numerical representations (e.g., embeddings) [Wang et al., 2018]; *shape induction*, in order to learn and formalise inherent patterns in the knowledge graph as constraints [Mihindukulasooriya et al., 2018]; and *contextual knowledge graph embeddings* that provide numeric representations of nodes and edges that vary with time, place, etc. [Kazemi et al., 2019]. In the intersection of deductive and inductive knowledge, we mention the topics of *entailment-aware knowledge graph embeddings* [Guo et al., 2016, Demeester et al., 2016], that incorporate rules and/or ontologies when computing plausibility; *expressive graph neural networks* proven capable of complex classification analogous to expressive ontology languages [Barceló et al., 2020]; as well as further advances on *rule and axiom mining*, allowing to extract symbolic, deductive

representations from the knowledge graphs [Galárraga et al., 2015, Bühmann et al., 2016]. Further challenges arise when considering the creation, enrichment, refinement and publication of knowledge graphs, which call for further works on topics such as *automated quality assessment (and repair)*, *distantly-supervised extraction frameworks*, *efficient access protocols*, and *anonymisation*, to name but a few.

Aside from specific topics, more general challenges for knowledge graphs include *scalability*, particularly for deductive and inductive reasoning; *quality*, not only in terms of data, but also the models induced from knowledge graphs; *diversity*, such as managing contextual or multi-modal data; *dynamicty*, considering temporal or streaming data; and finally *usability*, which is key to increasing adoption. Though techniques are continuously being proposed to address these challenges, they are unlikely to ever be completely “solved”; rather they serve as dimensions along which knowledge graphs, and their techniques, tools, etc., will continue to mature.

Given the availability of open knowledge graphs whose quality continues to improve, as well as the growing adoption of enterprise knowledge graphs in various industries, future research on knowledge graphs has the potential to foster key advancements in broad aspects of society. Here we have highlighted just some examples of future research directions of importance to this pursuit.

Bibliography

- Serge Abiteboul. 1997. [Querying Semi-Structured Data](#). In *Database Theory - ICDT '97, 6th International Conference, Delphi, Greece, January 8-10, 1997, Proceedings*, Foto N. Afrati and Phokion G. Kolaitis (Eds.). Lecture Notes in Computer Science, vol. 1186. Springer, 1–18.
- Sushant Agarwal, Simon Steyskal, Franjo Antunovic, and Sabrina Kirrane. 2018. [Legislative Compliance Assessment: Framework, Model and GDPR Instantiation](#). In *Privacy Technologies and Policy - 6th Annual Privacy Forum, APF 2018, Barcelona, Spain, June 13-14, 2018, Revised Selected Papers*, Manel Medina, Andreas Mittrakas, Kai Rannenberg, Erich Schweighofer, and Nikolaos Tsouroulas (Eds.). Lecture Notes in Computer Science, vol. 11079. Springer, 131–149.
- Rakesh Agrawal, Tomasz Imieliński, and Arun Swami. 1993. [Mining association rules between sets of items in large databases](#). In *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data, Washington, DC, USA, May 26-28, 1993*, Peter Buneman and Sushil Jajodia (Eds.). ACM Press, 207–216.
- Jinhyun Ahn, Dong-Hyuk Im, Jae-Hong Eom, Nansu Zong, and Hong-Gee Kim. 2015. [G-Diff: A Grouping Algorithm for RDF Change Detection on MapReduce](#). In *Semantic Technology - 4th Joint International Conference, JIST 2014, Chiang Mai, Thailand, November 9-11, 2014. Revised Selected Papers*, Thepchai Supnithi, Takahira Yamaguchi, Jeff Z. Pan, Vilas Wuwongse, and Marut Buranarach (Eds.). Lecture Notes in Computer Science, vol. 8943. Springer, 230–235.
- Adnan Akhter, Axel-Cyrille Ngonga Ngomo, and Muhammad Saleem. 2018. [An Empirical Evaluation of RDF Graph Partitioning Techniques](#). In *Knowledge Engineering and Knowledge Management - 21st International Conference, EKAW 2018, Nancy, France, November 12-16, 2018, Proceedings*, Catherine Faron-Zucker, Chiara Ghidini, Amedeo Napoli, and Yannick Toussaint (Eds.). Lecture Notes in Computer Science, vol. 11313. Springer, 3–18.
- Keith Alexander, Richard Cyganiak, Michael Hausenblas, and Jun Zhao. 2009. [Describing Linked Datasets](#). In *Proceedings of the WWW2009 Workshop on Linked Data on the Web, LDOW 2009, Madrid, Spain, April 20, 2009*, Christian Bizer, Tom Heath, Tim Berners-Lee, and Michael Hausenblas (Eds.). CEUR Workshop Proceedings, vol. 538. Sun SITE Central Europe (CEUR). 10 pages.
- Renzo Angles and Claudio Gutierrez. 2008. [Survey of graph database models](#). *ACM Computing Surveys* 40(1), 1:1–1:39.
- Renzo Angles, Marcelo Arenas, Pablo Barceló, Aidan Hogan, Juan L. Reutter, and Domagoj Vrgoč. 2017. [Foundations of Modern Query Languages for Graph Databases](#). *ACM Computing Surveys* 50(5), 68:–

Give us feedback!

- 68:40.
- Renzo Angles, Marcelo Arenas, Pablo Barceló, Peter A. Boncz, George H. L. Fletcher, Claudio Gutierrez, Tobias Lindaaker, Marcus Paradies, Stefan Plantikow, Juan F. Sequeda, Oskar van Rest, and Hannes Voigt. 2018. [G-CORE: A Core for Future Graph Query Languages](#). In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018*, Gautam Das, Christopher M. Jermaine, and Philip A. Bernstein (Eds.). ACM Press, 1421–1432.
- Renzo Angles, Harsh Thakkar, and Dominik Tomaszuk. 2019. [RDF and Property Graphs Interoperability: Status and Issues](#). In *Proceedings of the 13th Alberto Mendelzon International Workshop on Foundations of Data Management, Asunción, Paraguay, June 3-7, 2019*, Aidan Hogan and Tova Milo (Eds.). CEUR Workshop Proceedings, vol. 2369. Sun SITE Central Europe (CEUR). 11 pages.
- Renzo Angles. 2018. [The Property Graph Database Model](#). In *Proceedings of the 12th Alberto Mendelzon International Workshop on Foundations of Data Management, Cali, Colombia, May 21–25, 2018*, Dan Olteanu and Barbara Poblete (Eds.). CEUR Workshop Proceedings, vol. 2100. Sun SITE Central Europe (CEUR). 10 pages.
- Mikel Egaña Aranguren, Erick Antezana, Martin Kuiper, and Robert Stevens. 2008. [Ontology Design Patterns for bio-ontologies: a case study on the Cell Cycle Ontology](#). *BMC Bioinformatics* 9(5), pS1.
- Marcelo Arenas, Alexandre Bertails, Eric Prud'hommeaux, and Juan Sequeda. 2012. [A Direct Mapping of Relational Data to RDF, W3C Recommendation 27 September 2012](#). W3C Recommendation. World Wide Web Consortium. September 27, 2012.
- Marcelo Arenas, Bernardo Cuenca Grau, Evgeny Kharlamov, Sarunas Marciuska, and Dmitriy Zheleznyakov. 2016. [Faceted search over RDF-based knowledge graphs](#). *Journal of Web Semantics* 37–38, 55–74.
- Alessandro Artale, Diego Calvanese, Roman Kontchakov, and Michael Zakharyashev. 2009. [The DL-Lite Family and Relations](#). *Journal of Artificial Intelligence Research* 36, 1–69.
- Sören Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and Zachary Ives. 2007. [DBpedia: A Nucleus for a Web of Open Data](#). In *The Semantic Web, 6th International Semantic Web Conference, 2nd Asian Semantic Web Conference, ISWC 2007 + ASWC 2007, Busan, Korea, November 11-15, 2007*, Karl Aberer, Key-Sun Choi, Natasha Fridman Noy, Dean Allemang, Kyung-Il Lee, Lyndon J. B. Nixon, Jennifer Golbeck, Peter Mika, Diana Maynard, Riichiro Mizoguchi, Guus Schreiber, and Philippe Cudré-Mauroux (Eds.). Lecture Notes in Computer Science, vol. 4825. Springer, 722–735.
- Franz Baader, Ian Horrocks, Carsten Lutz, and Ulrike Sattler. 2017. [An Introduction to Description Logic](#). Cambridge University Press.
- Nguyen Bach and Sameer Badaskar. 2007. [A Review of Relation Extraction](#). Carnegie Mellon University.
- Ricardo Baeza-Yates. 2018. [Bias on the Web](#). *Communications of the ACM* 61(6), 54–61.
- Collin F. Baker, Charles J. Fillmore, and John B. Lowe. 1998. [The Berkeley FrameNet Project](#). In *36th Annual Meeting of the Association for Computational Linguistics and 17th International Conference on Computational Linguistics, COLING-ACL'98, August 10-14, 1998, Université de Montréal, Montréal, Quebec, Canada. Proceedings of the Conference*, Christian Boitet and Pete Whitelock (Eds.). Morgan Kaufmann / The Association for Computational Linguistics, 86–90.
- René Ronald Bakker. 1987. *Knowledge Graphs: Representation and Structuring of Scientific Knowledge*. Ph.D. dissertation. University of Twente, Enschede, Netherlands.
- Ivana Balazevic, Carl Allen, and Timothy M. Hospedales. 2019. [Multi-relational Poincaré Graph Embeddings](#). In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, 8–14 December 2019, Vancouver, BC, Canada*, Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett (Eds.), 4465–4475.
- Ivana Balazevic, Carl Allen, and Timothy M. Hospedales. 2019. [Tucker: Tensor Factorization for Knowledge Graph Completion](#). In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019*, Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.). The Association for Computational Linguistics, 5184–5193.
- Ivana Balazevic, Carl Allen, and Timothy M. Hospedales. 2019. [Hypernetwork Knowledge Graph Embeddings](#). In *Artificial Neural Networks and Machine Learning - ICANN 2019 - 28th International Conference on Artificial Neural Networks, Munich, Germany, September 17-19, 2019, Proceedings*. [Give us feedback!](#)

- Workshop and Special Sessions*, Igor V. Tetko, Vera Kurková, Pavel Karpov, and Fabian J. Theis (Eds.). Lecture Notes in Computer Science, vol. 11731. Springer, 553–565.
- Michele Banko, Michael J. Cafarella, Stephen Soderland, Matthew Broadhead, and Oren Etzioni. 2007. [Open Information Extraction from the Web](#). In *IJCAI 2007, Proceedings of the 20th International Joint Conference on Artificial Intelligence, Hyderabad, India, January 6-12, 2007*, Manuela M. Veloso (Ed.). AAAI Press, 2670–2676.
- Pablo Barceló, Egor V. Kostylev, Mikael Monet, Jorge Pérez, Juan Reutter, and Juan Pablo Silva. 2020. [The Logical Expressiveness of Graph Neural Networks](#). In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26–30, 2020*. OpenReview.net. 20 pages.
- Hannah Bast and Björn Buchhold. 2013. [An index for efficient semantic full-text search](#). In *22nd ACM International Conference on Information and Knowledge Management, CIKM'13, San Francisco, CA, USA, October 27 - November 1, 2013*, Qi He, Arun Iyengar, Wolfgang Nejdl, Jian Pei, and Rajeev Rastogi (Eds.). ACM Press, 369–378.
- Carlo Batini and Monica Scannapieco. 2016. [Data and Information Quality - Dimensions, Principles and Techniques](#). Data-Centric Systems and Applications. Springer.
- Carlo Batini, Anisa Rula, Monica Scannapieco, and Gianluigi Viscusi. 2015. [From Data Quality to Big Data Quality](#). *Journal of Database Management* 26(1), 60–82.
- Luigi Bellomarini, Emanuel Sallinger, and Georg Gottlob. 2018. [The Vadalogue System: Datalog-based Reasoning for Knowledge Graphs](#). *Proceedings of the VLDB Endowment* 11(9), 975–987.
- Luigi Bellomarini, Daniele Fakhoury, Georg Gottlob, and Emanuel Sallinger. 2019. [Knowledge Graphs and Enterprise AI: The Promise of an Enabling Technology](#). In *35th IEEE International Conference on Data Engineering, ICDE 2019, Macao, China, April 8-11, 2019*. IEEE Computer Society, 26–37.
- Claus Bendtsen and Slavé Petrovski. 2019. [How data and AI are helping unlock the secrets of disease](#). AstraZeneca Blog. November 1, 2019.
- Michael K. Bergman. 2019. [A Common Sense View of Knowledge Graphs](#). Adaptive Information, Adaptive Innovation, Adaptive Infrastructure Blog. July 1, 2019.
- Tim Berners-Lee and Dan Connolly. 2011. [Notation3 \(N3\): A readable RDF syntax](#). W3C Team Submission. World Wide Web Consortium. March 28, 2011.
- Tim Berners-Lee, James Hendler, and Ora Lassila. 2001. [The Semantic Web](#). *Scientific American* 284(5), 34–43.
- Tim Berners-Lee. 2006. [Linked Data](#). W3C Design Issues. July, 2006.
- Anant P. Bhardwaj, Souvik Bhattacharjee, Amit Chavan, Amol Deshpande, Aaron J. Elmore, Samuel Madden, and Aditya G. Parameswaran. 2015. [DataHub: Collaborative Data Science & Dataset Version Management at Scale](#). In *CIDR 2015, Seventh Biennial Conference on Innovative Data Systems Research, Asilomar, CA, USA, January 4-7, 2015, Online Proceedings*. www.cidrdb.org. 7 pages.
- Daniel M. Bikel, Richard M. Schwartz, and Ralph M. Weischedel. 1999. [An Algorithm that Learns What's in a Name](#). *Machine Learning* 34(1–3), 211–231.
- Stefan Bischof, Stefan Decker, Thomas Krennwallner, Nuno Lopes, and Axel Polleres. 2012. [Mapping between RDF and XML with XSPARQL](#). *Journal of Web Semantics* 1(3), 147–185.
- Christian Bizer, Jens Lehmann, Georgi Kobilarov, Sören Auer, Christian Becker, Richard Cyganiak, and Sebastian Hellmann. 2009. [DBpedia - A crystallization point for the Web of Data](#). *Journal of Web Semantics* 7(3), 154–165.
- Eva Blomqvist and Kurt Sandkuhl. 2005. [Patterns in Ontology Engineering: Classification of Ontology Patterns](#). In *ICEIS 2005, Proceedings of the Seventh International Conference on Enterprise Information Systems, Miami, USA, May 25-28, 2005*, Chin-Sheng Chen, Joaquim Filipe, Isabel Seruca, and José Cordeiro (Eds.), vol. 3, 413–416.
- Eva Blomqvist, Azam Seil Sepour, and Valentina Presutti. 2012. [Ontology Testing - Methodology and Tool](#). In *Knowledge Engineering and Knowledge Management - 18th International Conference, EKAW 2012, Galway City, Ireland, October 8-12, 2012. Proceedings*, Annette ten Teije, Johanna Völker, Siegfried Handschuh, Heiner Stuckenschmidt, Mathieu d'Aquin, Andriy Nikolov, Nathalie Aussenac-Gilles, and Nathalie Hernandez (Eds.). Lecture Notes in Computer Science, vol. 7603. Springer, 216–226.
- Eva Blomqvist, Karl Hammar, and Valentina Presutti. 2016. [Engineering Ontologies with Patterns – The eXtreme Design Methodology](#). In *Ontology Engineering with Ontology Design Patterns - Foundations and Applications*, Pascal Hitzler, Aldo Gangemi, Krzysztof Janowicz, Adila Krisnadhi, and Valentina Presutti (Eds.). Studies on the Semantic Web, vol. 25. IOS Press, 23–50.

- Kurt Bollacker, Patrick Tufts, Tomi Pierce, and Robert Cook. 2007. [A platform for scalable, collaborative, structured information integration](#). In *Intl. Workshop on Information Integration on the Web (IIWeb'07)*, Ullas Nambiar and Zaiqing Nie (Eds.). 6 pages.
- Kurt Bollacker, Robert Cook, and Patrick Tufts. 2007. [Freebase: A Shared Database of Structured General Human Knowledge](#). In *Proceedings of the Twenty-Second AAAI Conference on Artificial Intelligence*, July 22-26, 2007, Vancouver, British Columbia, Canada. AAAI Press, 1962–1963.
- Piero A. Bonatti and Sabrina Kirrane. 2019. [Big Data and Analytics in the Age of the GDPR](#). In *2019 IEEE International Congress on Big Data, BigData Congress 2019, Milan, Italy, July 8-13, 2019*, Elisa Bertino, Carl K. Chang, Peter Chen, Ernesto Damiani, Michael Goul, and Katsunori Oyama (Eds.). IEEE Computer Society, 7–16.
- Piero A. Bonatti, Aidan Hogan, Axel Polleres, and Luigi Sauro. 2011. [Robust and scalable Linked Data reasoning incorporating provenance and trust annotations](#). *Journal of Web Semantics* 9(2), 165–201.
- Piero Andrea Bonatti, Stefan Decker, Axel Polleres, and Valentina Presutti. 2018. [Knowledge Graphs: New Directions for Knowledge Representation on the Semantic Web \(Dagstuhl Seminar 18371\)](#). *Dagstuhl Reports* 8(9), 29–111.
- Piero Bonatti, Sabrina Kirrane, Iliana Mineva Petrova, Luigi Sauro, and Eva Schlehahn. 2019. [The SPECIAL Usage Policy Language, version 1.0](#). Unofficial Draft. Vienna University of Economics and Business. December 31, 2019.
- Iovka Boneva, Jose Emilio Labra Gayo, and Eric G. Prud'hommeaux. 2017. [Semantics and Validation of Shapes Schemas for RDF](#). In *The Semantic Web - ISWC 2017 - 16th International Semantic Web Conference, Vienna, Austria, October 21-25, 2017, Proceedings, Part I*, Claudia d'Amato, Miriam Fernández, Valentina A. M. Tamma, Freddy Lécué, Philippe Cudré-Mauroux, Juan F. Sequeda, Christoph Lange, and Jeff Heflin (Eds.). Lecture Notes in Computer Science, vol. 10587. Springer, 104–120.
- Iovka Boneva, Jérémie Dusart, Daniel Fernández-álvarez, and José Emilio Labra Gayo. 2019. [Shape Designer for ShEx and SHACL constraints](#). In *Proceedings of the ISWC 2019 Satellite Tracks (Posters & Demonstrations, Industry, and Outrageous Ideas) co-located with 18th International Semantic Web Conference (ISWC 2019), Auckland, New Zealand, October 26-30, 2019*, Mari Carmen Suárez-Figueroa, Gong Cheng, Anna Lisa Gentile, Christophe Guéret, C. Maria Keet, and Abraham Bernstein (Eds.). CEUR Workshop Proceedings, vol. 2456. Sun SITE Central Europe (CEUR), 269–272.
- Angela Bonifati, Wim Martens, and Thomas Timm. 2017. [An Analytical Study of Large SPARQL Query Logs](#). *Proceedings of the VLDB Endowment* 11(2), 149–161.
- Antoine Bordes, Nicolas Usunier, Alberto García-Durán, Jason Weston, and Oksana Yakhnenko. 2013. [Translating Embeddings for Modeling Multi-relational Data](#). In *Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013. Proceedings of a meeting held December 5-8, 2013, Lake Tahoe, Nevada, United States*, Christopher J. C. Burges, Léon Bottou, Zoubin Ghahramani, and Kilian Q. Weinberger (Eds.), 2787–2795.
- Gerlof Bouma. 2009. [Normalized \(Pointwise\) Mutual Information in Collocation Extraction](#). In *Von der Form zur Bedeutung: Texte automatisch verarbeiten - From Form to Meaning: Processing Texts Automatically, Proceedings of the Biennial GSCL Conference 2009*, Christian Chiarcos, Richard Eckart de Castilho, and Manfred Stede (Eds.). Gunter Narr Verlag, 31–40.
- Ronald J. Brachman and Hector J. Levesque. 1986. [The Knowledge Level of a KBMS](#). In *On Knowledge Base Management Systems: Integrating Artificial Intelligence and Database Technologies, Book resulting from the Islamorada Workshop 1985 (Islamorada, FL, USA)*, Michael L. Brodie and John Mylopoulos (Eds.). Topics in Information Systems. Springer, 9–12.
- Ronald J. Brachman and James G. Schmolze. 1985. [An Overview of the KL-ONE Knowledge Representation System](#). *Cognitive Science* 9(2), 171–216.
- Ronald J. Brachman. 1977. [A Structural Paradigm for Representing Knowledge](#). Ph.D. dissertation. Harvard University.
- Patricia Branum and Bethany Sehon. 2019. [Knowledge Graph Pilot Improves Data Quality While Providing a Customer 360 View](#). Invited talk at the Knowledge Graph Conference.
- Dan Brickley and R. V. Guha. 2014. [RDF Schema 1.1](#). W3C Recommendation. World Wide Web Consortium. February 25, 2014.
- Joan Bruna, Wojciech Zaremba, Arthur Szlam, and Yann LeCun. 2014. [Spectral Networks and Locally Connected Networks on Graphs](#). In *2nd International Conference on Learning Representations, ICLR*

- 2014, Banff, AB, Canada, April 14-16, 2014, *Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). OpenReview.net. 14 pages.
- Bruce G. Buchanan and Edward A. Feigenbaum. 1978. [Dendral and Meta-Dendral: Their Applications Dimension](#). *Artificial Intelligence* 11(1-2), 5-24.
- Lorenz Bühmann, Jens Lehmann, and Patrick Westphal. 2016. [DL-Learner – A framework for inductive learning on the Semantic Web](#). *Journal of Web Semantics* 39, 15-24.
- Carlos Buil-Aranda, Marcelo Arenas, Óscar Corcho, and Axel Polleres. 2013. [Federating queries in SPARQL 1.1: Syntax, semantics and evaluation](#). *Journal of Web Semantics* 18(1), 1-17.
- Carlos Buil-Aranda, Aidan Hogan, Jürgen Umbrich, and Pierre-Yves Vandenbussche. 2013. [SPARQL Web-Querying Infrastructure: Ready for Action?](#). In *The Semantic Web - ISWC 2013 - 12th International Semantic Web Conference, Sydney, NSW, Australia, October 21-25, 2013, Proceedings, Part II*, Harith Alani, Lalana Kagal, Achille Fokoue, Paul Groth, Chris Biemann, Josian Xavier Parreira, Lora Aroyo, Natasha Noy, Chris Welty, and Krzysztof Janowicz (Eds.). Lecture Notes in Computer Science, vol. 8219. Springer, 277-293.
- Paul Buitelaar, Philipp Cimiano, and Bernardo Magnini (Eds.). 2005. *Ontology learning from text: methods, evaluation and applications*. Frontiers in Artificial Intelligence and Applications, vol. 123. IOS Press.
- Razvan C. Bunescu and Raymond J. Mooney. 2005. [Subsequence Kernels for Relation Extraction](#). In *Advances in Neural Information Processing Systems 18 [Neural Information Processing Systems, NIPS 2005, December 5-8, 2005, Vancouver, British Columbia, Canada]*, Yair Weiss, Bernhard Schölkopf, and John Platt (Eds.), vol. 18. The MIT Press, 171-178.
- Razvan C. Bunescu and Raymond J. Mooney. 2007. [Learning to Extract Relations from the Web using Minimal Supervision](#). In *ACL 2007, Proceedings of the 45th Annual Meeting of the Association for Computational Linguistics, June 23-30, 2007, Prague, Czech Republic*, John A. Carroll, Antal van den Bosch, and Annie Zaenen (Eds.). The Association for Computational Linguistics, 576-583.
- Elena Cabrio, Alessio Palmero Arosio, and Serena Villata. 2014. [These Are Your Rights - A Natural Language Processing Approach to Automated RDF Licenses Generation](#). In *The Semantic Web: Trends and Challenges - 11th International Conference, ESWC 2014, Anissaras, Crete, Greece, May 25-29, 2014. Proceedings*, Valentina Presutti, Claudia d'Amato, Fabien Gandon, Mathieu d'Aquin, Stephen Staab, and Anna Tordia (Eds.). Lecture Notes in Computer Science, vol. 8465. Springer, 255-269.
- Michael J. Cafarella, Alon Y. Halevy, Daisy Zhe Wang, Eugene Wu, and Yang Zhang. 2008. [WebTables: exploring the power of tables on the web](#). *Proceedings of the VLDB Endowment* 1(1), 538-549.
- HongYun Cai, Vincent W. Zheng, and Kevin Chen-Chuan Chang. 2018. [A Comprehensive Survey of Graph Embedding: Problems, Techniques, and Applications](#). *IEEE Transactions on Knowledge and Data Engineering* 30(9), 1616-1637.
- Alison Callahan, Jose Cruz-Toledo, Peter Ansell, and Michel Dumontier. 2013. [Bio2RDF Release 2: Improved Coverage, Interoperability and Provenance of Life Science Linked Data](#). In *The Semantic Web: Semantics and Big Data, 10th International Conference, ESWC 2013, Montpellier, France, May 26-30, 2013. Proceedings*, Philipp Cimiano, Óscar Corcho, Valentina Presutti, Laura Hollink, and Sebastian Rudolph (Eds.). Lecture Notes in Computer Science, vol. 7882. Springer, 200-212.
- Nicholas J. Car, Paul J. Box, and Ashley Sommer. 2019. [The Location Index: A Semantic Web Spatial Data Infrastructure](#). In *The Semantic Web - 16th International Conference, ESWC 2019, Portorož, Slovenia, June 2-6, 2019, Proceedings*, Pascal Hitzler, Miriam Fernández, Krzysztof Janowicz, Amrapali Zaveri, Alasdair J. G. Gray, Vanessa López, Armin Haller, and Karl Hammar (Eds.). Lecture Notes in Computer Science, vol. 11503. Springer, 543-557.
- Šejla Čebirić, François Goasdoué, Haridimos Kondylakis, Dimitris Kotzinos, Ioana Manolescu, Georgia Troullinou, and Mussab Zneika. 2019. [Summarizing semantic graphs: a survey](#). *The Very Large Data Base Journal* 28(3), 295-327.
- Stefano Ceri, Georg Gottlob, and Letizia Tanca. 1989. [What you Always Wanted to Know About Datalog \(And Never Dared to Ask\)](#). *IEEE Transactions on Knowledge and Data Engineering* 1(1), 146-166.
- Spencer Chang. 2018. [Scaling Knowledge Access and Retrieval at Airbnb](#). AirBnB Medium Blog. September 4, 2018.
- Laura Chiticariu, Yunyao Li, and Frederick R. Reiss. 2013. [Rule-Based Information Extraction is Dead! Long Live Rule-Based Information Extraction Systems!](#). In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, EMNLP 2013, 18-21 October 2013, Grand Hyatt Seattle, Seattle, Washington, USA, A meeting of SIGDAT, a Special Interest Group of the ACL*, David

- Yarowsky, Timothy Baldwin, Anna Korhonen, Karen Livescu, and Steven Bethard (Eds.). The Association for Computational Linguistics, 827–832.
- Laura Chiticariu, Marina Danilevsky, Yunyao Li, Frederick Reiss, and Huaiyu Zhu. 2018. [SystemT: Declarative Text Understanding for Enterprise](#). In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2018, New Orleans, Louisiana, USA, June 1-6, 2018, Volume 3 (Industry Papers)*, Srinivas Bangalore, Jennifer Chu-Carroll, and Yunyao Li (Eds.). The Association for Computational Linguistics, 76–83.
- Giovanni Luca Ciampaglia, Prashant Shiralkar, Luis M Rocha, Johan Bollen, Filippo Menczer, and Alessandro Flammini. 2015. [Computational fact checking from knowledge networks](#). *PLOS One* 10(6), pe0128193.
- Aaron Clemmer and Stephen Davies. 2011. [Smeagol: A "Specific-to-General" Semantic Web Query Interface Paradigm for Novices](#). In *Database and Expert Systems Applications - 22nd International Conference, DEXA 2011, Toulouse, France, August 29 - September 2, 2011. Proceedings, Part I*, Abdelkader Hameurlain, Stephen W. Liddle, Klaus-Dieter Schewe, and Xiaofang Zhou (Eds.). Springer, 288–302.
- Michael Cochez, Petar Ristoski, Simone Paolo Ponzetto, and Heiko Paulheim. 2017. [Biased Graph Walks for RDF Graph Embeddings](#). In *Proceedings of the 7th International Conference on Web Intelligence, Mining and Semantics, WIMS 2017, Amantea, Italy, June 19-22, 2017*, Rajendra Akerkar, Alfredo Cuzzocrea, Jannong Cao, and Mohand-Said Hacid (Eds.). ACM Press, 21:1–21:12.
- Michael Cochez, Petar Ristoski, Simone Paolo Ponzetto, and Heiko Paulheim. 2017. [Global RDF Vector Space Embeddings](#). In *The Semantic Web - ISWC 2017 - 16th International Semantic Web Conference, Vienna, Austria, October 21-25, 2017, Proceedings, Part I*, Claudia d'Amato, Miriam Fernández, Valentina A. M. Tamma, Freddy Lécué, Philippe Cudré-Mauroux, Juan F. Sequeda, Christoph Lange, and Jeff Heflin (Eds.). Lecture Notes in Computer Science, vol. 10587. Springer, 190–207.
- Diego Collarana, Mikhail Galkin, Christoph Lange, Irlán Grangel-González, Maria-Esther Vidal, and Sören Auer. 2016. [FuhSen: A Federated Hybrid Search Engine for Building a Knowledge Graph On-Demand \(Short Paper\)](#). In *On the Move to Meaningful Internet Systems: OTM 2016 Conferences - Confederated International Conferences: CoopIS, C&TC, and ODBASE 2016, Rhodes, Greece, October 24-28, 2016, Proceedings*, Christophe Debruyne, Hervé Panetto, Robert Meersman, Tharam S. Dillon, eva Kühn, Declan O'Sullivan, and Claudio Agostino Ardagna (Eds.). Lecture Notes in Computer Science, vol. 10033. Springer, 752–761.
- Michael Collins and Yoram Singer. 1999. [Unsupervised Models for Named Entity Classification](#). In *Joint SIGDAT Conference on Empirical Methods in Natural Language Processing and Very Large Corpora, EMNLP 1999, College Park, MD, USA, June 21-22, 1999*. The Association for Computational Linguistics. 11 pages.
- Dan Connolly. 2007. [Gleaning Resource Descriptions from Dialects of Languages \(GRDDL\)](#), *W3C Recommendation 11 September 2007*. W3C Recommendation. World Wide Web Consortium. September 11, 2007.
- Mariano P. Consens and Alberto O. Mendelzon. 1990. [GraphLog: a Visual Formalism for Real Life Recursion](#). In *Proceedings of the Ninth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, April 2-4, 1990, Nashville, Tennessee, USA*, Daniel J. Rosenkrantz and Yehoshua Sagiv (Eds.). ACM Press, 404–416.
- Olivier Corby and Catherine Faron-Zucker. 2010. [The KGRAM Abstract Machine for Knowledge Graph Querying](#). In *2010 IEEE/WIC/ACM International Conference on Web Intelligence, WI 2010, Toronto, Canada, August 31 - September 3, 2010, Main Conference Proceedings*, Jimmy Xiangji Huang, Irwin King, Vijay V. Raghavan, and Stefan Rueger (Eds.). IEEE Computer Society, 338–341.
- Francesco Corcoglioniti, Marco Rospocher, and Alessio Palmero Arosio. 2016. [Frame-Based Ontology Population with PIKES](#). *IEEE Transactions on Knowledge and Data Engineering* 28(12), 3261–3275.
- Julien Corman, Juan L. Reutter, and Ognjen Savković. 2018. [Semantics and Validation of Recursive SHACL](#). In *The Semantic Web - ISWC 2018 - 17th International Semantic Web Conference, Monterey, CA, USA, October 8-12, 2018, Proceedings, Part I*, Denny Vrandečić, Kalina Bontcheva, Mari Carmen Suárez-Figueroa, Valentina Presutti, Irene Celino, Marta Sabou, Lucie-Aimée Kaffee, and Elena Simperl (Eds.). Lecture Notes in Computer Science, vol. 11136. Springer, 318–336.
- Julien Corman, Fernando Florenzano, Juan L. Reutter, and Ognjen Savkovic. 2019. [Validating SHACL Constraints over a SPARQL Endpoint](#). In *The Semantic Web - ISWC 2019 - 18th International Semantic Web Conference, Heraklion, Crete, Greece, October 13-17, 2019, Proceedings, Part I*, Michael Cochez, Petar Ristoski, Simone Paolo Ponzetto, and Heiko Paulheim (Eds.). Lecture Notes in Computer Science, vol. 11818. Springer, 318–336.

- Web Conference, Auckland, New Zealand, October 26-30, 2019, *Proceedings, Part I*, Chiara Ghidini, Olaf Hartig, Maria Maleshkova, Vojtech Svátek, Isabel F. Cruz, Aidan Hogan, Jie Song, Maxime Lefrançois, and Fabien Gandon (Eds.). Lecture Notes in Computer Science, vol. 11778. Springer, 145–163.
- Luca Costabello, Serena Villata, Nicolas Delaforge, and Fabien Gandon. 2012. [Linked Data Access Goes Mobile: Context-Aware Authorization for Graph Stores](#). In *WWW2012 Workshop on Linked Data on the Web, Lyon, France, 16 April, 2012*, Christian Bizer, Tom Heath, Tim Berners-Lee, and Michael Hausenblas (Eds.). CEUR Workshop Proceedings, vol. 937. Sun SITE Central Europe (CEUR). 8 pages.
- Kino Coursey and Rada Mihalcea. 2009. [Topic Identification Using Wikipedia Graph Centrality](#). In *Human Language Technologies: Conference of the North American Chapter of the Association of Computational Linguistics, Proceedings, May 31 - June 5, 2009, Boulder, Colorado, USA, Short Papers*, Mari Ostendorf, Michael Collins, Shri Narayanan, Douglas W. Oard, and Lucy Vanderwende (Eds.). The Association for Computational Linguistics, 117–120.
- Simon Cox, Chris Little, Jerry R. Hobbs, and Feng Pan. 2017. [Time Ontology in OWL](#). W3C Recommendation / OGC 16-071r2. World Wide Web Consortium and Open Geospatial Consortium. October 19, 2017.
- Eric Crestan and Patrick Pantel. 2011. [Web-scale table census and classification](#). In *Proceedings of the Forth International Conference on Web Search and Web Data Mining, WSDM 2011, Hong Kong, China, February 9-12, 2011*, Irwin King, Wolfgang Nejdl, and Hang Li (Eds.). ACM Press, 545–554.
- Richard Cyganiak, David Wood, and Markus Lanthaler. 2014. [RDF 1.1 Concepts and Abstract Syntax](#). W3C Recommendation. World Wide Web Consortium. February 25, 2014.
- Enrico Daga, Valentina Presutti, and Alberto Salvati. 2008. <http://ontologydesignpatterns.org> and [Evaluation WikiFlow](#). In *Proceedings of the 5th Workshop on Semantic Web Applications and Perspectives (SWAP2008), Rome, Italy, December 15-17, 2008*, Aldo Gangemi, Johannes Keizer, Valentina Presutti, and Heiko Stoermer (Eds.). CEUR Workshop Proceedings, vol. 426. Sun SITE Central Europe (CEUR), 1–11.
- Jeff Dalgliesh. 2016. [How the Enterprise Knowledge Graph Connects Oil and Gas Data Silos](#). Maana Blog. May 20, 2016.
- Claudia d'Amato, Steffen Staab, Andrea G. B. Tettamanzi, Duc Minh Tran, and Fabien L. Gandon. 2016. [Ontology enrichment by discovering multi-relational association rules from ontological knowledge bases](#). In *Proceedings of the 31st Annual ACM Symposium on Applied Computing, Pisa, Italy, April 4-8, 2016*, Sascha Ossowski (Ed.). ACM Press, 333–338.
- Claudia d'Amato, Andrea G. B. Tettamanzi, and Minh Duc Tran. 2016. [Evolutionary Discovery of Multi-relational Association Rules from Ontological Knowledge Bases](#). In *Knowledge Engineering and Knowledge Management - 20th International Conference, EKAW 2016, Bologna, Italy, November 19-23, 2016, Proceedings*, Eva Blomqvist, Paolo Ciancarini, Francesco Poggi, and Fabio Vitali (Eds.). Lecture Notes in Computer Science, vol. 10024. Springer, 113–128.
- Fariz Darari, Werner Nutt, Giuseppe Pirrò, and Simon Razniewski. 2018. [Completeness Management for RDF Data Sources](#). *ACM Transactions on the Web* 12(3), 18:1–18:53.
- Souripriya Das, Seema Sundara, and Richard Cyganiak. 2012. [R2RML: RDB to RDF Mapping Language, W3C Recommendation 27 September 2012](#). W3C Recommendation. World Wide Web Consortium. September 27, 2012.
- Ankur Dave, Alekh Jindal, Li Erran Li, Reynold Xin, Joseph Gonzalez, and Matei Zaharia. 2016. [GraphFrames: an integrated API for mixing graph and relational queries](#). In *Proceedings of the Fourth International Workshop on Graph Data Management Experiences and Systems, Redwood Shores, CA, USA, June 24 - 24, 2016*, Peter A. Boncz and Josep-Lluís Larriba-Pey (Eds.). ACM Press, 2:1–2:8.
- Gerard de Melo. 2015. [Lexvo.org: Language-Related Information for the Linguistic Linked Data Cloud](#). *Semantic Web Journal* 6(4), 393–400.
- Luc De Raedt, Bart Vandersmissen, Marc Denecker, and Maurice Bruynooghe. 1991. [A hybrid approach to learning and its knowledge representation](#). In *Proceedings of the third COGNITIVA symposium on At the crossroads of artificial intelligence, cognitive science, and neuroscience*, Teuvo Kohonen and Françoise Fogelman-Soulie (Eds.). Elsevier, 109–116.
- Luc De Raedt (Ed.). 2008. [Logical and Relational Learning](#). Springer.
- Marina De Vos, Sabrina Kirrane, Julian Padget, and Ken Satoh. 2019. [ODRL policy modelling and compliance checking](#). In *Rules and Reasoning - Third International Joint Conference, RuleML+RR 2019*,

- Bolzano, Italy, September 16-19, 2019, *Proceedings*, Paul Fodor, Marco Montali, Diego Calvanese, and Dumitru Roman (Eds.). Lecture Notes in Computer Science, vol. 11784. Springer, 36–51.
- Remy Delanaux, Angela Bonifati, Marie-Christine Rousset, and Romuald Thion. 2018. [Query-Based Linked Data Anonymization](#). In *The Semantic Web - ISWC 2018 - 17th International Semantic Web Conference, Monterey, CA, USA, October 8-12, 2018, Proceedings, Part I*, Denny Vrandečić, Kalina Bontcheva, Mari Carmen Suárez-Figueroa, Valentina Presutti, Irene Celino, Marta Sabou, Lucie-Aimée Kaffee, and Elena Simperl (Eds.). Lecture Notes in Computer Science, vol. 11136. Springer, 530–546.
- Thomas Demeester, Tim Rocktäschel, and Sebastian Riedel. 2016. [Lifted Rule Injection for Relation Embeddings](#). In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, EMNLP 2016, Austin, Texas, USA, November 1-4, 2016*, Jian Su, Xavier Carreras, and Kevin Duh (Eds.). The Association for Computational Linguistics, 1389–1399.
- Dong Deng, Yu Jiang, Guoliang Li, Jian Li, and Cong Yu. 2013. [Scalable Column Concept Determination for Web Tables Using Large Knowledge Bases](#). *Proceedings of the VLDB Endowment* 6(13), 1606–1617.
- Tim Dettmers, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. 2018. [Convolutional 2D Knowledge Graph Embeddings](#). In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, (AAAI-18), the 30th innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2-7, 2018*, Sheila A. McIlraith and Kilian Q. Weinberger (Eds.). AAAI Press, 1811–1818.
- Deepika Devarajan. 2017. [Happy Birthday Watson Discovery](#). IBM Cloud Blog. December 21, 2017.
- Gonzalo I. Diaz, Marcelo Arenas, and Michael Benedikt. 2016. [SPARQLByE: Querying RDF data by example](#). *Proceedings of the VLDB Endowment* 9(13), 1533–1536.
- Dennis Diefenbach, Andreas Both, Kamal Singh, and Pierre Maret. 2020. [Towards a question answering system over the Semantic Web](#). *Semantic Web Journal* 11(3), 421–439.
- Rose Dieng, Alain Giboin, Paul-André Tourtier, and Olivier Corby. 1992. [Knowledge Acquisition for Explainable, Multi-Expert, Knowledge-Based Design Systems](#). In *Current Developments in Knowledge Acquisition - EKAW'92, 6th European Knowledge Acquisition Workshop, Heidelberg and Kaiserslautern, Germany, May 18-22, 1992*, Thomas Wetter, Klaus-Dieter Althoff, John H. Boose, Brian R. Gaines, and Marc Linster (Eds.). Lecture Notes in Computer Science, vol. 599. Springer, 298–317.
- Anastasia Dimou, Miel Vander Sande, Jason Slepicka, Pedro A. Szekely, Erik Mannens, Craig A. Knoblock, and Rik Van de Walle. 2014. [Mapping Hierarchical Sources into RDF Using the RML Mapping Language](#). In *2014 IEEE International Conference on Semantic Computing, Newport Beach, CA, USA, June 16-18, 2014*. IEEE Computer Society, 151–158.
- Renata Queiroz Dividino, Sergej Sizov, Steffen Staab, and Bernhard Schueler. 2009. [Querying for provenance, trust, uncertainty and other meta knowledge in RDF](#). *Journal of Web Semantics* 7(3), 204–219.
- Xin Dong, Evgeniy Gabrilovich, Jeremy Heitz, Wilko Horn, Ni Lao, Kevin Murphy, Thomas Strohmman, Shaohua Sun, and Wei Zhang. 2014. [Knowledge vault: a web-scale approach to probabilistic knowledge fusion](#). In *The 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '14, New York, NY, USA - August 24 - 27, 2014*, Sofus A. Macskassy, Claudia Perlich, Jure Leskovec, Wei Wang, and Rayid Ghani (Eds.). ACM Press, 601–610.
- Xin Luna Dong. 2019. [Building a Broad Knowledge Graph for Products](#). In *35th IEEE International Conference on Data Engineering, ICDE 2019, Macao, China, April 8-11, 2019*. IEEE Computer Society, 25–25.
- Uwe Draisbach and Felix Naumann. 2011. [A generalization of blocking and windowing algorithms for duplicate detection](#). In *2011 International Conference on Data and Knowledge Engineering, ICDKE 2011, Milano, Italy, September 6, 2011*, Ji Zhang and Giovanni Livraga (Eds.). IEEE Computer Society, 18–24.
- Martin Dürst and Michel Suignard. 2005. [Internationalized Resource Identifiers \(IRIs\)](#). RFC. Internet Engineering Task Force. January, 2005.
- Arnab Dutta, Christian Meilicke, and Heiner Stuckenschmidt. 2014. [Semantifying Triples from Open Information Extraction Systems](#). In *STAIRS 2014 - Proceedings of the 7th European Starting AI Researcher Symposium, Prague, Czech Republic, August 18-22, 2014*, Ulle Endriss and João Leite (Eds.). Frontiers in Artificial Intelligence and Applications, vol. 264. IOS Press, 111–120.
- Arnab Dutta, Christian Meilicke, and Heiner Stuckenschmidt. 2015. [Enriching Structured Knowledge with Open Information](#). In *Proceedings of the 24th International Conference on World Wide Web, WW* [Give us feedback!](#)

- 2015, Florence, Italy, May 18-22, 2015, Aldo Gangemi, Stefano Leonardi, and Alessandro Panconesi (Eds.). ACM Press, 267–277.
- Cynthia Dwork. 2006. [Differential Privacy](#). In *Automata, Languages and Programming, 33rd International Colloquium, ICALP 2006, Venice, Italy, July 10-14, 2006, Proceedings, Part II*, Michele Bugliesi, Bart Preneel, Vladimiro Sassone, and Ingo Wegener (Eds.). Lecture Notes in Computer Science, vol. 4052. Springer, 1–12.
- Eugene Dynkin. 1965. *Markov processes: Volume 1*. Springer.
- Julian Eberius, Katrin Braunschweig, Markus Hentsch, Maik Thiele, Ahmad Ahmadov, and Wolfgang Lehner. 2015. [Building the Dresden Web Table Corpus: A Classification Approach](#). In *2nd IEEE/ACM International Symposium on Big Data Computing, BDC 2015, Limassol, Cyprus, December 7-10, 2015*, Ioan Raicu, Omer F. Rana, and Rajkumar Buyya (Eds.). IEEE Computer Society, 41–50.
- Mikel Egaña, Alan Rector, Robert Stevens, and Erick Antezana. 2008. [Applying Ontology Design Patterns in Bio-ontologies](#). In *Knowledge Engineering: Practice and Patterns, 16th International Conference, EKAW 2008, Acitrezza, Italy, September 29 - October 2, 2008. Proceedings*, Aldo Gangemi and Jérôme Euzenat (Eds.). Lecture Notes in Computer Science, vol. 5268. Springer, 7–16.
- Lisa Ehrlinger and Wolfram Wöß. 2016. [Towards a Definition of Knowledge Graphs](#). In *Joint Proceedings of the Posters and Demos Track of the 12th International Conference on Semantic Systems - SEMANTiCS2016 and the 1st International Workshop on Semantic Change & Evolving Semantics (SuCESS'16) co-located with the 12th International Conference on Semantic Systems (SEMANTiCS 2016), Leipzig, Germany, September 12-15, 2016*, Michael Martin, Martí Cuquet, and Erwin Folmer (Eds.). CEUR Workshop Proceedings, vol. 1695. Sun SITE Central Europe (CEUR). 4 pages.
- Shady Elbassuoni, Maya Ramanath, Ralf Schenkel, Marcin Sydow, and Gerhard Weikum. 2009. [Language-model-based ranking for queries on RDF-graphs](#). In *Proceedings of the 18th ACM Conference on Information and Knowledge Management, CIKM 2009, Hong Kong, China, November 2-6, 2009*, David Wai-Lok Cheung, Il-Yeol Song, Wesley W. Chu, Xiaohua Hu, and Jimmy J. Lin (Eds.). ACM Press, 977–986.
- Basil Ell, Andreas Harth, and Elena Simperl. 2014. [SPARQL Query Verbalization for Explaining Semantic Search Engine Queries](#). In *The Semantic Web: Trends and Challenges - 11th International Conference, ESWC 2014, Anissaras, Crete, Greece, May 25-29, 2014. Proceedings*, Valentina Presutti, Claudia d'Amato, Fabien Gandon, Mathieu d'Aquin, Stephen Staab, and Anna Tordia (Eds.). Lecture Notes in Computer Science, vol. 8465. Springer, 426–441.
- Orri Erling. 2012. [Virtuoso, a Hybrid RDBMS/Graph Column Store](#). *IEEE Data Engineering Bulletin* 35(1), 3–8.
- Ivan Ermilov and Axel-Cyrille Ngonga Ngomo. 2016. [TAIPAN: Automatic Property Mapping for Tabular Data](#). In *Knowledge Engineering and Knowledge Management - 20th International Conference, EKAW 2016, Bologna, Italy, November 19-23, 2016, Proceedings*, Eva Blomqvist, Paolo Ciancarini, Francesco Poggi, and Fabio Vitali (Eds.). Lecture Notes in Computer Science, vol. 10024. Springer, 163–179.
- Diego Esteves, Anisa Rula, Aniketh Janardhan Reddy, and Jens Lehmann. 2018. [Toward Veracity Assessment in RDF Knowledge Bases: An Exploratory Analysis](#). *Journal of Data and Information Quality* 9(3), 16:1–16:26.
- Ernesto Estrada. 2011. *The Structure of Complex Networks: Theory and Applications*. Oxford University Press, Inc..
- Oren Etzioni, Michael J. Cafarella, Doug Downey, Stanley Kok, Ana-Maria Popescu, Tal Shaked, Stephen Soderland, Daniel S. Weld, and Alexander Yates. 2004. [Web-scale information extraction in know-itall: \(preliminary results\)](#). In *Proceedings of the 13th international conference on World Wide Web, WWW 2004, New York, NY, USA, May 17-20, 2004*, Stuart I. Feldman, Mike Uretsky, Marc Najork, and Craig E. Wills (Eds.). ACM Press, 100–110.
- Oren Etzioni, Anthony Fader, Janara Christensen, Stephen Soderland, and Mausam. 2011. [Open Information Extraction: The Second Generation](#). In *IJCAI 2011, Proceedings of the 22nd International Joint Conference on Artificial Intelligence, Barcelona, Catalonia, Spain, July 16-22, 2011*, Toby Walsh (Ed.). IJCAI/AAAI, 3–10.
- Anthony Fader, Stephen Soderland, and Oren Etzioni. 2011. [Identifying Relations for Open Information Extraction](#). In *Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing, EMNLP 2011, 27-31 July 2011, John McIntyre Conference Centre, Edinburgh, UK, A meeting of SIGDAT, a Special Interest Group of the ACL, Regina Barzilay and Mark Johnson (Eds.)*. The Association for Computational Linguistics, 1535–1545.

- Wenfei Fan, Xin Wang, and Yinghui Wu. 2013. [Diversified Top-\\$k\\$ Graph Pattern Matching](#). *Proceedings of the VLDB Endowment* 6(13), 1510–1521.
- Nicola Fanizzi, Claudia d’Amato, and Floriana Esposito. 2008. [DL-FOIL Concept Learning in Description Logics](#). In *Inductive Logic Programming, 18th International Conference, ILP 2008, Prague, Czech Republic, September 10-12, 2008, Proceedings*, Filip Zelezný and Nada Lavrac (Eds.). Lecture Notes in Computer Science, vol. 5194. Springer, 107–121.
- Michael Färber, Frederic Bartscherer, Carsten Menne, and Achim Rettinger. 2018. [Linked data quality of DBpedia, Freebase, OpenCyc, Wikidata, and YAGO](#). *Semantic Web Journal* 9(1), 77–129.
- Michael Färber. 2019. The Microsoft Academic Knowledge Graph: A Linked Data Source with 8 Billion Triples of Scholarly Data. In *The Semantic Web - ISWC 2019 - 18th International Semantic Web Conference, Auckland, New Zealand, October 26-30, 2019, Proceedings, Part II*, Chiara Ghidini, Olaf Hartig, Maria Maleshkova, Vojtech Svátek, Isabel F. Cruz, Aidan Hogan, Jie Song, Maxime Lefrançois, and Fabien Gandon (Eds.). Lecture Notes in Computer Science, vol. 11779. Springer, 113–129.
- Dieter Fensel, Umutcan Simsek, Kevin Angele, Elwin Huaman, Elias Kärle, Oleksandra Panasiuk, Ioan Toma, Jürgen Umbrich, and Alexander Wahler. 2020. [Knowledge Graphs - Methodology, Tools and Selected Use Cases](#). Springer.
- Mariano Fernández, Asunción Gómez-Pérez, and Natalia Juristo. 1997. [METHONTOLOGY: from Ontological Art towards Ontological Engineering](#). In *Ontological Engineering, Papers from the AAAI Spring Symposium, Technical Report SS-97-06, 1997*, Adam Farquhar and Michael Gruninger (Eds.). AAAI Press, 33–40.
- Javier D. Fernández, Miguel A. Martínez-Prieto, Claudio Gutiérrez, Axel Polleres, and Mario Arias. 2013. [Binary RDF representation for publication and exchange \(HDT\)](#). *Journal of Web Semantics* 19, 22–41.
- Javier D. Fernández, Sabrina Kirrane, Axel Polleres, and Simon Steyskal. 2017. [Self-Enforcing Access Control for Encrypted RDF](#). In *The Semantic Web - 14th International Conference, ESWC 2017, Portorož, Slovenia, May 28 - June 1, 2017, Proceedings, Part I*, Eva Blomqvist, Diana Maynard, Aldo Gangemi, Rinke Hoekstra, Pascal Hitzler, and Olaf Hartig (Eds.). Lecture Notes in Computer Science, vol. 10249. Springer, 607–622.
- Emilio Ferrara, Pasquale De Meo, Giacomo Fiumara, and Robert Baumgartner. 2014. [Web data extraction, applications and techniques: A survey](#). *Knowledge-based Systems* 70, 301–323.
- Sébastien Ferré. 2017. [Sparklis: An expressive query builder for SPARQL endpoints with guidance in natural language](#). *Semantic Web Journal* 8(3), 405–418.
- Charles J. Fillmore. 1976. [Frame semantics and the nature of language](#). *Annals of the New York Academy of Sciences* 280(1), 20–32.
- Jenny Rose Finkel, Trond Grenager, and Christopher D. Manning. 2005. [Incorporating Non-local Information into Information Extraction Systems by Gibbs Sampling](#). In *ACL 2005, 43rd Annual Meeting of the Association for Computational Linguistics, Proceedings of the Conference, 25-30 June 2005, University of Michigan, USA*, Kevin Knight, Hwee Tou Ng, and Kemal Oflazer (Eds.). The Association for Computational Linguistics, 363–370.
- Sergio Flesca, Giuseppe Manco, Elio Masciari, Eugenio Rende, and Andrea Tagarelli. 2004. [Web wrapper induction: a brief survey](#). *AI Communications* 17(2), 57–61.
- Giorgos Flouris, Irini Fundulaki, Maria Michou, and Grigoris Antoniou. 2010. [Controlling Access to RDF Graphs](#). In *Future Internet - FIS 2010 - Third Future Internet Symposium, Berlin, Germany, September 20-22, 2010. Proceedings*, Arne-Jørgen Berre, Asunción Gómez-Pérez, Kurt Tutschku, and Dieter Fensel (Eds.). Lecture Notes in Computer Science, vol. 6369. Springer, 107–117.
- Charles Forgy. 1982. [Rete: A Fast Algorithm for the Many Patterns/Many Objects Match Problem](#). *Artificial Intelligence* 19(1), 17–37.
- Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. 2018. [Cypher: An Evolving Query Language for Property Graphs](#). In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018*, Gautam Das, Christopher M. Jermaine, and Philip A. Bernstein (Eds.). ACM Press, 1433–1445.
- Gottlob Frege. 1879. *Begriffsschrift*. Halle.
- André Freitas, João Gabriel Oliveira, Seán O’Riain, Edward Curry, and João Carlos Pereira da Silva. 2011. [Treo: Best-Effort Natural Language Queries over Linked Data](#). In *Natural Language Processing and Information Systems - 16th International Conference on Applications of Natural Language*

- Information Systems, NLDB 2011, Alicante, Spain, June 28-30, 2011. Proceedings*, Rafael Muñoz, Andrés Montoyo, and Elisabeth Métais (Eds.). Lecture Notes in Computer Science, vol. 6716. Springer, 286–289.
- Christian Fürber and Martin Hepp. 2011. [SWIQA - a semantic web information quality assessment framework](#). In *19th European Conference on Information Systems, ECIS 2011, Helsinki, Finland, June 9-11, 2011*, Virpi Kristiina Tuunainen, Matti Rossi, and Joe Nandhakumar (Eds.). Association for Information Systems, p76.
- Mohamed H. Gad-Elrab, Daria Stepanova, Jacopo Urbani, and Gerhard Weikum. 2016. [Exception-Enriched Rule Learning from Knowledge Graphs](#). In *The Semantic Web - ISWC 2016 - 15th International Semantic Web Conference, Kobe, Japan, October 17-21, 2016, Proceedings, Part I*, Paul T. Groth, Elena Simperl, Alasdair J. G. Gray, Marta Sabou, Markus Krötzsch, Freddy Lécué, Fabian Flöck, and Yolanda Gil (Eds.). Lecture Notes in Computer Science, vol. 9981. Springer, 234–251.
- Luis Antonio Galárraga, Christina Teflioudi, Katja Hose, and Fabian Suchanek. 2013. [AMIE: association rule mining under incomplete evidence in ontological knowledge bases](#). In *22nd International World Wide Web Conference, WWW '13, Rio de Janeiro, Brazil, May 13-17, 2013*, Daniel Schwabe, Virgílio A. F. Almeida, Hartmut Glaser, Ricardo Baeza-Yates, and Sue B. Moon (Eds.). ACM Press, 413–422.
- Luis Galárraga, Chistina Teflioudi, Katja Hose, and Fabian M. Suchanek. 2015. [Fast rule mining in ontological knowledge bases with AMIE+](#). *The Very Large Data Base Journal* 24(6), 707–730.
- Alban Galland, Serge Abiteboul, Amélie Marian, and Pierre Senellart. 2010. [Corroborating Information from Disagreeing Views](#). In *Proceedings of the Third International Conference on Web Search and Web Data Mining, WSDM 2010, New York, NY, USA, February 4-6, 2010*, Brian D. Davison, Torsten Suel, Nick Craswell, and Bing Liu (Eds.). ACM Press, 131–140.
- Fabien Gandon and Guus Schreiber. 2014. [RDF 1.1 XML Syntax, W3C Recommendation 25 February 2014](#). W3C Recommendation. World Wide Web Consortium. February 25, 2014.
- Aldo Gangemi, Valentina Presutti, Diego Reforgiato Recupero, Andrea Giovanni Nuzzolese, Francesco Draicchio, and Misael Mongiovì. 2017. [Semantic Web Machine Reading with FRED](#). *Semantic Web Journal* 8(6), 873–893.
- Aldo Gangemi. 2005. [Ontology design patterns for semantic web content](#). In *The Semantic Web - ISWC 2005, 4th International Semantic Web Conference, ISWC 2005, Galway, Ireland, November 6-10, 2005, Proceedings*, Yolanda Gil, Enrico Motta, V. Richard Benjamins, and Mark A. Musen (Eds.). Lecture Notes in Computer Science, vol. 3729. Springer, 262–276.
- Claire Gardent, Anastasia Shimorina, Shashi Narayan, and Laura Perez-Beltrachini. 2017. [The WebNLG challenge: Generating text from RDF data](#). In *Proceedings of the 10th International Conference on Natural Language Generation, INLG 2017, Santiago de Compostela, Spain, September 4-7, 2017*, José M. Alonso, Alberto Bugarín, and Ehud Reiter (Eds.). The Association for Computational Linguistics, 124–133.
- Michael Gelfond and Vladimir Lifschitz. 1988. The Stable Model Semantics for Logic Programming. In *Logic Programming, Proceedings of the Fifth International Conference and Symposium, Seattle, Washington, USA, August 15-19, 1988 (2 Volumes)*, Robert A. Kowalski and Kenneth A. Bowen (Eds.). The MIT Press, 1070–1080.
- James Geller, Soon Ae Chun, and Yoo Jung An. 2008. [Toward the Semantic Deep Web](#). *IEEE Computer* 41(9), 95–97.
- Anna Lisa Gentile, Ziqi Zhang, and Fabio Ciravegna. 2014. [Self Training Wrapper Induction with Linked Data](#). In *Text, Speech and Dialogue - 17th International Conference, TSD 2014, Brno, Czech Republic, September 8-12, 2014. Proceedings*, Petr Sojka, Ales Horák, Ivan Kopecek, and Karel Pala (Eds.). Lecture Notes in Computer Science, vol. 8655. Springer, 285–292.
- Anna Lisa Gentile, Daniel Gruhl, Petar Ristoski, and Steve Welch. 2019. [Personalized Knowledge Graphs for the Pharmaceutical Domain](#). In *The Semantic Web - ISWC 2019 - 18th International Semantic Web Conference, Auckland, New Zealand, October 26-30, 2019, Proceedings, Part II*, Chiara Ghidini, Olaf Hartig, Maria Maleshkova, Vojtech Svátek, Isabel F. Cruz, Aidan Hogan, Jie Song, Maxime Lefrançois, and Fabien Gandon (Eds.). Lecture Notes in Computer Science, vol. 11779. Springer, 400–417.
- Daniel Gerber, Diego Esteves, Jens Lehmann, Lorenz Bühmann, Ricardo Usbeck, Axel-Cyrille Ngonga Ngomo, and René Speck. 2015. [DeFacto-temporal and multilingual deep fact validation](#). *Journal of Web Semantics* 35, 85–101.

- Sabrina Gerbracht. 2008. [Possibilities to Encrypt an RDF-Graph](#). In *2008 3rd International Conference on Information and Communication Technologies: From Theory to Applications*. IEEE Computer Society. 6 pages.
- Lise Getoor and Ben Taskar (Eds.). 2007. [Introduction to Statistical Relational Learning](#). Adaptive Computation and Machine Learning. The MIT Press.
- Mark Giereth. 2005. [On Partial Encryption of RDF-Graphs](#). In *The Semantic Web - ISWC 2005, 4th International Semantic Web Conference, ISWC 2005, Galway, Ireland, November 6-10, 2005, Proceedings*, Yolanda Gil, Enrico Motta, V. Richard Benjamins, and Mark A. Musen (Eds.). Lecture Notes in Computer Science, vol. 3729. Springer, 308–322.
- Yolanda Gil, Simon Miles, Khalid Belhajjame, Daniel Garijo, Graham Klyne, Paolo Missier, Stian Soiland-Reyes, and Stephan Zednik. 2013. [PROV Model Primer](#). W3C Working Group Note. World Wide Web Consortium. April 30, 2013.
- José M. Giménez-García, Antoine Zimmermann, and Pierre Maret. 2017. [NdFluents: An Ontology for Annotated Statements with Inference Preservation](#). In *The Semantic Web - 14th International Conference, ESWC 2017, Portorož, Slovenia, May 28 - June 1, 2017, Proceedings, Part I*, Eva Blomqvist, Diana Maynard, Aldo Gangemi, Rinke Hoekstra, Pascal Hitzler, and Olaf Hartig (Eds.). Lecture Notes in Computer Science, vol. 10249. Springer, 638–654.
- Birte Glimm. 2011. [Using SPARQL with RDFS and OWL Entailment](#). In *Reasoning Web. Semantic Technologies for the Web of Data - 7th International Summer School 2011, Galway, Ireland, August 23-27, 2011, Tutorial Lectures*, Axel Polleres, Claudia d'Amato, Marcelo Arenas, Siegfried Handschuh, Paula Kroner, Sascha Ossowski, and Peter F. Patel-Schneider (Eds.). Lecture Notes in Computer Science, vol. 6848. Springer, 137–201.
- Xavier Glorot, Antoine Bordes, Jason Weston, and Yoshua Bengio. 2013. [A Semantic Matching Energy Function for Learning with Multi-relational Data](#). In *1st International Conference on Learning Representations, ICLR 2013, Scottsdale, Arizona, USA, May 2-4, 2013, Workshop Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). OpenReview.net. 4 pages.
- Patrick Golden and Ryan B. Shaw. 2016. [Nanopublication beyond the sciences: the PeriodO period gazetteer](#). *PeerJ Computer Science* 2, pe44.
- Asunción Gómez-Pérez, Mariano Fernández-López, and Oscar Corcho. 2006. [Ontological Engineering: with examples from the areas of Knowledge Management, e-Commerce and the Semantic Web](#). Advanced Information and Knowledge Processing. Springer.
- Rafael S. Gonçalves, Matthew Horridge, Rui Li, Yu Liu, Mark A. Musen, Csongor I. Nyulas, Evelyn Obamos, Dhananjay Shrouthy, and David Temple. 2019. [Use of OWL and Semantic Web Technologies at Pinterest](#). In *The Semantic Web - ISWC 2019 - 18th International Semantic Web Conference, Auckland, New Zealand, October 26-30, 2019, Proceedings, Part II*, Chiara Ghidini, Olaf Hartig, Maria Maleshkova, Vojtech Svátek, Isabel F. Cruz, Aidan Hogan, Jie Song, Maxime Lefrançois, and Fabien Gandon (Eds.). Lecture Notes in Computer Science, vol. 11779. Springer, 418–435.
- Larry González and Aidan Hogan. 2018. [Modelling Dynamics in Semantic Web Knowledge Graphs with Formal Concept Analysis](#). In *Proceedings of the 2018 World Wide Web Conference on World Wide Web, WWW 2018, Lyon, France, April 23-27, 2018*, Pierre-Antoine Champin, Fabien L. Gandon, Mounia Lalmas, and Panagiotis G. Ipeirotis (Eds.). ACM Press, 1175–1184.
- Simon Gottschalk and Elena Demidova. 2018. [EventKG: A Multilingual Event-Centric Temporal Knowledge Graph](#). In *The Semantic Web - 15th International Conference, ESWC 2018, Heraklion, Crete, Greece, June 3-7, 2018, Proceedings*, Aldo Gangemi, Roberto Navigli, Maria-Esther Vidal, Pascal Hitzler, Troncy Raphaël, Laura Hollink, Anna Tordai, and Mehwish Alam (Eds.). Lecture Notes in Computer Science, vol. 10843. Springer, 272–287.
- Guido Governatori, Ho-Pun Lam, Antonino Rotolo, Serena Villata, and Fabien Gandon. 2013. [Heuristics for Licenses Composition](#). In *Legal Knowledge and Information Systems - JURIX 2013: The Twenty-Sixth Annual Conference, December 11-13, 2013, University of Bologna, Italy*, Kevin D. Ashley (Ed.). Frontiers in Artificial Intelligence and Applications, vol. 259. IOS Press, 77–86.
- Ralph Grishman. 2012. [Information Extraction: Capabilities and Challenges](#). Notes prepared for the 2012 International Winter School in Language and Speech Technologies.
- Paul T. Groth, Antonis Loizou, Alasdair J. G. Gray, Carole A. Goble, Lee Harland, and Steve Pettifer. 2014. [API-centric Linked Data integration: The Open PHACTS Discovery Platform case study](#). *Journal of Web Semantics* 29, 12–18.

- Michael Grüninger and Mark S. Fox. 1995. [Methodology for the Design and Evaluation of Ontologies](#). In *Workshop on Basic Ontological Issues in Knowledge Sharing, IJCAI-95, Montreal*. 10 pages.
- Michael Grüninger and Mark S. Fox. 1995. [The role of competency questions in enterprise engineering](#). In *Benchmarking – Theory and practice*, (Ed.). Springer, 22–31.
- Nicola Guarino, Daniel Oberle, and Steffen Staab. 2009. [What Is an Ontology?](#). In *Handbook on Ontologies*, Steffen Staab and Rudi Studer (Eds.). International Handbooks on Information Systems. Springer, 1–17.
- Ramanathan V. Guha, Rob McCool, and Eric Miller. 2003. [Semantic search](#). In *Proceedings of the Twelfth International World Wide Web Conference, WWW2003, Budapest, Hungary, 20-24 May 2003*, Gusztáv Hencsey, Bebo White, Yih-Farn Robin Chen, László Kovács, and Steve Lawrence (Eds.). ACM Press, 700–709.
- Ramanathan V. Guha, Rob McCool, and Richard Fikes. 2004. [Contexts for the Semantic Web](#). In *The Semantic Web - ISWC 2004: Third International Semantic Web Conference, Hiroshima, Japan, November 7-11, 2004. Proceedings*, Frank van Harmelen, Sheila McIlraith, and Dimitri Plexousakis (Eds.). Lecture Notes in Computer Science, vol. 3298. Springer, 32–46.
- Shu Guo, Quan Wang, Lihong Wang, Bin Wang, and Li Guo. 2016. [Jointly Embedding Knowledge Graphs and Logical Rules](#). In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, EMNLP 2016, Austin, Texas, USA, November 1-4, 2016*, Jian Su, Xavier Carreras, and Kevin Duh (Eds.). The Association for Computational Linguistics, 192–202.
- Shu Guo, Quan Wang, Lihong Wang, Bin Wang, and Li Guo. 2018. [Knowledge Graph Embedding With Iterative Guidance From Soft Rules](#). In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, (AAAI-18), the 30th innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2-7, 2018*, Sheila A. McIlraith and Kilian Q. Weinberger (Eds.). AAAI Press, 4816–4823.
- Sonal Gupta and Christopher D. Manning. 2014. [Improved Pattern Learning for Bootstrapped Entity Extraction](#). In *Proceedings of the Eighteenth Conference on Computational Natural Language Learning, CoNLL 2014, Baltimore, Maryland, USA, June 26-27, 2014*, Roser Morante and Wen-tau Yih (Eds.). The Association for Computational Linguistics, 98–108.
- Claudio Gutiérrez, Carlos A. Hurtado, and Alejandro A. Vaisman. 2007. [Introducing Time into RDF](#). *IEEE Transactions on Knowledge and Data Engineering* 19(2), 207–218.
- Florian Haag, Steffen Lohmann, Stephan Siek, and Thomas Ertl. 2015. [QueryVOWL: A Visual Query Notation for Linked Data](#). In *The Semantic Web: ESWC 2015 Satellite Events - ESWC 2015 Satellite Events Portorož, Slovenia, May 31 - June 4, 2015, Revised Selected Papers*, Fabien Gandon, Christophe Guéret, Serena Villata, John G. Breslin, Catherine Faron-Zucker, and Antoine Zimmermann (Eds.). Lecture Notes in Computer Science, vol. 9341. Springer, 387–402.
- Juha Hakala. 2010. [Persistent identifiers – an overview](#). PersID Technical Report. September 3, 2010.
- Ferras Hamad, Isaac Liu, and Xian Xing Zhang. 2018. [Food Discovery with Uber Eats: Building a Query Understanding Engine](#). Uber Engineering Blog. june 10, 2018.
- William L. Hamilton, Payal Bajaj, Marinka Zitnik, Dan Jurafsky, and Jure Leskovec. 2018. [Embedding Logical Queries on Knowledge Graphs](#). In *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, 3-8 December 2018, Montréal, Canada*, Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett (Eds.), 2030–2041.
- Tony Hammond, Michele Pasin, and Evangelos Theodoridis. 2017. [Data integration and disintegration: Managing Springer Nature SciGraph with SHACL and OWL](#). In *Proceedings of the ISWC 2017 Posters & Demonstrations and Industry Tracks co-located with 16th International Semantic Web Conference (ISWC 2017), Vienna, Austria, October 23rd - to - 25th, 2017*, Nadeschda Nikitina, Dezhao Song, Achille Fokoue, and Peter Haase (Eds.). CEUR Workshop Proceedings, vol. 1963. Sun SITE Central Europe (CEUR). 2 pages.
- Steve Harris, Andy Seaborne, and Eric Prud'hommeaux. 2013. [SPARQL 1.1 Query Language](#). W3C Recommendation. World Wide Web Consortium. March 21, 2013.
- Andreas Harth. 2010. [VisiNav: A system for visual search and navigation on web data](#). *Journal of Web Semantics* 8(4), 348–354.
- Olaf Hartig and Carlos Buil-Aranda. 2016. [Bindings-Restricted Triple Pattern Fragments](#). In *On the Move to Meaningful Internet Systems: OTM 2016 Conferences - Confederated International Conferences: Coop*

- C&TC, and ODBASE 2016, Rhodes, Greece, October 24-28, 2016, *Proceedings*, Christophe Debruyne, Hervé Panetto, Robert Meersman, Tharam S. Dillon, Eva Kühn, Declan O'Sullivan, and Claudio Agostino Ardagna (Eds.). Lecture Notes in Computer Science, vol. 10033. Springer, 762–779.
- Olaf Hartig and Bryan Thompson. 2014. [Foundations of an Alternative Approach to Reification in RDF](#). CoRR abs/1406.3399. 14 pages.
- Olaf Hartig, Christian Bizer, and Johann Christoph Freytag. 2009. [Executing SPARQL Queries over the Web of Linked Data](#). In *The Semantic Web - ISWC 2009, 8th International Semantic Web Conference, ISWC 2009, Chantilly, VA, USA, October 25-29, 2009. Proceedings*, Abraham Bernstein, David R. Karger, Tom Heath, Lee Feigenbaum, Diana Maynard, Enrico Motta, and Krishnaprasad Thirunarayan (Eds.). Lecture Notes in Computer Science, vol. 5823. Springer, 293–309.
- Olaf Hartig, Ian Letter, and Jorge Pérez. 2017. [A Formal Framework for Comparing Linked Data Fragments](#). In *The Semantic Web - ISWC 2017 - 16th International Semantic Web Conference, Vienna, Austria, October 21-25, 2017, Proceedings, Part I*, Claudia d'Amato, Miriam Fernández, Valentina A. M. Tamma, Freddy Lécué, Philippe Cudré-Mauroux, Juan F. Sequeda, Christoph Lange, and Jeff Heflin (Eds.). Lecture Notes in Computer Science, vol. 10587. Springer, 364–382.
- Olaf Hartig. 2017. [Foundations of RDF* and SPARQL* – An Alternative Approach to Statement-Level Metadata in RDF](#). In *Proceedings of the 11th Alberto Mendelzon International Workshop on Foundations of Data Management and the Web, Montevideo, Uruguay, June 7-9, 2017*, Juan L. Reutter and Divesh Srivastava (Eds.). CEUR Workshop Proceedings, vol. 1912. Sun SITE Central Europe (CEUR). 11 pages.
- Patrick J. Hayes and Peter F. Patel-Schneider. 2014. [RDF 1.1 Semantics](#). W3C Recommendation. World Wide Web Consortium. February 25, 2014.
- Qi He, Bee-Chung Chen, and Deepak Agarwal. 2016. [Building The LinkedIn Knowledge Graph](#). LinkedIn Blog. October 6, 2016.
- Marti A. Hearst. 1992. [Automatic Acquisition of Hyponyms from Large Text Corpora](#). In *14th International Conference on Computational Linguistics, COLING 1992, Nantes, France, August 23-28, 1992*, 539–545.
- Tom Heath and Christian Bizer. 2011. [Linked Data: Evolving the Web into a Global Data Space \(1st Edition\)](#). Synthesis Lectures on the Semantic Web: Theory and Technology, vol. 1. Morgan & Claypool.
- Tom Heath and Enrico Motta. 2008. [Revyu: Linking reviews and ratings into the Web of Data](#). *Journal of Web Semantics* 6(4), 266–273.
- Stefan Heindorf, Martin Potthast, Benno Stein, and Gregor Engels. 2016. [Vandalism Detection in Wikidata](#). In *Proceedings of the 25th ACM International Conference on Information and Knowledge Management, CIKM 2016, Indianapolis, IN, USA, October 24-28, 2016*, Snehasis Mukhopadhyay, ChengXiang Zhai, Elisa Bertino, Fabio Crestani, Javed Mostafa, Jie Tang, Luo Si, Xiaofang Zhou, Yi Chang, Yunyao Li, and Parikshit Sondhi (Eds.). ACM Press, 327–336.
- Benjamin Heitmann, Felix Hermesen, and Stefan Decker. 2017. [k-RDF-Neighbourhood Anonymity: Combining Structural and Attribute-based Anonymisation for Linked Data](#). In *Proceedings of the 5th Workshop on Society, Privacy and the Semantic Web - Policy and Technology (PrivOn2017) co-located with 16th International Semantic Web Conference (ISWC 2017), Vienna, Austria, October 22, 2017*, Christopher Brewster, Michelle Cheatham, Mathieu d'Aquin, Stefan Decker, and Sabrina Kirrane (Eds.). CEUR Workshop Proceedings, vol. 1951. Sun SITE Central Europe (CEUR). 16 pages.
- Sebastian Hellmann, Jens Lehmann, Sören Auer, and Martin Brümmer. 2013. [Integrating NLP Using Linked Data](#). In *The Semantic Web - ISWC 2013 - 12th International Semantic Web Conference, Sydney, NSW, Australia, October 21-25, 2013, Proceedings, Part II*, Harith Alani, Lalana Kagal, Achille Fokoue, Paul Groth, Chris Biemann, Josian Xavier Parreira, Lora Aroyo, Natasha Noy, Chris Welty, and Krzysztof Janowicz (Eds.). Lecture Notes in Computer Science, vol. 8219. Springer, 98–113.
- Remko Helms and Kees Buijsrogge. 2005. [Knowledge Network Analysis: A Technique to Analyze Knowledge Management Bottlenecks in Organizations](#). In *16th International Workshop on Database and Expert Systems Applications (DEXA 2005), 22-26 August 2005, Copenhagen, Denmark*. IEEE Computer Society, 410–414.
- James A. Hendler, Jeanne Holm, Chris Musialek, and George Thomas. 2012. [US Government Linked Open Data: Semantic.data.gov](#). *IEEE Intelligent Systems* 27(3), 25–31.
- Cory Henson, Stefan Schmid, Anh Tuan Tran, and Antonios Karatzoglou. 2019. [Using a Knowledge Graph of Scenes to Enable Search of Autonomous Driving Data](#). In *Proceedings of the ISWC 2019 Satellite Tracks (Posters & Demonstrations, Industry, and Outrageous Ideas) co-located with 18th International Semantic Web Conference (ISWC 2019), Auckland, New Zealand, October 26-30, 2019*, Mari Carm

- Suárez-Figueroa, Gong Cheng, Anna Lisa Gentile, Christophe Guéret, C. Maria Keet, and Abraham Bernstein (Eds.). CEUR Workshop Proceedings, vol. 2456. Sun SITE Central Europe (CEUR), 313–314.
- Daniel Hernández, Aidan Hogan, and Markus Krötzsch. 2015. [Reifying RDF: What Works Well With Wikidata?](#). In *Proceedings of the 11th International Workshop on Scalable Semantic Web Knowledge Base Systems co-located with 14th International Semantic Web Conference (ISWC 2015), Bethlehem, PA, USA, October 11, 2015*, Thorsten Liebig and Achille Fokoue (Eds.). CEUR Workshop Proceedings, vol. 1457. Sun SITE Central Europe (CEUR), 32–47.
- Frank L. Hitchcock. 1927. [The Expression of a Tensor or a Polyadic as a Sum of Products](#). *Journal of Mathematics and Physics* 6(1–4), 164–189.
- Pascal Hitzler and Adila Krisnadhi. 2018. [A Tutorial on Modular Ontology Modeling with Ontology Design Patterns: The Cooking Recipes Ontology](#). CoRR abs/1808.08433. 22 pages.
- Pascal Hitzler, Markus Krötzsch, and Sebastian Rudolph. 2010. *Foundations of Semantic Web Technologies*. Chapman and Hall / CRC Press.
- Pascal Hitzler, Markus Krötzsch, Bijan Parsia, Peter F. Patel-Schneider, and Sebastian Rudolph. 2012. [OWL 2 Web Ontology Language Primer \(Second Edition\)](#). W3C Recommendation. World Wide Web Consortium. December 11, 2012.
- Vinh Thinh Ho, Daria Stepanova, Mohamed H. Gad-Elrab, Evgeny Kharlamov, and Gerhard Weikum. 2018. [Rule Learning from Knowledge Graphs Guided by Embedding Models](#). In *The Semantic Web - ISWC 2018 - 17th International Semantic Web Conference, Monterey, CA, USA, October 8-12, 2018, Proceedings, Part I*, Denny Vrandečić, Kalina Bontcheva, Mari Carmen Suárez-Figueroa, Valentina Presutti, Irene Celino, Marta Sabou, Lucie-Aimée Kaffee, and Elena Simperl (Eds.). Lecture Notes in Computer Science, vol. 11136. Springer, 72–90.
- Cornelis Hoede. 1995. [On the ontology of knowledge graphs](#). In *Conceptual Structures: Applications, Implementation and Theory, Third International Conference on Conceptual Structures, ICCS '95, Santa Cruz, California, USA, August 14-18, 1995, Proceedings*, Gerard Ellis, Robert Levinson, William Rich, and John F. Sowa (Eds.). Lecture Notes in Computer Science, vol. 954. Springer, 308–322.
- Johannes Hoffart, Fabian M. Suchanek, Klaus Berberich, Edwin Lewis-Kelham, Gerard de Melo, and Gerhard Weikum. 2011. [YAGO2: Exploring and Querying World Knowledge in Time, Space, Context, and Many Languages](#). In *Proceedings of the 20th International Conference on World Wide Web, WWW 2011, Hyderabad, India, March 28 - April 1, 2011 (Companion Volume)*, Sadagopan Srinivasan, Krithi Ramamritham, Arun Kumar, M. P. Ravindra, Elisa Bertino, and Ravi Kumar (Eds.). ACM Press, 229–232.
- Raphael Hoffmann, Congle Zhang, Xiao Ling, Luke S. Zettlemoyer, and Daniel S. Weld. 2011. [Knowledge-Based Weak Supervision for Information Extraction of Overlapping Relations](#). In *The 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, Proceedings of the Conference, 19-24 June, 2011, Portland, Oregon, USA*, Dekang Lin, Yuji Matsumoto, and Rada Mihalcea (Eds.). The Association for Computational Linguistics, 541–550.
- Aidan Hogan. 2017. [Canonical Forms for Isomorphic and Equivalent RDF Graphs: Algorithms for Learning and Labelling Blank Nodes](#). *ACM Transactions on the Web* 11(4), 22:1–22:62.
- Aidan Hogan. 2020. [Knowledge Graphs: Research Directions](#). In *Reasoning Web. Declarative Artificial Intelligence - 16th International Summer School 2020, Oslo, Norway, June 24–26, 2020, Tutorial Lectures*, Marco Manna and Andreas Pieris (Eds.). Lecture Notes in Computer Science, vol. 12258. Springer, 223–253.
- Aidan Hogan. 2020. [The Web of Data](#). Springer.
- Aidan Hogan, Andreas Harth, Alexandre Passant, Stefan Decker, and Axel Polleres. 2010. [Weaving the Pedantic Web](#). In *Proceedings of the WWW2010 Workshop on Linked Data on the Web, LDOW 2010, Raleigh, USA, April 27, 2010*, Christian Bizer, Tom Heath, Tim Berners-Lee, and Michael Hausenblas (Eds.). CEUR Workshop Proceedings, vol. 628. Sun SITE Central Europe (CEUR). 10 pages.
- Aidan Hogan, Jürgen Umbrich, Andreas Harth, Richard Cyganiak, Axel Polleres, and Stefan Decker. 2012. [An empirical survey of Linked Data conformance](#). *Journal of Web Semantics* 14, 14–44.
- Aidan Hogan, Antoine Zimmermann, Jürgen Umbrich, Axel Polleres, and Stefan Decker. 2012. [Scalable and distributed methods for entity matching, consolidation and disambiguation over Linked Data corpora](#). *Journal of Web Semantics* 10, 76–110.
- Aidan Hogan, Marcelo Arenas, Alejandro Mallea, and Axel Polleres. 2014. [Everything you always wanted to know about blank nodes](#). *Journal of Web Semantics* 27–28, 42–69.

- Aidan Hogan, Juan L. Reutter, and Adrián Soto. 2020. [In-Database Graph Analytics with Recursive SPARQL](#). In *The Semantic Web - ISWC 2020 - 19th International Semantic Web Conference, Athens, Greece, November 2-6, 2020, Proceedings, Part I*, Jeff Z. Pan, Valentina A. M. Tamma, Claudia d'Amato, Krzysztof Janowicz, Bo Fu, Axel Polleres, Oshani Seneviratne, and Lalana Kagal (Eds.). Lecture Notes in Computer Science, vol. 12506. Springer, 511–528.
- Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutiérrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan F. Sequeda, Steffen Staab, and Antoine Zimmermann. 2021. [Knowledge Graphs](#). *ACM Computing Surveys* 54(4), 71:1–71:37.
- Ian Horrocks and Peter F. Patel-Schneider. 2004. [Reducing OWL entailment to description logic satisfiability](#). *Journal of Web Semantics* 1(4), 345–357.
- Ian Horrocks, Peter F. Patel-Schneider, Harold Boley, Said Tabet, Benjamin Grosz, and Mike Dean. 2004. [SWRL: A Semantic Web Rule Language Combining OWL and RuleML](#). W3C Member Submission. May 21, 2004.
- Sen Hu, Lei Zou, Jeffrey Xu Yu, Haixun Wang, and Dongyan Zhao. 2018. [Answering Natural Language Questions by Subgraph Matching over Knowledge Graphs](#). *IEEE Transactions on Knowledge and Data Engineering* 30(5), 824–837.
- Xiao Huang, Jingyuan Zhang, Dingcheng Li, and Ping Li. 2019. [Knowledge Graph Embedding Based Question Answering](#). In *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining, WSDM 2019, Melbourne, VIC, Australia, February 11-15, 2019*, J. Shane Culpepper, Alistair Moffat, Paul N. Bennett, and Kristina Lerman (Eds.). ACM Press, 105–113.
- Andy Hunt and Dave Thomas. 2003. [The Trip-Packing Dilemma](#). *IEEE Software* 20(3), 106–107.
- Rana Hussein, Dingqi Yang, and Philippe Cudré-Mauroux. 2018. [Are Meta-Paths Necessary?: Revisiting Heterogeneous Graph Embeddings](#). In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management, CIKM 2018, Torino, Italy, October 22-26, 2018*, Alfredo Cuzzocrea, James Allan, Norman W. Paton, Divesh Srivastava, Rakesh Agrawal, Andrei Z. Broder, Mohammed J. Zaki, K. Selçuk Candan, Alexandros Labrinidis, Assaf Schuster, and Haixun Wang (Eds.). ACM Press, 437–446.
- Dylan Hutchison, Bill Howe, and Dan Suciu. 2017. [LaraDB: A Minimalist Kernel for Linear and Relational Algebra Computation](#). In *Proceedings of the 4th ACM SIGMOD Workshop on Algorithms and Systems for MapReduce and Beyond, BeyondMR@SIGMOD 2017, Chicago, IL, USA, May 19, 2017*, Foto N. Afrati and Jacek Sroka (Eds.). ACM Press, 2:1–2:10.
- Eero Hyvönen, Eetu Mäkelä, Tomi Kauppinen, Olli Alm, Jussi Kurki, Tuukka Ruotsalo, Katri Seppälä, Joeli Takala, Kimmo Puputti, Heini Kuittinen, Kim Viljanen, Jouni Tuominen, Tuomas Palonen, Matias Frosterus, Reetta Sinkkilä, Panu Paakkari, Joonas Laitio, and Katariina Nyberg. 2009. [CultureSampo: A National Publication System of Cultural Heritage on the Semantic Web 2.0](#). In *The Semantic Web: Research and Applications, 6th European Semantic Web Conference, ESWC 2009, Heraklion, Crete, Greece, May 31-June 4, 2009, Proceedings*, Lora Aroyo, Paolo Traverso, Fabio Ciravegna, Philipp Cimiano, Tom Heath, Eero Hyvönen, Riichiro Mizoguchi, Eyal Oren, Marta Sabou, and Elena Paslaru Bontas Simperl (Eds.), vol. 5554. Springer, 851–856.
- Andreea Iana, Steffen Jung, Philipp Naeser, Aliaksandr Birukou, Sven Hertling, and Heiko Paulheim. 2019. [Building a Conference Recommender System Based on SciGraph and WikiCFP](#). In *Semantic Systems. The Power of AI and Knowledge Graphs - 15th International Conference, SEMANTiCS 2019, Karlsruhe, Germany, September 9-12, 2019, Proceedings*, Maribel Acosta, Philippe Cudré-Mauroux, Maria Maleshkova, Tassilo Pellegrini, Harald Sack, and York Sure-Vetter (Eds.). Lecture Notes in Computer Science, vol. 11702. Springer, 117–123.
- Renato Iannella and Serena Villata. 2018. [ODRL Information Model 2.2](#). W3C Recommendation. World Wide Web Consortium. February 15, 2018.
- International Health Terminology Standards Development Organisation. 2019. [SNOMED CT Editorial Guide](#). July 31, 2019.
- Alexandru Iosup, Tim Hegeman, Wing Lung Ngai, Stijn Heldens, Arnau Prat-Pérez, Thomas Manhardt, Hassan Chafi, Mihai Capota, Narayanan Sundaram, Michael J. Anderson, Ilie Gabriel Tanase, Yinglong Xia, Lifeng Nai, and Peter A. Boncz. 2016. [LDBC Graphalytics: A Benchmark for Large-Scale Graph on Parallel and Distributed Platforms](#). *Proceedings of the VLDB Endowment* 9(13), 1317–1328.

- Robert Isele, Anja Jentzsch, and Christian Bizer. 2011. [Efficient multidimensional blocking for link discovery without losing recall](#). In *Proceedings of the 14th International Workshop on the Web and Databases 2011, WebDB 2011, Athens, Greece, June 12, 2011*, Amélie Marian and Vasilis Vassalos (Eds.). 6 pages.
- P. James. 1992. Knowledge Graphs. In *Linguistic Instruments in Knowledge Engineering*, Reind P. van de Riet and Robert A. Meersman (Eds.). Elsevier, 97–117.
- Daniel Janke and Steffen Staab. 2018. [Storing and Querying Semantic Data in the Cloud](#). In *Reasoning Web. Learning, Uncertainty, Streaming, and Scalability - 14th International Summer School 2018, Esch-sur-Alzette, Luxembourg, September 22-26, 2018, Tutorial Lectures*, Claudia d'Amato and Martin Theobald (Eds.). Lecture Notes in Computer Science, vol. 11078. Springer, 173–222.
- Daniel Janke, Steffen Staab, and Matthias Thimm. 2018. [Impact analysis of data placement strategies on query efforts in distributed RDF stores](#). *Journal of Web Semantics* 50, 21–48.
- Krzysztof Janowicz, Pascal Hitzler, Benjamin Adams, Dave Kolas, and Charles Vardeman. 2014. [Five stars of Linked Data vocabulary use](#). *Semantic Web Journal* 5(3), 173–176.
- Krzysztof Janowicz, Bo Yan, Blake Regalia, Rui Zhu, and Gengchen Mai. 2018. [Debiasing Knowledge Graphs: Why Female Presidents are not like Female Popes](#). In *Proceedings of the ISWC 2018 Posters & Demonstrations, Industry and Blue Sky Ideas Tracks co-located with 17th International Semantic Web Conference (ISWC 2018), Monterey, USA, October 8th - to - 12th, 2018*, Marieke van Erp, Medha Atre, Vanessa López, Kavitha Srinivas, and Carolina Fortuna (Eds.). CEUR Workshop Proceedings, vol. 2180. Sun SITE Central Europe (CEUR). 5 pages.
- Nandish Jayaram, Sidharth Goyal, and Chengkai Li. 2015. [VIIQ: Auto-Suggestion Enabled Visual Interface for Interactive Graph Query Formulation](#). *Proceedings of the VLDB Endowment* 8(12), 1940–1943.
- Nandish Jayaram, Arijit Khan, Chengkai Li, Xifeng Yan, and Ramez Elmasri. 2015. [Querying Knowledge Graphs by Example Entity Tuples](#). *IEEE Transactions on Knowledge and Data Engineering* 27(10), 2797–2811.
- Guoliang Ji, Shizhu He, Liheng Xu, Kang Liu, and Jun Zhao. 2015. [Knowledge graph embedding via dynamic mapping matrix](#). In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing of the Asian Federation of Natural Language Processing, ACL 2015, July 26-31, 2015, Beijing, China, Volume 1: Long Papers*. The Association for Computational Linguistics, 687–696.
- Yun-fei Jiang and Ning Ma. 2002. [A Plan Recognition Algorithm Based on Plan Knowledge Graph](#). *Journal of Software* 13(4), 686–692.
- Dan Jurafsky and James H. Martin. 2021. [Speech and Language Processing](#).
- David Jurgens and Roberto Navigli. 2014. [It's All Fun and Games until Someone Annotates: Video Games with a Purpose for Linguistic Annotation](#). *Transactions of the Association for Computational Linguistics* 2, 449–464.
- Tobias Käfer, Ahmed Abdelrahman, Jürgen Umbrich, Patrick O'Byrne, and Aidan Hogan. 2013. [Observing Linked Data Dynamics](#). In *The Semantic Web: Semantics and Big Data, 10th International Conference, ESWC 2013, Montpellier, France, May 26-30, 2013. Proceedings*, Philipp Cimiano, Óscar Corcho, Valentina Presutti, Laura Hollink, and Sebastian Rudolph (Eds.). Lecture Notes in Computer Science, vol. 7882. Springer, 213–227.
- Lucie-Aimée Kaffee, Alessandro Piscopo, Pavlos Vougiouklis, Elena Simperl, Leslie Carr, and Lydia Pintscher. 2017. [A Glimpse into Babel: An Analysis of Multilinguality in Wikidata](#). In *Proceedings of the 13th International Symposium on Open Collaboration, OpenSym 2017, Galway, Ireland, August 23-25, 2017*, Lorraine Morgan (Ed.). ACM Press, 14:1–14:5.
- Hans Kamp. 1981. A Theory of Truth and Semantic Representation. In *Formal Methods in the Study of Language*, Jeroen A. G. Groenendijk, Theo M. V. Janssen, and Martin B. J. Stokhof (Eds.). Mathematical Centre Tracts, vol. 136. Mathematisch Centrum Amsterdam, 277–322.
- Elias Kärle, Umutcan Simsek, Oleksandra Panasiuk, and Dieter Fensel. 2018. [Building an Ecosystem for the Tyrolean Tourism Knowledge Graph](#). *CoRR* abs/1805.05744. 8 pages.
- Gjergji Kasneci, Fabian M. Suchanek, Georgiana Ifrim, Maya Ramanath, and Gerhard Weikum. 2008. [NAGA: Searching and Ranking Knowledge](#). In *Proceedings of the 24th International Conference on Data Engineering, ICDE 2008, April 7-12, 2008, Cancún, Mexico*, Gustavo Alonso, José A. Blakeley, and Arbee L. P. Chen (Eds.). IEEE Computer Society, 953–962.

- Andreas Kasten, Ansgar Scherp, Frederik Armknecht, and Matthias Krause. 2013. [Towards Search on Encrypted Graph Data](#). In *Proceedings of the Workshop on Society, Privacy and the Semantic Web - Policy and Technology (PrivOn2013) co-located with the 12th International Semantic Web Conference (ISWC 2013), Sydney, Australia, October 22, 2013*, Stefan Decker, Jim Hendler, and Sabrina Kirrane (Eds.). CEUR Workshop Proceedings, vol. 1121. Sun SITE Central Europe (CEUR), 46–57.
- Seyed Mehran Kazemi and David Poole. 2018. [Simple Embedding for Link Prediction in Knowledge Graphs](#). In *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, 3-8 December 2018, Montréal, Canada*, Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett (Eds.), 4289–4300.
- Seyed Mehran Kazemi, Rishab Goel, Kshitij Jain, Ivan Kobyzev, Akshay Sethi, Peter Forsyth, and Pascal Poupart. 2019. [Relational Representation Learning for Dynamic \(Knowledge\) Graphs: A Survey](#). CoRR abs/1905.11485.
- C. Maria Keet and Agnieszka Lawrynowicz. 2016. [Test-driven development of ontologies](#). In *The Semantic Web. Latest Advances and New Domains - 13th International Conference, ESWC 2016, Heraklion, Crete, Greece, May 29 - June 2, 2016, Proceedings*, Harald Sack, Eva Blomqvist, Mathieu d'Aquin, Chiara Ghidini, Simone Paolo Ponzetto, and Christoph Lange (Eds.). Lecture Notes in Computer Science, vol. 9678. Springer, 642–657.
- C. Maria Keet. 2018. [An Introduction to Ontology Engineering](#). College Publications.
- Mayank Kejriwal, Craig A. Knoblock, and Pedro Szekely (Eds.). 2021. *Knowledge Graphs: Fundamentals, Techniques, and Applications*. The MIT Press.
- Elisa F. Kendall and Deborah L. McGuinness. 2019. [Ontology Engineering](#). Synthesis Lectures on the Semantic Web: Theory and Technology, vol. 9. Morgan & Claypool.
- Michael Kifer and Harold Boley. 2013. [RIF Overview \(Second Edition\)](#). W3C Working Group Note. World Wide Web Consortium. February 5, 2013.
- Thomas N. Kipf and Max Welling. 2017. [Semi-Supervised Classification with Graph Convolutional Networks](#). In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net. 14 pages.
- Sabrina Kirrane, Ahmed Abdelrahman, Alessandra Mileo, and Stefan Decker. 2013. [Secure Manipulation of Linked Data](#). In *The Semantic Web - ISWC 2013 - 12th International Semantic Web Conference, Sydney, NSW, Australia, October 21-25, 2013, Proceedings, Part I*, Harith Alani, Lalana Kagal, Achille Fokoue, Paul Groth, Chris Biemann, Josian Xavier Parreira, Lora Aroyo, Natasha Noy, Chris Welty, and Krzysztof Janowicz (Eds.). Lecture Notes in Computer Science, vol. 8218. Springer, 248–263.
- Sabrina Kirrane, Alessandra Mileo, and Stefan Decker. 2017. [Access control and the Resource Description Framework: A survey](#). *Semantic Web Journal* 8(2), 311–352.
- Jon M. Kleinberg. 1999. [Hubs, authorities, and communities](#). *ACM Computing Surveys* 31(4es), p5.
- Peter Kluegl, Martin Atzmueller, and Frank Puppe. 2009. [TextMarker: A Tool for Rule-Based Information Extraction](#). In *UIMA-GSCL Workshop*, 233–240.
- Holger Knublauch and Dimitris Kontokostas. 2017. [Shapes Constraint Language \(SHACL\)](#). W3C Recommendation. World Wide Web Consortium. June 20, 2017.
- Holger Knublauch, James A. Hendler, and Kingsley Idehen. 2011. [SPIN – Overview and Motivation](#). W3C Member Submission. February 22, 2011.
- Hanna Köpcke and Erhard Rahm. 2010. [Frameworks for entity matching: A comparison](#). *Data and Knowledge Engineering* 69(2), 197–210.
- Arun Krishnan. 2018. [Making search easier: How Amazon's Product Graph is helping customers find products more easily](#). Amazon Blog. August 17, 2018.
- Adila Krisnadhi and Pascal Hitzler. 2016. [A Core Pattern for Events](#). In *Advances in Ontology Design and Patterns [revised and extended versions of the papers presented at the 7th edition of the Workshop on Ontology and Semantic Web Patterns, WOP@ISWC 2016, Kobe, Japan, 18th October 2016]*, Karl Hammar, Pascal Hitzler, Adila Krisnadhi, Agnieszka Lawrynowicz, Andrea Giovanni Nuzzolese, and Monika Solanki (Eds.). Studies on the Semantic Web, vol. 32. IOS Press, 29–37.
- Adila Krisnadhi and Pascal Hitzler. 2016. [Modeling With Ontology Design Patterns: Chess Games As a Worked Example](#). In *Ontology Engineering with Ontology Design Patterns - Foundations and Applications*, Pascal Hitzler, Aldo Gangemi, Krzysztof Janowicz, Adila Krisnadhi, and Valentina Presutti (Eds.). Studies on the Semantic Web, vol. 25. IOS Press, 3–21.

- Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2017. [ImageNet classification with deep convolutional neural networks](#). *Communications of the ACM* 60(6), 84–90.
- Markus Krötzsch, Maximilian Marx, Ana Ozaki, and Veronika Thost. 2018. [Attributed Description Logics: Reasoning on Knowledge Graphs](#). In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, IJCAI 2018, July 13-19, 2018, Stockholm, Sweden*, Jérôme Lang (Ed.). IJCAI/AAAI, 5309–5313.
- Peter Kümmel. 1973. [An Algorithm of Limited Syntax Based on Language Universals](#). In *Computational And Mathematical Linguistics: Proceedings of the 5th International Conference on Computational Linguistics, COLING 1973, Pisa, Italy, August 27 - September 1, 1973*, Antonio Zampolli and Nicoletta Calzolari (Eds.). The Association for Computational Linguistics, 225–248.
- H. T. Kung. 1982. [Why Systolic Architectures?](#). *IEEE Computer* 15(1), 37–46.
- Jose Emilio Labra Gayo, Eric Prud'hommeaux, Iovka Boneva, and Dimitris Kontokostas. 2018. [Validating RDF Data](#). *Synthesis Lectures on the Semantic Web: Theory and Technology*, vol. 7. Morgan & Claypool.
- Jose Emilio Labra Gayo, Herminio García-González, Daniel Fernández-Alvarez, and Eric Prud'hommeaux. 2019. [Challenges in RDF Validation](#). In *Current Trends in Semantic Web Technologies: Theory and Practice*, Giner Alor-Hernández, José Luis Sánchez-Cervantes, Alejandro Rodríguez-González, and Rafael Valencia-García (Eds.). Studies in Computational Intelligence. Springer, 121–151.
- Guillaume Lample, Miguel Ballesteros, Sandeep Subramanian, Kazuya Kawakami, and Chris Dyer. 2016. [Neural Architectures for Named Entity Recognition](#). In *NAACL HLT 2016, The 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, San Diego California, USA, June 12-17, 2016*, Kevin Knight, Ani Nenkova, and Owen Rambow (Eds.). The Association for Computational Linguistics, 260–270.
- Ni Lao and William W. Cohen. 2010. [Relational retrieval using a combination of path-constrained random walks](#). *Machine Learning* 81(1), 53–67.
- Jens Lehmann, Tim Furche, Giovanni Grasso, Axel-Cyrille Ngonga Ngomo, Christian Schallhart, Andrew Jon Sellers, Christina Unger, Lorenz Bühmann, Daniel Gerber, Konrad Höffner, David Liu, and Sören Auer. 2012. [deqa: Deep Web Extraction for Question Answering](#). In *The Semantic Web - ISWC 2012 - 11th International Semantic Web Conference, Boston, MA, USA, November 11-15, 2012, Proceedings, Part II*, Philippe Cudré-Mauroux, Jeff Heflin, Evren Sirin, Tania Tudorache, Jérôme Euzenat, Manfred Hauswirth, Josiane Xavier Parreira, Jim Hendler, Guus Schreiber, Abraham Bernstein, and Eva Blomqvist (Eds.). Lecture Notes in Computer Science, vol. 7650. Springer, 131–147.
- Jens Lehmann, Robert Isele, Max Jakob, Anja Jentzsch, Dimitris Kontokostas, Pablo N. Mendes, Sebastian Hellmann, Mohamed Morsey, Patrick van Kleef, Sören Auer, and Christian Bizer. 2015. [DBpedia - A large-scale, multilingual knowledge base extracted from Wikipedia](#). *Semantic Web Journal* 6(2), 167–195.
- Oliver Lehmberg, Dominique Ritze, Robert Meusel, and Christian Bizer. 2016. [A Large Public Corpus of Web Tables containing Time and Context Metadata](#). In *Proceedings of the 25th International Conference on World Wide Web, WWW 2016, Montreal, Canada, April 11-15, 2016, Companion Volume*, Jacqueline Bourdeau, Jim Hendler, Roger Nkambou, Ian Horrocks, and Ben Y. Zhao (Eds.). ACM Press, 75–76.
- Yuangui Lei, Victoria Uren, and Enrico Motta. 2007. [A framework for evaluating semantic metadata](#). In *Proceedings of the Fourth International Conference on Knowledge Capture*, Derek Sleeman and Ken Barker (Eds.). ACM Press, 135–142.
- Douglas B. Lenat. 1995. [CYC: A large-scale investment in knowledge infrastructure](#). *Communications of the ACM* 38(11), 33–38.
- Mark Levene and Alexandra Poulouvasilis. 1989. The Hypernode Model: A Graph-Theoretic Approach to Integrating Data and Computation. In *Workshop on Foundations of Models and Languages for Data and Objects, Aigen, Austria, 25.-29. September 1989*, Andreas Heuer (Ed.). Informatik-Berichte des IfI, vol. 89-2. Technische Universität Clausthal, 55–77.
- Ninghui Li, Tiancheng Li, and Suresh Venkatasubramanian. 2007. [t-closeness: Privacy beyond k-anonymity and l-diversity](#). In *Proceedings of the 23rd International Conference on Data Engineering, ICDE 2007, The Marmara Hotel, Istanbul, Turkey, April 15-20, 2007*, Rada Chirkova, Asuman Dogac, M. Tamer Özsu, and Timos K. Sellis (Eds.). IEEE Computer Society, 106–115.
- Girija Limaye, Sunita Sarawagi, and Soumen Chakrabarti. 2010. [Annotating and Searching Web Tables Using Entities, Types and Relationships](#). *Proceedings of the VLDB Endowment* 3(1), 1338–1347.

- Zhiyuan Lin and Mahesh Tripunitara. 2017. [Graph Automorphism-Based, Semantics-Preserving Security for the Resource Description Framework \(RDF\)](#). In *Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy, CODASPY 2017, Scottsdale, AZ, USA, March 22-24, 2017*, Gail-Joon Ahn, Alexander Pretschner, and Gabriel Ghinita (Eds.). ACM Press, 337–348.
- Yankai Lin, Zhiyuan Liu, Maosong Sun, Yang Liu, and Xuan Zhu. 2015. [Learning entity and relation embeddings for knowledge graph completion](#). In *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence, January 25-30, 2015, Austin, Texas, USA*, Blai Bonet and Sven Koenig (Eds.). AAAI Press, 2181–2187.
- Xiao Ling and Daniel S. Weld. 2012. [Fine-Grained Entity Recognition](#). In *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence, July 22-26, 2012, Toronto, Ontario, Canada*, Jörg Hoffmann and Bart Selman (Eds.). AAAI Press, 94–100.
- Kun Liu and Evimaria Terzi. 2008. [Towards identity anonymization on graphs](#). In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2008, Vancouver, BC, Canada, June 10-12, 2008*, Jason Tsong-Li Wang (Ed.). ACM Press, 93–106.
- Yike Liu, Tara Safavi, Abhilash Dighe, and Danai Koutra. 2018. [Graph Summarization Methods and Applications: A Survey](#). *ACM Computing Surveys* 51(3), 62:1–62:x34.
- John W. Lloyd. 1984. *Foundations of logic programming*. Springer.
- Colin Lockard, Xin Luna Dong, Prashant Shiralkar, and Arash Einolghozati. 2018. [CERES: Distantly Supervised Relation Extraction from the Semi-Structured Web](#). *Proceedings of the VLDB Endowment* 11(10), 1084–1096.
- Dave Longley and Manu Sporny. 2019. [RDF Dataset Normalization, A Standard RDF Dataset Normalization Algorithm](#). W3C Community Group Draft Report. February 27, 2019.
- Yucheng Low, Joseph Gonzalez, Aapo Kyrola, Danny Bickson, Carlos Guestrin, and Joseph M. Hellerstein. 2012. [Distributed GraphLab: A Framework for Machine Learning in the Cloud](#). *Proceedings of the VLDB Endowment* 5(8), 716–727.
- Chunliang Lu, Lidong Bing, Wai Lam, Ki Chan, and Yuan Gu. 2013. [Web Entity Detection for Semi-structured Text Data Records with Unlabeled Data](#). *International Journal of Computational Linguistics and Applications* 4(2), 135–150.
- Chun Lu, Philippe Laublet, and Milan Stankovic. 2016. [Travel Attractions Recommendation with Knowledge Graphs](#). In *Knowledge Engineering and Knowledge Management - 20th International Conference, EKAW 2016, Bologna, Italy, November 19-23, 2016, Proceedings*, Eva Blomqvist, Paolo Ciancarini, Francesco Poggi, and Fabio Vitali (Eds.). Lecture Notes in Computer Science, vol. 10024. Springer, 416–431.
- Thomas Lukasiewicz, Maria Vanina Martinez, and Gerardo I. Simari. 2013. [Complexity of Inconsistency-Tolerant Query Answering in Datalog +/-](#). In *Informal Proceedings of the 26th International Workshop on Description Logics, Ulm, Germany, July 23 - 26, 2013*, Thomas Eiter, Birte Glimm, Yevgeny Kazakov, and Markus Krötzsch (Eds.). CEUR Workshop Proceedings, vol. 1014. Sun SITE Central Europe (CEUR), 791–803.
- Gang Luo, Xiaojiang Huang, Chin-Yew Lin, and Zaiqing Nie. 2015. [Joint Entity Recognition and Disambiguation](#). In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing, EMNLP 2015, Lisbon, Portugal, September 17-21, 2015*, Lluís Màrquez, Chris Callison-Burch, Jian Su, Daniele Pighin, and Yuval Marton (Eds.). The Association for Computational Linguistics, 879–888.
- Ricardo José Machado and Armando Freitas da Rocha. 1990. [The Combinatorial Neural Network: A Connectionist Model for Knowledge Based Systems](#). In *Uncertainty in Knowledge Bases, 3rd International Conference on Information Processing and Management of Uncertainty in Knowledge-Based Systems, IPMU '90, Paris, France, July 2-6, 1990, Proceedings*, Bernadette Bouchon-Meunier, Ronald R. Yager, and Lotfi A. Zadeh (Eds.). Lecture Notes in Computer Science, vol. 521. Springer, 578–587.
- Jayant Madhavan, David Ko, Lucja Kot, Vignesh Ganapathy, Alex Rasmussen, and Alon Y. Halevy. 2008. [Google's Deep Web crawl](#). *Proceedings of the VLDB Endowment* 1(2), 1241–1252.
- Farzaneh Mahdisoltani, Joanna Biega, and Fabian M. Suchanek. 2015. [YAGO3: A Knowledge Base from Multilingual Wikipedias](#). In *CIDR 2015, Seventh Biennial Conference on Innovative Data Systems Research, Asilomar, CA, USA, January 4-7, 2015, Online Proceedings*. www.cidrdb.org. 11 pages.
- Pierre Mailliot and Carlos Bobed. 2018. [Measuring structural similarity between RDF graphs](#). In *Proceedings of the 33rd Annual ACM Symposium on Applied Computing, SAC 2018, Pau, France, April*

- 09-13, 2018, Hisham M. Haddad, Roger L. Wainwright, and Richard Chbeir (Eds.). ACM Press, 1960–1967.
- Grzegorz Malewicz, Matthew H. Austern, Aart J. C. Bik, James C. Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. 2010. [Pregel: a system for large-scale graph processing](#). In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2010, Indianapolis, Indiana, USA, June 6-10, 2010*, Ahmed K. Elmagarmid and Divyakant Agrawal (Eds.). ACM Press, 135–146.
- Stanislav Malyshev, Markus Krötzsch, Larry González, Julius Gonsior, and Adrian Bielefeldt. 2018. [Getting the most out of Wikidata: Semantic technology usage in Wikipedia's knowledge graph](#). In *The Semantic Web - ISWC 2018 - 17th International Semantic Web Conference, Monterey, CA, USA, October 8-12, 2018, Proceedings, Part II*, Denny Vrandečić, Kalina Bontcheva, Mari Carmen Suárez-Figueroa, Valentina Presutti, Irene Celino, Marta Sabou, Lucie-Aimée Kaffee, and Elena Simperl (Eds.). Lecture Notes in Computer Science, vol. 11137. Springer, 376–394.
- Ezio Marchi and Osvaldo Miguel. 1974. [On the structure of the teaching-learning interactive process](#). *International Journal of Game Theory* 3, 83–99.
- Jose L. Martínez-Rodríguez, Ivan López-Arévalo, and Ana B. Rios-Alvarado. 2018. [OpenIE-based approach for Knowledge Graph construction from text](#). *Expert Systems With Applications* 113, 339–355.
- Jose L. Martínez-Rodríguez, Aidan Hogan, and Ivan Lopez-Arevalo. 2020. [Information Extraction meets the Semantic Web: A Survey](#). *Semantic Web Journal* 11(2), 255–335.
- Ricardo Alonso Maturana, Elena Alvarado-Cortes, Susana López-Sola, María Ortega Martínez-Losa, and Pablo Hermoso-González. 2018. [La Rioja Turismo: The Construction and Exploitation of a Queryable Tourism Knowledge Graph](#). In *Current Trends in Web Engineering - ICWE 2018 International Workshops, MATWEP, EnWot, KD-WEB, WEOD, TourismKG, Cáceres, Spain, June 5, 2018, Revised Selected Papers*, Cesare Pautasso, Fernando Sánchez-Figueroa, Kari Systä, and Juan Manuel Murillo Rodriguez (Eds.). Lecture Notes in Computer Science, vol. 11153. Springer, 213–220.
- Cynthia Matuszek, John Cabral, Michael J. Witbrock, and John De Oliveira. 2006. [An Introduction to the Syntax and Content of Cyc](#). In *Formalizing and Compiling Background Knowledge and Its Applications to Knowledge Representation and Question Answering, Papers from the 2006 AAAI Spring Symposium, Technical Report SS-06-05, Stanford, California, USA, March 27-29, 2006*, Chitta Baral (Ed.). AAAI Press, 44–49.
- Mausam, Michael Schmitz, Stephen Soderland, Robert Bart, and Oren Etzioni. 2012. [Open Language Learning for Information Extraction](#). In *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning, EMNLP-CoNLL 2012, July 12-14, 2012, Jeju Island, Korea, Jun'ichi Tsujii, James Henderson, and Marius Pasca (Eds.)*. The Association for Computational Linguistics, 523–534.
- Mausam. 2016. [Open Information Extraction Systems and Downstream Applications](#). In *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence, IJCAI 2016, New York, NY, USA, 9-15 July 2016*, Subbarao Kambhampati (Ed.). IJCAI/AAAI, 4074–4077.
- Diana Maynard, Kalina Bontcheva, and Isabelle Augenstein. 2016. [Natural Language Processing for the Semantic Web](#). Morgan & Claypool.
- John McCarthy. 1990. *Formalizing Common Sense: Papers by John McCarthy*. Greenwood Publishing Group.
- John McCarthy. 1993. [Notes on Formalizing Context](#). In *Proceedings of the 13th International Joint Conference on Artificial Intelligence. Chambéry, France, August 28 - September 3, 1993*, Ruzena Bajcsy (Ed.). Morgan Kaufmann, 555–562.
- Edgar Meij. 2019. Understanding News using the Bloomberg Knowledge Graph. Invited talk at the Big Data Innovators Gathering (TheWebConf).
- Pablo N. Mendes, Max Jakob, and Christian Bizer. 2012. [DBpedia: A Multilingual Cross-domain Knowledge Base](#). In *Proceedings of the Eighth International Conference on Language Resources and Evaluation, LREC 2012, Istanbul, Turkey, May 23-25, 2012*, Nicoletta Calzolari, Khalid Choukri, Thierry Declerck, Mehmet Ugur Dogan, Bente Maegaard, Joseph Mariani, Jan Odijk, and Stelios Piperidis (Eds.). European Language Resources Association (ELRA), 1813–1817.
- Pablo N. Mendes, Hannes Mühleisen, and Christian Bizer. 2012. [Sieve: linked data quality assessment and fusion](#). In *Proceedings of the 2012 Joint EDBT/ICDT Workshops, Berlin, Germany, March 30, 2012*, Divesh Srivastava and Ismail Ari (Eds.). ACM Press, 116–123.
- Nandana Mihindukulasooriya, Mohammad Rifat Ahmmad Rashid, Giuseppe Rizzo, Raúl García-Castro, Óscar Corcho, and Marco Torchiano. 2018. [RDF shape induction using knowledge base profiling](#). In *Proceedings of the 33rd Annual ACM Symposium on Applied Computing, SAC 2018, Pau, France, April 1-5, 2018*, Divesh Srivastava and Ismail Ari (Eds.). ACM Press, 116–123.

- 09-13, 2018, Hisham M. Haddad, Roger L. Wainwright, and Richard Chbeir (Eds.). ACM Press, 1952–1959.
- Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. [Efficient estimation of word representations in vector space](#). In *1st International Conference on Learning Representations, ICLR 2013, Scottsdale, Arizona, USA, May 2-4, 2013, Workshop Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). OpenReview.net. 12 pages.
- George A. Miller and Christiane Fellbaum. 2007. [WordNet then and now](#). *Language Resources and Evaluation* 41(2), 209–214.
- Justin J. Miller. 2013. [Graph Database Applications and Concepts with Neo4j](#). In *Proceedings of the Southern Association for Information Systems Conference, Atlanta, GA, USA March 23rd-24th, 2013*. Association for Information Systems, 141–147.
- Marvin Minsky. 1974. [A Framework for representing knowledge](#). *MIT-AI Memo 306, Santa Monica*. 76 pages.
- Mike Mintz, Steven Bills, Rion Snow, and Daniel Jurafsky. 2009. [Distant supervision for relation extraction without labeled data](#). In *ACL 2009, Proceedings of the 47th Annual Meeting of the Association for Computational Linguistics and the 4th International Joint Conference on Natural Language Processing of the AFNLP, 2-7 August 2009, Singapore*, Keh-Yih Su, Jian Su, and Janyce Wiebe (Eds.). The Association for Computational Linguistics, 1003–1011.
- Tom M. Mitchell, William W. Cohen, Estevam R. Hruschka Jr., Partha P. Talukdar, Bo Yang, Justin Betteridge, Andrew Carlson, Bhavana Dalvi Mishra, Matt Gardner, Bryan Kisiel, Jayant Krishnamurthy, Ni Lao, Kathryn Mazaitis, Thahir Mohamed, Nandapandula Nakashole, Emmanouil A. Platanios, Alan Ritter, Mehdi Samadi, Burr Settles, Richard C. Wang, Derry Wijaya, Abhinav Gupta, Xinlei Chen, Abulhair Saparov, Malcolm Greaves, and Joel Welling. 2018. [Never-ending learning](#). *Communications of the ACM* 61(5), 103–115.
- Federico Monti, Davide Boscaini, Jonathan Masci, Emanuele Rodolà, Jan Svoboda, and Michael M. Bronstein. 2017. [Geometric Deep Learning on Graphs and Manifolds Using Mixture Model CNNs](#). In *2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, Honolulu, HI, USA, July 21-26, 2017*. IEEE Computer Society, 5425–5434.
- Elena Montiel-Ponsoda, Víctor Rodríguez-Doncel, and Jorge Gracia. 2017. [Building the Legal Knowledge Graph for Smart Compliance Services in Multilingual Europe](#). In *Proceedings of the 1st Workshop on Technologies for Regulatory Compliance co-located with the 30th International Conference on Legal Knowledge and Information Systems (JURIX 2017), Luxembourg, December 13, 2017*, Víctor Rodríguez-Doncel, Pompeu Casanovas, and Jorge González-Conejero (Eds.). CEUR Workshop Proceedings, vol. 2049. Sun SITE Central Europe (CEUR), 15–17.
- Benjamin Moreau, Patricia Serrano-Alvarado, Matthieu Perrin, and Emmanuel Desmontils. 2019. [Modelling the Compatibility of Licenses](#). In *The Semantic Web - 16th International Conference, ESWC 2019, Portorož, Slovenia, June 2-6, 2019, Proceedings*, Pascal Hitzler, Miriam Fernández, Krzysztof Janowicz, Amrapali Zaveri, Alasdair J. G. Gray, Vanessa López, Armin Haller, and Karl Hammar (Eds.). Lecture Notes in Computer Science, vol. 11503. Springer, 255–269.
- José Moreno-Vega and Aidan Hogan. 2018. [GraFa: Scalable Faceted Browsing for RDF Graphs](#). In *The Semantic Web - ISWC 2018 - 17th International Semantic Web Conference, Monterey, CA, USA, October 8-12, 2018, Proceedings, Part I*, Denny Vrandečić, Kalina Bontcheva, Mari Carmen Suárez-Figueroa, Valentina Presutti, Irene Celino, Marta Sabou, Lucie-Aimée Kaffee, and Elena Simperl (Eds.). Lecture Notes in Computer Science, vol. 11136. Springer, 301–317.
- Andrea Moro and Roberto Navigli. 2013. Integrating Syntactic and Semantic Analysis into the Open Information Extraction Paradigm. In *IJCAI 2013, Proceedings of the 23rd International Joint Conference on Artificial Intelligence, Beijing, China, August 3-9, 2013*, Francesca Rossi (Ed.). IJCAI/AAAI, 2148–2154.
- Andrea Moro, Alessandro Raganato, and Roberto Navigli. 2014. [Entity Linking meets Word Sense Disambiguation: a Unified Approach](#). *Transactions of the Association for Computational Linguistics* 2, 231–244.
- Boris Motik, Rob Shearer, and Ian Horrocks. 2009. [Hypertableau Reasoning for Description Logics](#). *Journal of Artificial Intelligence Research* 36, 165–228.
- Boris Motik, Bernardo Cuenca Grau, Ian Horrocks, Zhe Wu, Achille Fokoue, and Carsten Lutz. 2012. [OWL 2 Web Ontology Language Profiles \(Second Edition\)](#). W3C Recommendation. World Wide Web Consortium. December 11, 2012.

- Varish Mulwad, Tim Finin, and Anupam Joshi. 2013. [Semantic Message Passing for Generating Linked Data from Tables](#). In *The Semantic Web - ISWC 2013 - 12th International Semantic Web Conference, Sydney, NSW, Australia, October 21-25, 2013, Proceedings, Part I*, Harith Alani, Lalana Kagal, Achille Fokoue, Paul Groth, Chris Biemann, Josian Xavier Parreira, Lora Aroyo, Natasha Noy, Chris Welty, and Krzysztof Janowicz (Eds.). Lecture Notes in Computer Science, vol. 8218. Springer, 363–378.
- Chris Mungall, Alan Ruttenberg, Ian Horrocks, and David Osumi-Sutherland. 2012. [OBO Flat File Format 1.4 Syntax and Semantics](#). Editor's Draft. May, 2012.
- Sergio Muñoz, Jorge Pérez, and Claudio Gutiérrez. 2009. [Simple and Efficient Minimal RDFS](#). *Journal of Web Semantics* 7(3), 220–234.
- Emir Muñoz, Aidan Hogan, and Alessandra Mileo. 2014. [Using Linked Data to mine RDF from Wikipedia's tables](#). In *Seventh ACM International Conference on Web Search and Data Mining, WSDM 2014, New York, NY, USA, February 24-28, 2014*, Ben Carterette, Fernando Diaz, Carlos Castillo, and Donald Metzler (Eds.). ACM Press, 533–542.
- David Nadeau and Satoshi Sekine. 2007. [A survey of named entity recognition and classification](#). *Linguistic Investigations* 30(1), 3–26.
- Ndapandula Nakashole, Tomasz Tylenda, and Gerhard Weikum. 2013. [Fine-grained Semantic Typing of Emerging Entities](#). In *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics, ACL 2013, 4-9 August 2013, Sofia, Bulgaria, Volume 1: Long Papers*. The Association for Computational Linguistics, 1488–1497.
- Arvind Narayanan and Vitaly Shmatikov. 2009. [De-anonymizing Social Networks](#). In *30th IEEE Symposium on Security and Privacy (S&P 2009), 17-20 May 2009, Oakland, California, USA*. IEEE Computer Society, 173–187.
- Roberto Navigli and Simone Paolo Ponzetto. 2012. [BabelNet: The automatic construction, evaluation and application of a wide-coverage multilingual semantic network](#). *Artificial Intelligence* 193, 217–250.
- Roberto Navigli. 2009. [Word Sense Disambiguation: A Survey](#). *ACM Computing Surveys* 41(2), 1–69.
- Markus Nentwig, Michael Hartung, Axel-Cyrille Ngonga Ngomo, and Erhard Rahm. 2017. [A survey of current link discovery frameworks](#). *Semantic Web Journal* 8(3), 419–436.
- Sebastian Neumaier and Axel Polleres. 2019. [Enabling Spatio-Temporal Search in Open Data](#). *Journal of Web Semantics* 55, 21–36.
- Sebastian Neumaier, Jürgen Umbrich, Josiane Xavier Parreira, and Axel Polleres. 2016. [Multi-level Semantic Labelling of Numerical Values](#). In *The Semantic Web - ISWC 2016 - 15th International Semantic Web Conference, Kobe, Japan, October 17-21, 2016, Proceedings, Part I*, Paul T. Groth, Elena Simperl, Alasdair J. G. Gray, Marta Sabou, Markus Krötzsch, Freddy Lécué, Fabian Flöck, and Yolanda Gil (Eds.). Lecture Notes in Computer Science, vol. 9981. Springer, 428–445.
- David Newman. 2019. Knowledge Graphs and AI: The Future of Financial Data.
- Axel-Cyrille Ngonga Ngomo and Sören Auer. 2011. [LIMES - A Time-Efficient Approach for Large-Scale Link Discovery on the Web of Data](#). In *IJCAI 2011, Proceedings of the 22nd International Joint Conference on Artificial Intelligence, Barcelona, Catalonia, Spain, July 16-22, 2011*, Toby Walsh (Ed.). IJCAI/AAAI, 2312–2317.
- Axel-Cyrille Ngonga Ngomo, Lorenz Bühmann, Christina Unger, Jens Lehmann, and Daniel Gerber. 2013. [Sorry, I don't speak SPARQL: translating SPARQL queries into natural language](#). In *22nd International World Wide Web Conference, WWW '13, Rio de Janeiro, Brazil, May 13-17, 2013*, Daniel Schwabe, Virgílio A. F. Almeida, Hartmut Glaser, Ricardo Baeza-Yates, and Sue B. Moon (Eds.). ACM Press, 977–988.
- Axel-Cyrille Ngonga Ngomo. 2012. [Link discovery with guaranteed reduction ratio in affine spaces with Minkowski measures](#). In *The Semantic Web - ISWC 2012 - 11th International Semantic Web Conference, Boston, MA, USA, November 11-15, 2012, Proceedings, Part I*, Philippe Cudré-Mauroux, Jeff Heflin, Evren Sirin, Tania Tudorache, Jérôme Euzenat, Manfred Hauswirth, Josiane Xavier Parreira, Jim Hendler, Guus Schreiber, Abraham Bernstein, and Eva Blomqvist (Eds.). Lecture Notes in Computer Science, vol. 7649. Springer, 378–393.
- Axel-Cyrille Ngonga Ngomo. 2013. [ORCHID–reduction-ratio-optimal computation of geo-spatial distances for link discovery](#). In *The Semantic Web - ISWC 2013 - 12th International Semantic Web Conference, Sydney, NSW, Australia, October 21-25, 2013, Proceedings, Part I*, Harith Alani, Lalana Kagal, Achille Fokoue, Paul Groth, Chris Biemann, Josian Xavier Parreira, Lora Aroyo, Natasha Noy, Chris Welty, and Krzysztof Janowicz (Eds.). Lecture Notes in Computer Science, vol. 8218. Springer, 395–410.

- Vinh Nguyen, Olivier Bodenreider, and Amit Sheth. 2014. [Don't Like RDF Reification?: Making Statements About Statements Using Singleton Property](#). In *23rd International World Wide Web Conference, WWW '14, Seoul, Republic of Korea, April 7-11, 2014*, Chin-Wan Chung, Andrei Z. Broder, Kyuseok Shim, and Torsten Suel (Eds.). ACM Press, 759–770.
- Dat Ba Nguyen, Martin Theobald, and Gerhard Weikum. 2016. [J-NERD: Joint Named Entity Recognition and Disambiguation with Rich Linguistic Features](#). *Transactions of the Association for Computational Linguistics* 4, 215–229.
- Maximilian Nickel and Volker Tresp. 2013. [Tensor factorization for multi-relational learning](#). In *Machine Learning and Knowledge Discovery in Databases - European Conference, ECML PKDD 2013, Prague, Czech Republic, September 23-27, 2013, Proceedings, Part III*, Hendrik Blockeel, Kristian Kersting, Siegfried Nijssen, and Filip Zelezny (Eds.). Lecture Notes in Computer Science, vol. 8190. Springer, 617–621.
- Maximilian Nickel, Kevin Murphy, Volker Tresp, and Evgeniy Gabrilovich. 2016. [A Review of Relational Machine Learning for Knowledge Graphs](#). *Proceedings of the IEEE* 104(1), 11–33.
- Maximilian Nickel, Lorenzo Rosasco, and Tomaso A. Poggio. 2016. [Holographic Embeddings of Knowledge Graphs](#). In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence, February 12-17, 2016, Phoenix, Arizona, USA*, Dale Schuurmans and Michael P. Wellman (Eds.). AAAI Press, 1955–1961.
- Natalya F. Noy and Deborah L. McGuinness. 2001. [Ontology Development 101: A Guide to Creating Your First Ontology](#). Stanford Knowledge Systems Laboratory. March, 2001.
- Natasha Noy, Yuqing Gao, Anshu Jain, Anant Narayanan, Alan Patterson, and Jamie Taylor. 2019. [Industry-scale Knowledge Graphs: Lessons and Challenges](#). *ACM Queue* 17(2). 20 pages.
- Sri Nurdianti and Cornelis Hoede. 2012. [25 Years of Development of Knowledge Graph Theory: the Results and the Challenge](#). Memorandum 1876, University of Twente. September, 2012.
- Ekpe Okorafor and Atish Ray. 2019. [The path from data to knowledge](#). Accenture Applied Intelligence Blog. June 19, 2019.
- Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. [The PageRank Citation Ranking: Bringing order to the Web](#). Stanford InfoLab. November, 1999.
- Jeff Z. Pan, Guido Vetere, José Manuel Gómez-Pérez, and Honghan Wu (Eds.). 2017. [Exploiting Linked Data and Knowledge Graphs in Large Organisations](#). Springer.
- Oleksandra Panasiuk, Simon Steyskal, Giray Havur, Anna Fensel, and Sabrina Kirrane. 2018. [Modeling and Reasoning over Data Licenses](#). In *The Semantic Web: ESWC 2018 Satellite Events - ESWC 2018 Satellite Events, Heraklion, Crete, Greece, June 3-7, 2018, Revised Selected Papers*, Aldo Gangemi, Anna Lisa Gentile, Andrea Giovanni Nuzzolese, Sebastian Rudolph, Maria Maleshkova, Heiko Paulheim, Jeff Z. Pan, and Mehwish Alam (Eds.). Lecture Notes in Computer Science, vol. 11155. Springer, 218–222.
- Harshvardhan J. Pandit, Axel Polleres, Beatriz Esteves, Bert Bos, Bud Bruegger, Elmar Kiesling, Eva Schlehahn, David Hickey, Fajar J. Ekaputra, Georg P. Krog, Javier D. Fernández, Mark Lizar, Paul Ryan, Piero Bonatti, Ramisa Gachpaz Hamed, Rigo Wenning, Rob Brennan, and Simon Steyskal. 2021. [Data Privacy Vocabulary \(DPV\) version 0.3](#). Draft Community Group Report. World Wide Web Consortium. October 1, 2021.
- Vicky Papavasileiou, Giorgos Flouris, Iirini Fundulaki, Dimitris Kotzinos, and Vassilis Christophides. 2013. [High-level change detection in RDF\(S\) KBs](#). *ACM Transactions on Database Systems* 38(1), 1:1–1:42.
- Namyong Park, Andrey Kan, Xin Luna Dong, Tong Zhao, and Christos Faloutsos. 2019. [Estimating Node Importance in Knowledge Graphs Using Graph neural Networks](#). In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2019, Anchorage, AK, USA, August 4-8, 2019*, Ankur Teredesai, Vipin Kumar, Ying Li, Rómer Rosales, Evimaria Terzi, and George Karypis (Eds.). ACM Press, 596–606.
- Namyong Park, Andrey Kan, Xin Luna Dong, Tong Zhao, and Christos Faloutsos. 2020. [MultiImport: Inferring Node Importance in a Knowledge Graph from Multiple Input Signals](#). In *KDD '20: The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, CA, USA, August 23-27, 2020*, Rajesh Gupta, Yan Liu, Jiliang Tang, and B. Aditya Prakash (Eds.). ACM Press, 503–512.
- Jeff Pasternack and Dan Roth. 2010. [Knowing What to Believe \(when You Already Know Something\)](#). In *COLING 2010, 23rd International Conference on Computational Linguistics, Proceedings of the Conference, 23-27 August 2010, Beijing, China*, Chu-Ren Huang and Dan Jurafsky (Eds.). Tsinghua University Press, 877–885.

- Jeff Pasternack and Dan Roth. 2011. [Making Better Informed Trust Decisions with Generalized Fact-Finding](#). In *IJCAI 2011, Proceedings of the 22nd International Joint Conference on Artificial Intelligence, Barcelona, Catalonia, Spain, July 16-22, 2011*, Toby Walsh (Ed.). IJCAI/AAAI, 2324–2329.
- Heiko Paulheim and Christian Bizer. 2013. [Type inference on noisy RDF data](#). In *The Semantic Web - ISWC 2013 - 12th International Semantic Web Conference, Sydney, NSW, Australia, October 21-25, 2013, Proceedings, Part II*, Harith Alani, Lalana Kagal, Achille Fokoue, Paul Groth, Chris Biemann, Josian Xavier Parreira, Lora Aroyo, Natasha Noy, Chris Welty, and Krzysztof Janowicz (Eds.). Lecture Notes in Computer Science, vol. 8219. Springer, 510–525.
- Heiko Paulheim. 2017. [Knowledge graph refinement: A survey of approaches and evaluation methods](#). *Semantic Web Journal* 8(3), 489–508.
- Heiko Paulheim. 2018. [How much is a Triple? Estimating the Cost of Knowledge Graph Creation](#). In *Proceedings of the ISWC 2018 Posters & Demonstrations, Industry and Blue Sky Ideas Tracks co-located with 17th International Semantic Web Conference (ISWC 2018), Monterey, USA, October 8th - to - 12th, 2018*, Marieke van Erp, Medha Atre, Vanessa López, Kavitha Srinivas, and Carolina Fortuna (Eds.). CEUR Workshop Proceedings, vol. 2180. Sun SITE Central Europe (CEUR). 4 pages.
- Chaveevan Pechsiri and Rapepun Piriyaikul. 2010. [Explanation Knowledge Graph Construction Through Causality Extraction from Texts](#). *Journal of Computer Science and Technology* 25(5), 1055–1070.
- Charles S. Peirce. 1878. How to Make Our Ideas Clear. *Popular Science Monthly* 12, 286–302.
- Tassilo Pellegrini, Giray Havur, Simon Steyskal, Oleksandra Panasiuk, Anna Fensel, Victor Mireles, Thomas Thurner, Axel Polleres, Sabrina Kirrane, and Andrea Schönhofer. 2019. DALICC: A License Management Framework for Digital Assets. In *Proceedings of the Internationales Rechtsinformatik Symposium (IRIS)*. 10 pages.
- Thomas Pellissier Tanon, Denny Vrandečić, Sebastian Schaffert, Thomas Steiner, and Lydia Pintscher. 2016. [From Freebase to Wikidata: The Great Migration](#). In *Proceedings of the 25th International Conference on World Wide Web, WWW 2016, Montreal, Canada, April 11-15, 2016*, Jacqueline Bourdeau, Jim Hendler, Roger Nkambou, Ian Horrocks, and Ben Y. Zhao (Eds.). ACM Press, 1419–1428.
- Thomas Pellissier Tanon, Daria Stepanova, Simon Razniewski, Paramita Mirza, and Gerhard Weikum. 2017. [Completeness-Aware Rule Learning from Knowledge Graphs](#). In *The Semantic Web - ISWC 2017 - 16th International Semantic Web Conference, Vienna, Austria, October 21-25, 2017, Proceedings, Part I*, Claudia d'Amato, Miriam Fernández, Valentina A. M. Tamma, Freddy Lécué, Philippe Cudré-Mauroux, Juan F. Sequeda, Christoph Lange, and Jeff Heflin (Eds.). Lecture Notes in Computer Science, vol. 10587. Springer, 507–525.
- Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. [Glove: Global vectors for word representation](#). In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL*, Alessandro Moschitti, Bo Pang, and Walter Daelemans (Eds.). The Association for Computational Linguistics, 1532–1543.
- Silvio Peroni, David M. Shotton, and Fabio Vitali. 2017. [One Year of the OpenCitations Corpus – Releasing RDF-Based Scholarly Citation Data into the Public Domain](#). In *The Semantic Web - ISWC 2017 - 16th International Semantic Web Conference, Vienna, Austria, October 21-25, 2017, Proceedings, Part II*, Claudia d'Amato, Miriam Fernández, Valentina A. M. Tamma, Freddy Lécué, Philippe Cudré-Mauroux, Juan F. Sequeda, Christoph Lange, and Jeff Heflin (Eds.). Lecture Notes in Computer Science, vol. 10588. Springer, 184–192.
- Silvio Peroni. 2016. [A simplified agile methodology for ontology development](#). In *OWL: - Experiences and Directions - Reasoner Evaluation - 13th International Workshop, OWLED 2016, and 5th International Workshop, ORE 2016, Bologna, Italy, November 20, 2016, Revised Selected Papers*, Mauro Dragoni, María Poveda-Villalón, and Ernesto Jiménez-Ruiz (Eds.). Lecture Notes in Computer Science, vol. 10161. Springer, 55–69.
- David Peterson, Shudi Gao, Ashok Malhotra, C. M. Sperberg-McQueen, Henry S. Thompson, and Paul V. Biron. 2012. [W3C XML Schema Definition Language \(XSD\) 1.1 Part 2: Datatypes](#). W3C Recommendation. World Wide Web Consortium. April 5, 2012.
- Giulio Petrucci, Chiara Ghidini, and Marco Rospocher. 2016. [Ontology learning in the deep](#). In *Knowledge Engineering and Knowledge Management - 20th International Conference, EKAW 2016, Bologna, Italy, November 19-23, 2016, Proceedings*, Eva Blomqvist, Paolo Ciancarini, Francesco Poggi, and Fabio Vitali (Eds.). Lecture Notes in Computer Science, vol. 10024. Springer, 480–495.

- Minh-Duc Pham, Linnea Passing, Orri Erling, and Peter A. Boncz. 2015. [Deriving an Emergent Relational Schema from RDF Data](#). In *Proceedings of the 24th International Conference on World Wide Web, WWW 2015, Florence, Italy, May 18-22, 2015*, Aldo Gangemi, Stefano Leonardi, and Alessandro Panconesi (Eds.). ACM Press, 864–874.
- H. Sofia Pinto, C. Tempich, and Steffen Staab. 2009. [Ontology Engineering and Evolution in a Distributed World Using DILIGENT](#). In *Handbook on Ontologies*, Steffen Staab and Rudi Studer (Eds.). International Handbooks on Information Systems. Springer, 153–176.
- Alessandro Piscopo and Elena Simperl. 2018. [Who Models the World?: Collaborative Ontology Creation and User Roles in Wikidata](#). *Proceedings of the ACM on Human-Computer Interaction 2(CSCW)*, 141:1–141:18.
- Alessandro Piscopo, Lucie-Aimée Kaffee, Chris Phethean, and Elena Simperl. 2017. [Provenance Information in a Collaborative Knowledge Graph: An Evaluation of Wikidata External References](#). In *The Semantic Web - ISWC 2017 - 16th International Semantic Web Conference, Vienna, Austria, October 21-25, 2017, Proceedings, Part I*, Claudia d'Amato, Miriam Fernández, Valentina A. M. Tamma, Freddy Lécué, Philippe Cudré-Mauroux, Juan F. Sequeda, Christoph Lange, and Jeff Heflin (Eds.). Lecture Notes in Computer Science, vol. 10587. Springer, 542–558.
- R. J. Pittman, Amit Srivastava, Sanjika Hewavitharana, Ajinkya Kale, and Saab Mansour. 2017. [Cracking the Code on Conversational Commerce](#). eBay Blog. April 6, 2017.
- Aleksander Pivk, Philipp Cimiano, York Sure, Matjaz Gams, Vladislav Rajkovic, and Rudi Studer. 2007. [Transforming arbitrary tables into logical form with TARTAR](#). *Data and Knowledge Engineering* 60(3), 567–595.
- Roel Poppinga. 2003. [Knowledge Graphs and Network Text Analysis](#). *Social Science Information* 42(91), 91–106.
- Valentina Presutti, Enrico Daga, Aldo Gangemi, and Eva Blomqvist. 2009. [eXtreme Design with Content Ontology Design Patterns](#). In *Proceedings of the Workshop on Ontology Patterns (WOP 2009), collocated with the 8th International Semantic Web Conference (ISWC-2009), Washington D.C., USA, 25 October, 2009*, Eva Blomqvist, Kurt Sandkuhl, François Scharffe, and Vojtech Svátek (Eds.). CEUR Workshop Proceedings, vol. 516. Sun SITE Central Europe (CEUR). 15 pages.
- Eric Prud'hommeaux and Gavin Carothers. 2014. [RDF 1.1 Turtle – Terse RDF Triple Language, W3C Recommendation 25 February 2014](#). W3C Recommendation. World Wide Web Consortium. February 25, 2014.
- Eric Prud'hommeaux, Jose Emilio Labra Gayo, and Harold Solbrig. 2014. [Shape Expressions: An RDF Validation and Transformation Language](#). In *Proceedings of the 10th International Conference on Semantic Systems, SEMANTICS 2014, Leipzig, Germany, September 4-5, 2014*, Harald Sack, Agata Filipowska, Jens Lehmann, and Sebastian Hellmann (Eds.). ACM Press, 32–40.
- Jay Pujara, Hui Miao, Lise Getoor, and William W. Cohen. 2013. [Knowledge Graph Identification](#). In *The Semantic Web - ISWC 2013 - 12th International Semantic Web Conference, Sydney, NSW, Australia, October 21-25, 2013, Proceedings, Part I*, Harith Alani, Lalana Kagal, Achille Fokoue, Paul Groth, Chris Biemann, Josian Xavier Parreira, Lora Aroyo, Natasha Noy, Chris Welty, and Krzysztof Janowicz (Eds.). Lecture Notes in Computer Science, vol. 8218. Springer, 542–557.
- Guilin Qi, Huajun Chen, Kang Liu, Haofen Wang, Qiu Ji, and Tianxing Wu. 2021. [Knowledge Graph](#). Database Management & Information Retrieval. Springer.
- Ross Quillian. 1963. [A notation for representing conceptual information: An application to semantics and mechanical English paraphrasing](#). Systems Development Corp..
- Stephan Rabanser, Oleksandr Shchur, and Stephan Günnemann. 2017. [Introduction to Tensor Decompositions and their Applications in Machine Learning](#). CoRR abs/1711.10781. 13 pages.
- Roy Rada. 1986. [Gradualness eases refinement of medical knowledge](#). *Medical Informatics* 11(1), 59–73.
- Filip Radulovic, Raúl García-Castro, and Asunción Gómez-Pérez. 2015. [Towards the Anonymisation of RDF Data](#). In *The 27th International Conference on Software Engineering and Knowledge Engineering, SEKE 2015, Wyndham Pittsburgh University Center, Pittsburgh, PA, USA, July 6-8, 2015*, Haiping Xu (Ed.). KSI Research Inc. and Knowledge Systems Institute Graduate School, 646–651.
- Yves Raimond, Christopher Sutton, and Mark B. Sandler. 2009. [Interlinking Music-Related Data on the Web](#). *IEEE MultiMedia* 16(2), 52–63.
- Yves Raimond, Tristan Ferne, Michael Smethurst, and Gareth Adams. 2014. [The BBC World Service Archive prototype](#). *Journal of Web Semantics* 27–28, 2–9.

- Alain T. Rappaport and Albert M. Gouyet. 1988. [Dynamic, interactive display system for a knowledge base](#). US Patent US4752889A. June 21, 1988.
- Lev-Arie Ratinov and Dan Roth. 2009. [Design Challenges and Misconceptions in Named Entity Recognition](#). In *Proceedings of the Thirteenth Conference on Computational Natural Language Learning, CoNLL 2009, Boulder, Colorado, USA, June 4-5, 2009*, Suzanne Stevenson and Xavier Carreras (Eds.). The Association for Computational Linguistics, 147–155.
- Pavan Reddivari, Tim Finin, and Anupam Joshi. 2005. [Policy-based access control for an RDF store](#). In *Policy Management for the Web, A workshop held at the 14th International World Wide Web Conference Tuesday 10 May 2005, Chiba Japan*, Lalana Kagal, Tim Finin, and James Hendler (Eds.), 78–81.
- Raymond Reiter. 1987. [A Theory of Diagnosis from First Principles](#). *Artificial Intelligence* 32(1), 57–95.
- Xiang Ren, Ahmed El-Kishky, Chi Wang, Fangbo Tao, Clare R. Voss, and Jiawei Han. 2015. [ClusType: Effective Entity Recognition and Typing by Relation Phrase-Based Clustering](#). In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Sydney, NSW, Australia, August 10-13, 2015*, Longbing Cao, Chengqi Zhang, Thorsten Joachims, Geoffrey I. Webb, Dragos D. Margineantu, and Graham Williams (Eds.). ACM Press, 995–1004.
- Xiang Ren, Zeqiu Wu, Wenqi He, Meng Qu, Clare R. Voss, Heng Ji, Tarek F. Abdelzaher, and Jiawei Han. 2017. [CoType: Joint Extraction of Typed Entities and Relations with Knowledge Bases](#). In *Proceedings of the 26th International Conference on World Wide Web, WWW 2017, Perth, Australia, April 3-7, 2017*, Rick Barrett, Rick Cummings, Eugene Agichtein, and Evgeniy Gabrilovich (Eds.). ACM Press, 1015–1024.
- Juan L. Reutter, Adrián Soto, and Domagoj Vrgoc. 2015. [Recursion in SPARQL](#). In *The Semantic Web - ISWC 2015 - 14th International Semantic Web Conference, Bethlehem, PA, USA, October 11-15, 2015, Proceedings, Part I*, Marcelo Arenas, Óscar Corcho, Elena Paslaru Bontas Simperl, Markus Strohmaier, Mathieu d'Aquin, Kavitha Srinivas, Paul T. Groth, Michel Dumontier, Jeff Heflin, Krishnaprasad Thirunarayan, and Stephen Staab (Eds.). Lecture Notes in Computer Science, vol. 9366. Springer, 19–35.
- Sebastian Riedel, Limin Yao, and Andrew McCallum. 2010. [Modeling Relations and Their Mentions without Labeled Text](#). In *Machine Learning and Knowledge Discovery in Databases, European Conference, ECML PKDD 2010, Barcelona, Spain, September 20-24, 2010, Proceedings, Part III*, José L. Balcázar, Francesco Bonchi, Aristides Gionis, and Michèle Sebag (Eds.). Lecture Notes in Computer Science, vol. 6323. Springer, 148–163.
- Petar Ristoski and Heiko Paulheim. 2016. [RDF2Vec: RDF Graph Embeddings for Data Mining](#). In *The Semantic Web - ISWC 2016 - 15th International Semantic Web Conference, Kobe, Japan, October 17-21, 2016, Proceedings, Part I*, Paul T. Groth, Elena Simperl, Alasdair J. G. Gray, Marta Sabou, Markus Krötzsch, Freddy Lécué, Fabian Flöck, and Yolanda Gil (Eds.). Lecture Notes in Computer Science, vol. 9981. Springer, 498–514.
- Richard H. Richens. 1958. [Interlingual Machine Translation](#). *The Computer Journal* 1(3), 144–147.
- Giuseppe Rizzo, Claudia d'Amato, Nicola Fanizzi, and Floriana Esposito. 2017. [Terminological Cluster Trees for Disjointness Axiom Discovery](#). In *The Semantic Web - 14th International Conference, ESWC 2017, Portorož, Slovenia, May 28 - June 1, 2017, Proceedings, Part I*, Eva Blomqvist, Diana Maynard, Aldo Gangemi, Rinke Hoekstra, Pascal Hitzler, and Olaf Hartig (Eds.). Lecture Notes in Computer Science, vol. 10249. Springer, 184–201.
- Giuseppe Rizzo, Nicola Fanizzi, and Claudia d'Amato. 2020. [Class expression induction as concept space exploration: From DL-Foil to DL-Focl](#). *Future Generation Computer Systems* 108, 256–272.
- Giuseppe Rizzo, Claudia d'Amato, and Nicola Fanizzi. 2021. [An unsupervised approach to disjointness learning based on terminological cluster trees](#). *Semantic Web Journal* 12(3), 423–447.
- Tim Rocktäschel and Sebastian Riedel. 2017. [End-to-end Differentiable Proving](#). In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, 4-9 December 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.), 3788–3800.
- Marko A. Rodriguez. 2015. [The Gremlin graph traversal machine and language](#). In *Proceedings of the 15th Symposium on Database Programming Languages, Pittsburgh, PA, USA, October 25-30, 2015*, James Cheney and Thomas Neumann (Eds.). ACM Press, 1–10.
- Stephen Roller, Douwe Kiela, and Maximilian Nickel. 2018. [Hearst Patterns Revisited: Automatic Hypernym Detection from Large Text Corpora](#). In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, ACL 2018, Melbourne, Australia, July 15-20, 2018, Volume*

- Short Papers*, Iryna Gurevych and Yusuke Miyao (Eds.). The Association for Computational Linguistics, 358–363.
- Marco Rospocher, Marieke van Erp, Piek Vossen, Antske Fokkens, Itziar Aldabe, German Rigau, Aitor Soroa, Thomas Ploeger, and Tessel Bogaard. 2016. [Building event-centric knowledge graphs from news](#). *Journal of Web Semantics* 37–38, 132–151.
- Jacobo Rouces, Gerard de Melo, and Katja Hose. 2015. [Framebase: Representing \$n\$ -ary relations using semantic frames](#). In *The Semantic Web. Latest Advances and New Domains - 12th European Semantic Web Conference, ESWC 2015, Portoroz, Slovenia, May 31 - June 4, 2015. Proceedings*, Fabien Gandon, Marta Sabou, Harald Sack, Claudia d'Amato, Philippe Cudré-Mauroux, and Antoine Zimmermann (Eds.). Lecture Notes in Computer Science, vol. 9088. Springer, 505–521.
- Sebastian Rudolph, Markus Krötzsch, and Pascal Hitzler. 2008. [Description Logic Reasoning with Decision Diagrams: Compiling SHIQ to Disjunctive Datalog](#). In *The Semantic Web - ISWC 2008, 7th International Semantic Web Conference, ISWC 2008, Karlsruhe, Germany, October 26-30, 2008. Proceedings*, Amit P. Sheth, Steffen Staab, Mike Dean, Massimo Paolucci, Diana Maynard, Timothy W. Finin, and Krishnaprasad Thirunarayan (Eds.). Lecture Notes in Computer Science, vol. 5318. Springer, 435–450.
- Anisa Rula, Matteo Palmonari, Andreas Harth, Steffen Stadtmüller, and Andrea Maurino. 2012. [On the Diversity and Availability of Temporal Information in Linked Open Data](#). In *The Semantic Web - ISWC 2012 - 11th International Semantic Web Conference, Boston, MA, USA, November 11-15, 2012, Proceedings, Part I*, Philippe Cudré-Mauroux, Jeff Heflin, Evren Sirin, Tania Tudorache, Jérôme Euzenat, Manfred Hauswirth, Josiane Xavier Parreira, Jim Hendler, Guus Schreiber, Abraham Bernstein, and Eva Blomqvist (Eds.). Lecture Notes in Computer Science, vol. 7649. Springer, 492–507.
- Anisa Rula, Luca Panziera, Matteo Palmonari, and Andrea Maurino. 2014. [Capturing the Currency of DBpedia Descriptions and Get Insight into their Validity](#). In *Proceedings of the 5th International Workshop on Consuming Linked Data, COLD 2014 co-located with the 13th International Semantic Web Conference (ISWC 2014), Riva del Garda, Italy, October 20, 2014*, Olaf Hartig, Aidan Hogan, and Juan F. Sequeda (Eds.). CEUR Workshop Proceedings, vol. 1264. Sun SITE Central Europe (CEUR). 12 pages.
- Anisa Rula, Matteo Palmonari, Simone Rubinacci, Axel-Cyrille Ngonga Ngomo, Jens Lehmann, Andrea Maurino, and Diego Esteves. 2019. [TISCO: Temporal scoping of facts](#). *Journal of Web Semantics* 54, 72–86.
- Owen Sacco and Alexandre Passant. 2011. [A Privacy Preference Ontology \(PPO\) for Linked Data](#). In *WWW2011 Workshop on Linked Data on the Web, Hyderabad, India, March 29, 2011*, Christian Bizer, Tom Heath, Tim Berners-Lee, and Michael Hausenblas (Eds.). CEUR Workshop Proceedings, vol. 813. Sun SITE Central Europe (CEUR). 5 pages.
- Ali Sadeghian, Mohammadreza Armandpour, Patrick Ding, and Patrick Wang. 2019. [DRUM: End-To-End Differentiable Rule Mining On Knowledge Graphs](#). In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, 8–14 December 2019, Vancouver, BC, Canada*, Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett (Eds.), 15321–15331.
- Tomás Sáez and Aidan Hogan. 2018. [Automatically Generating Wikipedia Info-boxes from Wikidata](#). In *Companion of the The Web Conference 2018 on The Web Conference 2018, WWW 2018, Lyon, France, April 23-27, 2018*, Pierre-Antoine Champin, Fabien L. Gandon, Mounia Lalmas, and Panagiotis G. Ipeirotis (Eds.). ACM Press, 1823–1830.
- Muhammad Saleem, Muhammad Intizar Ali, Aidan Hogan, Qaiser Mehmood, and Axel-Cyrille Ngonga Ngomo. 2015. [LSQ: The Linked SPARQL Queries Dataset](#). In *The Semantic Web - ISWC 2015 - 14th International Semantic Web Conference, Bethlehem, PA, USA, October 11-15, 2015, Proceedings, Part II*, Marcelo Arenas, Óscar Corcho, Elena Paslaru Bontas Simperl, Markus Strohmaier, Mathieu d'Aquin, Kavitha Srinivas, Paul T. Groth, Michel Dumontier, Jeff Heflin, Krishnaprasad Thirunarayan, and Stephen Staab (Eds.). Lecture Notes in Computer Science, vol. 9367. Springer, 261–269.
- Mehdi Samadi, Partha Talukdar, Manuela Veloso, and Manuel Blum. 2016. [ClaimEval: Integrated and Flexible Framework for Claim Evaluation Using Credibility of Sources](#). In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence, February 12-17, 2016, Phoenix, Arizona, USA*, Dale Schuurmans and Michael P. Wellman (Eds.). AAAI Press, 222–228.

- Pierangela Samarati and Latanya Sweeney. 1998. *Protecting privacy when disclosing information: k-anonymity and its enforcement through generalization and suppression*. Computer Science Laboratory, SRI International.
- Georgios M. Santipantakis, Apostolos Glenis, Christos Doukeridis, Akrivi Vlachou, and George A. Vouros. 2019. *stLD: towards a spatio-temporal link discovery framework*. In *Proceedings of the International Workshop on Semantic Big Data, SBD@SIGMOD 2019, Amsterdam, The Netherlands, July 5, 2019*, Sven Groppe and Le Gruenwald (Eds.). ACM Press, 4:1–4:6.
- Eugene Santos Jr. and Eugene S. Santos. 1999. *A framework for building knowledge-bases under uncertainty*. *Journal of Experimental & Theoretical Artificial Intelligence* 11(2), 265–286.
- Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2009. *The Graph Neural Network Model*. *IEEE Transactions on Neural Networks* 20(1), 61–80.
- Max Schmachtenberg, Christian Bizer, and Heiko Paulheim. 2014. *Adoption of the Linked Data Best Practices in Different Topical Domains*. In *The Semantic Web - ISWC 2014 - 13th International Semantic Web Conference, Riva del Garda, Italy, October 19-23, 2014. Proceedings, Part I*, Peter Mika, Tania Tudorache, Abraham Bernstein, Christopher A. Welty, Craig A. Knoblock, Denny Vrandečić, Paul T. Groth, Natasha Fridman Noy, Krzysztof Janowicz, and Carole A. Goble (Eds.). Lecture Notes in Computer Science, vol. 8796. Springer, 245–260.
- Manfred Schmidt-Schauß and Gert Smolka. 1991. *Attributive Concept Descriptions with Complements*. *Artificial Intelligence* 48(1), 1–26.
- Michael Schneider and Geoff Sutcliffe. 2011. *Reasoning in the OWL 2 Full Ontology Language Using First-Order Automated Theorem Proving*. In *Automated Deduction - CADE-23 - 23rd International Conference on Automated Deduction, Wrocław, Poland, July 31 - August 5, 2011. Proceedings*, Nikolaj Bjørner and Viorica Sofronie-Stokkermans (Eds.). Lecture Notes in Computer Science, vol. 6803. Springer, 461–475.
- E. W. Schneider. 1973. *Course Modularization Applied: The Interface System and Its Implications For Sequence Control and Data Analysis*. Human Resources Research Organization, Alexandria, VA. November, 1973.
- Christoph Schuetz, Loris Bozzato, Bernd Neumayr, Michael Schrefl, and Luciano Serafini. 2021. *Knowledge Graph OLAP: A Multidimensional Model and Query Operations for Contextualized Knowledge Graphs*. *Semantic Web Journal* 12(4).
- Philipp Seifer, Johannes Härtel, Martin Leinberger, Ralf Lämmel, and Steffen Staab. 2019. *Empirical study on the usage of graph query languages in open source Java projects*. In *Proceedings of the 12th ACM SIGPLAN International Conference on Software Language Engineering, SLE 2019, Athens, Greece, October 20-22, 2019*, Oscar Nierstrasz, Jeff Gray, and Bruno C. d. S. Oliveira (Eds.). ACM Press, 152–166.
- Juan F. Sequeda, Marcelo Arenas, and Daniel P. Miranker. 2012. *On directly mapping relational databases to RDF and OWL*. In *Proceedings of the 21st World Wide Web Conference 2012, WWW 2012, Lyon, France, April 16-20, 2012*, Alain Mille, Fabien L. Gandon, Jacques Misselis, Michael Rabinovich, and Steffen Staab (Eds.). ACM Press, 649–658.
- Juan F. Sequeda, Marcelo Arenas, and Daniel P. Miranker. 2014. *OBDA: Query Rewriting or Materialization? In Practice, Both!*. In *The Semantic Web - ISWC 2014 - 13th International Semantic Web Conference, Riva del Garda, Italy, October 19-23, 2014. Proceedings, Part I*, Peter Mika, Tania Tudorache, Abraham Bernstein, Christopher A. Welty, Craig A. Knoblock, Denny Vrandečić, Paul T. Groth, Natasha Fridman Noy, Krzysztof Janowicz, and Carole A. Goble (Eds.). Lecture Notes in Computer Science, vol. 8796. Springer, 535–551.
- Juan F. Sequeda, Willard J. Briggs, Daniel P. Miranker, and Wayne P. Heideman. 2019. *A Pay-as-you-go Methodology to Design and Build Enterprise Knowledge Graphs from Relational Databases*. In *The Semantic Web - ISWC 2019 - 18th International Semantic Web Conference, Auckland, New Zealand, October 26-30, 2019. Proceedings, Part II*, Chiara Ghidini, Olaf Hartig, Maria Maleshkova, Vojtech Svátek, Isabel F. Cruz, Aidan Hogan, Jie Song, Maxime Lefrançois, and Fabien Gandon (Eds.). Lecture Notes in Computer Science, vol. 11779. Springer, 526–545.
- Luciano Serafini and Martin Homola. 2012. *Contextualized Knowledge Repositories for the Semantic Web*. *Journal of Web Semantics* 12, 64–87.
- Stephan Seufert, Patrick Ernst, Srikanth J. Bedathur, Sarath Kumar Kondreddi, Klaus Berberich, and Gerhard Weikum. 2016. *Instant Espresso: Interactive Analysis of Relationships in Knowledge Graphs*. In *Proceedings of the 25th International Conference on World Wide Web, WWW 2016, Montreal, Canada*

- April 11-15, 2016, *Companion Volume*, Jacqueline Bourdeau, Jim Hendler, Roger Nkambou, Ian Horrocks, and Ben Y. Zhao (Eds.). ACM Press, 251–254.
- Nigel Shadbolt and Kieron O'Hara. 2013. [Linked Data in Government](#). *IEEE Inteternet Computing* 17(4), 72–77.
- Mohamed Ahmed Sherif and Axel-Cyrille Ngonga Ngomo. 2015. [Semantic Quran](#). *Semantic Web Journal* 6(4), 339–345.
- Mohamed Ahmed Sherif and Axel-Cyrille Ngonga Ngomo. 2018. [A systematic survey of point set distance measures for link discovery](#). *Semantic Web Journal* 9(5), 589–604.
- Baoxu Shi and Tim Weninger. 2016. [Discriminative predicate path mining for fact checking in knowledge graphs](#). *Knowledge-based Systems* 104, 123–133.
- Cogan Shimizu, Quinn Hirt, and Pascal Hitzler. 2019. [MODL: A Modular Ontology Design Library](#). CoRR abs/1904.05405. 12 pages.
- Solomon Eyal Shimony, Carmel Domshlak, and Eugene Santos Jr.. 1997. [Cost-Sharing in Bayesian Knowledge Bases](#). In *UAI '97: Proceedings of the Thirteenth Conference on Uncertainty in Artificial Intelligence, Brown University, Providence, Rhode Island, USA, August 1-3, 1997*, Dan Geiger and Prakash P. Shenoy (Eds.). Morgan Kaufmann, 421–428.
- Prashant Shiralkar, Alessandro Flammini, Filippo Menczer, and Giovanni Luca Ciampaglia. 2017. [Finding streams in knowledge graphs to support fact checking](#). In *2017 IEEE International Conference on Data Mining, ICDM 2017, New Orleans, LA, USA, November 18-21, 2017*, Vijay Raghavan, Srinivas Aluru, George Karypis, Lucio Miele, and Xindong Wu (Eds.). IEEE Computer Society, 859–864.
- Saurabh Shrivastava. 2017. [Bring rich knowledge of people, places, things and local businesses to your apps](#). Bing Blogs. July 12, 2017.
- Rôney Reis C. Silva, Bruno C. Leal, Felipe T. Brito, Vânia M. P. Vidal, and Javam C. Machado. 2017. [A Differentially Private Approach for Querying RDF Data of Social Networks](#). In *Proceedings of the 21st International Database Engineering & Applications Symposium, IDEAS 2017, Bristol, United Kingdom, July 12-14, 2017*, Bipin C. Desai, Jun Hong, and Richard McClatchey (Eds.). ACM Press, 74–81.
- Amit Singhal. 2012. [Introducing the Knowledge Graph: things, not strings](#). Google Blog. May 16, 2012.
- Martin G. Skjæveland, Daniel P. Lupp, Leif Harald Karlsen, and Henrik Forssell. 2018. [Practical Ontology Pattern Instantiation, Discovery, and Maintenance with Reasonable Ontology Templates](#). In *The Semantic Web - ISWC 2018 - 17th International Semantic Web Conference, Monterey, CA, USA, October 8-12, 2018, Proceedings, Part I*, Denny Vrandečić, Kalina Bontcheva, Mari Carmen Suárez-Figueroa, Valentina Presutti, Irene Celino, Marta Sabou, Lucie-Aimée Kaffee, and Elena Simperl (Eds.). Lecture Notes in Computer Science, vol. 11136. Springer, 477–494.
- Jennifer Sleeman and Tim Finin. 2013. [Type Prediction for Efficient Coreference Resolution in Heterogeneous Semantic Graphs](#). In *2013 IEEE Seventh International Conference on Semantic Computing, Irvine, CA, USA, September 16-18, 2013*. IEEE Computer Society, 78–85.
- Alisa Smirnova and Philippe Cudré-Mauroux. 2019. [Relation Extraction Using Distant Supervision: A Survey](#). *ACM Computing Surveys* 51(5), 106:1–106:35.
- Richard Socher, Danqi Chen, Christopher D Manning, and Andrew Ng. 2013. [Reasoning with neural tensor networks for knowledge base completion](#). In *Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013. Proceedings of a meeting held December 5-8, 2013, Lake Tahoe, Nevada, United States*, Christopher J. C. Burges, Léon Bottou, Zoubin Ghahramani, and Kilian Q. Weinberger (Eds.), 926–934.
- Arnaud Soulet, Arnaud Giacometti, Béatrice Markhoff, and Fabian M. Suchanek. 2018. [Representativeness of Knowledge Bases with the Generalized Benford's Law](#). In *The Semantic Web - ISWC 2018 - 17th International Semantic Web Conference, Monterey, CA, USA, October 8-12, 2018, Proceedings, Part I*, Denny Vrandečić, Kalina Bontcheva, Mari Carmen Suárez-Figueroa, Valentina Presutti, Irene Celino, Marta Sabou, Lucie-Aimée Kaffee, and Elena Simperl (Eds.). Lecture Notes in Computer Science, vol. 11136. Springer, 374–390.
- John Sowa. 1979. [Semantics of Conceptual Graphs](#). In *17th Annual Meeting of the Association for Computational Linguistics, 29 June - 1 July 1979, University of California at San Diego, La Jolla, CA, USA*, Norman K. Sondheim (Ed.). The Association for Computational Linguistics, 39–44.
- John Sowa. 1987. Semantic Networks. In *Encyclopedia of Cognitive Science*, Stuart C. Shapiro (Ed.). John Wiley & Sons.
- Blerina Spahiu, Riccardo Porrini, Matteo Palmonari, Anisa Rula, and Andrea Maurino. 2016. [ABSTAT: Ontology-Driven Linked Data Summaries with Pattern Minimalization](#). In *The Semantic Web - ES*

- 2016 Satellite Events, Heraklion, Crete, Greece, May 29 - June 2, 2016, *Revised Selected Papers*, Harald Sack, Giuseppe Rizzo, Nadine Steinmetz, Dunja Mladenec, Sören Auer, and Christoph Lange (Eds.). Lecture Notes in Computer Science, vol. 9989. Springer, 381–395.
- Alessandro Sperduti and Antonina Starita. 1997. [Supervised neural networks for the classification of structures](#). *IEEE Transactions on Neural Networks* 8(3), 714–735.
- Manu Sporny, Gregg Kellogg, Markus Lanthaler, Dave Longley, and Niklas Lindström. 2014. *JSON-LD 1.0, A JSON-based Serialization for Linked Data*, W3C Recommendation 16 January 2014. W3C Recommendation. World Wide Web Consortium. January 16, 2014.
- Rajan Srikanth and Matthias Jarke. 1989. [The Design of Knowledge-Based Systems for Managing Ill-Structured Software Projects](#). *Decision Support Systems* 5(4), 425–447.
- Claus Stadler, Jens Lehmann, Konrad Höffner, and Sören Auer. 2012. [LinkedGeoData: A core for a web of spatial open data](#). *Semantic Web Journal* 3(4), 333–354.
- Simon Steyskal and Axel Polleres. 2014. [Defining expressive access policies for linked data using the ODRL ontology 2.0](#). In *Proceedings of the 10th International Conference on Semantic Systems, SEMANTICS 2014, Leipzig, Germany, September 4-5, 2014*, Harald Sack, Agata Filipowska, Jens Lehmann, and Sebastian Hellmann (Eds.). ACM Press, 20–23.
- Patrick Stickler. 2005. *CBD – Concise Bounded Description*, W3C Member Submission 3 June 2005. W3C Member Submission. June 3, 2005.
- Radu Stoica, George H. L. Fletcher, and Juan F. Sequeda. 2019. [On Directly Mapping Relational Databases to Property Graphs](#). In *Proceedings of the 13th Alberto Mendelzon International Workshop on Foundations of Data Management, Asunción, Paraguay, June 3-7, 2019*, Aidan Hogan and Tova Milo (Eds.). CEUR Workshop Proceedings, vol. 2369. Sun SITE Central Europe (CEUR). 4 pages.
- Frans N. Stokman and Pieter H. de Vries. 1988. [Structuring knowledge in a graph](#). In *Human-Computer Interaction*, Gerrit C. van der Veer and Gijsbertus Mulder (Eds.). Springer, 186–206.
- Umberto Straccia. 2009. [A Minimal Deductive System for General Fuzzy RDF](#). In *Web Reasoning and Rule Systems, Third International Conference, RR 2009, Chantilly, VA, USA, October 25-26, 2009, Proceedings*, Axel Polleres and Terrance Swift (Eds.). Lecture Notes in Computer Science, vol. 5837. Springer, 166–181.
- Philip Stutz, Daniel Strebel, and Abraham Bernstein. 2016. [Signal/Collect12](#). *Semantic Web Journal* 7(2), 139–166.
- Fabian M. Suchanek, Gjergji Kasneci, and Gerhard Weikum. 2007. [YAGO: A core of semantic knowledge](#). In *Proceedings of the 16th International Conference on World Wide Web, WWW 2007, Banff, Alberta, Canada, May 8-12, 2007*, Carey L. Williamson, Mary Ellen Zurko, Peter F. Patel-Schneider, and Prashant J. Shenoy (Eds.). ACM Press, 697–706.
- Fabian M. Suchanek, Gjergji Kasneci, and Gerhard Weikum. 2008. [YAGO: A Large Ontology from Wikipedia and WordNet](#). *Journal of Web Semantics* 6(3), 203–217.
- Fabian M. Suchanek, Jonathan Lajus, Armand Bosch, and Gerhard Weikum. 2019. [Knowledge Representation and Rule Mining in Entity-Centric Knowledge Bases](#). In *Reasoning Web. Explainable Artificial Intelligence - 15th International Summer School 2019, Bolzano, Italy, September 20-24, 2019, Tutorial Lectures*, Markus Krötzsch and Daria Stepanova (Eds.). Lecture Notes in Computer Science, vol. 11810. Springer, 110–152.
- Yizhou Sun and Jiawei Han. 2012. [Mining Heterogeneous Information Networks: Principles and Methodologies](#). Synthesis Lectures on Data Mining and Knowledge Discovery, vol. 3. Morgan & Claypool.
- Yizhou Sun, Jiawei Han, Xifeng Yan, Philip S. Yu, and Tianyi Wu. 2011. [Pathsim: Meta path-based top-k similarity search in heterogeneous information networks](#). *Proceedings of the VLDB Endowment* 4(11), 992–1003.
- Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. 2019. [RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space](#). In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net.
- Mihai Surdeanu, Julie Tibshirani, Ramesh Nallapati, and Christopher D. Manning. 2012. [Multi-instance Multi-label Learning for Relation Extraction](#). In *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning, EMNLP-CoNLL 2012, July 12-14, 2012, Jeju Island, Korea*, Jun'ichi Tsujii, James Henderson, and Marius Pasca (Eds.). The Association for Computational Linguistics, 455–465.

- Zafar Habeeb Syed, Michael Röder, and Axel-Cyrille Ngonga Ngomo. 2018. [FactCheck: Validating RDF Triples Using Textual Evidence](#). In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management, CIKM 2018, Torino, Italy, October 22-26, 2018*, Alfredo Cuzzocrea, James Allan, Norman W. Paton, Divesh Srivastava, Rakesh Agrawal, Andrei Z. Broder, Mohammed J. Zaki, K. Selçuk Candan, Alexandros Labrinidis, Assaf Schuster, and Haixun Wang (Eds.). ACM Press, 1599–1602.
- Zafar Habeeb Syed, Michael Röder, and Axel-Cyrille Ngonga Ngomo. 2019. [Unsupervised Discovery of Corroborative Paths for Fact Validation](#). In *The Semantic Web - ISWC 2019 - 18th International Semantic Web Conference, Auckland, New Zealand, October 26-30, 2019, Proceedings, Part I*, Chiara Ghidini, Olaf Hartig, Maria Maleshkova, Vojtech Svátek, Isabel F. Cruz, Aidan Hogan, Jie Song, Maxime Lefrançois, and Fabien Gandon (Eds.). Lecture Notes in Computer Science, vol. 11778. Springer, 630–646.
- James Joseph Sylvester. 1878. Chemistry and Algebra. *Nature* 17, p284.
- Jeremy Tandy, Ivan Herman, and Gregg Kellogg. 2015. [Generating RDF from Tabular Data on the Web, W3C Recommendation 17 December 2015](#). W3C Recommendation. World Wide Web Consortium. December 17, 2015.
- Jeni Tennison and Gregg Kellogg. 2015. [Metadata Vocabulary for Tabular Data, W3C Recommendation 17 December 2015](#). W3C Recommendation. World Wide Web Consortium. December 17, 2015.
- The R Foundation. 1992. [The R Project for Statistical Computing](#). Online at <https://www.r-project.org/>.
- The UniProt Consortium. 2014. [UniProt: a hub for protein information](#). *Nucleic Acids Research* 43(D1), D204–D212.
- Bryan B. Thompson, Mike Personick, and Martyn Cutcher. 2014. [The Bigdata® RDF Graph Database](#). In *Linked Data Management*, Andreas Harth, Katja Hose, and Ralf Schenkel (Eds.). CRC Press, 193–237.
- Katherine Thornton, Harold Solbrig, Gregory S. Stupp, José Emilio Labra Gayo, Daniel Mietchen, Eric Prud'hommeaux, and Andra Waagmeester. 2019. [Using Shape Expressions \(ShEx\) to Share RDF Data Models and to Guide Curation with Rigorous Validation](#). In *The Semantic Web - 16th International Conference, ESWC 2019, Portorož, Slovenia, June 2-6, 2019, Proceedings*, Pascal Hitzler, Miriam Fernández, Krzysztof Janowicz, Amrapali Zaveri, Alasdair J. G. Gray, Vanessa López, Armin Haller, and Karl Hammar (Eds.). Lecture Notes in Computer Science, vol. 11503. Springer, 606–620.
- Felice Tobin. 2017. [Thomson Reuters Launches first of its kind Knowledge Graph Feed allowing Financial Services customers to accelerate their AI and Digital Strategies](#). Thomson Reuters Press Release. October 23, 2017.
- Dominik Tomaszuk, Renzo Angles, Lukasz Szeremeta, Karol Litman, and Diego Cisterna. 2019. [Serialization for Property Graphs](#). In *Beyond Databases, Architectures and Structures. Paving the Road to Smart Data Processing and Analysis - 15th International Conference, BDAS 2019, Ustroń, Poland, May 28-31, 2019, Proceedings*, Stanisław Kozielski, Dariusz Mrozek, Paweł Kasprowski, Bożena Malysiak-Mrozek, and Daniel Kostrzewa (Eds.). Communications in Computer and Information Science, vol. 1018. Springer, 57–69.
- Gerald Töpper, Magnus Knuth, and Harald Sack. 2012. [DBpedia ontology enrichment for inconsistency detection](#). In *I-SEMANTICS 2012 - 8th International Conference on Semantic Systems, I-SEMANTICS '12, Graz, Austria, September 5-7, 2012*, Valentina Presutti and Helena Sofia Pinto (Eds.). ACM Press, 33–40.
- Jeffrey Travers and Stanley Milgram. 1969. [An Experimental Study of the Small World Problem](#). *Sociometry* 32(4), 425–443.
- Théo Trouillon, Johannes Welbl, Sebastian Riedel, Éric Gaussier, and Guillaume Bouchard. 2016. [Complex Embeddings for Simple Link Prediction](#). In *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, Maria-Florina Balcan and Kilian Q. Weinberger (Eds.). JMLR Workshop and Conference Proceedings, vol. 48. JMLR.org, 2071–2080.
- Ledyard R. Tucker. 1964. The extension of factor analysis to three-dimensional matrices. In *Contributions to Mathematical Psychology*, H. Gulliksen and N. Frederiksen (Eds.). Holt, Rinehart and Winston, 110–127.
- Giovanni Tummarello, Christian Morbidoni, Reto Bachmann-Gmür, and Orri Erling. 2007. [RDFSyc: Efficient Remote Synchronization of RDF Models](#). In *The Semantic Web, 6th International Semantic Web Conference, 2nd Asian Semantic Web Conference, ISWC 2007 + ASWC 2007, Busan, Korea, November 11-15, 2007*, Karl Aberer, Key-Sun Choi, Natasha Fridman Noy, Dean Allemang, Kyung

- Lee, Lyndon J. B. Nixon, Jennifer Golbeck, Peter Mika, Diana Maynard, Riichiro Mizoguchi, Guus Schreiber, and Philippe Cudré-Mauroux (Eds.). *Lecture Notes in Computer Science*, vol. 4825. Springer, 537–551.
- Octavian Udrea, Diego Reforgiato Recupero, and V. S. Subrahmanian. 2010. [Annotated RDF](#). *ACM Transactions on Computational Logics* 11(2), 10:1–10:41.
- Jacopo Urbani, Spyros Kotoulas, Jason Maassen, Frank van Harmelen, and Henri E. Bal. 2012. [WebPIE: A Web-scale Parallel Inference Engine using MapReduce](#). *Journal of Web Semantics* 10, 59–75.
- Hernán Vargas, Carlos Buil Aranda, Aidan Hogan, and Claudia López. 2019. [RDF Explorer: A Visual SPARQL Query Builder](#). In *The Semantic Web - ISWC 2019 - 18th International Semantic Web Conference, Auckland, New Zealand, October 26-30, 2019, Proceedings, Part I*, Chiara Ghidini, Olaf Hartig, Maria Maleshkova, Vojtech Svátek, Isabel F. Cruz, Aidan Hogan, Jie Song, Maxime Lefrançois, and Fabien Gandon (Eds.). *Lecture Notes in Computer Science*, vol. 11778. Springer, 647–663.
- Shikhar Vashishth, Prince Jain, and Partha Talukdar. 2018. [CESI: Canonicalizing Open Knowledge Bases using Embeddings and Side Information](#). In *Proceedings of the 2018 World Wide Web Conference on World Wide Web, WWW 2018, Lyon, France, April 23-27, 2018*, Pierre-Antoine Champin, Fabien L. Gandon, Mounia Lalmas, and Panagiotis G. Ipeirotis (Eds.). ACM Press, 1317–1327.
- Paola Velardi, Stefano Faralli, and Roberto Navigli. 2013. [OntoLearn Reloaded: A Graph-Based Algorithm for Taxonomy Induction](#). *Computational Linguistics* 39(3), 665–707.
- Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. [Graph Attention Networks](#). In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net. 12 pages.
- Ruben Verborgh, Miel Vander Sande, Pieter Colpaert, Sam Coppens, Erik Mannens, and Rik Van de Walle. 2014. [Web-Scale Querying through Linked Data Fragments](#). In *Proceedings of the Workshop on Linked Data on the Web, co-located with the 23rd International World Wide Web Conference (WWW 2014), Seoul, Korea, April 8, 2014*, Christian Bizer, Tom Heath, Sören Auer, and Tim Berners-Lee (Eds.). CEUR Workshop Proceedings, vol. 1184. Sun SITE Central Europe (CEUR). 10 pages.
- Ruben Verborgh, Miel Vander Sande, Olaf Hartig, Joachim Van Herwegen, Laurens De Vocht, Ben De Meester, Gerald Haesendonck, and Pieter Colpaert. 2016. [Triple Pattern Fragments: A low-cost knowledge graph interface for the Web](#). *Journal of Web Semantics* 37–38, 184–206.
- Serena Villata and Fabien Gandon. 2012. [Licenses Compatibility and Composition in the Web of Data](#). In *Proceedings of the Third International Workshop on Consuming Linked Data, COLD 2012, Boston, MA, USA, November 12, 2012*, Juan F. Sequeda, Andreas Harth, and Olaf Hartig (Eds.). CEUR Workshop Proceedings, vol. 905. Sun SITE Central Europe (CEUR). 12 pages.
- Serena Villata, Nicolas Delaforge, Fabien Gandon, and Amelie Gyrard. 2011. [An Access Control Model for Linked Data](#). In *On the Move to Meaningful Internet Systems: OTM 2011 Workshops - Confederated International Workshops and Posters: E12N+NSF ICE, ICSP+INBAST, ISDE, ORM, OTMA, SWWS+MONET+SeDeS, and VADER 2011, Hersonissos, Crete, Greece, October 17-21, 2011. Proceedings*, Robert Meersman, Tharam S. Dillon, and Pilar Herrero (Eds.). *Lecture Notes in Computer Science*, vol. 7046. Springer, 454–463.
- Johanna Völker, Daniel Fleischhacker, and Heiner Stuckenschmidt. 2015. [Automatic Acquisition of Class Disjointness](#). *Journal of Web Semantics* 35(P2), 124–139.
- Julius Volz, Christian Bizer, Martin Gaedke, and Georgi Kobilarov. 2009. [Discovering and Maintaining Links on the Web of Data](#). In *The Semantic Web - ISWC 2009, 8th International Semantic Web Conference, ISWC 2009, Chantilly, VA, USA, October 25-29, 2009. Proceedings*, Abraham Bernstein, David R. Karger, Tom Heath, Lee Feigenbaum, Diana Maynard, Enrico Motta, and Krishnaprasad Thirunarayan (Eds.). *Lecture Notes in Computer Science*, vol. 5823. Springer, 650–665.
- Denny Vrandečić and Markus Krötzsch. 2014. [Wikidata: A Free Collaborative Knowledgebase](#). *Communications of the ACM* 57(10), 78–85.
- Andreas Wagner, Duc Thanh Tran, Günter Ladwig, Andreas Harth, and Rudi Studer. 2012. [Top-\\$k\\$ Linked Data Query Processing](#). In *The Semantic Web: Research and Applications - 9th Extended Semantic Web Conference, ESWC 2012, Heraklion, Crete, Greece, May 27-31, 2012. Proceedings*, Elena Simperl, Philipp Cimiano, Axel Polleres, Óscar Corcho, and Valentina Presutti (Eds.). *Lecture Notes in Computer Science*, vol. 7295. Springer, 56–71.

- Claudia Wagner, Eduardo Graells-Garrido, David García, and Filippo Menczer. 2016. [Women through the glass ceiling: gender asymmetries in Wikipedia](#). *EPJ Data Science* 5(1), p5.
- Zhen Wang, Jianwen Zhang, Jianlin Feng, and Zheng Chen. 2014. [Knowledge Graph Embedding by Translating on Hyperplanes](#). In *Proceedings of the Twenty-Eighth AAAI Conference on Artificial Intelligence, July 27 -31, 2014, Québec City, Québec, Canada*, Carla E. Brodley and Peter Stone (Eds.). AAAI Press, 1112–1119.
- Quan Wang, Bin Wang, and Li Guo. 2015. [Knowledge Base Completion Using Embeddings and Rules](#). In *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2015, Buenos Aires, Argentina, July 25-31, 2015*, Qiang Yang and Michael J. Wooldridge (Eds.). IJCAI/AAAI, 1859–1866.
- Quan Wang, Zhendong Mao, Bin Wang, and Li Guo. 2017. [Knowledge Graph Embedding: A Survey of Approaches and Applications](#). *IEEE Transactions on Knowledge and Data Engineering* 29(12), 2724–2743.
- Meng Wang, Ruijie Wang, Jun Liu, Yihe Chen, Lei Zhang, and Guilin Qi. 2018. [Towards Empty Answers in SPARQL: Approximating Querying with RDF Embedding](#). In *The Semantic Web - ISWC 2018 - 17th International Semantic Web Conference, Monterey, CA, USA, October 8-12, 2018, Proceedings, Part I*, Denny Vrandečić, Kalina Bontcheva, Mari Carmen Suárez-Figueroa, Valentina Presutti, Irene Celino, Marta Sabou, Lucie-Aimée Kaffee, and Elena Simperl (Eds.). Lecture Notes in Computer Science, vol. 11136. Springer, 513–529.
- Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S. Yu. 2019. [Heterogeneous Graph Attention Network](#). In *The World Wide Web Conference, WWW 2019, San Francisco, CA, USA, May 13-17, 2019*, Ling Liu, Ryen W. White, Amin Mantrach, Fabrizio Silvestri, Julian J. McAuley, Ricardo Baeza-Yates, and Leila Zia (Eds.). ACM Press, 2022–2032.
- Gerhard Weikum and Martin Theobald. 2010. [From information to knowledge: harvesting entities and relationships from web sources](#). In *Proceedings of the Twenty-Ninth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems, PODS 2010, June 6-11, 2010, Indianapolis, Indiana, USA*, Jan Paredaens and Dirk Van Gucht (Eds.). ACM Press, 65–76.
- Robert West, Evgeniy Gabrilovich, Kevin Murphy, Shaohua Sun, Rahul Gupta, and Dekang Lin. 2014. [Knowledge Base Completion via Search-Based Question Answering](#). In *23rd International World Wide Web Conference, WWW '14, Seoul, Republic of Korea, April 7-11, 2014*, Chin-Wan Chung, Andrei Z. Broder, Kyuseok Shim, and Torsten Suel (Eds.). ACM Press, 515–526.
- Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, Niklas Blomberg, Jan-Willem Boiten, Luiz Bonino da Silva Santos, Philip E. Bourne, Jildau Bouwman, Anthony J. Brookes, Tim Clark, Mercè Crosas, Ingrid Dillo, Olivier Dumon, Scott Edmunds, Chris T. Evelo, Richard Finkers, Alejandra Gonzalez-Beltran, Alasdair J.G. Gray, Paul Groth, Carole Goble, Jeffrey S. Grethe, Jaap Heringa, Peter A.C 't Hoen, Rob Hooft, Tobias Kuhn, Ruben Kok, Joost Kok, Scott J. Lusher, Maryann E. Martone, Albert Mons, Abel L. Packer, Bengt Persson, Philippe Rocca-Serra, Marco Roos, Rene van Schaik, Susanna-Assunta Sansone, Erik Schultes, Thierry Sengstag, Ted Slater, George Strawn, Morris A. Swertz, Mark Thompson, Johan van der Lei, Erik van Mulligen, Jan Velterop, Andra Waagmeester, Peter Wittenburg, Katherine Wolstencroft, Jun Zhao, and Barend Mons. 2016. [The FAIR Guiding Principles for scientific data management and stewardship](#). *Scientific Data* 3. 9 pages.
- William A. Woods. 1975. [What's in a Link: Foundations for Semantic Networks](#). In *Representation and Understanding*, Daniel G. Bobrow and Allan Collins (Eds.). Studies in Cognitive Science. Elsevier, 35–82.
- Gong-Qing Wu, Ying He, and Xuegang Hu. 2018. [Entity Linking: An Issue to Extract Corresponding Entity With Knowledge Base](#). *IEEE Access* 6, 6220–6231.
- Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. 2019. [A Comprehensive Survey on Graph Neural Networks](#). *CoRR* abs/1901.00596. 22 pages.
- Marcin Wylot, Manfred Hauswirth, Philippe Cudré-Mauroux, and Sherif Sakr. 2018. [RDF Data Storage and Query Processing Schemes: A Survey](#). *ACM Computing Surveys* 51(4), 84:1–84:36.
- Guohui Xiao, Diego Calvanese, Roman Kontchakov, Domenico Lembo, Antonella Poggi, Riccardo Rosati, and Michael Zakharyashev. 2018. [Ontology-Based Data Access: A Survey](#). In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, IJCAI 2018, July 13-19, 2018, Stockholm, Sweden*, Jérôme Lang (Ed.). IJCAI/AAAI, 5511–5519.

- Reynold S. Xin, Joseph E. Gonzalez, Michael J. Franklin, and Ion Stoica. 2013. [GraphX: a resilient distributed graph system on Spark](#). In *First International Workshop on Graph Data Management Experiences and Systems, GRADES 2013, co-located with SIGMOD/PODS 2013, New York, NY, USA, June 24, 2013*, Peter A. Boncz and Thomas Neumann (Eds.). CWI/ACM, 2:1–2:6.
- Reynold S. Xin, Josh Rosen, Matei Zaharia, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2013. [Shark: SQL and rich analytics at scale](#). In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2013, New York, NY, USA, June 22–27, 2013*, Kenneth A. Ross, Divesh Srivastava, and Dimitris Papadias (Eds.). ACM Press, 13–24.
- Wei Xu, Raphael Hoffmann, Le Zhao, and Ralph Grishman. 2013. [Filling Knowledge Base Gaps for Distant Supervision of Relation Extraction](#). In *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics, ACL 2013, 4–9 August 2013, Sofia, Bulgaria, Volume 2: Short Papers*. The Association for Computational Linguistics, 665–670.
- Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. [How Powerful are Graph Neural Networks?](#). In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6–9, 2019*. OpenReview.net. 17 pages.
- Bishan Yang, Wen-tau Yih, Xiaodong He, Jianfeng Gao, and Li Deng. 2015. [Embedding Entities and Relations for Learning and Inference in Knowledge Bases](#). In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7–9, 2015, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). 12 pages.
- Fan Yang, Zhilin Yang, and William W. Cohen. 2017. [Differentiable Learning of Logical Rules for Knowledge Base Reasoning](#). In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, 4–9 December 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.), 2319–2328.
- Luwei Yang, Zhibo Xiao, Wen Jiang, Yi Wei, Yi Hu, and Hao Wang. 2020. [Dynamic Heterogeneous Graph Embedding Using Hierarchical Attentions](#). In *Advances in Information Retrieval - 42nd European Conference on IR Research, ECIR 2020, Lisbon, Portugal, April 14–17, 2020, Proceedings, Part II*, Joemon M. Jose, Emine Yilmaz, João Magalhães, Pablo Castells, Nicola Ferro, Mário J. Silva, and Flávio Martins (Eds.). Lecture Notes in Computer Science, vol. 12036. Springer, 425–432.
- Taha Yasseri, Robert Sumi, András Rung, András Kornai, and János Kertész. 2012. [Dynamics of Conflicts in Wikipedia](#). *PLOS One* 7(6). 12 pages.
- Xiaoxin Yin, Jiawei Han, and Philip S. Yu. 2008. [Truth discovery with multiple conflicting information providers on the web](#). *IEEE Transactions on Knowledge and Data Engineering* 20(6), 796–808.
- Dani Yogatama, Daniel Gillick, and Nevena Lazic. 2015. [Embedding Methods for Fine Grained Entity Type Classification](#). In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing of the Asian Federation of Natural Language Processing, ACL 2015, July 26–31, 2015, Beijing, China, Volume 2: Short Papers*. The Association for Computational Linguistics, 291–296.
- Amrapali Zaveri, Anisa Rula, Andrea Maurino, Ricardo Pietrobon, Jens Lehmann, and Sören Auer. 2016. [Quality assessment for Linked Data: A Survey](#). *Semantic Web Journal* 7(1), 63–93.
- Fuzheng Zhang, Nicholas Jing Yuan, Defu Lian, Xing Xie, and Wei-Ying Ma. 2016. [Collaborative knowledge base embedding for recommender systems](#). In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13–17, 2016*, Balaji Krishnapuram, Mohak Shah, Alexander J. Smola, Charu C. Aggarwal, Dou Shen, and Rajeev Rastogi (Eds.). ACM Press, 353–362.
- Weizhen Zhang, Han Cao, Fei Hao, Lu Yang, Muhib Ahmad, and Yifei Li. 2019. [The Chinese Knowledge Graph on Domain-Tourism](#). In *Advanced Multimedia and Ubiquitous Engineering, MUE/FutureTech 2019*, James J. Park, Laurence T. Yang, Young-Sik Jeong, and Fei Hao (Eds.). Lecture Notes in Electrical Engineering, vol. 590. Springer, 20–27.
- Lei Zhang. 2002. [Knowledge Graph Theory and Structural Parsing](#). Ph.D. dissertation. University of Twente, Enschede, Netherlands.
- Mingbo Zhao, Tommy WS Chow, Zhao Zhang, and Bing Li. 2015. [Automatic image annotation via compact graph based semi-supervised learning](#). *Knowledge-based Systems* 76, 148–165.
- Suncong Zheng, Feng Wang, Hongyun Bao, Yuexing Hao, Peng Zhou, and Bo Xu. 2017. [Joint Extraction of Entities and Relations Based on a Novel Tagging Scheme](#). In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, Canada, July 30 - August*

- Volume 1: *Long Papers*, Regina Barzilay and Min-Yen Ka (Eds.). The Association for Computational Linguistics, 1227–1236.
- Weiguo Zheng, Jeffrey Xu Yu, Lei Zou, and Hong Cheng. 2018. [Question Answering Over Knowledge Graphs: Question Understanding Via Template Decomposition](#). *Proceedings of the VLDB Endowment* 11(11), 1373–1386.
- Bin Zhou and Jian Pei. 2008. [Preserving Privacy in Social Networks Against Neighborhood Attacks](#). In *Proceedings of the 24th International Conference on Data Engineering, ICDE 2008, April 7-12, 2008, Cancún, Mexico*, Gustavo Alonso, José A. Blakeley, and Arbee L. P. Chen (Eds.). IEEE Computer Society, 506–515.
- Bin Zhou and Jian Pei. 2011. [The \$k\$ -anonymity and \$l\$ -diversity approaches for privacy preservation in social networks against neighborhood attacks](#). *Knowledge and Information Systems* 28(1), 47–77.
- Guodong Zhou, Jian Su, Jie Zhang, and Min Zhang. 2005. [Exploring Various Knowledge in Relation Extraction](#). In *ACL 2005, 43rd Annual Meeting of the Association for Computational Linguistics, Proceedings of the Conference, 25-30 June 2005, University of Michigan, USA*, Kevin Knight, Hwee Tou Ng, and Kemal Oflazer (Eds.). The Association for Computational Linguistics, 427–434.
- Antoine Zimmermann, Nuno Lopes, Axel Polleres, and Umberto Straccia. 2012. [A General Framework for Representing, Reasoning and Querying with Annotated Semantic Web Data](#). *Journal of Web Semantics* 12, 72–95.
- Lei Zou, Lei Chen, and M. Tamer Özsu. 2009. [K-Automorphism: A General Framework For Privacy Preserving Network Publication](#). *Proceedings of the VLDB Endowment* 2(1), 946–957.

Appendix A

Background

We now discuss the broader historical context that has paved the way for the modern advent of knowledge graphs, and the definitions of the notion of “knowledge graph” that have been proposed both before and after the announcement of the Google Knowledge Graph [Singhal, 2012]. We remark that the discussion presented here builds upon (but does not subsume) previous discussion by Ehrlinger and Wöß [2016] and Bergman [2019], which we refer to for further details. Though our goal is to be comprehensive, the list of historical references should not be considered exhaustive.

A.1. HISTORICAL PERSPECTIVE

The lineage of knowledge graphs can be traced back to the origins of diagrammatic forms of knowledge representation: a tradition going back at least as far as Aristotle (~350 BC), followed by notions such as Euler circles and Venn diagrams that helped humans to reason through visual insights. Centuries later, a variety of researchers – particularly Sylvester [1878], Peirce [1878] and Frege [1879] – independently devised formal diagrammatic systems that not only facilitate reasoning, but also codify reasoning; in other words, their goal was to use diagrams as formal systems.

With the advent of digital computers, programs began to be used to perform formal reasoning and to code representations of knowledge. These developments can be traced back to works such as those of Richens [1958], Quillian [1963], and Travers and Milgram [1969], which focused on formal representations for natural language, information, and knowledge. These early works were limited (at least by modern standards) by the poor computational resources available. From the formal (logical) point of view, a number of influential developments took place in the 70’s, including the introduction of *frames* by Minsky [1974], the formalisation of *semantic networks* by Brachman [1977] and Woods [1975], and the proposal of *conceptual graphs* by Sowa [1979]. These works tried to integrate formal logic with diagrammatic representations of knowledge by giving a (more-or-less) formal semantics to graph representations. But as Sowa [1979] later wrote in the entry “*Semantic networks*”

Give us feedback!

the Encyclopedia of Cognitive Science: “Woods (1975) and McDermott (1976) observed, the semantic networks themselves have no well-defined semantics. Standard predicate calculus does have a precisely defined, model theoretic semantics; it is adequate for describing mathematical theories with a closed set of axioms. But the real world is messy, incompletely explored, and full of unexpected surprises.”

From this era of exploration and attempts to define programs to simulate the visual and formal reasoning of humans, the following key notions were established that are still of relevance today:

- knowledge representation using diagrams (often graphs) and visual means;
- computational procedures and algorithms to perform formal reasoning;
- combinations of formal (logical) and statistical forms of reasoning;
- relevance of diverse types of data (e.g., images, audio) as knowledge sources.

These works on conceptual graphs, semantic networks, and frames were direct predecessors of Description Logics, which aimed to give a well-defined semantics to these earlier notions towards building practical reasoning systems for decidable logics. Description Logics stem from the KL-ONE system proposed by Brachman and Schmolze [1985], and the “*attributive concept descriptions with complements*” language (aka *ALC*) proposed by Schmidt-Schauß and Smolka [1991]. Description Logics would be further explored in later years (see Section 4.3.2) and formed the underpinnings of the Web Ontology Language (OWL) standard [Hitzler et al., 2012]. Together with the Resource Description Framework (RDF) [Cyganiak et al., 2014], OWL would become one of the main building blocks of the Semantic Web [Berners-Lee et al., 2001], within which many of the formative ideas and standards underlying knowledge graphs would later be developed, including not only RDF and OWL, but also RDFS [Brickley and Guha, 2014], SPARQL [Harris et al., 2013], Linked Data principles [Berners-Lee, 2006], Shape Expressions [Labra Gayo et al., 2018], and indeed, many of the other concepts, standards and techniques discussed in this book. Most of the open knowledge graphs discussed in Section 10.1 – including BabelNet [Navigli and Ponzetto, 2012], DBpedia [Lehmann et al., 2015], Freebase [Bollacker et al., 2007a], Wikidata [Vrandečić and Krötzsch, 2014], YAGO [Suchanek et al., 2007], etc. – have either emerged from the Semantic Web community, or would later adopt the standards it proposes.

A.2. “KNOWLEDGE GRAPHS”: PRE-2012

Long before the 2012 announcement of the Google Knowledge Graph, various authors had used the phrase “knowledge graph” in publications stretching back to the 40’s, but with unrelated meaning. To the best of our knowledge, the first reference to a “knowledge graph” of relevance to the modern meaning was in a paper by Schneider [1973] in the area of computerised instructional systems for education, where a knowledge graph – in his case a directed graph whose nodes are units of knowledge (concepts) that a student should acquire, and whose edges denote dependencies between such units of knowledge – is used to represent and store an instructional course on a computer. An analogous notion of a “knowledge graph” was used by Marchi and Miguel [1974] to study paths through the knowledge units of an instructional course that yield the highest payoffs for teachers and students in a game-theoretic sense. Around the same time, in a paper on linguistics, Kümmel [1973] describes a numerical representation of knowledge, with “radicals” – referring to some symbol with meaning – forming the nodes of a knowledge graph.

Further authors were to define instantiations of knowledge graphs in the 80’s. Rada [1986] defines a knowledge graph in the context of medical expert systems, where domain knowledge is defined as a weighted graph, over which a “gradual” learning process is applied to refine knowledge by making small change to weights. Bakker [1987] defines a knowledge graph with the purpose of cumulatively representing content gleaned from medical and sociological texts, with a focus on causal relationships. Work on knowledge graphs from the same group would continue over the years, with contributions by Stokman and de Vries [1988] further introducing mereological (*part of*) and instantiation (*is a*) relations to the knowledge graph, and thereafter by James [1992], Hoede [1995], Zhang [2002], Popping [2003], amongst others, in the decades that followed [Nurdiati and Hoede, 2012]. The notion of knowledge graph used in such works considered a fixed number of relations. Other authors pursued their own parallel notions of knowledge graphs towards the end of the 80’s. Rappaport and Gouyet [1988] describe a user interface for visualising a knowledge-base – composed of fa

Give us feedback!

and rules – using a knowledge graph that connects related elements of the knowledge-base. [Srikanth and Jarke \[1989\]](#) use the notion of a knowledge graph to represent the entities and relations involved in projects, particularly software projects, where partitioning techniques are applied to the knowledge graph to modularise the knowledge required in the project.

Continuing to the 90's, the notion of a “knowledge graph” would again arise in different, seemingly independent settings. [De Raedt et al. \[1990\]](#) propose a knowledge graph as a directed graph composed of a taxonomy of instances being related with weighted edges to a taxonomy of classes; they use symbolic learning to extract such knowledge graphs from examples. [Machado and Freitas da Rocha \[1990\]](#) define a knowledge graph as an acyclic, weighted *and-or* graph,³⁹ defining fuzzy dependencies that connect observations to hypotheses through intermediary nodes. These knowledge graphs are elicited from domain experts and can be used to generate neural networks for selecting hypotheses from input observations. Knowledge graphs were again later used by [Dieng et al. \[1992\]](#) to represent the results of knowledge acquisition from experts. [Shimony et al. \[1997\]](#) rather define a knowledge graph based on a *Bayesian knowledge base* – i.e., a Bayesian network that permits directed cycles – over which Bayesian inference can be applied. This definition was further built upon in a later work by [Santos Jr. and Santos \[1999\]](#).

note 39 An *and-or* graph of sub-goals on which

Moving to the 00's, [Jiang and Ma \[2002\]](#) introduce the notion of “plan knowledge graphs” where nodes represent goals and edges dependencies between goals, further encoding supporting degrees that can change upon further evidence. Search algorithms are then defined on the graph to determine a plan for a particular goal. [Helms and Buijsrogge \[2005\]](#) propose a knowledge graph to represent the flow of knowledge in an organisation, with nodes representing knowledge actors (creators, sharers, users), edges representing knowledge flow from one actor to another, and edge weights indicating the “velocity” (delay of flow) and “viscosity” (the depth of knowledge transferred). Graph algorithms are then proposed to find bottlenecks in knowledge flow. [Kasneci et al. \[2008\]](#) propose a search engine for knowledge graphs, defined to be weighted directed edge-labelled graphs, where weights denote confidence scores based on the centrality of source documents from which the edge/relation was extracted. From the same group, [Elbassuoni et al. \[2009\]](#) adopt a similar notion of a knowledge graph, adding edge attributes to include keywords from the source, a count of supporting sources, etc., showing how the graph can be queried. [Coursey and Mihalcea \[2009\]](#) construct a knowledge graph from Wikipedia, where nodes represent Wikipedia articles and categories, while edges represent the proximity of nodes. Given an input text, entity linking and centrality measures are applied over the knowledge graph to determine relevant Wikipedia categories for the text.

Concluding with the 10's (prior to 2012), [Pechsiri and Piriyaikul \[2010\]](#) use knowledge graphs to capture “explanation knowledge” – the knowledge of why something is the way it is – by representing events as nodes and causal relationships as edges, claiming that this graphical notation offers more intuitive explanations to users; their work focuses on extracting such graphs from text. [Corby and Faron-Zucker \[2010\]](#) use the phrase “knowledge graph” in a general way to denote any graph encoding knowledge, proposing an abstract machine for querying such graphs.

Other phrases were used to represent similar notions by other authors, including “information graphs” [[Kümmel, 1973](#)], “information networks” [[Sun et al., 2011](#)], “knowledge networks” [[Ciampaglia et al., 2015](#)], as well as “semantic networks” [[Brachman, 1977](#), [Woods, 1975](#), [Navigli and Ponzetto, 2012](#)] and “conceptual graphs” [[Sowa, 1979](#)], as mentioned previously. Here we exclusively considered works that (happen to) use the phrase “knowledge graph” prior to Google's announcement of their knowledge graph in 2012, where we see that many works had independently coined this phrase for different purposes. Similar to the current practice, all of the works of this period consider a knowledge graph to be formed of a set of nodes denoting entities of interest and a set of edges denoting relations between those entities, with different entities and relations being considered in different works. Some works add extra elements to these knowledge graphs, such as edge weights, edge labels, or other metadata [[Elbassuoni et al., 2009](#)]. Other trends include knowledge acquisition from experts [[Rada, 1986](#), [Machado and Freitas da Rocha, 1990](#), [Dieng et al., 1992](#)] and knowledge extraction from text [[Bakker, 1987](#), [Stokman and de Vries, 1988](#), [James, 1992](#), [Hoede, 1995](#)], combinations of symbolic and inductive methods [[Machado and Freitas da Rocha, 1990](#), [De Raedt et al., 1990](#), [Shimony et al., 1997](#), [Santos Jr. and Santos, 1999](#)], as well as the use of rules [[Rappaport and Gouyet, 1988](#)], ontologies [[Hoede, 1995](#)], graph analytics [[Srikanth and Jarke, 1989](#), [Helms and Buijsrogge, 2005](#), [Kasneci et al., 2008](#)], learning [[Rada, 1986](#), [De Raedt et al., 1990](#), [Shimony et al., 1997](#), [Santos Jr. and Santos, 1999](#)], amongst other techniques. Later papers

Give us feedback!

(2008–2010) by [Kasneji et al. \[2008\]](#), [Elbassuoni et al. \[2009\]](#), [Coursey and Mihalcea \[2009\]](#) and [Corby and Faron-Zucker \[2010\]](#) introduce notions of “knowledge graph” that are more similar to the current practice.

However, some trends are not reflected in current practice. Of note is that many of the knowledge graphs defined in this period consider edges as denoting a form of dependence or causality, where $x \rightarrow y$ may denote that x is a prerequisite for y [[Schneider, 1973](#), [Marchi and Miguel, 1974](#), [Jiang and Ma, 2002](#)] or that x leads to y [[Rada, 1986](#), [Bakker, 1987](#), [Rappaport and Gouyet, 1988](#), [Machado and Freitas da Rocha, 1990](#), [Shimony et al., 1997](#), [Jiang and Ma, 2002](#)]. In some cases *and-or* graphs are used to denote conjunctions or disjunctions of such relations [[Machado and Freitas da Rocha, 1990](#)], while in other cases edges are weighted to assign a belief to a relation [[Machado and Freitas da Rocha, 1990](#), [Jiang and Ma, 2002](#), [Rada, 1986](#)]. Papers from 1970–2000 tend to have worked with small graphs, which contrasts with modern practice where knowledge graphs can reach scales of millions or billions of nodes [[Noy et al., 2019](#)]: during this period, computational resources were more limited [[Schneider, 1973](#)], and fewer sources of structured data were readily available, meaning that the knowledge graphs were often sourced solely from human experts [[Rada, 1986](#), [Machado and Freitas da Rocha, 1990](#), [Dieng et al., 1992](#)] or from text [[Bakker, 1987](#), [Stokman and de Vries, 1988](#), [James, 1992](#), [Hoede, 1995](#)].

A.3. “KNOWLEDGE GRAPHS”: 2012 ONWARDS

Google Knowledge Graph was announced in 2012 [[Singhal, 2012](#)]. This initial announcement was targeted at a broad audience, mainly motivating the knowledge graph and describing applications that it would enable, where the knowledge graph itself is described as “[a graph] that understands real-world entities and their relationships to one another” [[Singhal, 2012](#)]. Mentions of “knowledge graphs” gained momentum in the research literature from that point. As noted by [Bergman \[2019\]](#), this announcement by Google was a watershed moment for adopting the phrase “knowledge graph”. However, given the informal nature of the announcement, a technical definition was lacking [[Ehrlinger and Wöß, 2016](#), [Bonatti et al., 2018](#)].

Given that knowledge graphs were gaining more and more attention not only in practice, but also in the academic literature, formal definitions were becoming a necessity in order to precisely characterise what they were, how they were structured, how they could be used, etc., and more generally to facilitate their study in a precise manner. We can determine four general categories of definitions that have emerged.

Category I: The first category simply defines the knowledge graph as a graph where nodes represent entities, and edges represent relationships between those entities. Often a directed edge-labelled graph is assumed (or analogously, a set of named binary relations, or a set of triples). This simple definition was popularised by seminal papers on knowledge graph embeddings [[Wang et al., 2014](#), [Lin et al., 2015](#)], being sufficient to represent the data model upon which these embeddings operate. As reflected in the later survey by [Wang et al. \[2017\]](#), the multitude of works that would follow on knowledge graph embeddings have continued to use this definition. Though simple, the *Category I* definition raised some doubts: How is a knowledge graph different from a graph (database)? Where does knowledge come into play?

Category II: A second common definition goes as follows: “a knowledge graph is a graph-structured knowledge base”. To the best of our knowledge, the earliest usages of this definition in the literature were by [Nickel et al. \[2016a\]](#) and [Seufert et al. \[2016\]](#) (interestingly in the formal notation of these initial papers, a knowledge graph is defined analogously to a directed edge-labelled graph). Such a definition raises the question: what, then, is a “knowledge base”? The phrase “knowledge base” was popularised in the 70’s (possibly earlier) in the context of rule-based expert systems [[Buchanan and Feigenbaum, 1978](#)], and later were used in the context of ontologies and other logical formalisms [[Brachman and Schmolze, 1985](#)]. The follow-up question then is: can we have a knowledge base (graph-structured or not) without a logical formalism while staying true to the original definitions? Looking in further detail, similar ambiguities have also existed regarding the definition of a “knowledge base” (KB). Of note: [Brachman and](#)

Levesque [1986] – reporting after a workshop on this issue – state that “if we ask what the KB tells us about the world, we are asking about its Knowledge Level”.

Category III: The third category of definitions outline more specific technical characteristics that a “knowledge graph” should comply with.

- In an influential survey on knowledge graph refinement, Paulheim [2017] lists four criteria that characterise the knowledge graphs considered for the paper. Specifically, he puts forward that a knowledge graph “mainly describes real world entities and their interrelations, organized in a graph; defines possible classes and relations of entities in a schema; allows for potentially interrelating arbitrary entities with each other; covers various topical domains”; he thus rules out ontologies without instances (e.g., DOLCE) and graphs of word senses (e.g., WordNet) as not meeting the first two criteria, while relational databases do not meet the third criterion (due to schema restrictions), and domain-specific graphs (e.g., Geonames) are considered to not meet the fourth criterion; this leaves graphs such as DBpedia, YAGO, Freebase, etc.
- Ehrlinger and Wöß [2016] also review definitions of “knowledge graph”, where they criticise the *Category II* definitions based on the argument that knowledge bases are often synonymous with ontologies⁴⁰, while knowledge graphs are not; they further criticise Google for calling its knowledge graph a “knowledge base”. After reviewing prior definitions of terms such as “knowledge base”, “ontology”, and “knowledge graph”, they propose their definition: “A knowledge graph acquires and integrates information into an ontology and applies a reasoner to derive new knowledge”. In the subsequent discussion, they remark that a knowledge graph is distinguished from an ontology (considered synonymous with a knowledge base) by the provision of reasoning capabilities.
- One of the most detailed technical definitions for a “knowledge graph” is provided by Bellomarini et al. [2019], who state: “A knowledge graph is a semi-structured data model characterized by three components: (i) a ground extensional component, that is, a set of relational constructs for schema and data (which can be effectively modeled as graphs or generalizations thereof); (ii) an intensional component, that is, a set of inference rules over the constructs of the ground extensional component; (iii) a derived extensional component that can be produced as the result of the application of the inference rules over the ground extensional component (with the so-called “reasoning” process).” They remark that ontologies and rules represent analogous structures, and that a knowledge graph is then a knowledge base extended with reasoning along similar lines to the definition provided by Ehrlinger and Wöß [2016].

note 40 Prior definitions contradict this conclusion

We refer to Bergman [2019] for a list of further definitions that fit *Category III*. While having a specific, technical definition for knowledge graphs provides a more solid foundation for their study, as Bergman [2019] remarks, many such definitions do not seem to fit the current practice of knowledge graphs. For example, it is not clear which of these definitions the Google Knowledge Graph itself – responsible for popularising the idea – would meet (if any). Many of the criteria proposed by such definitions are also orthogonal to the many works in the area of knowledge graph embeddings [Wang et al., 2017].

Category IV: While the previous three categories involve (sometimes conflicting) intensional definitions, the fourth category adopts an extensional definition of knowledge graphs, defining them in terms of prominent examples of knowledge graphs, such as DBpedia, Google’s Knowledge Graph, Freebase, YAGO, amongst others [Bonatti et al., 2018]. Arguably this category avoids the issue of defining a knowledge graph, rather than actually defining them.

These categories refer to definitions that have appeared in the academic literature. In terms of enterprise knowledge graphs, an important reference is the paper of Noy et al. [2019], which has been co-authored by leaders of knowledge graph projects from eBay, Facebook, Google, IBM, and Microsoft, and thus can be seen as representing a form of consensus amongst these companies – who have played a key role in the popularisation of knowledge graphs – on what a “knowledge graph” means in this setting. Specifically this paper states that “a knowledge graph describes objects of interest and connections between them”, and goes on to state that “many practical implementations impose constraints on the links in knowledge graphs by defining a schema or ontology”. They later add “Knowledge graphs and similar structures usually provide a shared substrate of knowledge within an organization, allowing c

Give us feedback!

ferent products and applications to use similar vocabulary and to reuse definitions and descriptions that others create. Furthermore, they usually provide a compact formal representation that developers can use to infer new facts and build up the knowledge”. We interpret this definition as corresponding to *Category I*, but further acknowledging that while not a necessary condition for a knowledge graph, ontologies and formal representations *usually* play a key role. The definition we provide at the outset of the paper is largely compatible with that of [Noy et al. \[2019\]](#).

Authors' Biographies

AIDAN HOGAN

Aidan Hogan is an Associate Professor at the Department of Computer Science, Universidad de Chile, where he also holds the position of Associate Researcher in the Millennium Institute for Foundational Research on Data (IMFD). He received a B.Eng. and Ph.D. from the National University of Ireland, Galway, in 2006 and 2011, respectively. His primary research interests centre on the Semantic Web and Knowledge Graphs. He is the author of over one hundred research publications on these topics, including two other books: “Reasoning Techniques for the Web of Data” and “The Web of Data”.

EVA BLOMQVIST

Eva Blomqvist is an Associate Professor at the Department of Computer and Information Science, Linköping University. She received a Ph.D. from Linköping University, Sweden, in 2009, in the area of Ontology Learning for the Semantic Web. After a postdoc at ISTC-CNR in Rome, Italy, she has been a member of the Semantic Web group at Linköping University since 2011. Her primary research interests include the Semantic Web and Knowledge Graphs, more specifically the development and use of ontologies as schemas for Knowledge Graphs. She is the author of over fifty research publications in the area, and has served as scientific program chair of several of the top conferences in the field.

MICHAEL COCHEZ

Michael Cochez is an Assistant Professor in the Knowledge Representation and Reasoning Group at the Computer Science department of the Vrije Universiteit, Amsterdam. He received his B.Sc. from the University of Antwerp, Belgium and his M.Sc. and Ph.D. degrees from the University of Jyväskylä, Finland. His research interests are in the intersection of Machine Learning and Knowledge Graphs.

CLAUDIA D'AMATO

Claudia d'Amato is an Associate Professor at the Department of Computer Science, University of Bari, Italy and a member of the Knowledge Acquisition and Machine Learning Lab. She also holds a habilitation as Full Professor for the scientific sectors: INF/01 and ING-INF/05. She received her Masters Degree and Ph.D. from the University of Bari, Italy, in 2003 and 2007, respectively. Over the years, she has also spent several invited-researcher stays in different international universities and research

[Give us feedback!](#)

institutes. Her primary research interests centre on Machine Learning for the Semantic Web and Knowledge Graphs. She is the author of over one hundred research publications on these topics.

GERARD DE MELO

Gerard de Melo is a Full Professor at the Hasso Plattner Institute for Digital Engineering and at the University of Potsdam, where he holds the Chair for Artificial Intelligence and Intelligent Systems and heads the corresponding research group. Previously, he was a faculty member at Rutgers University in New Jersey and at Tsinghua University in Beijing, and a Post-Doctoral Research Scholar at ICSI/UC Berkeley. He has published over 150 papers on natural language processing, knowledge graphs, and AI, and received a number of best paper awards.

CLAUDIO GUTIERREZ

Claudio Gutierrez is Full Professor at the Department of Computer Science, Universidad de Chile. He is also a Senior Researcher in the Millennium Institute for Foundational Research on Data (IMFD). His main research interests are the computational foundations of data and knowledge. He has worked and published extensively in the areas of the Semantic Web and Databases, fields in which he received test of time awards (ISWC and PODS). He also devotes time to research in the field of the History of Science and Technology.

SABRINA KIRrane

Sabrina Kirrane is an Assistant Professor at the Vienna University of Economics and Business Institute for Information Systems and New Media, where she is also a member of the Research Institute for Cryptoeconomics and the Sustainable Computing Lab. Her research interests include Security, Privacy, and Policy aspects of the Next Generation Internet (NGI), Distributed and Decentralised Systems, Big Data and Data Science, with a particular focus on policy representation and reasoning (e.g., access constraints, usage policies, regulatory obligations, societal norms, business processes), and the development of transparency and trust techniques for the Web.

JOSE EMILIO LABRA GAYO

Jose Emilio Labra Gayo is an Associate Professor at the University of Oviedo, Spain. He founded the WESO (Web Semantics Oviedo) research group in 2004, whose main goal is to apply semantic technologies to solve practical problems. He was a member of the W3C Data Shapes working group and is a member of the W3C Community Groups: Shape Expressions and SHACL. He is coauthor of the “Validating RDF data” book and maintains the ShEx and SHACL library SHaCLEX as well as the on-line tools RDFShape and Wikishape. Previously, he was coordinator of the Master in Web Engineering and Dean of the School of Computer Science Engineering at the University of Oviedo (2004–2012).

ROBERTO NAVIGLI

Roberto Navigli is a Full Professor of Computer Science at the Sapienza University of Rome, where he leads the Sapienza NLP Group. His research is focused on multilingual Natural Language Understanding, a field in which he received two grants of the European Research Council. In 2015 he received the META prize for groundbreaking work in overcoming language barriers with the BabelNet lexical-semantic knowledge graph, a project also highlighted in The Guardian and Time magazine, and winner of the Artificial Intelligence Journal prominent paper award 2017. He is the co-founder of Babelscape, a successful company which enables Natural Language Understanding in dozens of languages.

Give us feedback!

SEBASTIAN NEUMAIER

Sebastian Neumaier is a researcher in the Data Intelligence group at the St. Poelten University of Applied Sciences, Austria. He received an M.Sc. and Ph.D. from the Vienna University of Technology, in 2015 and 2019, respectively. His Ph.D. thesis is centred around methods to facilitate the integration and semantic enrichment of Open Data sources using Knowledge Graph technologies. His current research focuses on different aspects of semantic data management.

AXEL-CYRILLE NGONGA NGOMO

Axel-Cyrille Ngonga Ngomo is a Full Professor for Data Science at Paderborn University. He obtained his M.Sc., Ph.D. and habilitation from the University of Leipzig, where he also led the Agile Knowledge Engineering and Semantic Web Group. His research focuses on the automation of the life-cycle of knowledge graphs. Thus, his works include the development of approaches for the extraction, integration, fusion, storage, analysis and exploitation of knowledge graphs.

AXEL POLLERES

Axel Polleres heads the Institute for Data, Process and Knowledge Management of Vienna University of Economics and Business (WU Wien), which he joined in September 2013 as a Full Professor in the area of “Data and Knowledge Engineering”. He is also a faculty member of the Complexity Science Hub Vienna and was a visiting professor at Stanford University in 2018. He obtained his Ph.D. and habilitation from Vienna University of Technology. His research focuses on ontologies, query languages, logic programming, configuration technologies, Artificial Intelligence, Semantic Web, Linked Open Data, Knowledge Graphs and their applications for Knowledge Management. Moreover, he actively contributed to international standardisation efforts within the World Wide Web Consortium (W3C) where he co-chaired the W3C SPARQL working group.

SABBIR M. RASHID

Sabbir M. Rashid is a Ph.D. candidate at Rensselaer Polytechnic Institute (RPI) working with Deborah L. McGuinness on research related to data annotation and harmonisation, ontology engineering, knowledge representation, and various forms of reasoning. Prior to RPI, Sabbir completed a double major at Worcester Polytechnic Institute, where he received B.Sc. degrees in both Physics and Electrical & Computer Engineering. Much of his graduate studies at RPI have involved research related to data annotation and transformation using Semantic Data Dictionaries. His current work includes the application of deductive and abductive inference techniques over Linked Health Data, such as in the context of chronic diseases like diabetes.

ANISA RULA

Anisa Rula is an Assistant Professor in Computer Science at the Department of Information Engineering, University of Brescia since January 2021 and a researcher at the University of Bonn in the SDA group since January 2017. She obtained her doctoral degree in Computer Science from the University of Milano-Bicocca in 2014. Her research interests are in the intersection of semantic knowledge technologies and data quality with a particular focus on data integration. She is researching new solutions to data integration with respect to the quality of data modelling and efficient solutions for large-scale data sources. Recently she has been working on data understanding for large and complex datasets, on knowledge extraction, and on semantic data enrichment and refinement.

LUKAS SCHMELZEISEN

[Give us feedback!](#)

Lukas Schmelzeisen is a PhD candidate working with Steffen Staab in the Analytic Computing group at the University of Stuttgart, Germany. He holds a B.Sc. in Computer Science, which he received in 2015 at University of Koblenz–Landau. His main research interests are continuous representations of both natural language corpora and knowledge graphs. In particular, his current focus is on how such representations can be updated over time.

JUAN SEQUEDA

Juan Sequeda is the Principal Scientist at data.world. He joined through the acquisition of Capsenta, a company he founded as a spin-off from his research. His academic and industry work has been on designing and building Knowledge Graph for enterprise data integration where he has researched and developed technologies for semantic and graph data virtualisation, ontology and graph data modelling and schema mapping, and data integration methodologies. Juan holds a Ph.D. in Computer Science from the University of Texas at Austin. He is the recipient of the NSF Graduate Research Fellowship, received 2nd Place in the 2013 Semantic Web Challenge for his work on ConstituteProject.org, Best Student Research Paper at International Semantic Web Conference 2014 and the 2015 Best Transfer and Innovation Project awarded by the Institute for Applied Informatics. Juan bridges academia and industry through standardisation committees, being a co-chair of the Property Graph Schema Working Group, and past member of the Graph Query Languages task force of the Linked Data Benchmark Council (LDBC), as well as a past invited expert member and standards editor at the World Wide Web Consortium (W3C).

STEFFEN STAAB

Steffen Staab holds a Cyber Valley endowed chair for Analytic Computing at the University of Stuttgart, Germany, and a chair for Web and Computer Science at the University of Southampton, UK. Steffen is a fellow of the European Association for Artificial Intelligence. His research interests range from knowledge graphs and machine learning to the semantics of human–computer interaction. He is co-director of the Interchange Forum for Reflecting on Intelligent Systems (IRIS) at the University of Stuttgart.

ANTOINE ZIMMERMANN

Antoine Zimmermann is an Associate Professor at Mines Saint-Étienne in France. He received an M.Sc. and a Ph.D. degree from the University of Grenoble, France in 2004 and 2008 respectively. He spent two years at the Digital Enterprise Research Institute in Galway, Ireland, from 2009 to 2010, then one year at INSA Lyon, France, before getting a position at Mines Saint-Étienne, where he has been a permanent researcher since 2012. In 2021, he received his habilitation from Université Jean Monnet, Saint-Étienne. His research interests are related to the Semantic Web, more specifically on knowledge representation, knowledge engineering, reasoning, data management and context on the Web.

🐦 The End 🐦